



## **UE24CS352A: Machine Learning**

---

**Dr. Shylaja S S**

Professor

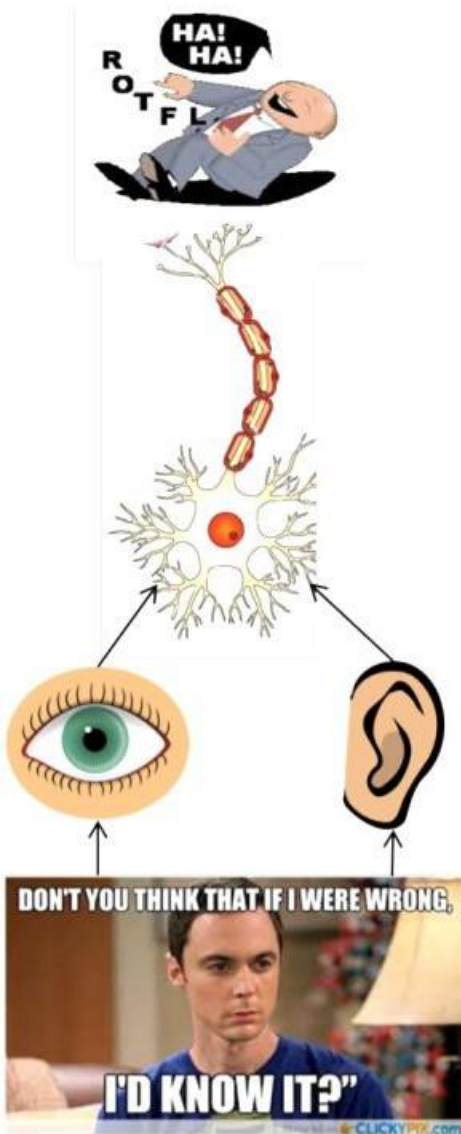
Department of Computer Science & Engineering

Acknowledgement : Dr. Rajanikanth K (Academic Advisor, PESU)  
Teaching Assistant (Arya Rajiv Chaloli)

- Most fundamental unit of ANN or DNN is called artificial neuron or just neuron.
- The inspiration comes from Biology, more specifically the brain.

Their name and structure are inspired by the human brain as it mimics the way biological neurons pass signals to one another.

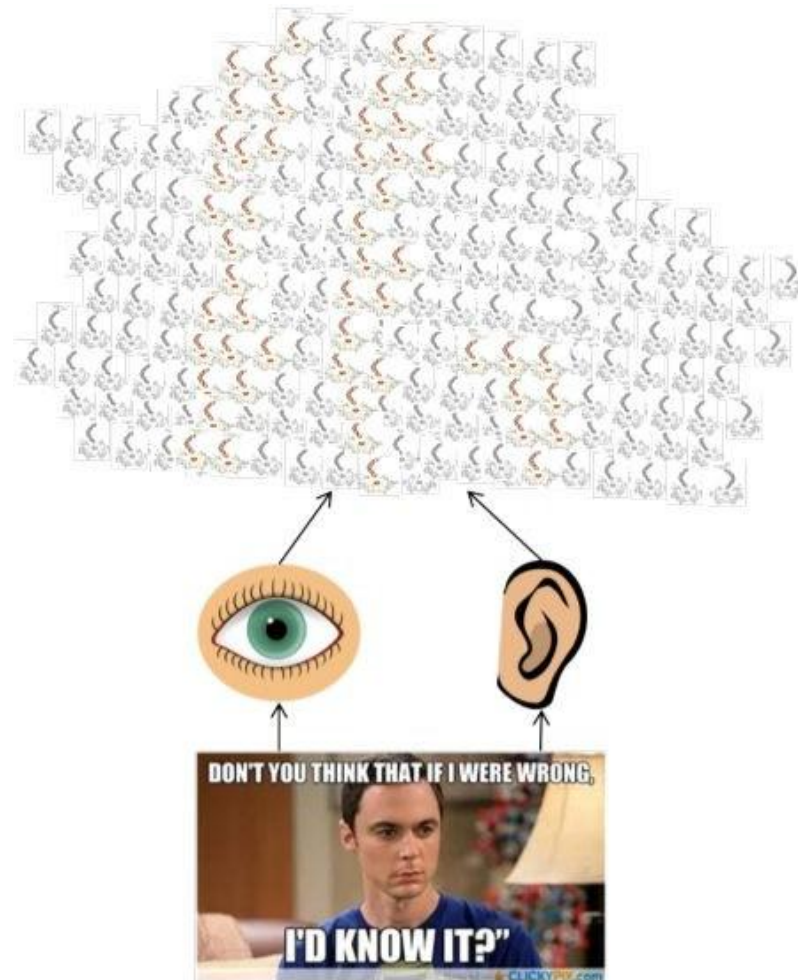
### Illustrating how a biological neuron works



In a real world scenario :

- Our sense organs interact with the outside world.
- They relay information to the neurons.
- The neurons (may) get activated and produces a response (laughter).

### Illustrating how a biological neuron works

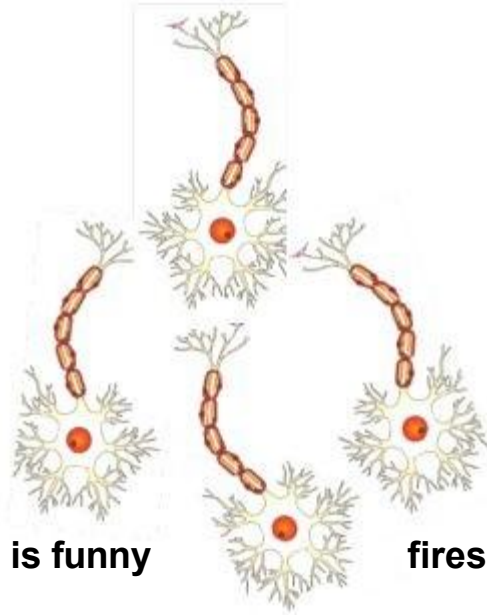


- It is not just a single neuron which does all this.
- There is a **massively parallel interconnected network of neurons**.
- The sense organs relay information to the lowest layer of neurons.
- Some of these neurons may fire (in red) in response to this information and in turn relay information to other neurons they are connected to.
- These neurons may also fire (again, in red) and the process continues.
- An average human brain has around 100 billion neurons!



### Illustrating how a biological neuron works

fires if at least 2 out of 3 inputs fired



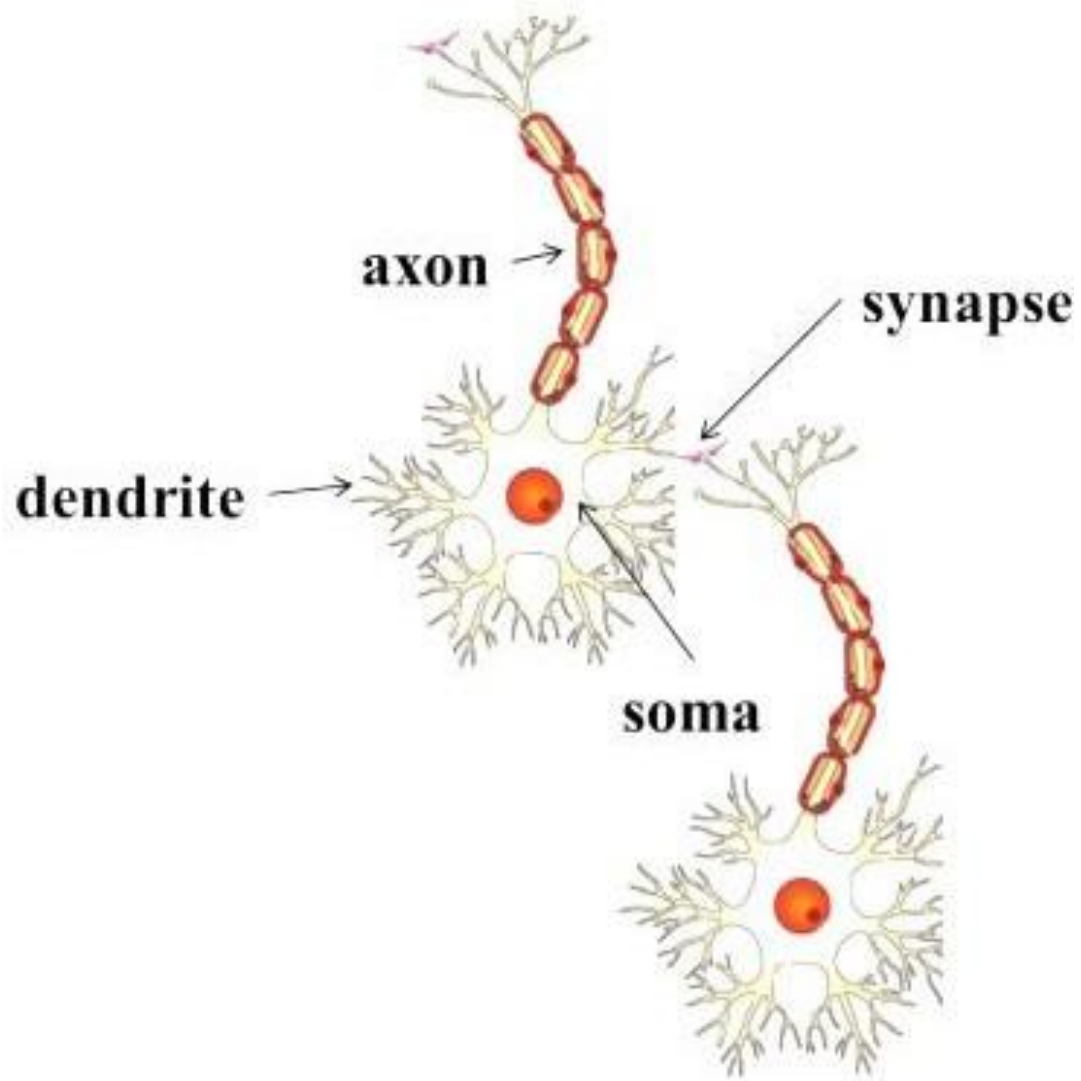
fires if visual is funny

fires if text is funny

fires if speech style is funny



- This massively parallel network also ensures that there is division of work.
- Each neuron may perform a certain role or respond to a certain stimulus.



**Dendrite:** Obtain information from other cells and carry that information to the cell body. (input)

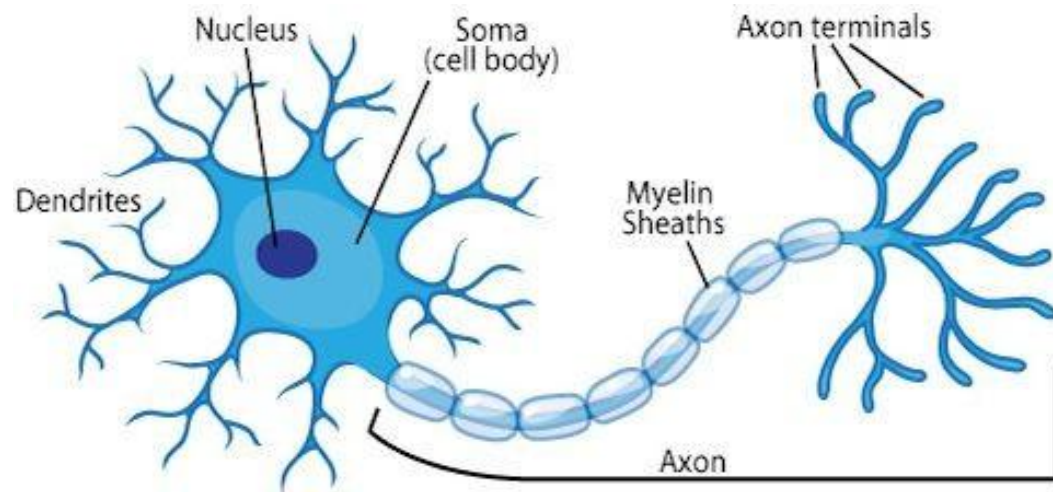
**Soma:** Carries genetic information, maintains the neuron's structure, and provides energy to drive activities. (processing)

**Axon:** Carries nerve impulses away from the cell body. (output)

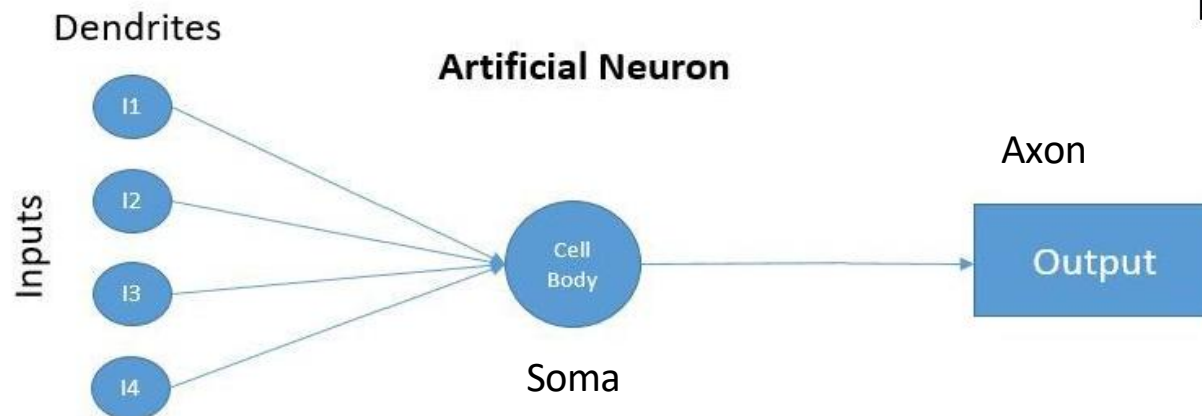
**Synapse:** Small pocket of space between two cells where they can pass messages to communicate. It is a point of connection between two neurons.

## Unit 2 - ANN

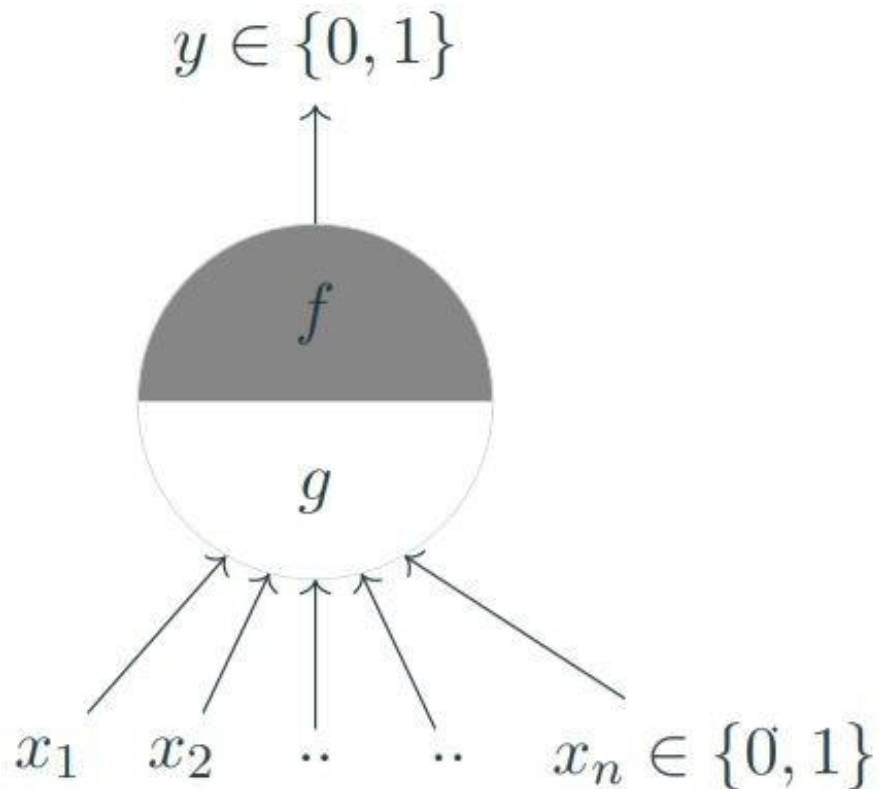
### Corresponding parts of a biological and artificial neuron



| Biological Neuron  | Artificial Neuron |
|--------------------|-------------------|
| Dendrites          | Input             |
| Cell Nucleus(Soma) | Node              |
| Axon               | Output            |
| Synapse            | Interconnections  |



### MP Neuron: McCulloch Pitts neuron - Thresholding logic



- McCulloch and Pitts proposed a highly simplified computational model of the neuron.
- $g$  aggregates the inputs and the function  $f$  takes a decision based on this aggregation.
- The inputs can be excitatory or inhibitory.
- $y = 0$  if any  $x_i$  is inhibitory, else

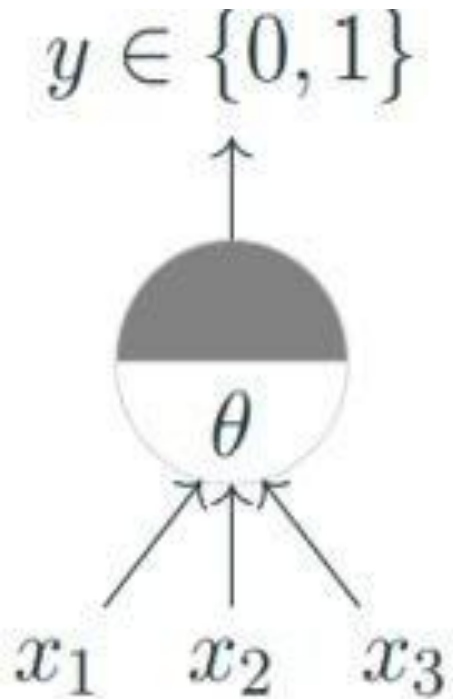
$$g(x_1, x_2, \dots, x_n) = g(\mathbf{x}) = \sum_{i=1}^n x_i$$

$$y = f(g(\mathbf{x})) = 1 \quad \text{if} \quad g(\mathbf{x}) \geq \theta$$
$$= 0 \quad \text{if} \quad g(\mathbf{x}) < \theta$$

- $\theta$  is called the thresholding parameter.

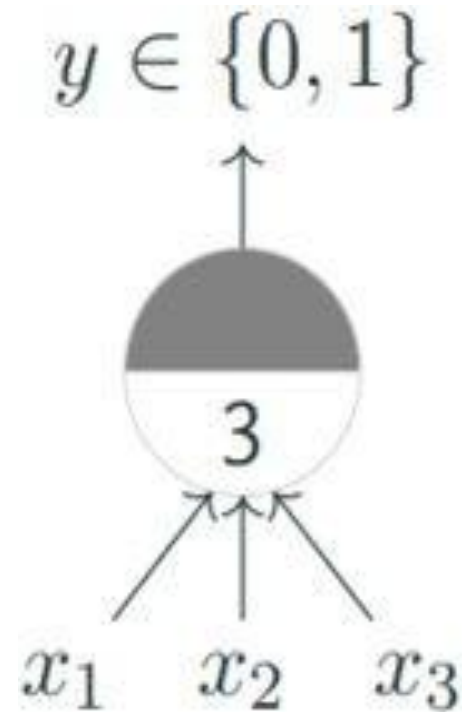
### MP Neuron: Representing some Boolean functions

MP unit



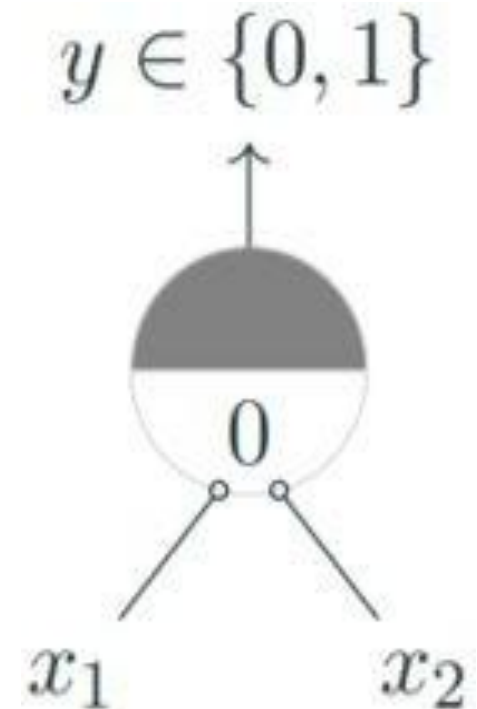
| x1 | x2 | x3 | y |
|----|----|----|---|
| 0  | 0  | 0  | 0 |
| 0  | 0  | 1  | 0 |
| 0  | 1  | 0  | 0 |
| 0  | 1  | 1  | 0 |
| 1  | 0  | 0  | 0 |
| 1  | 0  | 1  | 0 |
| 1  | 1  | 0  | 0 |
| 1  | 1  | 1  | 1 |

AND function



| x2 | x3 | y |
|----|----|---|
| 0  | 0  | 1 |
| 0  | 1  | 0 |
| 1  | 0  | 0 |
| 1  | 1  | 0 |

NOR function



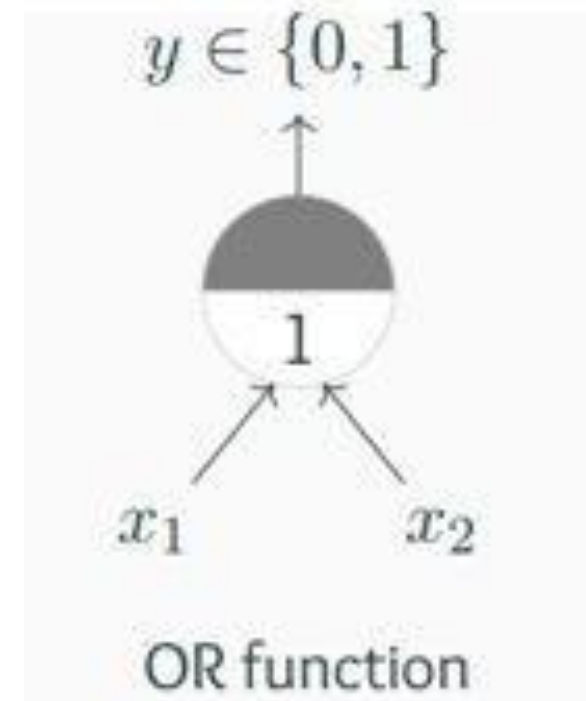
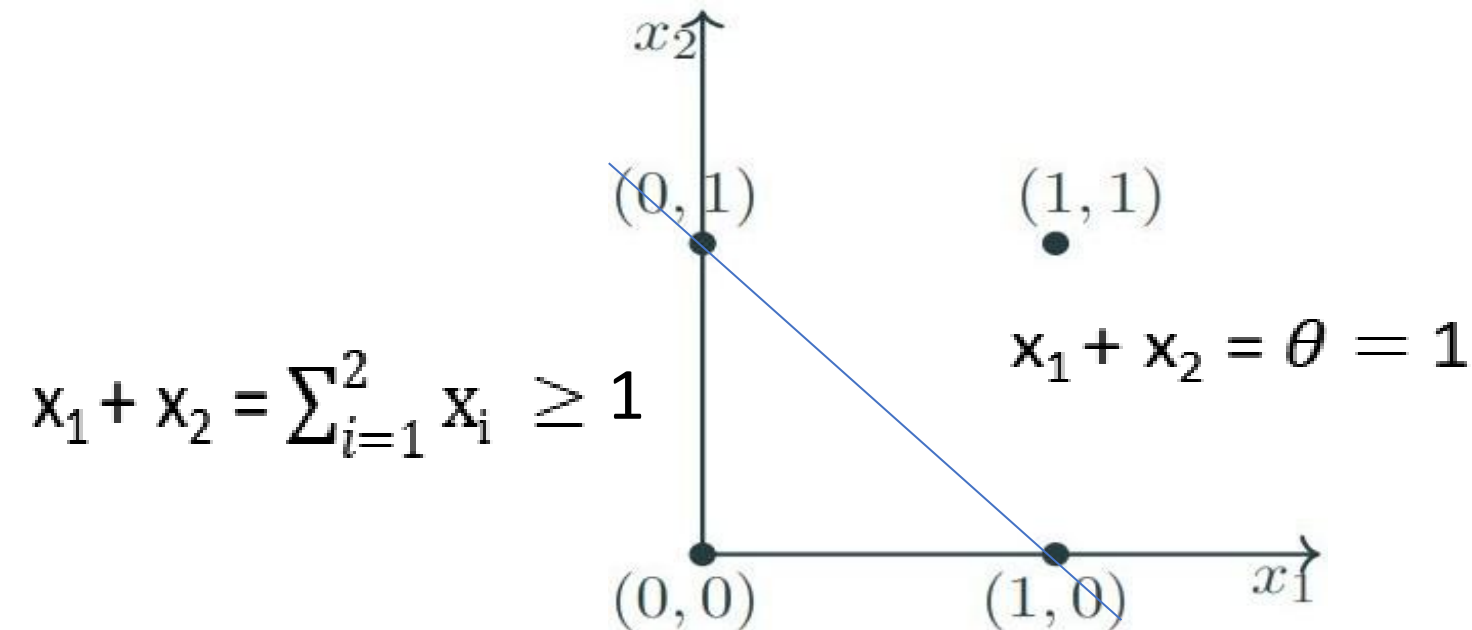
### MP Neuron: Geometric interpretation – OR function

A single MP neuron splits the input points (4 points for 2 binary inputs) into two halves.

Points lying on or above the line  $\sum_{i=1}^n x_i - \theta = 0$  and points lying below this line.

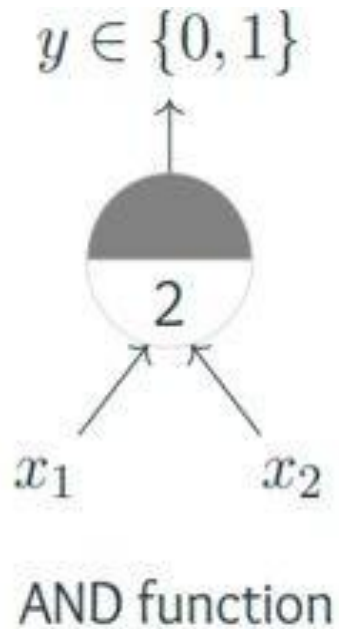
All inputs which produce an output 0 will be on one side  $(\sum_{i=1}^n x_i < \theta)$

All inputs which produce an output 1 will lie on the other side  $(\sum_{i=1}^n x_i \geq \theta)$

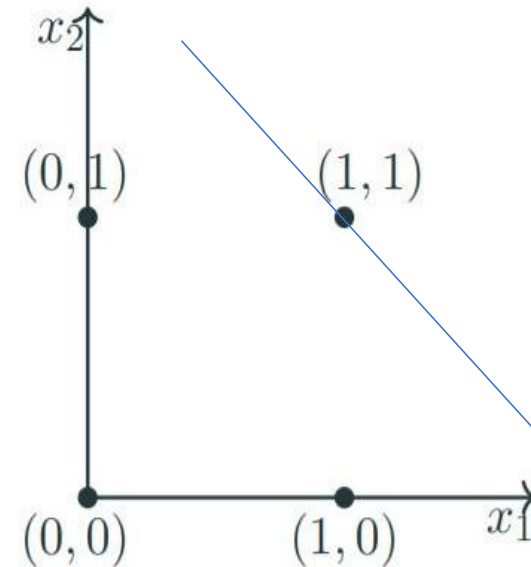




### MP Neuron: Geometric interpretation – AND function



$$x_1 + x_2 = \sum_{i=1}^2 x_i \geq 2$$



$$x_1 + x_2 = \theta = 2$$

A single MP Neuron can be used to represent Boolean functions which are linearly separable.

#### Linear separability (for Boolean functions)

There exists a line (plane) such that all inputs which produce a 1 lie on one side of the line (plane) and all inputs which produce a 0 lie on other side of the line (plane).

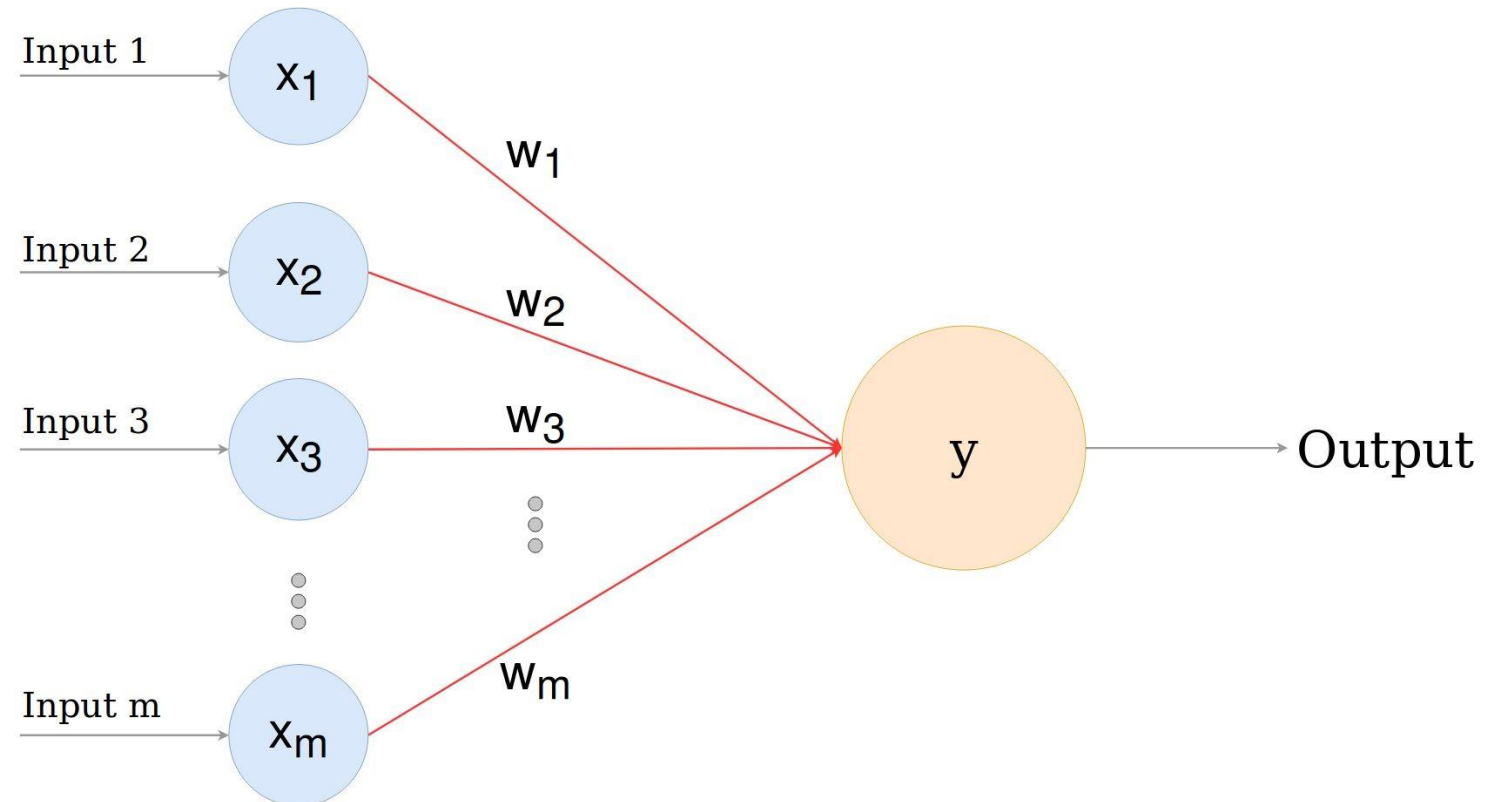


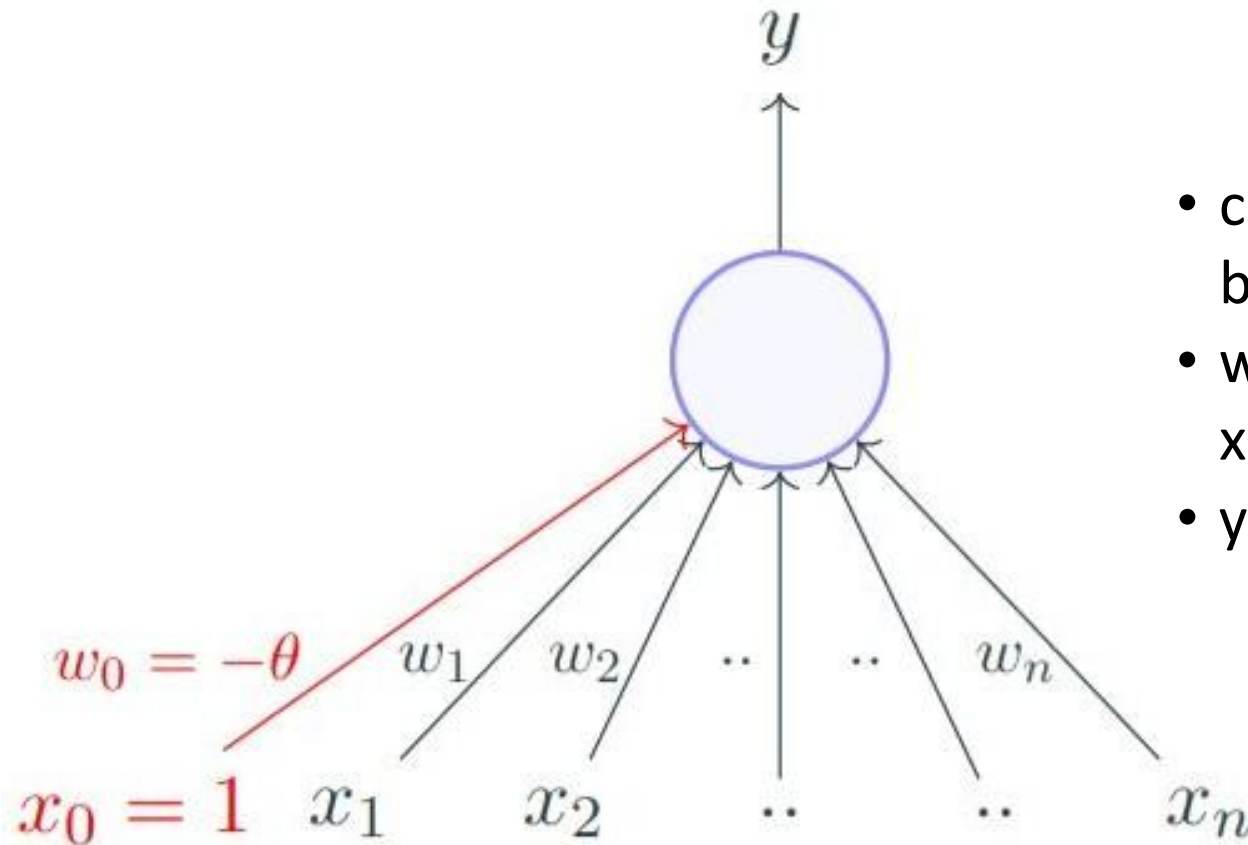
- Input must be binary (what if input attribute is real-valued or has more than 2 classes?)
- All inputs are treated equally (what if few input attributes are more important?)
- Threshold is hand coded (Can it be learned?)
- Works only for (boolean) functions which are linearly separable.
- The output is binary (categorical)

- Frank Rosenblatt proposed the classical perceptron model (1958).
- A more general computational model than MP neurons.

#### Primary differences:

- Introduction of numerical weights for inputs and a mechanism for learning these weights and the bias.
- Inputs are no longer limited to Boolean values.





- consider this single perceptron, we call  $w_0$  as bias and  $x_0 = 1$ .
- $w_1 \dots w_n$  are weights associated with inputs  $x_1 \dots x_n$ .
- $y$  is the output.

### The perceptron model

$$y = 1 \quad \text{if } \sum_{i=1}^n w_i * x_i \geq \theta$$
$$= 0 \quad \text{if } \sum_{i=1}^n w_i * x_i < \theta$$

$$y = 1 \quad \text{if } \sum_{i=1}^n w_i * x_i - \theta \geq 0$$
$$= 0 \quad \text{if } \sum_{i=1}^n w_i * x_i - \theta < 0$$

A mathematical convenience:

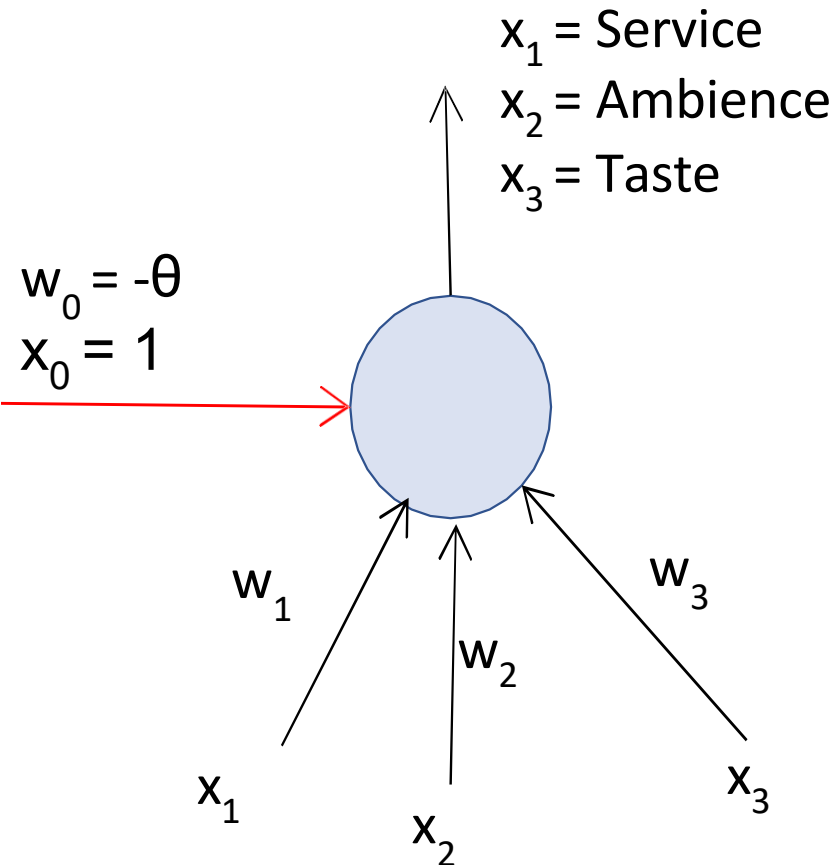
$$y = 1 \text{ if } \mathbf{w}^T \mathbf{x} + b \geq 0$$
$$= 0 \text{ if } \mathbf{w}^T \mathbf{x} + b < 0$$

where  $w_0 = -\theta$  and  $x_0 = 1$

$$y = 1 \quad \text{if } \sum_{i=0}^n w_i * x_i \geq 0$$
$$= 0 \quad \text{if } \sum_{i=0}^n w_i * x_i < 0$$

## Unit 2 - ANN

### An intuitive example



- Consider the task of predicting whether we would like to dine in at a restaurant.
- Assume our decision is based on 3 inputs.
- Based on our past dining experience (data), we give a high weight to taste when compared to the other inputs.
- $w_0$  is called the **bias** as it represents the prior (prejudice).
- A foodie may have a very low threshold and may dine at any restaurant irrespective of the service, ambience, taste [ $\theta = 0$ ].
- On the other hand, a food critic may only dine at restaurants with  $\geq 4$  star ratings for service, taste and ambience [ $\theta = 3$ ].

### Comparing MP neurons and perceptrons

$$y = 1 \quad \text{if} \quad \sum_{i=0}^n x_i \geq 0$$
$$= 0 \quad \text{if} \quad \sum_{i=0}^n x_i < 0$$

MP neuron

$$y = 1 \quad \text{if} \quad \sum_{i=0}^n w_i * x_i \geq 0$$
$$= 0 \quad \text{if} \quad \sum_{i=0}^n w_i * x_i < 0$$

Perceptron

#### Difference

- The weights (including threshold) can be learned and the inputs can be real valued for a perceptron.

#### Similarities

- Both perceptrons and MP neurons separates the input space into two halves.
- All inputs which produce a 1 lie on one side and all inputs which produce a 0 lie on the other side.
- The output can only be binary (0/1).



## Unit 2 - ANN

### Logical OR



$$y = 1 \quad \text{if} \quad \sum_{i=0}^n w_i * x_i \geq 0$$
$$= 0 \quad \text{if} \quad \sum_{i=0}^n w_i * x_i < 0$$

Perceptron equation

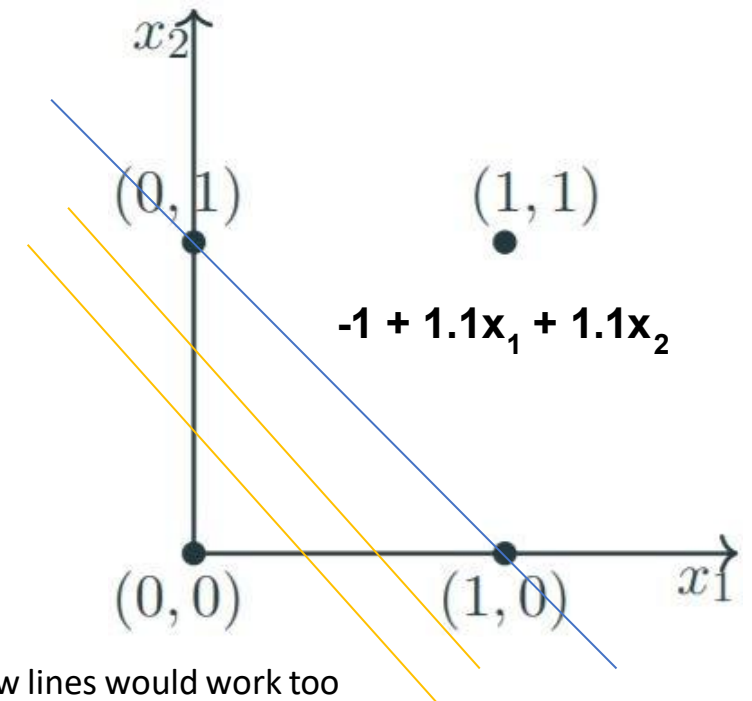
| X | Y | OR |
|---|---|----|
| 0 | 0 | 0  |
| 0 | 1 | 1  |
| 1 | 0 | 1  |
| 1 | 1 | 1  |

1.  $w_0 + w_1 \cdot 0 + w_2 \cdot 0 < 0 \Rightarrow w_0 < 0$
2.  $w_0 + w_1 \cdot 1 + w_2 \cdot 0 \geq 0 \Rightarrow w_1 \geq -w_0$
3.  $w_0 + w_1 \cdot 0 + w_2 \cdot 1 \geq 0 \Rightarrow w_2 \geq -w_0$
4.  $w_0 + w_1 \cdot 1 + w_2 \cdot 1 \geq 0 \Rightarrow w_2 + w_1 \geq -w_0$

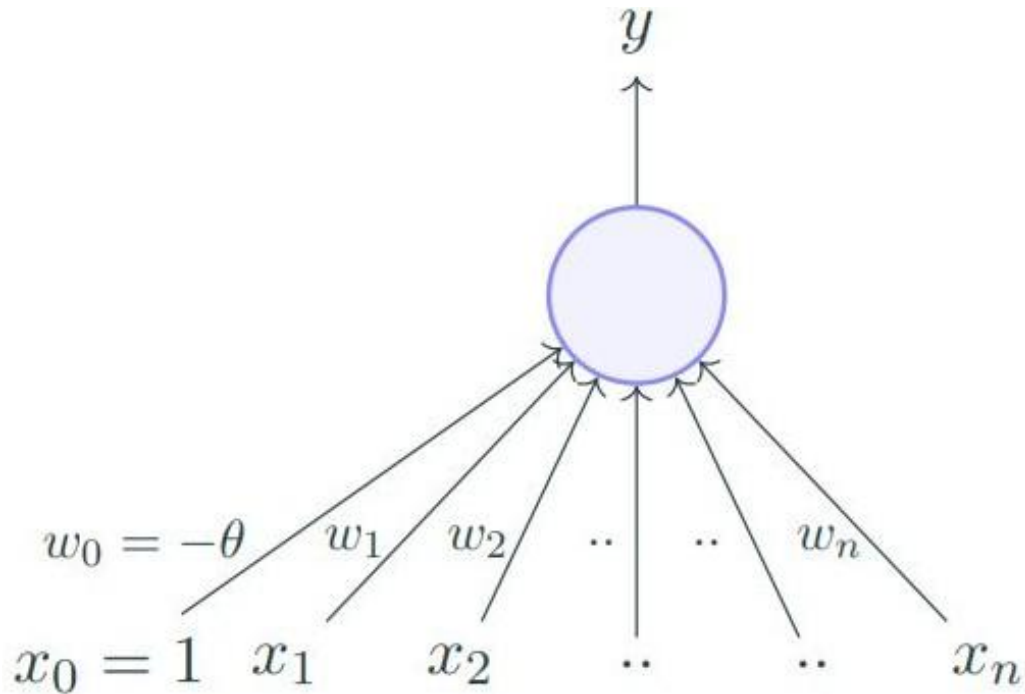
Framing the perceptron equation for each input as shown

One possible solution to this set of inequalities:

$$w_0 = 1, w_1 = 1.1, w_2 = 1.1$$



These yellow lines would work too



$x_1$  = isActorGood

$x_2$  = isDirectorNew

$x_3$  = IMDB rating

....

$x_n$  = Ticket Price for 2

- Let us consider our problem of deciding whether to watch a movie.
- Suppose we have a list of  $m$  movies and a label (class) associated with each movie indicating whether the user like the actor, director etc or not i.e. binary decision.
- Let's represent each movie with  $n$  features (Boolean / real(IMDB rating)).
- We will assume that the data is linearly separable and we want a perceptron to learn how to make this decision.
- Essentially, we want the perceptron to find the equation of this separating plane (or find the values of  $w_0, w_1, w_2, \dots, w_n$ ).

## Unit 2 - ANN

### Perceptron learning algorithm



$P \leftarrow$  inputs with label 1;

$N \leftarrow$  inputs with label 0;

Initialize  $\mathbf{w}$  randomly;

**while** !convergence **do**

    Pick random  $\mathbf{x} \in P \cup N$  ;

**if**  $\mathbf{x} \in P$  and  $\sum_{i=0}^n w_i * x_i < 0$  **then**

$\mathbf{w} = \mathbf{w} + \mathbf{x}$  ;

**end**

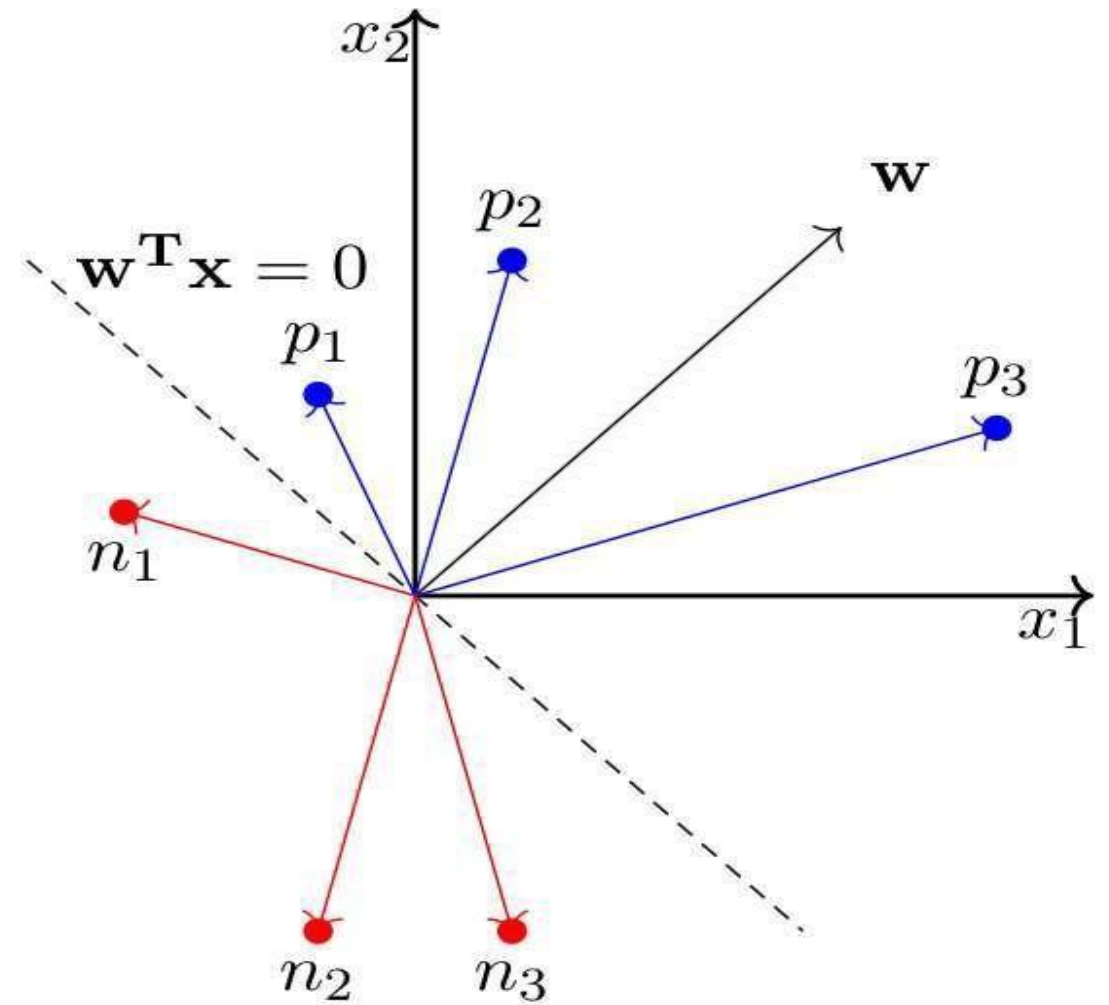
**if**  $\mathbf{x} \in N$  and  $\sum_{i=0}^n w_i * x_i \geq 0$  **then**

$\mathbf{w} = \mathbf{w} - \mathbf{x}$  ;

**end**

**end**

//the algorithm converges when all the inputs are classified correctly



### Perceptron learning algorithm

```
P ← inputs with label 1;  
N ← inputs with label 0;  
Initialize w randomly;  
while !convergence do  
    Pick random  $\mathbf{x} \in P \cup N$  ;  
    if  $\mathbf{x} \in P$  and  $\mathbf{w} \cdot \mathbf{x} < 0$  then  
        |  $\mathbf{w} = \mathbf{w} + \mathbf{x}$  ;  
    end  
    if  $\mathbf{x} \in N$  and  $\mathbf{w} \cdot \mathbf{x} \geq 0$  then  
        |  $\mathbf{w} = \mathbf{w} - \mathbf{x}$  ;  
    end  
end  
//the algorithm converges when all the  
inputs are classified correctly
```

$$\cos \alpha = \frac{\mathbf{w}^T \mathbf{x}}{\|\mathbf{w}\| \|\mathbf{x}\|}$$

For  $\mathbf{x} \in P$  if  $\mathbf{w} \cdot \mathbf{x} < 0$  then it means that the angle ( $\alpha$ ) between this  $\mathbf{x}$  and the current  $\mathbf{w}$  is greater than  $90^\circ$  (but we want  $\alpha$  to be less than  $90^\circ$ )

What happens to the new angle ( $\alpha_{new}$ ) when  $\mathbf{w}_{new} = \mathbf{w} + \mathbf{x}$

$$\begin{aligned} \cos(\alpha_{new}) &\propto \mathbf{w}_{new}^T \mathbf{x} \\ &\propto (\mathbf{w} + \mathbf{x})^T \mathbf{x} \\ &\propto \mathbf{w}^T \mathbf{x} + \mathbf{x}^T \mathbf{x} \\ &\propto \cos \alpha + \mathbf{x}^T \mathbf{x} \\ \cos(\alpha_{new}) &> \cos \alpha \end{aligned}$$

Thus  $\alpha_{new}$  will be less than  $\alpha$  and this is exactly what we want

### Perceptron learning algorithm

---

- For error in prediction of negative instances, the angle changes as follows (it is less than 90 degrees, it must be greater than 90 degrees)

$(\alpha_{new})$  when  $\mathbf{w}_{new} = \mathbf{w} - \mathbf{x}$

$$\begin{aligned}\cos(\alpha_{new}) &\propto \mathbf{w}_{new}^T \mathbf{x} \\ &\propto (\mathbf{w} - \mathbf{x})^T \mathbf{x} \\ &\propto \mathbf{w}^T \mathbf{x} - \mathbf{x}^T \mathbf{x} \\ &\propto \cos\alpha - \mathbf{x}^T \mathbf{x}\end{aligned}$$

$$\cos(\alpha_{new}) < \cos\alpha$$

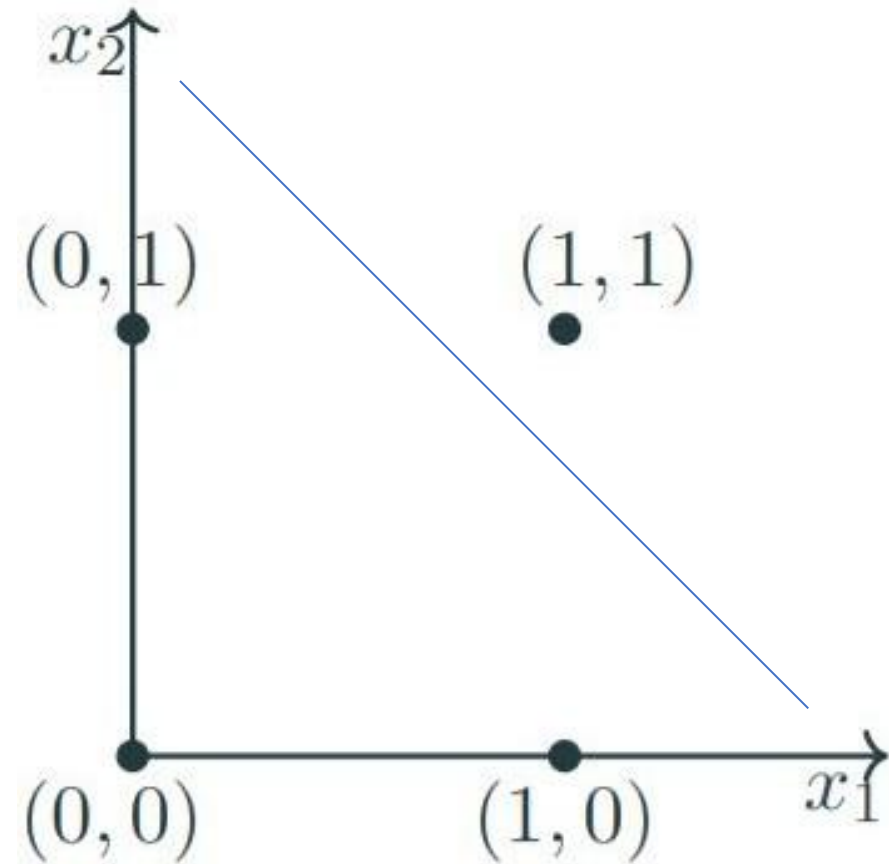
### Perceptron learning for linear surfaces

---

- A perceptron is a threshold linear unit (discrete-valued).
- **What is a linear unit?** A linear combination of weighted inputs (real-valued).
- A perceptron offers linear decision surfaces. This means a single perceptron can easily represent simple Boolean functions like AND, OR, NAND and NOR.

## Unit 2 - ANN

Can you find line that separates the 4 points in the truth table?



| X | Y | AND |
|---|---|-----|
| 0 | 0 | 0   |
| 0 | 1 | 0   |
| 1 | 0 | 0   |
| 1 | 1 | 1   |

$$y = 1 \quad \text{if} \quad \sum_{i=0}^n w_i * x_i \geq 0$$
$$= 0 \quad \text{if} \quad \sum_{i=0}^n w_i * x_i < 0$$

Perceptron equation

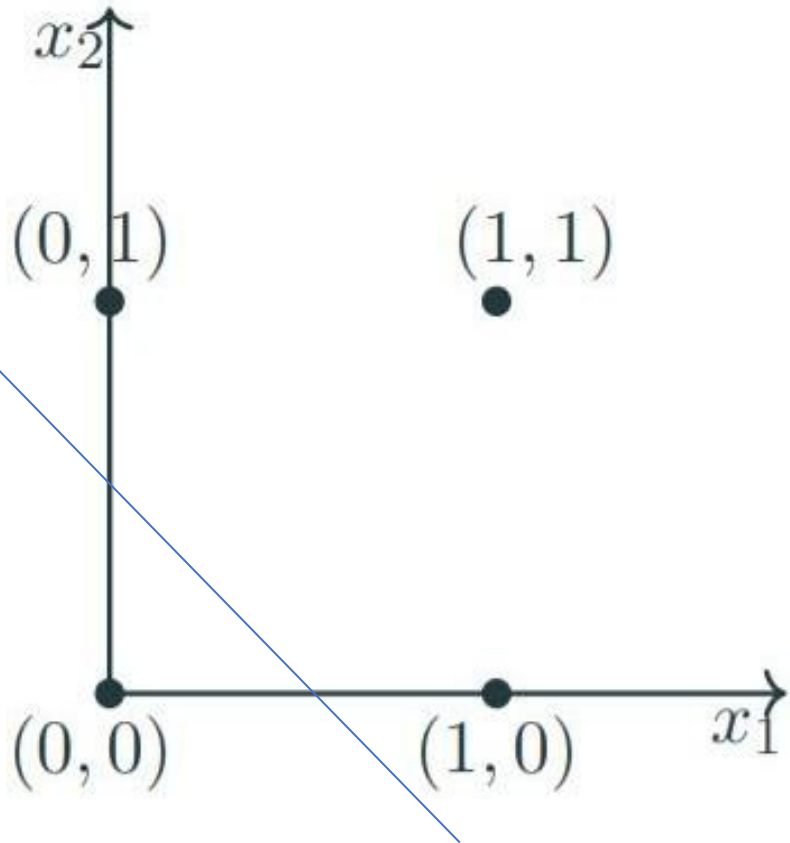
Can you find  $w_0$ ,  $w_1$  and  $w_2$ ?

Hint: one possible solution is  $w_0 = -0.8$ ,  $w_1 = w_2 = 0.5$



## Unit 2 - ANN

Can you find line that separates the 4 points in the truth table?



| X | Y | OR |
|---|---|----|
| 0 | 0 | 0  |
| 0 | 1 | 1  |
| 1 | 0 | 1  |
| 1 | 1 | 1  |

$$y = 1 \quad \text{if} \quad \sum_{i=0}^n w_i * x_i \geq 0$$
$$= 0 \quad \text{if} \quad \sum_{i=0}^n w_i * x_i < 0$$

Perceptron equation

Can you find  $w_0$ ,  $w_1$  and  $w_2$ ?

Hint: one possible solution is  $w_0 = -0.3$ ,  $w_1 = w_2 = 0.5$

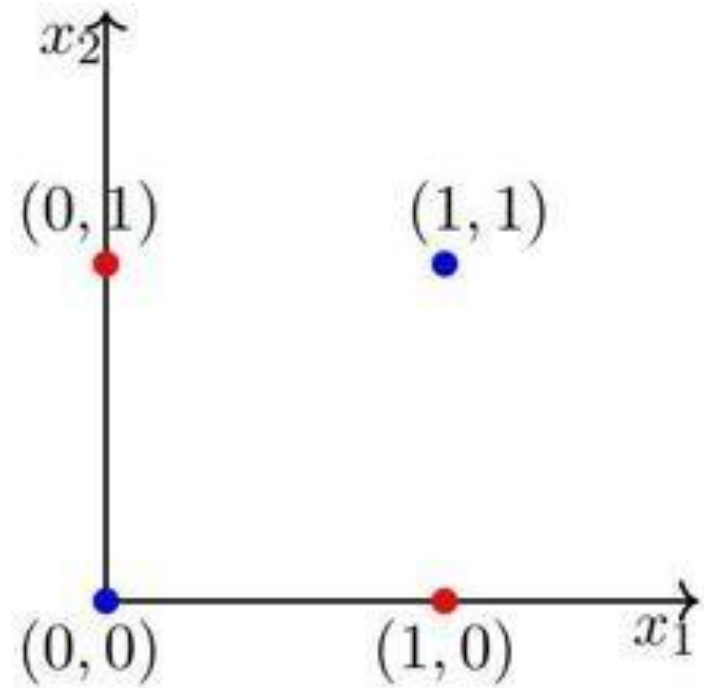
## Unit 2 - ANN

Can you find line that separates the 4 points in the truth table?

| X | Y | XOR |
|---|---|-----|
| 0 | 0 | 0   |
| 0 | 1 | 1   |
| 1 | 0 | 1   |
| 1 | 1 | 0   |

1.  $w_0 + w_1 \cdot 0 + w_2 \cdot 0 < 0 \Rightarrow w_0 < 0$
2.  $w_0 + w_1 \cdot 1 + w_2 \cdot 0 \geq 0 \Rightarrow w_2 \geq -w_0$
3.  $w_0 + w_1 \cdot 0 + w_2 \cdot 1 \geq 0 \Rightarrow w_1 \geq -w_0$
4.  $w_0 + w_1 \cdot 1 + w_2 \cdot 1 < 0 \Rightarrow w_2 + w_1 < -w_0$

The 4<sup>th</sup> condition contradicts the 2<sup>nd</sup> and 3<sup>rd</sup>.  
Hence, we cannot solve the system of inequations.



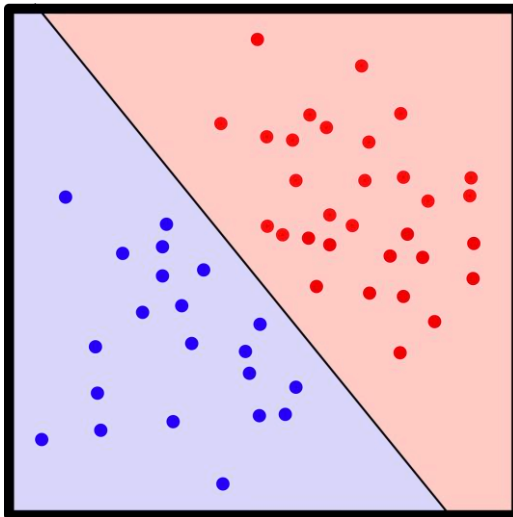
We cannot draw a line that separates the points!

$$y = 1 \quad \text{if} \quad \sum_{i=0}^n w_i * x_i \geq 0$$
$$= 0 \quad \text{if} \quad \sum_{i=0}^n w_i * x_i < 0$$

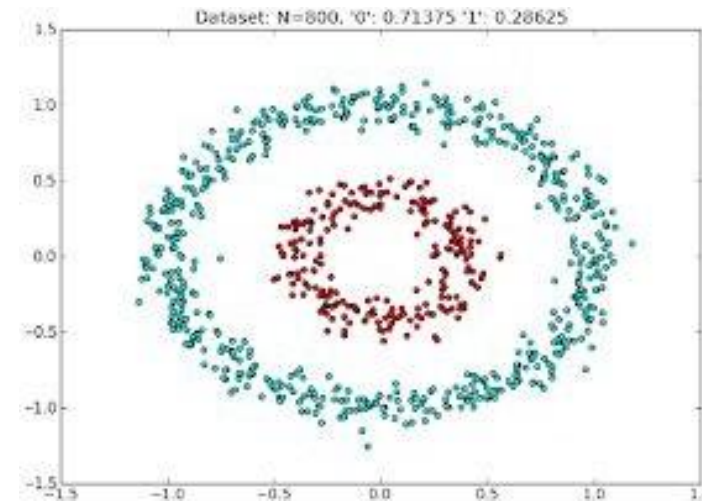
Perceptron equation

### When does a single perceptron fail?

- If data is not linearly separable as we just saw in the case of the XOR function, then a single perceptron is not enough to represent the functionality.
- In fact, most real world data is linearly inseparable. While a single perceptron cannot deal with such data, a network of perceptrons can!



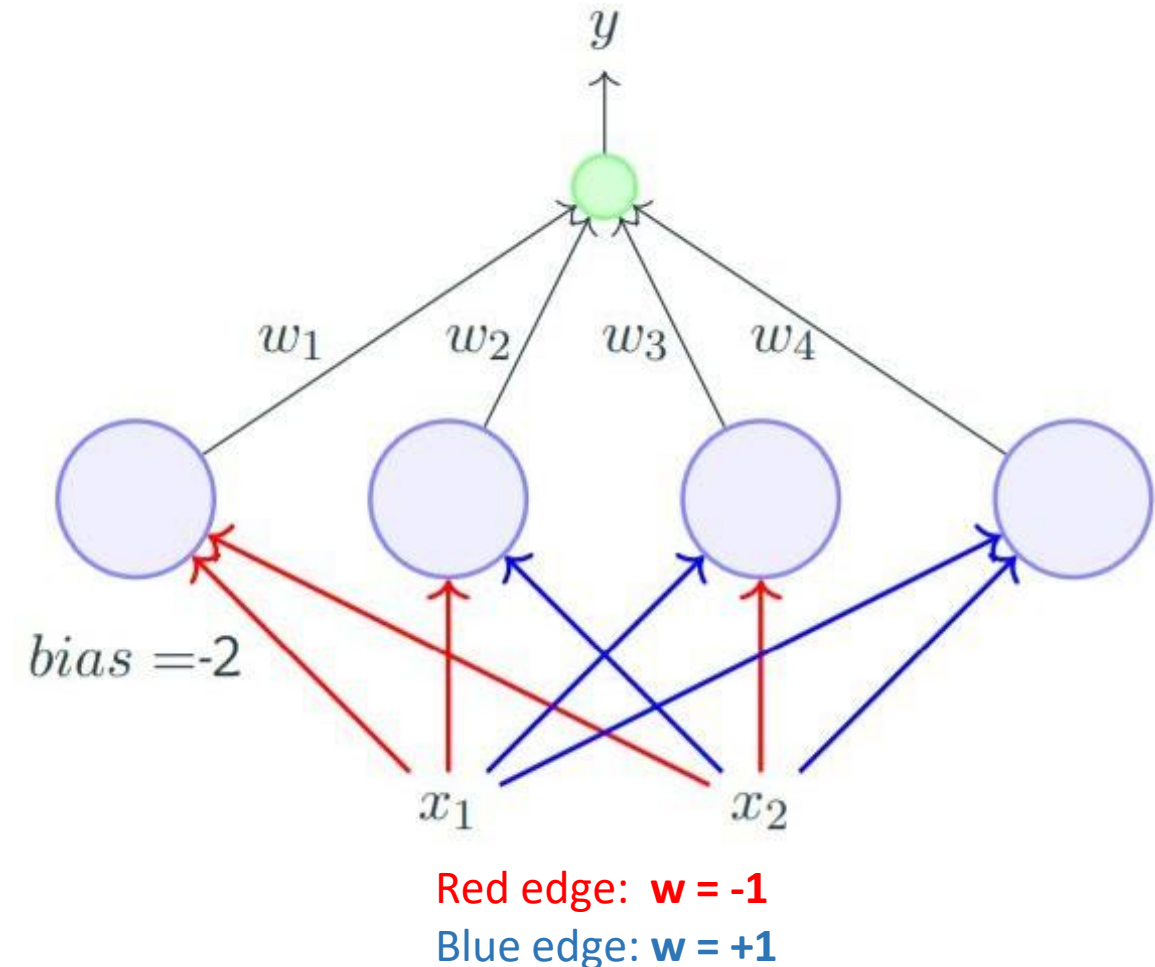
Linearly separable



Linearly inseparable

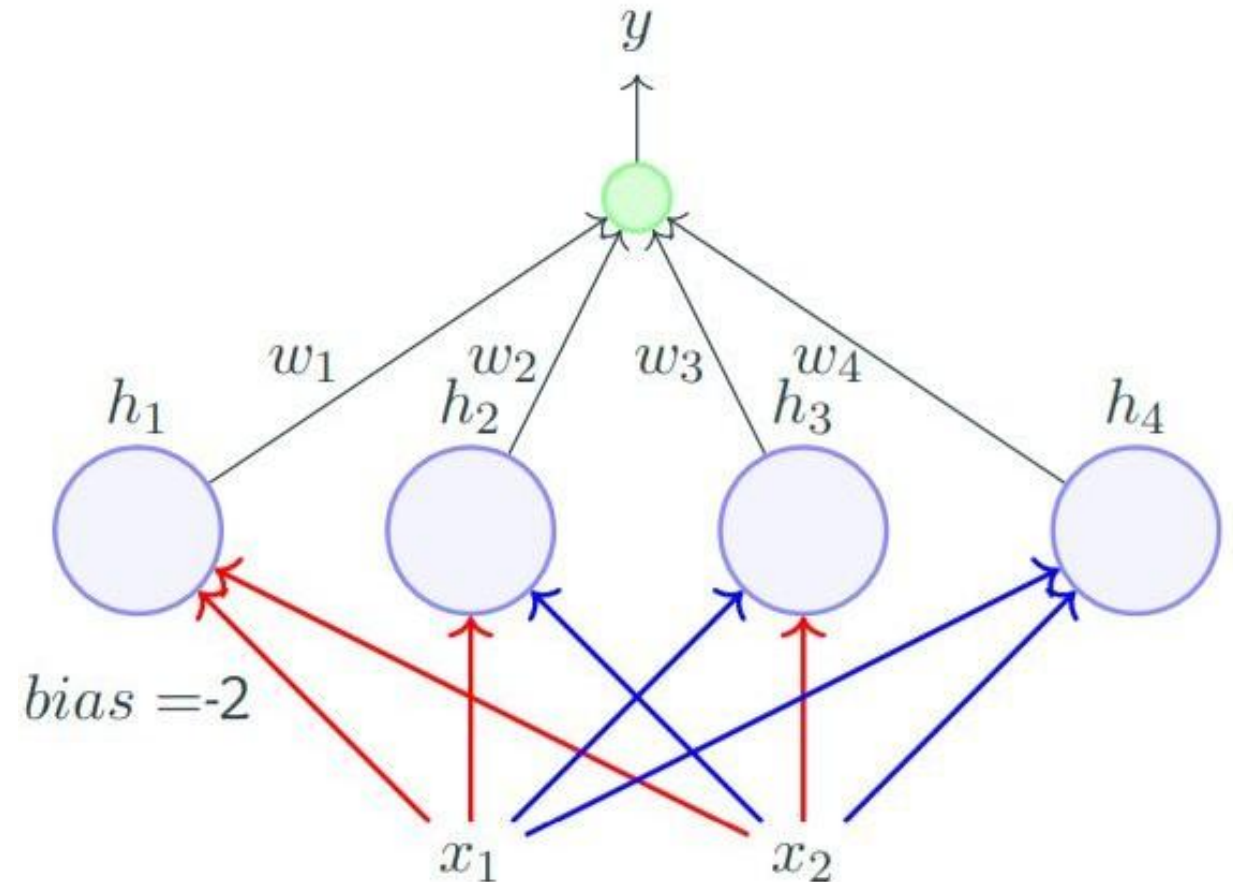
### Implementing any Boolean function using a network of perceptrons

- Assume True = +1 and False = -1.
- We consider 2 inputs and 4 perceptrons.
- Each input is connected to all the 4 perceptrons with specific weights.
- The bias ( $w_0$ ) of each perceptron is -2 (each perceptron will fire if the weighted sum of its input is  $\geq 2$ ).
- Each of these perceptrons is connected to an output perceptron by weights (need to be learned).
- The output of this perceptron ( $y$ ) is the output of this network.



### Implementing any Boolean function using a network of perceptrons

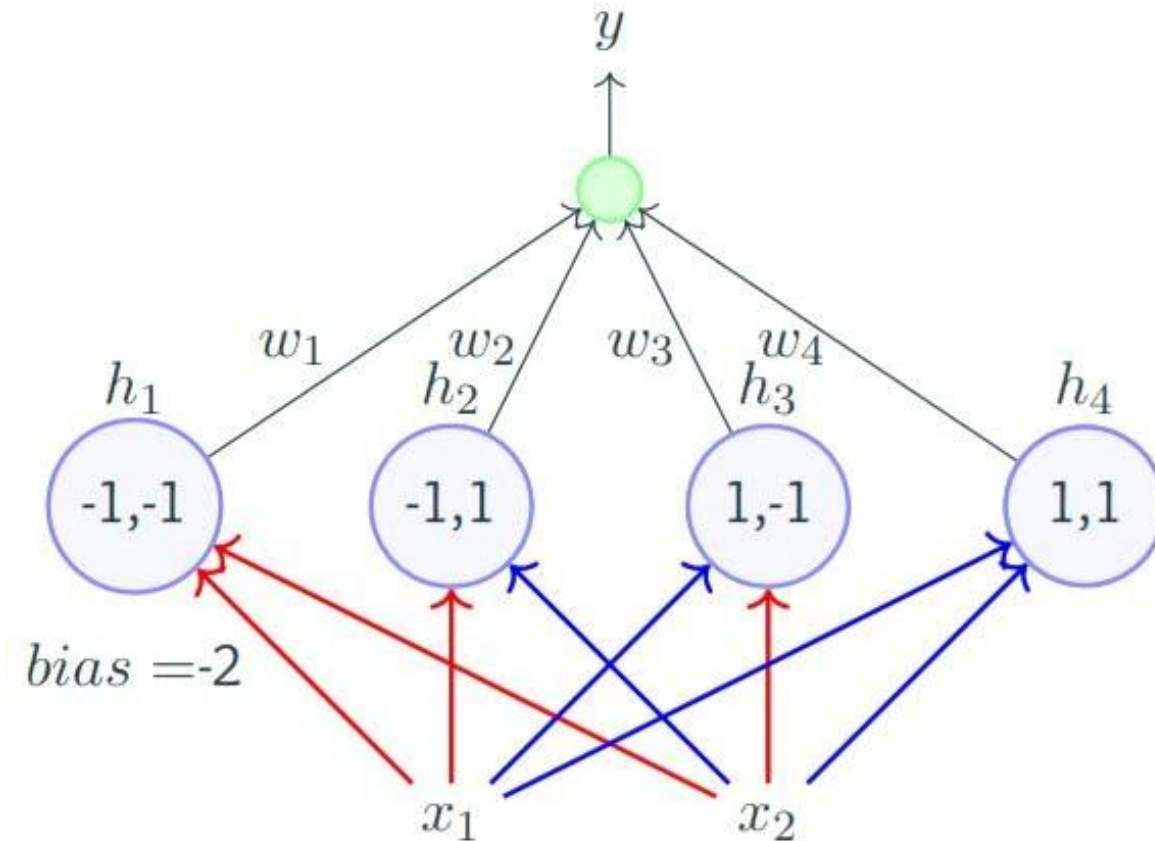
- The input layer contains the inputs ( $x_1$ ,  $x_2$ ).
- The middle layer containing the 4 perceptrons is called the hidden layer.
- The final layer containing one output neuron is called the output layer.
- The outputs of the 4 perceptrons in the hidden layer are denoted by  $h_1$ ,  $h_2$ ,  $h_3$ ,  $h_4$ .
- The red and blue edges are called layer 1 weights.
- $w_1$ ,  $w_2$ ,  $w_3$ ,  $w_4$  are called layer 2 weights.



3 layered network

### Implementing any Boolean function using a network of perceptrons

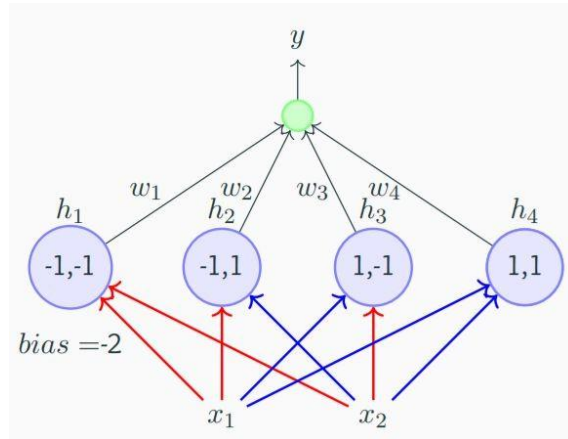
- We claim that this network can be used to implement any boolean function (linearly separable or not).
- In other words, we can find  $w_1, w_2, w_3, w_4$  such that the truth table of any boolean function can be represented by this network.
- Each perceptron in the middle layer fires only for a specific input (and no two perceptrons fire for the same input).
- Let us see why this network works by taking an example of the XOR function.





## Unit 2 - ANN

### Example of XOR function



| $x_1$ | $x_2$ | $XOR$ | $h_1$ | $h_2$ | $h_3$ | $h_4$ | $\sum_{i=1}^4 w_i h_i$ |
|-------|-------|-------|-------|-------|-------|-------|------------------------|
| 0     | 0     | 0     | 1     | 0     | 0     | 0     | $w_1$                  |
| 0     | 1     | 1     | 0     | 1     | 0     | 0     | $w_2$                  |
| 1     | 0     | 1     | 0     | 0     | 1     | 0     | $w_3$                  |
| 1     | 1     | 0     | 0     | 0     | 0     | 1     | $w_4$                  |

- Let  $w_0$  be the bias output of the neuron. It will fire if  $\sum_{i=1}^4 w_i h_i \geq w_0$ . Here,  $w_0 = 0$ .
- This results in the following four conditions to implement XOR:  $w_1 < w_0$ ,  $w_2 \geq w_0$ ,  $w_3 \geq w_0$ ,  $w_4 < w_0$ .
- Unlike before, there are no contradictions now and the system of inequalities can be satisfied.
- Each  $w_i$  is now responsible for one of the 4 possible inputs and can be adjusted to get the desired output for that input.
- Each boolean function will result in a different set of non-contradicting inequalities which can be satisfied by appropriately setting  $w_1$ ,  $w_2$ ,  $w_3$ ,  $w_4$ .





## **UE24CS352A: Machine Learning**

---

**Dr. Shylaja S S**

Department of Computer Science & Engineering

**[shylaja.sharath@pes.edu](mailto:shylaja.sharath@pes.edu)**



## UE24CS352A: Machine Learning

---

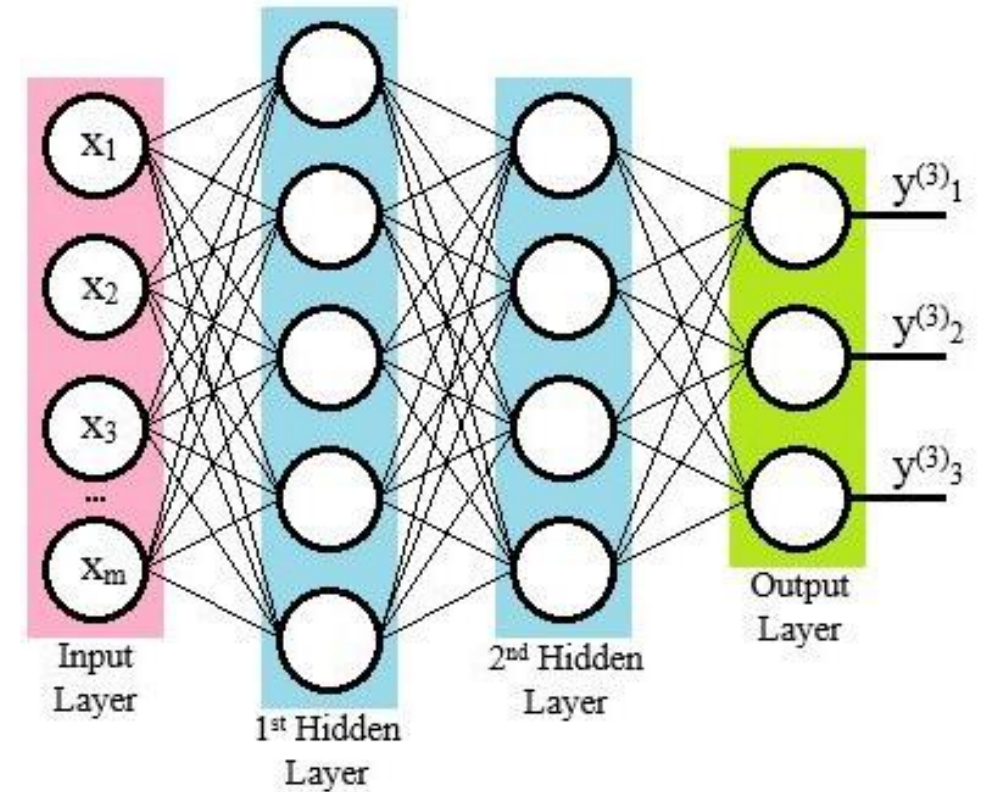
**Dr. Shylaja S S**

Professor

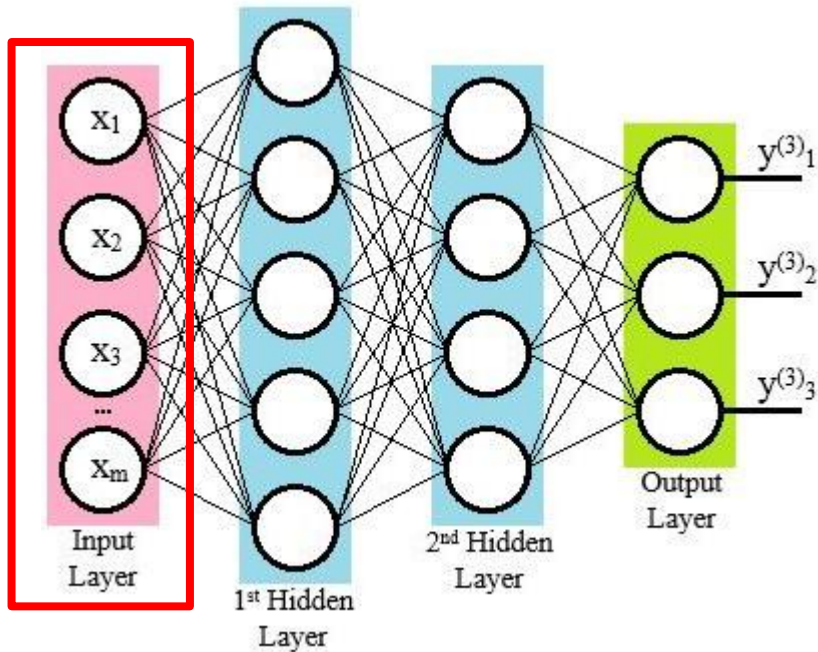
Department of Computer Science & Engineering

Acknowledgement : Dr. Rajanikanth K (Academic Advisor, PESU)  
Teaching Assistant (Arya Rajiv Chaloli)

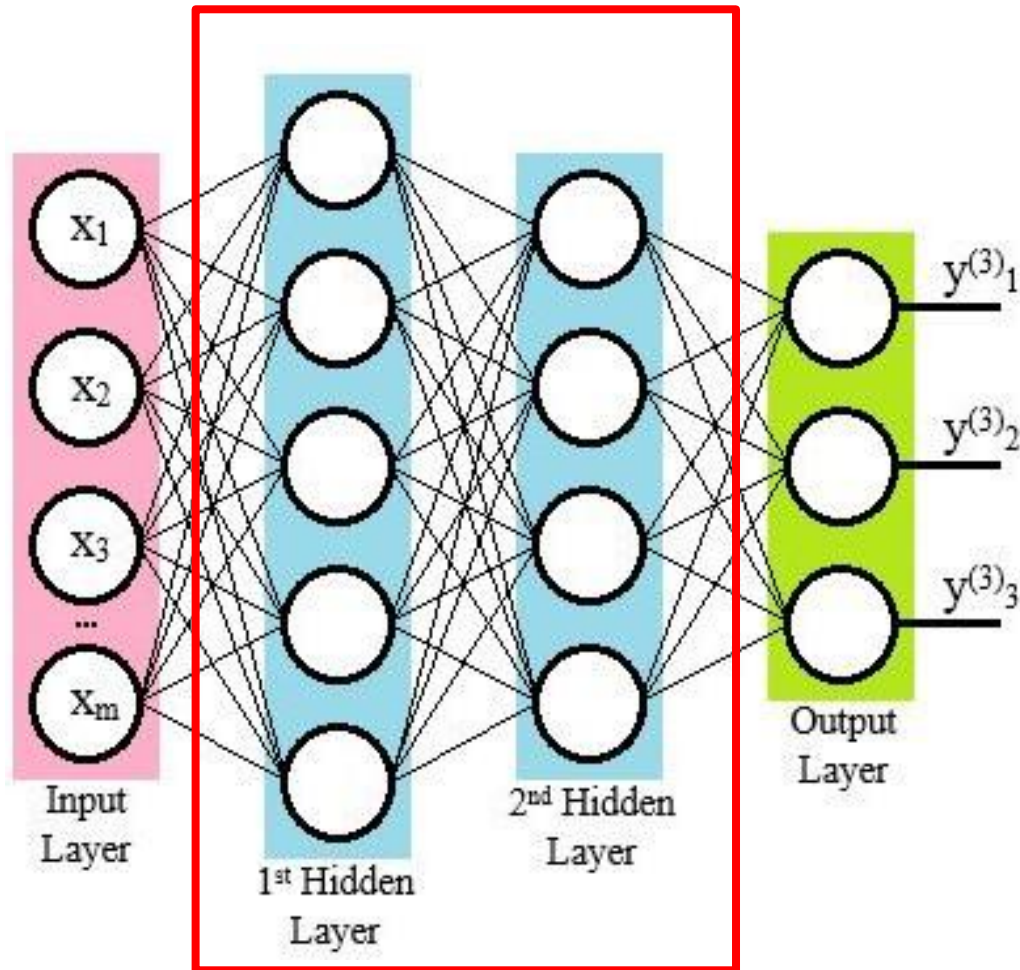
- We previously saw how a single neuron can't be used to learn complex functions.
- This is the reason our brain has billions of neurons, stacked in layers forming a network.
- Similarly, artificial neurons are arranged in layers.
- Each layer will be connected in such a way that information is passed from one layer to another.



- A typical ANN consists of the input layer, hidden layer(s) and output layer.
- If it has more than 1 hidden layer, it is called a deep ANN (DNN).
- A fully connected multi-layer neural network is called a Multilayer Perceptron (MLP).
- Each layer has a collection of neurons, and the neurons in a layer interact with all neurons in other layers.
- We use the term nodes or units to represent the neurons in an ANN.
- The number of layers and the number of neurons are referred to as hyperparameters of a neural network.

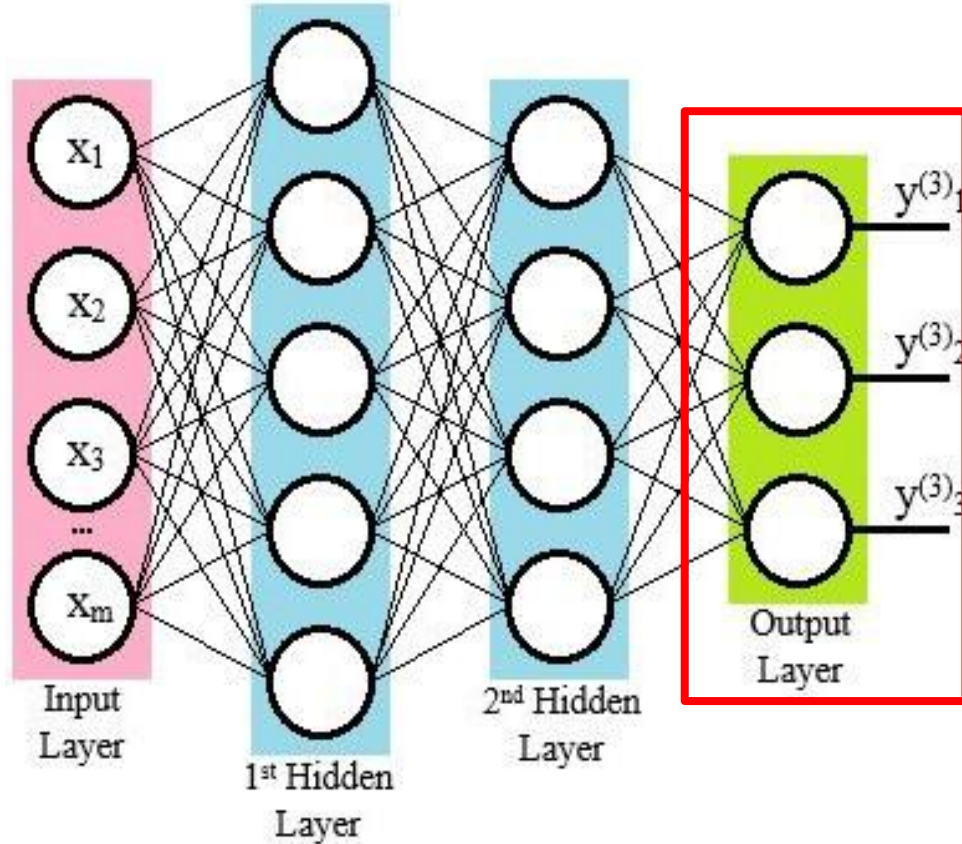


- The input layer is where we feed in inputs to the network.
- The number of units in input layer is the number of inputs we feed to the network.
- No computation is performed in the input layer



- Any layer between the input layer and the output layer is called hidden layers.
- It is the main core for learning, processes the input received from input layer.
- The hidden layer is responsible for deriving complex relationships between input and output. It identifies patterns in the dataset.
- There can be n number of hidden layer according to our use case.

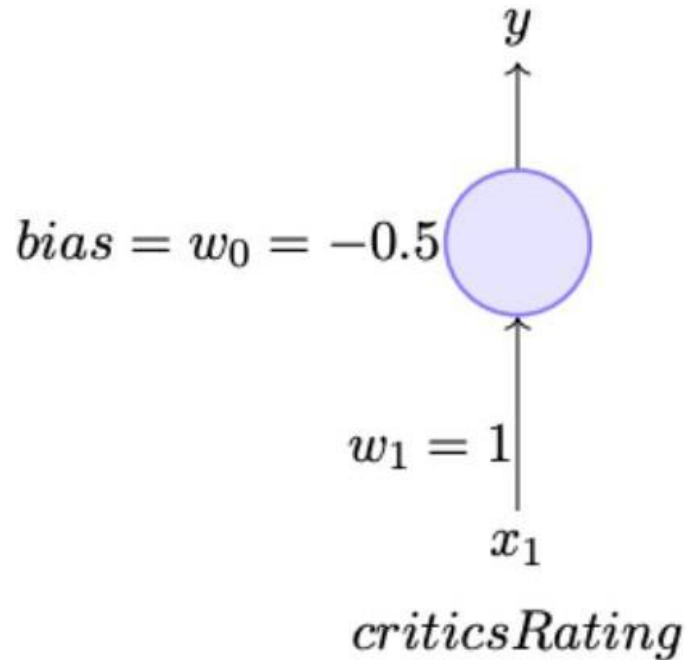




- After processing the input the hidden layer sends its results to the output layer.
- As the name suggests the output layer emits output

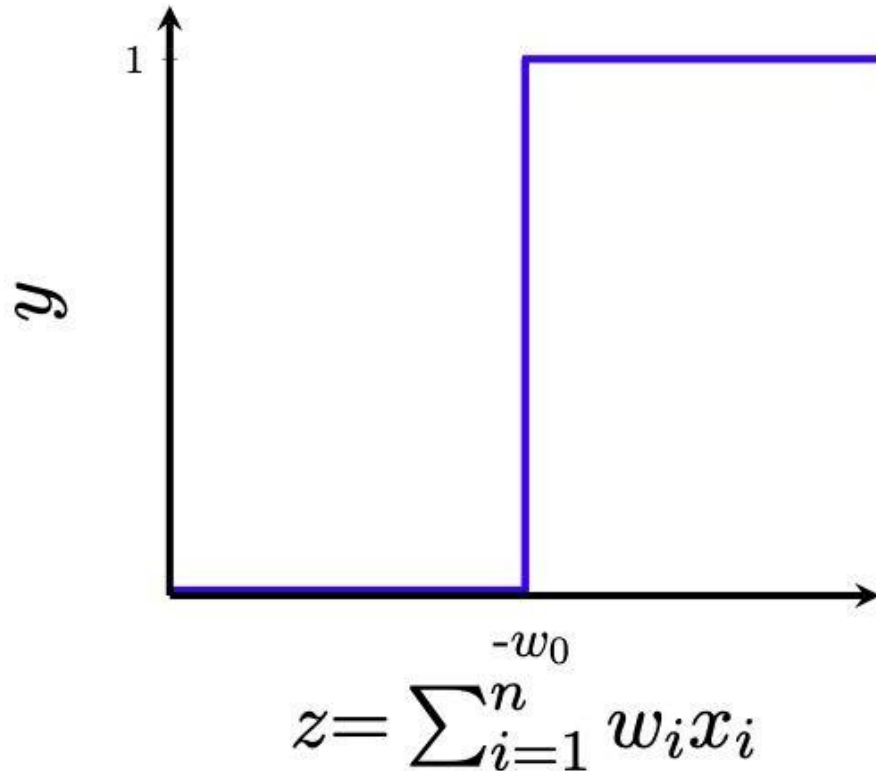


### Perceptron thresholding logic is harsh

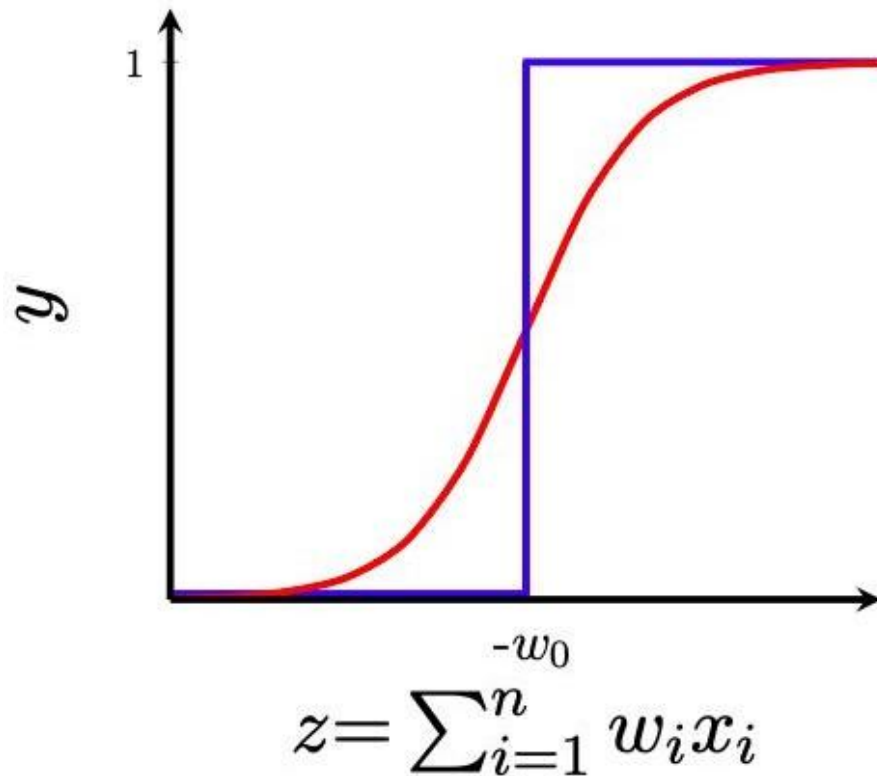


- The thresholding logic used by a perceptron is very harsh !
- For example, let us return to our problem of deciding whether we will like or dislike a movie
- Consider that we base our decision only on one input ( $x_1 = criticsRating$  which lies between 0 and 1)
- If the threshold is 0.5 ( $w_0 = -0.5$ ) and  $w_1 = 1$  then what would be the decision for a movie with  $criticsRating = 0.51$  ? (like)
- What about a movie with  $criticsRating = 0.49$  ? (dislike)
- It seems harsh that we would like a movie with rating 0.51 but not one with a rating of 0.49

### Perceptron thresholding logic is harsh



- This behavior is not a characteristic of the specific problem we chose or the specific weight and threshold that we chose
- It is a characteristic of the perceptron function itself which behaves like a step function
- There will always be this sudden change in the decision (from 0 to 1) when  $\sum_{i=1}^n w_i x_i$  crosses the threshold ( $-w_0$ )
- For most real world applications we would expect a smoother decision function which gradually changes from 0 to 1

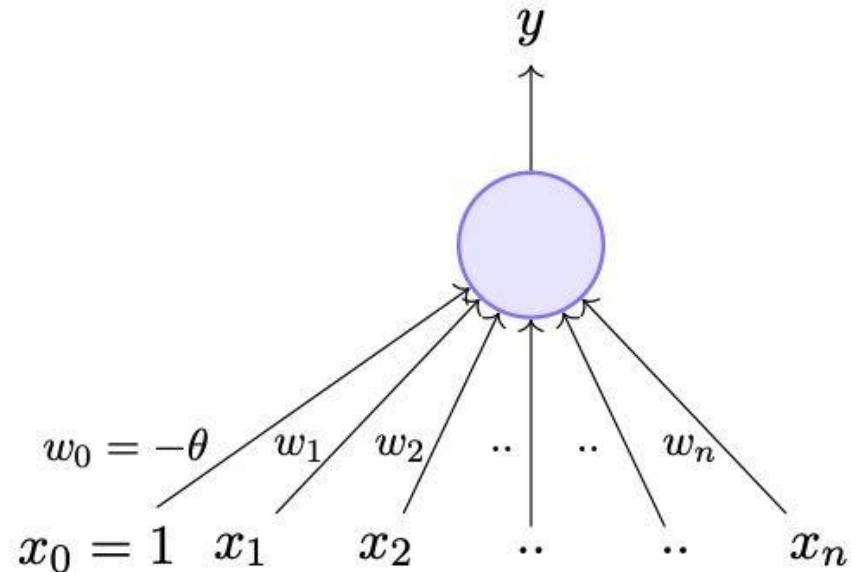


- Introducing sigmoid neurons where the output function is much smoother than the step function
- Here is one form of the sigmoid function called the logistic function

$$y = \frac{1}{1 + e^{-(w_0 + \sum_{i=1}^n w_i x_i)}}$$

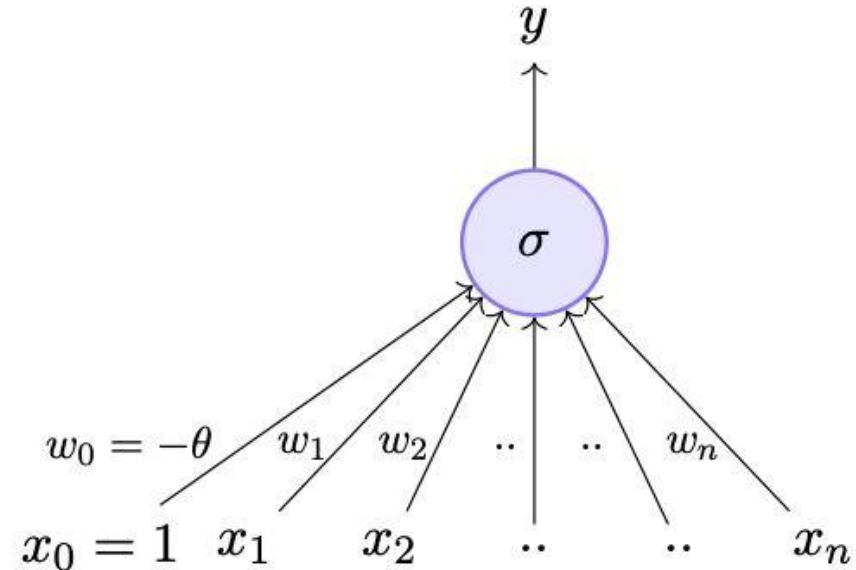
- We no longer see a sharp transition around the threshold  $-w_0$
- Also the output  $y$  is no longer binary but a real value between 0 and 1 which can be interpreted as a probability
- Instead of a like/dislike decision we get the probability of liking the movie

Perceptron



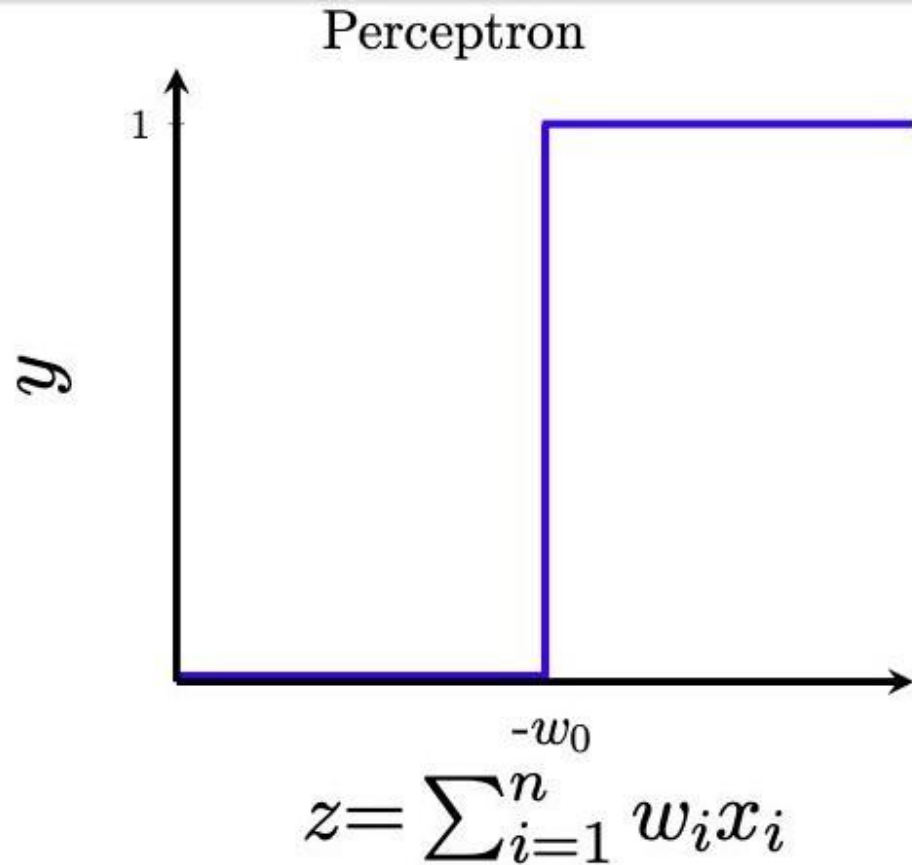
$$y = 1 \quad \text{if} \quad \sum_{i=0}^n w_i * x_i \geq 0$$
$$= 0 \quad \text{if} \quad \sum_{i=0}^n w_i * x_i < 0$$

Sigmoid (logistic) Neuron

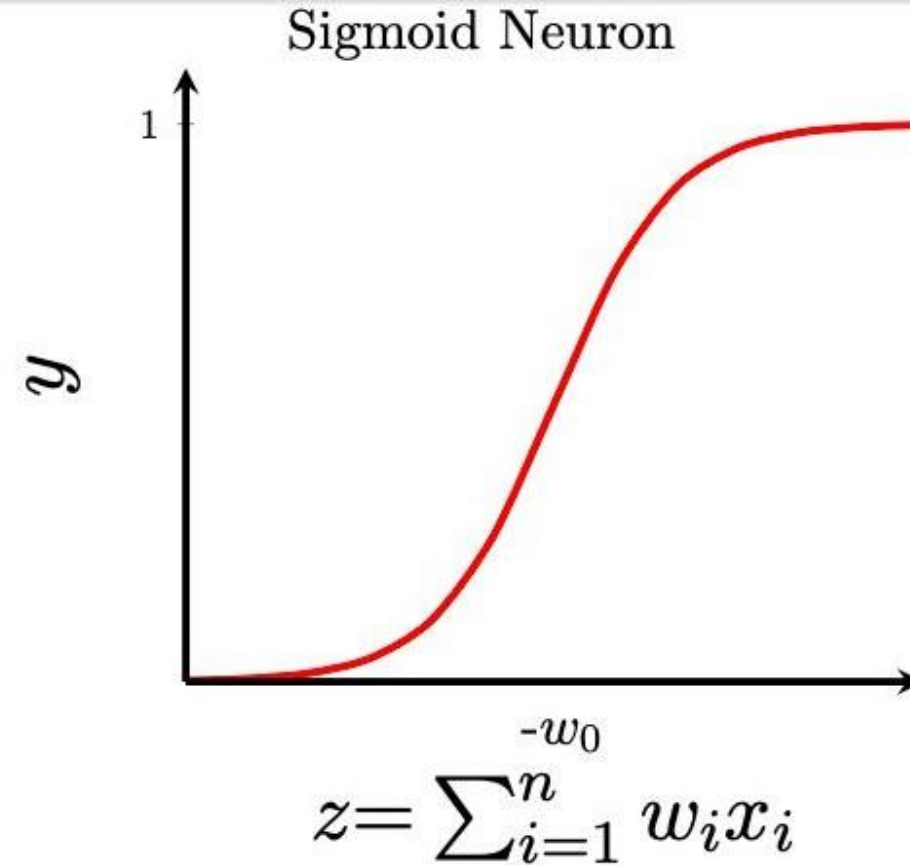


$$y = \frac{1}{1 + e^{-(\sum_{i=0}^n w_i x_i)}}$$

### Perceptron vs Sigmoid Neuron



Not smooth, not continuous (at  $w_0$ ), **not differentiable**

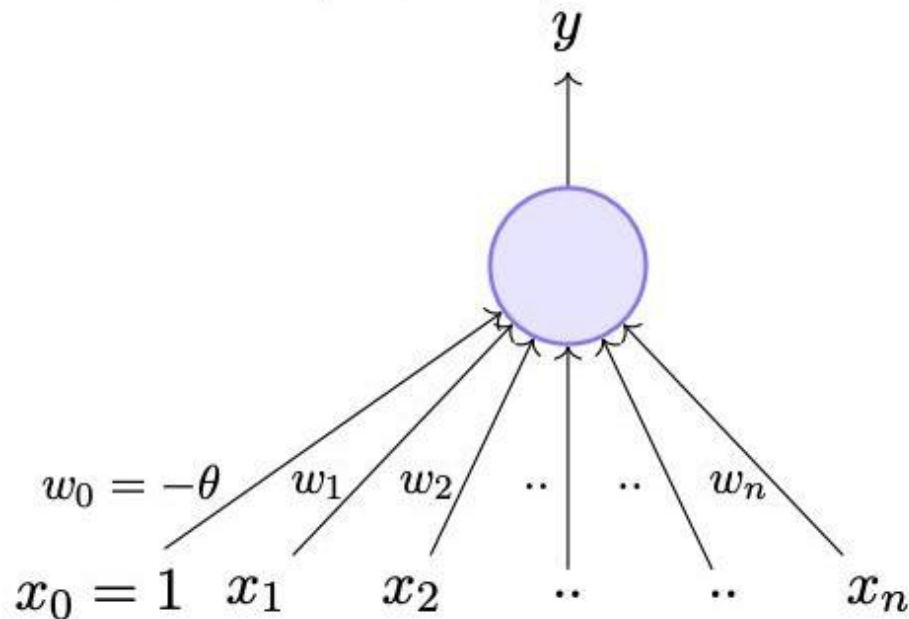


Smooth, continuous, **differentiable**



### Learning weights when output is real-valued

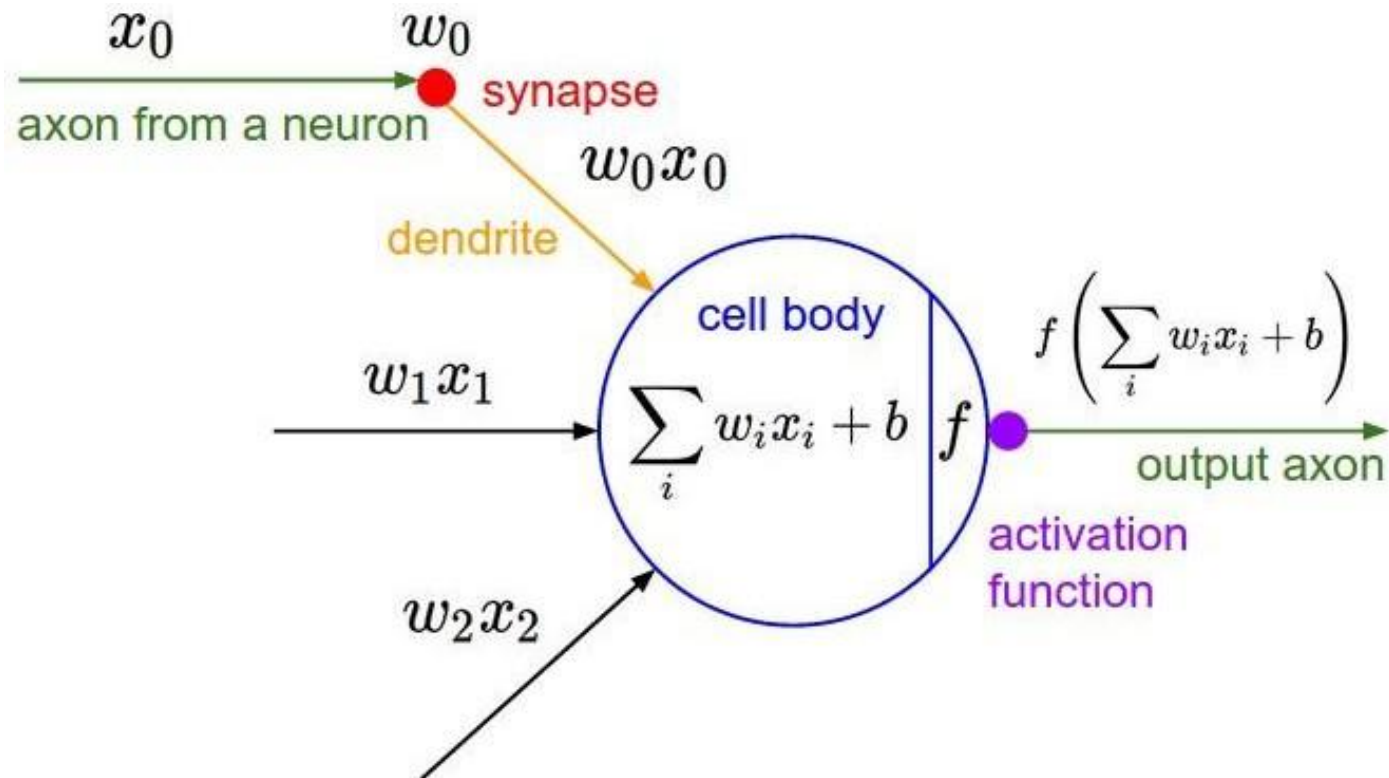
#### Sigmoid (logistic) Neuron



- Well, just as we had an algorithm for learning the weights of a perceptron, we also need a way of learning the weights of a sigmoid neuron



### Artificial neuron in an Artificial Neural Network



**The most fundamental unit of an ANN**

### Objective function/Cost function/Loss function/Error function

---

- The search for weights is driven by the objective of minimizing the error between prediction and true value.
- Because the error can be positive or negative, most often we take the square.
- Depending on the problem, suitable loss function can be chosen.
- Example :

a) Mean squared error, 
$$\mathbf{MSE} = \frac{1}{2} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

b) Cross Entropy - measures the performance of a classification model whose output is a probability value between 0 and 1.

$$L = - \sum_{i=1}^N y^{(i)} * \log \hat{y}^{(i)}$$

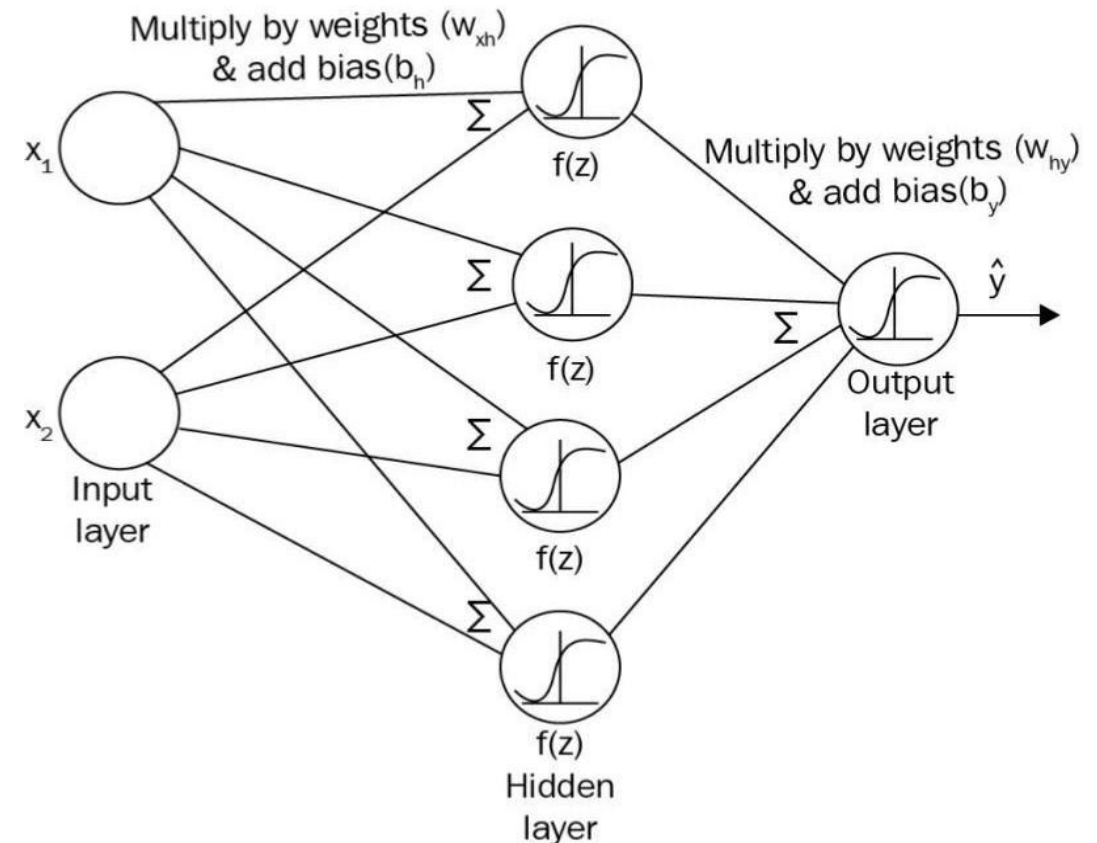
# Forward Propagation

**Process of moving from input to output layer, computing net sum and output at each neuron**

What will the dimension of the weight matrix be?

**Number of units in the previous layer**  
**X**  
**Number of units in the current layer**

- We first need to initialize the weight matrix.
- In the real world we don't know which inputs are relatively important hence we randomly initialize weights and biases.
- The inputs are then multiplied by weights, bias is added the result is propagated to the hidden layer where the activation function will be applied.



$x_{ji}$  = the  $i^{\text{th}}$  input to unit  $j$

$w_{ji}$  = the weight associated with the  $i^{\text{th}}$  input to unit  $j$

$net_j = \sum_i w_{ji} x_{ji}$  (the weighted sum of inputs for unit  $j$ )

$o_j$  = the output computed by unit  $j$

$t_j$  = the target output for unit  $j$

Applicable only to Neurons in  
Output Layer

$\sigma$  = the sigmoid function

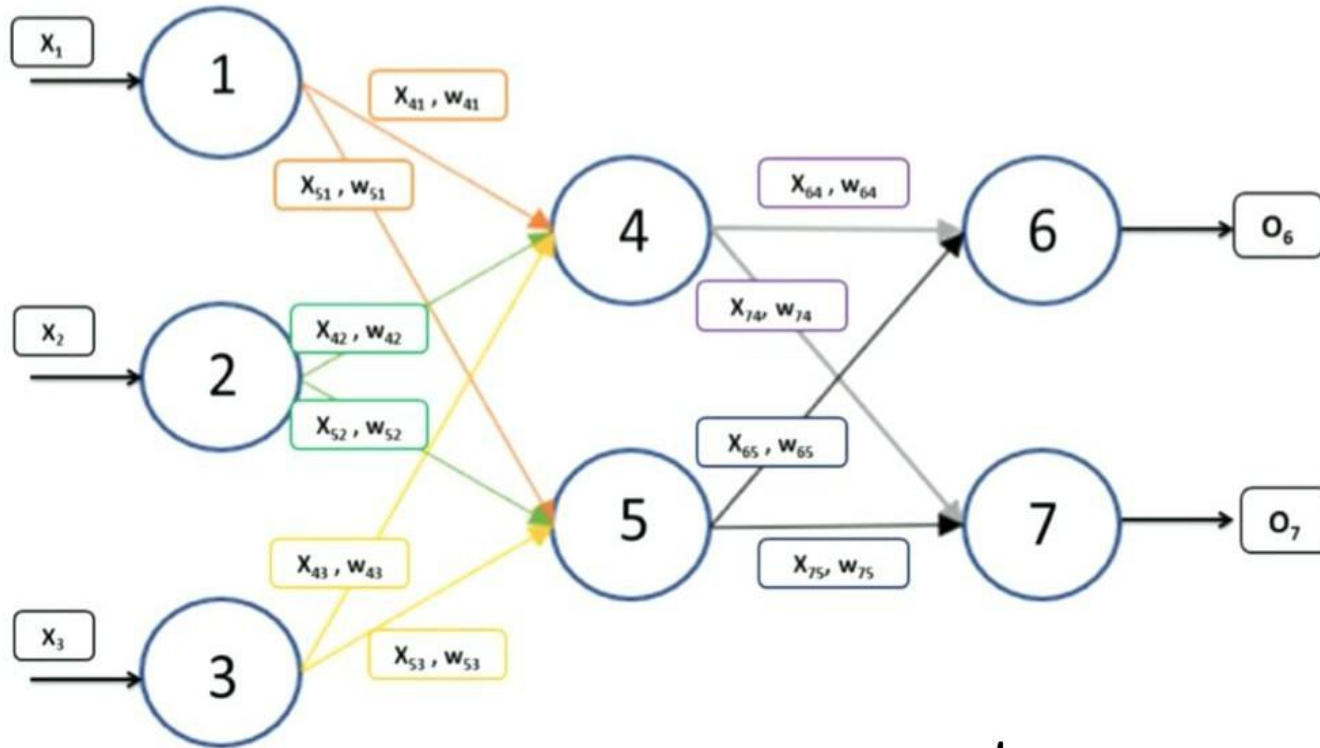
**outputs** = the set of units in the final layer of the network

**Downstream(j)** = the set of units whose immediate inputs include the output of unit  $j$

Applicable only to Neurons in  
Hidden Layer

## Unit 2 - ANN

### Calculation of a weight matrix



Downstream(4) = {6,7}  
Downstream(5) = {6,7}

outputs = {6, 7}

### Weight Matrix Shape

#neurons in previous layer  $\times$  #neurons in current layer

$$W_1 = \begin{bmatrix} w_{41} & w_{51} \\ w_{42} & w_{52} \\ w_{43} & w_{53} \end{bmatrix}_{3 \times 2}$$

$w_{ji}$

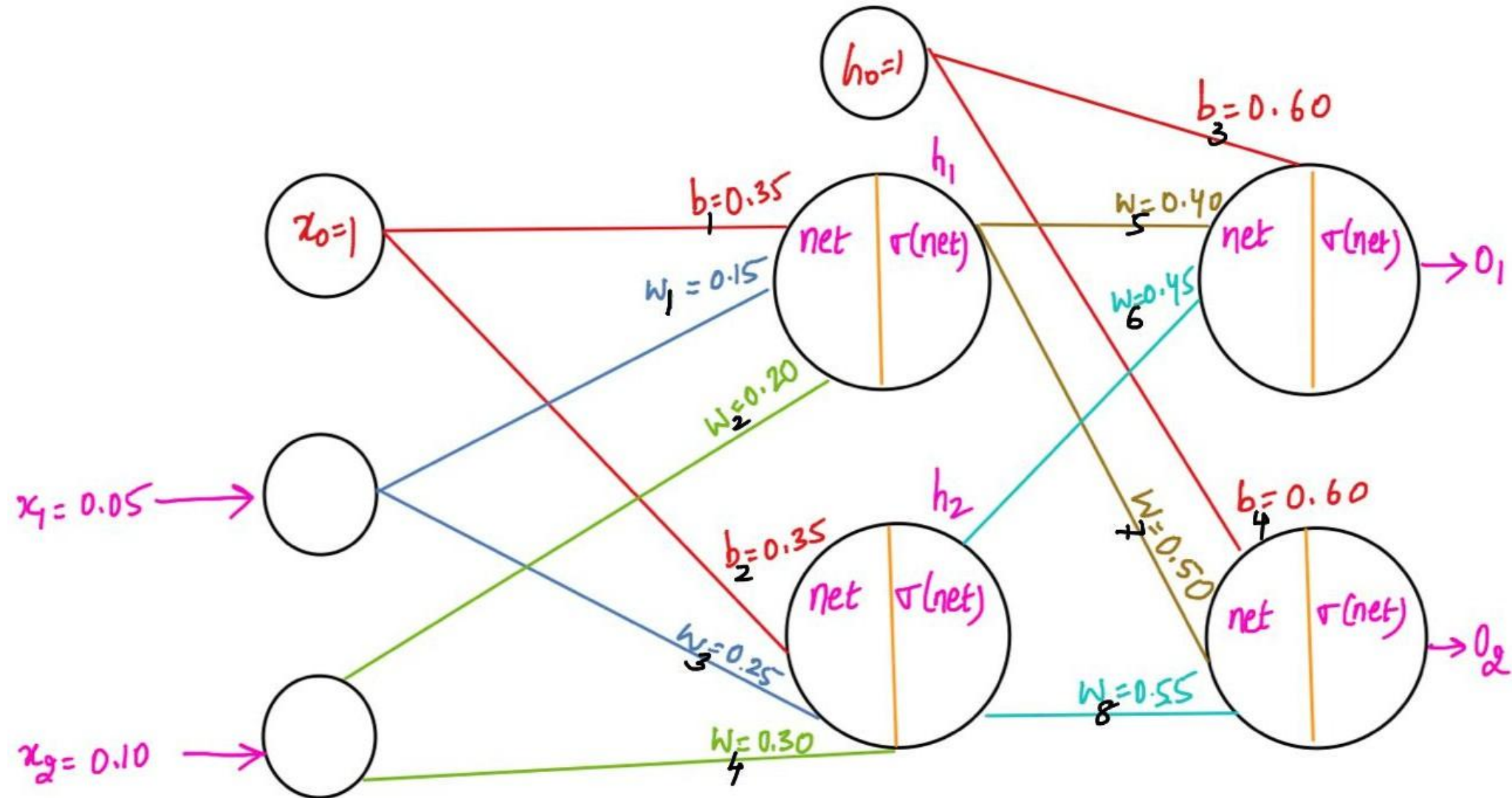
j: current layer neuron number  
i: previous layer neuron number



# Problem 1

## Unit 2 - ANN

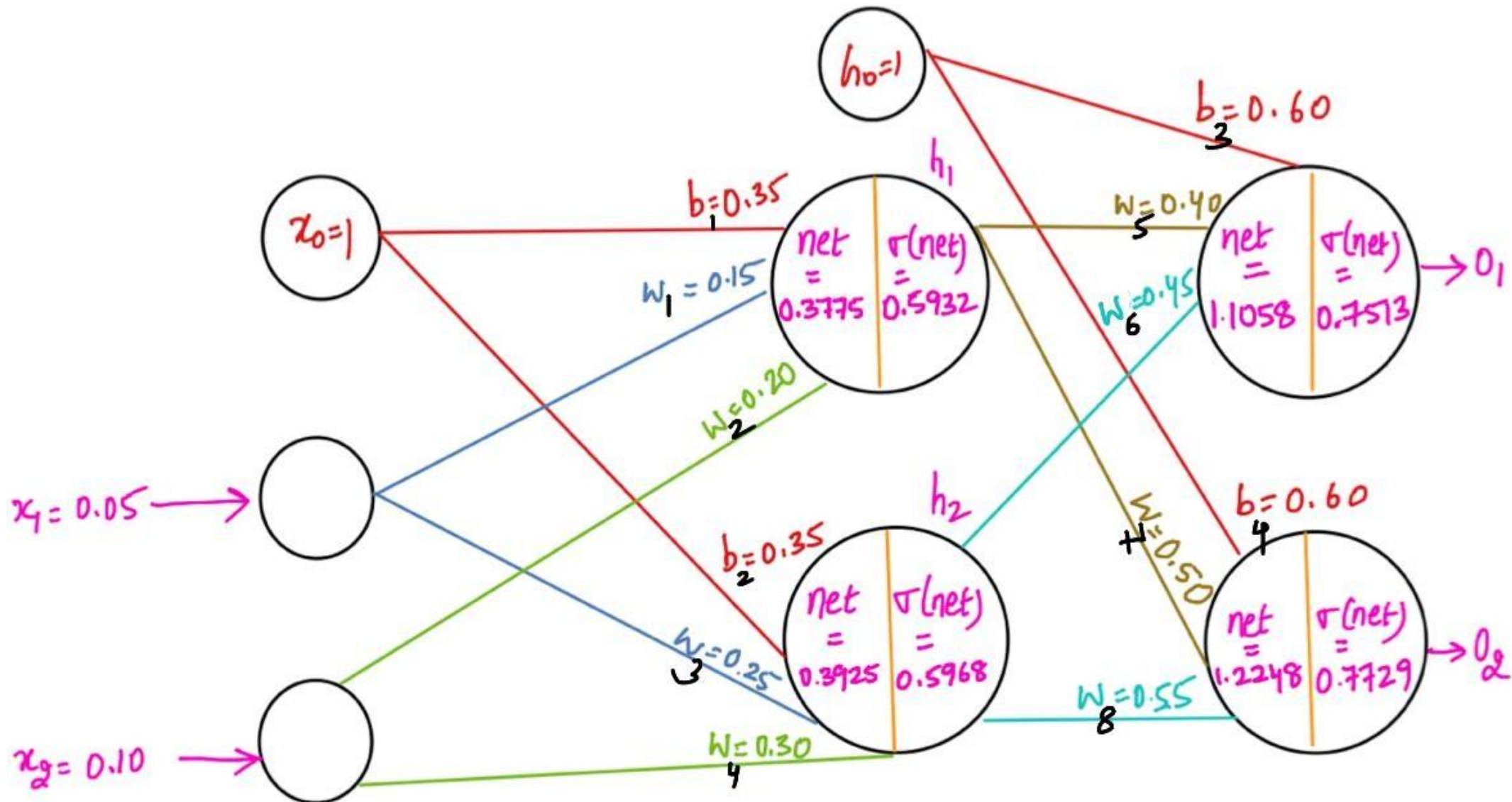
### Forward propagation --- Problem 1



t(target values at output neuron)

0.01

0.99



## Forward propagation Problem 1--- Solution

Output calculation at hidden layer

$$Z_1 = \underset{\substack{\downarrow \\ \text{net} \\ \text{sum}}}{(W_1)^T} * \underset{\substack{\downarrow \\ \text{Weight} \\ \text{matrix}}}{X} + \underset{\substack{\downarrow \\ \text{Input} \\ \text{vector}}}{b} \underset{\substack{\downarrow \\ \text{Bias} \\ \text{vector}}}{b}$$

$$\begin{bmatrix} \text{net}_1 \\ \text{net}_2 \end{bmatrix} = \begin{bmatrix} 0.15 & 0.20 \\ 0.25 & 0.30 \end{bmatrix} * \begin{bmatrix} 0.05 \\ 0.10 \end{bmatrix} + \begin{bmatrix} 0.35 \\ 0.35 \end{bmatrix}$$

$$\begin{bmatrix} \text{net}_1 \\ \text{net}_2 \end{bmatrix} = \begin{bmatrix} 0.3775 \\ 0.3925 \end{bmatrix}$$

$$a_1 = \sigma(Z_1) = \begin{bmatrix} \sigma(\text{net}_1) \\ \sigma(\text{net}_2) \end{bmatrix} = \begin{bmatrix} 0.5932 \\ 0.5968 \end{bmatrix}$$

output of activation function

Output calculation at output layer

$$Z_2 = (W_2)^T * a_1 + b$$

$$\begin{bmatrix} \text{net}_1 \\ \text{net}_2 \end{bmatrix} = \begin{bmatrix} 0.40 & 0.45 \\ 0.50 & 0.55 \end{bmatrix} * \begin{bmatrix} 0.5932 \\ 0.5968 \end{bmatrix} + \begin{bmatrix} 0.60 \\ 0.60 \end{bmatrix}$$

$$\begin{bmatrix} \text{net}_1 \\ \text{net}_2 \end{bmatrix} = \begin{bmatrix} 1.1058 \\ 1.2248 \end{bmatrix}$$

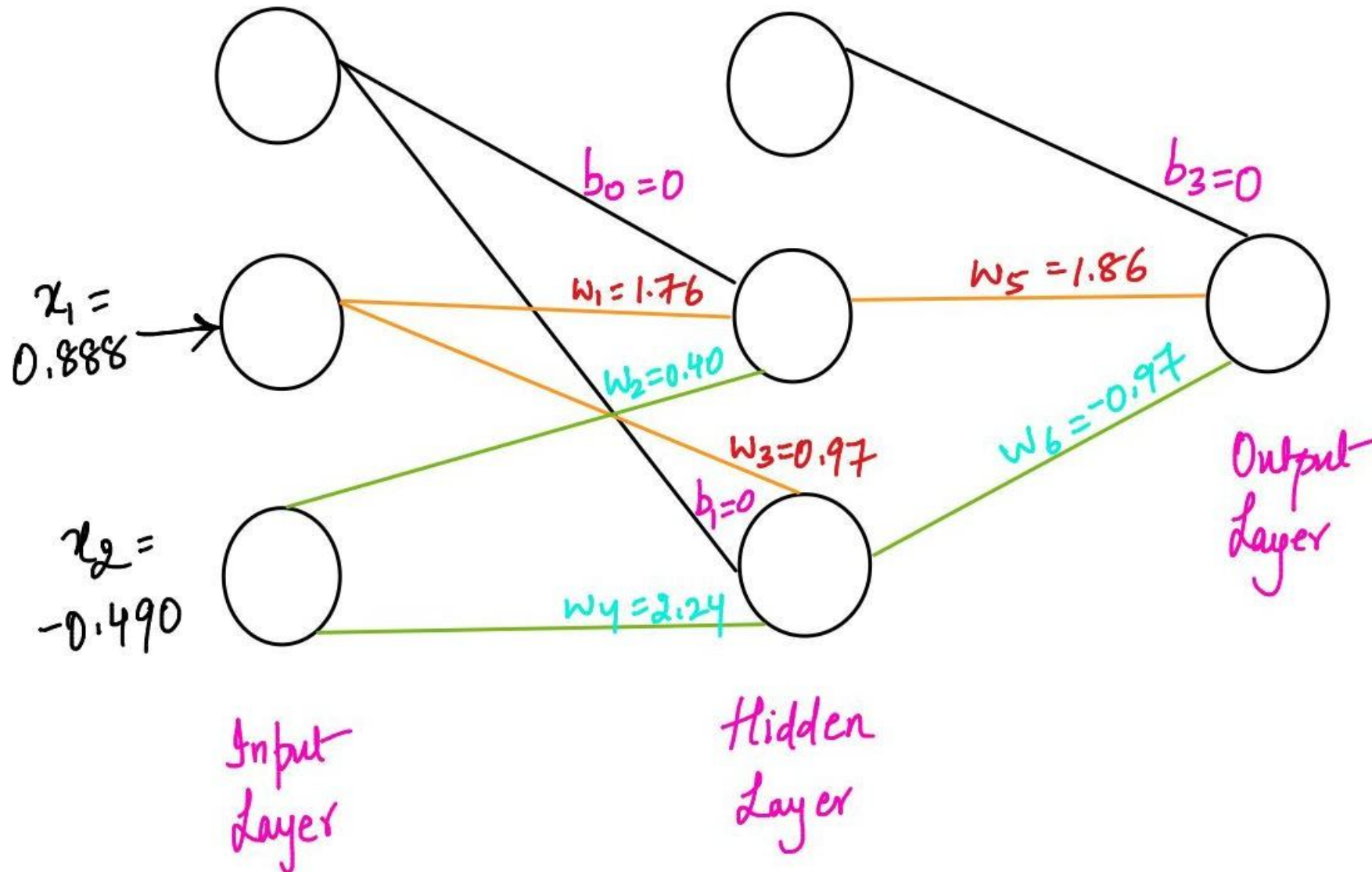
$$\hat{y} = \sigma(Z_2) = \begin{bmatrix} \sigma(\text{net}_1) \\ \sigma(\text{net}_2) \end{bmatrix} = \begin{bmatrix} 0.7513 \\ 0.7729 \end{bmatrix}$$

## Problem 2



## Unit 2 - ANN

### Forward propagation Problem - 2





## Unit 2 - ANN

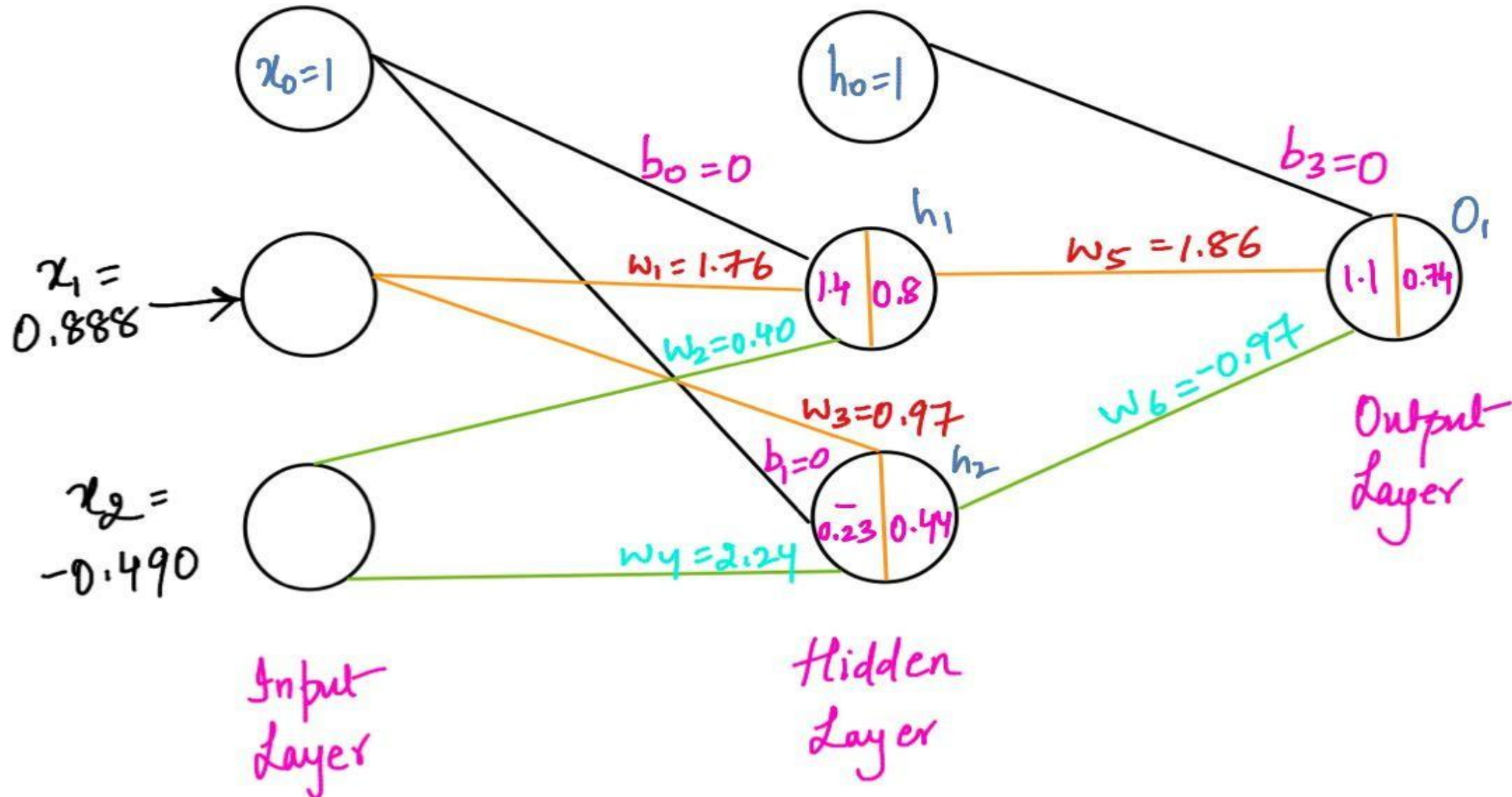
### Forward propagation Problem 2 - Solution



- During forward propagation at each node of hidden and output layer pre-activation and activation takes place.
- At the first node of the hidden layer, **a1**(pre-activation) is calculated first and then **h1**(activation) is calculated.
- **a1** is a weighted sum of inputs. Here, the weights are randomly generated.
- **a1** =  $w1*x1 + w2*x2 + b1 = 1.76*0.88 + 0.40*(-0.49) + 0 = 1.37$
- **h1** is the value of activation function applied on a1.  $h1 = \frac{1}{1 + e^{-a1}} = 0.8$  approx.
- **a2** =  $w3*x1 + w4*x2 + b2 = 0.97*0.88 + 2.24*(-0.49) + 0 = -2.29$  approx.  $h2 = \frac{1}{1 + e^{-a2}} = 0.44$  approx.
- For any layer after the first hidden layer, the input is output from the previous layer.
- **a3** =  $w5*h1 + w6*h2 + b3 = 1.86*0.8 + (-0.97)*0.44 + 0 = 1.1$  approx.  $h3 = \frac{1}{1 + e^{-a3}} = 0.74$  approx.

## Unit 2 - ANN

### Forward propagation Problem 2 - Solution



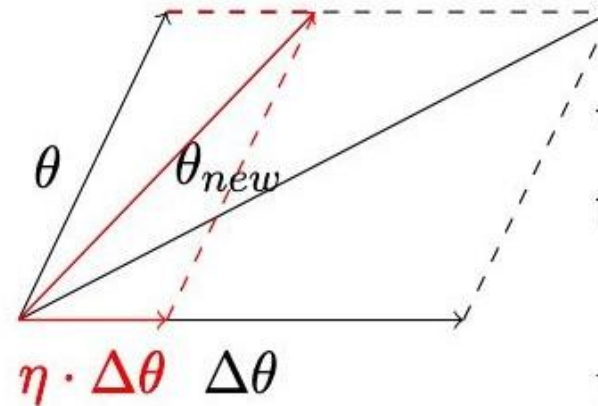
# Gradient Descent

**Goal :** To find a better way of traversing the error surface so that we can reach the minimum value quickly without resorting to brute force search!

vector of parameters,  
say, randomly initial-  
ized

$$\theta = [w, b]$$

change in the  
values of  $w, b$

$$\Delta\theta = [\Delta w, \Delta b]$$


We moved in the direc-  
tion of  $\Delta\theta$

Let us be a bit conservat-  
ive: move only by a small  
amount  $\eta$

$$\theta_{new} = \theta + \eta \cdot \Delta\theta$$

**Question:** What is the right  $\Delta\theta$  to use?

The answer comes from Taylor series

### Taylor series --- Recap

---

- Used to evaluate/approximate the value of a function at any point
- Taylor series of a function is an infinite sum of terms that are expressed in terms of the function's derivatives at a single point.
- Let's say we have a function with only 1 parameter,  $f(x)$ . Assume at a given point  $x$ , we want to evaluate the function at  $x+h$ , where  $h$  is a small step in the direction of  $x$ .
- We can use Taylor series to evaluate the value of function at  $x+h$  point.

$$f(x+h) = f(x) + hf'(x) + \frac{h^2}{2}f''(x) + \frac{h^3}{3!}f'''(x) + \dots$$

### Taylor series --- Recap

- Taylor formula for function of two variables :  $f(x,y)$

$$f(x+a, y+b) = f(x,y) + \frac{a^1}{1!} \frac{\partial f(x,y)}{\partial x} + \frac{b^1}{1!} \frac{\partial f(x,y)}{\partial y} + \frac{a^2}{2!} \frac{\partial^2 f(x,y)}{\partial x^2} + \frac{b^2}{2!} \frac{\partial^2 f(x,y)}{\partial y^2} + ab \frac{\partial^2 f(x,y)}{\partial x \partial y}$$

- We can also use vector notation to represent two-variable Taylor series as:

$$f(\vec{x} + \vec{u}) = f(\vec{x}) + (\vec{u})^T \cdot \nabla f(\vec{x}) + \frac{1}{2} (\vec{u})^T \cdot H_f(\vec{x}) \cdot \vec{u}$$

gradient of  $f$

$$\nabla f(\vec{x}) = \begin{bmatrix} \frac{\partial f(x,y)}{\partial x} \\ \frac{\partial f(x,y)}{\partial y} \end{bmatrix}$$

Hessian matrix ( $\nabla^2 f(\vec{x})$ )

$$H_f(\vec{x}) = \begin{bmatrix} \frac{\partial^2 f(x,y)}{\partial x^2} & \frac{\partial^2 f(x,y)}{\partial x \partial y} \\ \frac{\partial^2 f(x,y)}{\partial y \partial x} & \frac{\partial^2 f(x,y)}{\partial y^2} \end{bmatrix}$$

Let  $\vec{x} = \begin{bmatrix} x \\ y \end{bmatrix}$  ,  $\vec{u} = \begin{bmatrix} a \\ b \end{bmatrix}$

input vector      change in values



For our purpose, the function we want to approximate is a loss function.

For ease of notation, let  $\Delta\theta = u$ , then from Taylor series, we have,

$$\begin{aligned}\mathcal{L}(\theta + \eta u) &= \mathcal{L}(\theta) + \eta * u^T \nabla \mathcal{L}(\theta) + \frac{\eta^2}{2!} * u^T \nabla^2 \mathcal{L}(\theta) u + \frac{\eta^3}{3!} * \dots + \frac{\eta^4}{4!} * \dots \\ &= \mathcal{L}(\theta) + \eta * u^T \nabla \mathcal{L}(\theta) \quad [\eta \text{ is typically small, so } \eta^2, \eta^3, \dots \rightarrow 0]\end{aligned}$$

$$\mathcal{L}(\theta + \eta u) - \mathcal{L}(\theta) = u^T \nabla \mathcal{L}(\theta)$$

Note that the move  $(\eta u)$  would be favorable only if,

$$\mathcal{L}(\theta + \eta u) - \mathcal{L}(\theta) < 0 \quad [i.e., \text{ if the new loss is less than the previous loss}]$$

This implies,  $u^T \nabla \mathcal{L}(\theta) < 0$

Let  $\beta$  be the angle between  $u^T$  and  $\nabla \mathcal{L}(\theta)$ , then we know that,

$$u^T \nabla \mathcal{L}(\theta) < 0$$

This implies,  $k * \cos(\beta) < 0$

Thus,  $\mathcal{L}(\theta + \eta u) - \mathcal{L}(\theta) = u^T \nabla \mathcal{L}(\theta) = k * \cos(\beta)$  will be most negative when  $\cos(\beta) = -1$  i.e., when  $\beta$  is  $180^\circ$

#### Gradient Descent Rule

- The direction  $u$  that we intend to move in should be at  $180^\circ$  w.r.t. the gradient
- In other words, move in a direction opposite to the gradient

$$\sim \Delta \theta = u = -\nabla \mathcal{L}(\theta)$$

#### Step 1. The function we want to minimize

- Let  $v(x)$  be a function (e.g., a loss function in machine learning).
- Goal: find  $x$  that minimizes  $v(x)$ .

#### Step 2. The gradient gives the direction of steepest increase

- Gradient:

$$\nabla v(x) = \left( \frac{\partial v}{\partial x_1}, \frac{\partial v}{\partial x_2}, \dots \right)$$

- Properties:
  - Points in the direction of **maximum increase** of  $v$ .
  - Magnitude shows how fast  $v$  increases.

#### Step 3. Directional derivative — effect of moving in any direction

- For a direction  $u$  (unit vector), the rate of change is:

$$D_u v(x) = u \cdot \nabla v(x)$$

- If  $u \cdot \nabla v > 0$ :  $v$  increases along  $u$ .
- If  $u \cdot \nabla v < 0$ :  $v$  decreases along  $u$ .

#### Step 4. Choosing the best direction — steepest descent

- Maximum decrease happens in the direction of the **negative gradient**:

$$u = -\frac{\nabla v(x)}{\|\nabla v(x)\|}$$

- This gives the steepest drop in  $v(x)$ .

#### Step 5. Gradient descent update rule

- To update  $x$  step by step:

$$x_{\text{new}} = x_{\text{old}} - \eta \nabla v(x_{\text{old}})$$

Where:

- $\eta$  learning rate (step size).
- Repeating this moves  $x$  gradually toward a local minimum.

We would like move in the direction of gradient only by a small step  $\eta$ (eta) to avoid overshooting the minimum value.

$\eta$  is also called the learning rate.

So final equations become:

$$\begin{aligned}w_{t+1} &= w_t - \eta \nabla w_t \\b_{t+1} &= b_t - \eta \nabla b_t \\ \text{where, } \nabla w_t &= \frac{\partial \mathcal{L}(w, b)}{\partial w} \text{ at } w = w_t, b = b_t, \nabla b_t = \frac{\partial \mathcal{L}(w, b)}{\partial b} \text{ at } w = w_t, b = b_t\end{aligned}$$



---

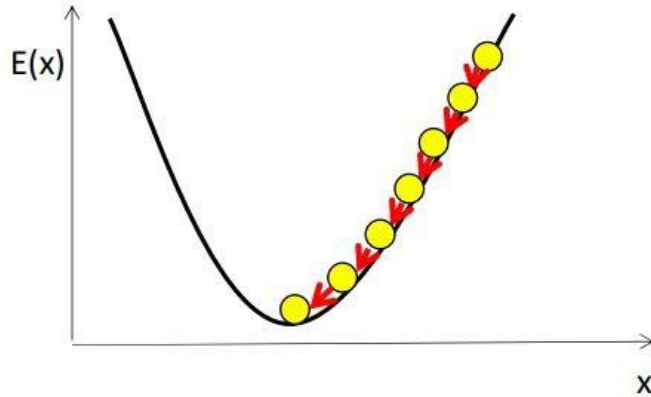
**Algorithm 1:** `gradient_descent()`

---

```
t ← 0;  
max_iterations ← 1000;  
while t < max_iterations do  
    |  $w_{t+1} \leftarrow w_t - \eta \nabla w_t;$   
    |  $b_{t+1} \leftarrow b_t - \eta \nabla b_t;$   
end
```

---

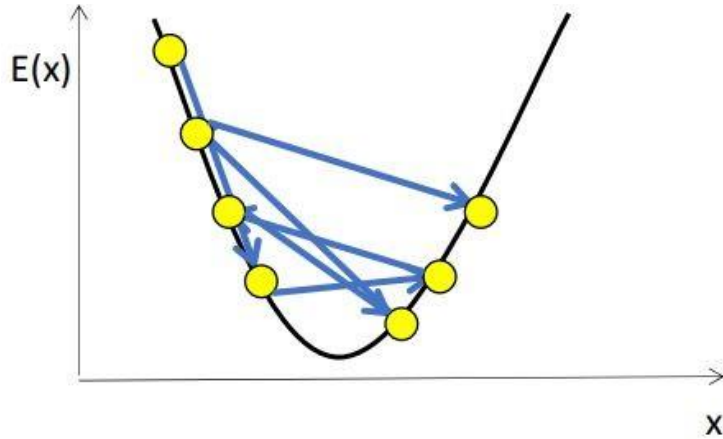
### Learning rate being too small



$$w_{t+1} = w_t - \eta \nabla w_t$$

- Suppose we keep our  $\eta$  too small.
- Let's start with a point  $x$ .
- If  $\eta$  is too small on each updation we are changing our  $x$  with some small value i.e taking small steps.
- Again, from the new point, we take small steps to get to the minimum.
- If  $\eta$  is small, gradient descent can be slow to reach minimum.

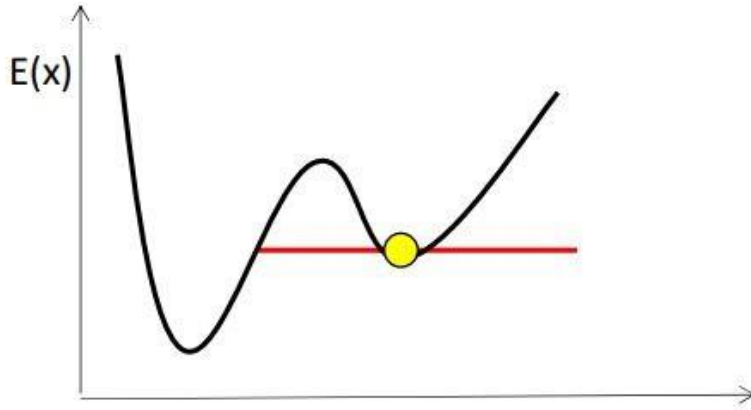
### Learning rate being too large



$$w_{t+1} = w_t - \eta \nabla w_t$$

- If  $\eta$  is too large, then gradient may overshoot the minimum or may never reach minimum.
- Suppose we start at point  $x'$ .
- If  $\eta$  is too large, we take huge steps. Let's say we take this step and reach a new point.
- On the subsequent updation, we take a huge step and go right past the minimum.
- This may continue and we may never end up reaching the minimum.

### Problem faced by gradient descent



$$w_{t+1} = w_t - \eta \nabla w_t$$

Suppose our  $x$  is already at local minimum, what would a gradient descent step do?

- At the local minimum, the slope would be zero. Hence, the derivative term is zero.
- It leaves our  $x$  unchanged.
- This is one problem with gradient descent – you may get stuck in a local minimum instead of reaching the global minimum.

### Is there any ideal value for Learning rate???

#### Tips for initial learning rate ?

- Tune learning rate [Try different values on a log scale: 0.0001, 0.001, 0.01, 0.1, 1.0]
- Run a few epochs with each of these and figure out a learning rate which works best
- Now do a finer search around this value [for example, if the best learning rate was 0.1 then now try some values around it: 0.05, 0.2, 0.3]
- Disclaimer: these are just heuristics ... no clear winner strategy

### Is there any ideal value for Learning rate???

---

- As an alternative to constant learning, one could use adaptive learning rates(also called **learning rate annealing**) to increase performance and reduce training time.
- The simplest and perhaps most used adaptation of learning rate during training are techniques that reduce the learning rate over time. These have the benefit of making large changes at the beginning of the training procedure when larger learning rate values are used, and decreasing the learning rate such that a smaller rate and therefore smaller training updates are made to weights later in the training procedure.
- This has the effect of quickly learning good weights early and fine tuning them later.



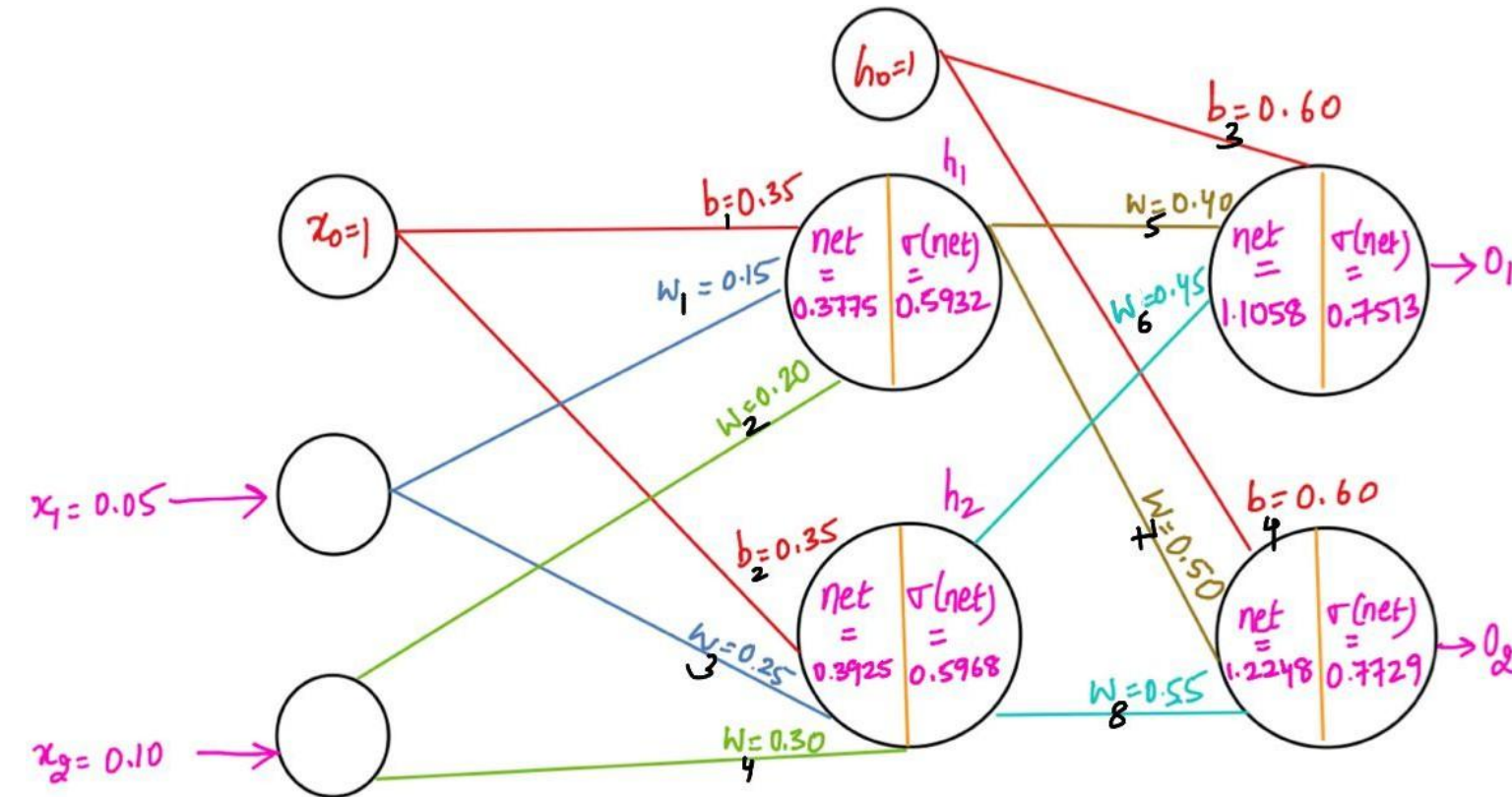
## Backward Propagation

**Process of modifying the parameters (weights, bias) with an objective of minimizing the loss/error in the predicted value at each neuron in the output layer.**

- How do we know whether the output generated by the neural network is correct?  
We need to define a **error function (cost)** and **backpropagate** the error.
- **Iteratively** process a set of training tuples and compare the network's prediction with the actual known target value.
- For each training tuple, the weights are modified to **minimize the loss function** between the network's prediction and the actual target value.
- Modifications are made in the “**backward**” direction i.e. from the output layer, through each hidden layer down to the first hidden layer, hence the name “**back propagation**”.

## Unit 2 - ANN

### Backward propagation



| Neuron # in Output layer (k) | Target values at output neuron (t) | Predicted values (O) | Error in prediction ( $t_k - O_k$ ) |
|------------------------------|------------------------------------|----------------------|-------------------------------------|
| 1                            | 0.01                               | 0.7513               | -0.7413                             |
| 2                            | 0.99                               | 0.7729               | 0.2171                              |

$$\text{MSE} = \frac{1}{2} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

$$E_d(\vec{w}) \equiv \frac{1}{2} \sum_{k \in \text{outputs}} (t_k - o_k)^2$$

### Backward propagation -- Computing Error at Output layer

| Neuron # in Output layer<br>( $k \in \text{outputs}$ )                      | Target values at output neuron<br>( $t$ ) | Predicted values<br>( $O$ ) | Error in prediction<br>( $t_k - O_k$ ) | $(t_k - O_k)^2$ |
|-----------------------------------------------------------------------------|-------------------------------------------|-----------------------------|----------------------------------------|-----------------|
| 1                                                                           | 0.01                                      | 0.7513                      | -0.7413                                | 0.5495          |
| 2                                                                           | 0.99                                      | 0.7729                      | 0.2171                                 | 0.0471          |
| $\sum_{k \in \text{outputs}} (t_k - o_k)^2$                                 |                                           |                             |                                        | 0.5966          |
| $E_d(\vec{w}) \equiv \frac{1}{2} \sum_{k \in \text{outputs}} (t_k - o_k)^2$ |                                           |                             |                                        | 0.2983          |

# Backward Propagation

Understanding weight update with a Toy example

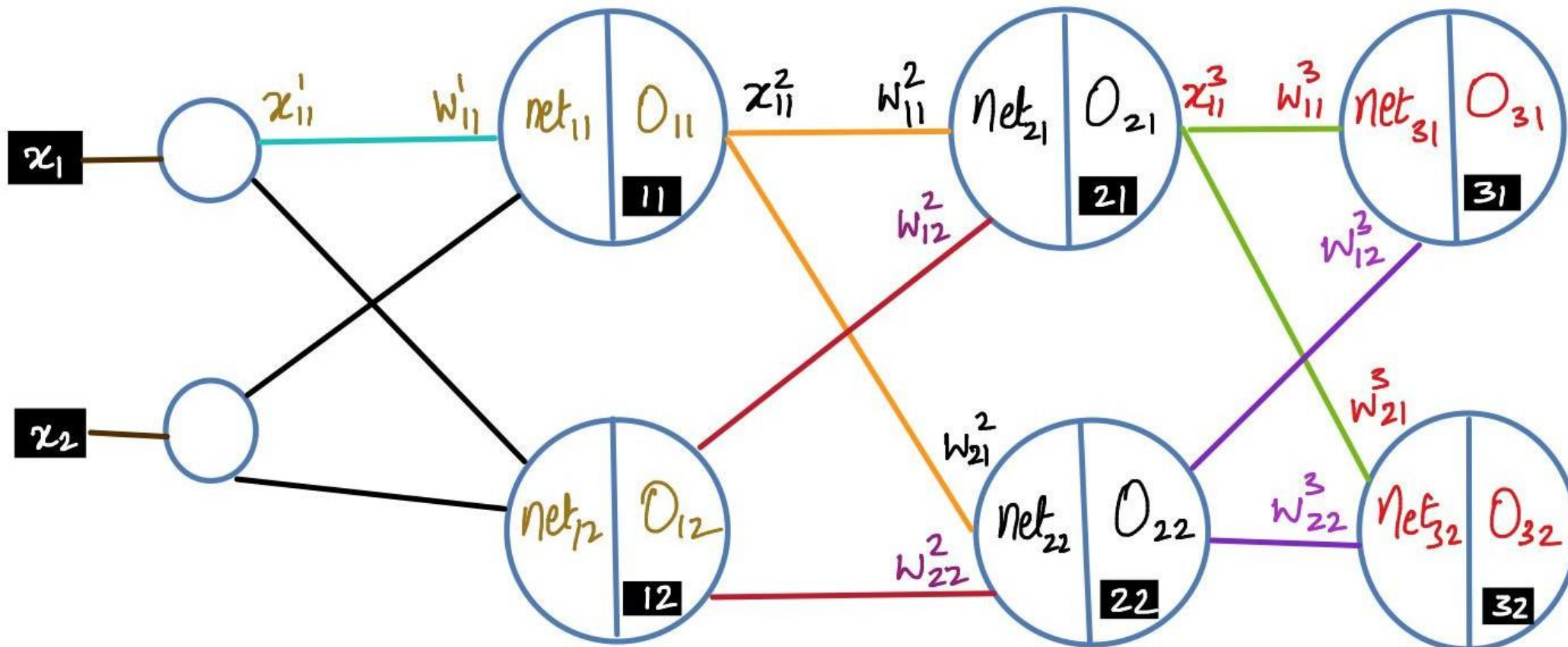
### Understanding Weight update using a Toy example

Weights are updated as using the gradient descent method.

$$\rightarrow w_{ji} \leftarrow w_{ji} + \Delta w_{ji}$$

where

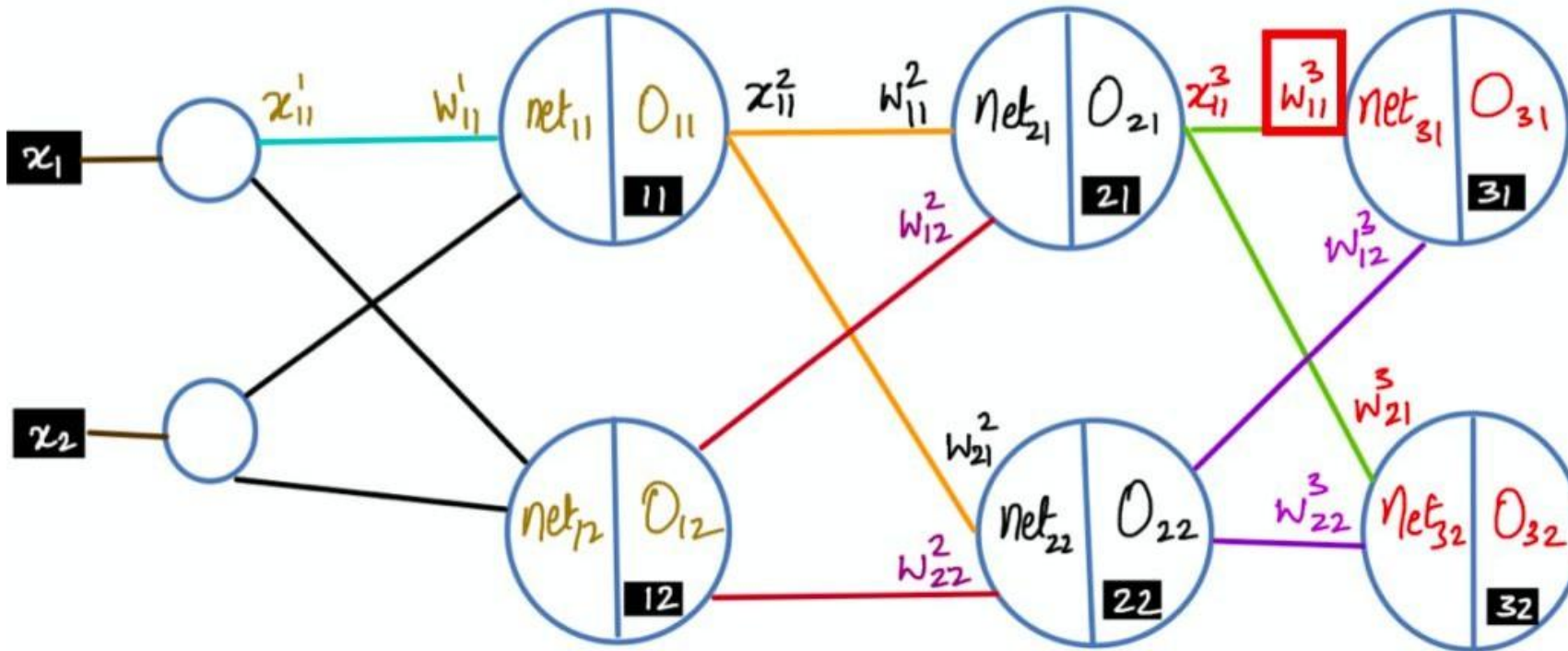
$$\Delta w_{ji} = -\eta \frac{\partial E_d}{\partial w_{ji}}$$





## Unit 2 - ANN

### Updating Weight at Output layer

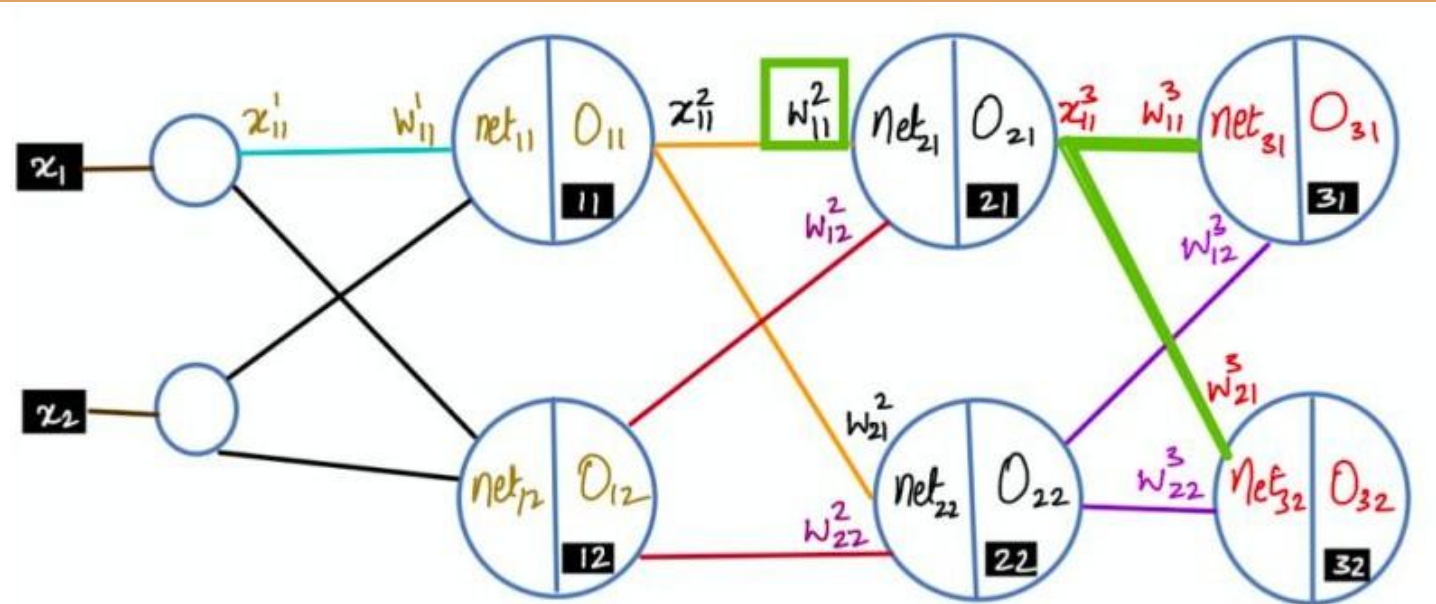


$$w_{11}^3 = w_{11}^3 + \Delta w_{11}^3, \quad \Delta w_{11}^3 = -\eta \frac{\partial E_d}{\partial w_{11}^3}$$

$$\frac{\partial E_d}{\partial w_{11}^3} = \frac{\partial E_d}{\partial O_{31}} \frac{\partial O_{31}}{\partial net_{31}} \frac{\partial net_{31}}{\partial w_{11}^3}$$

## Unit 2 - ANN

### Updating Weight at Hidden Layer 2



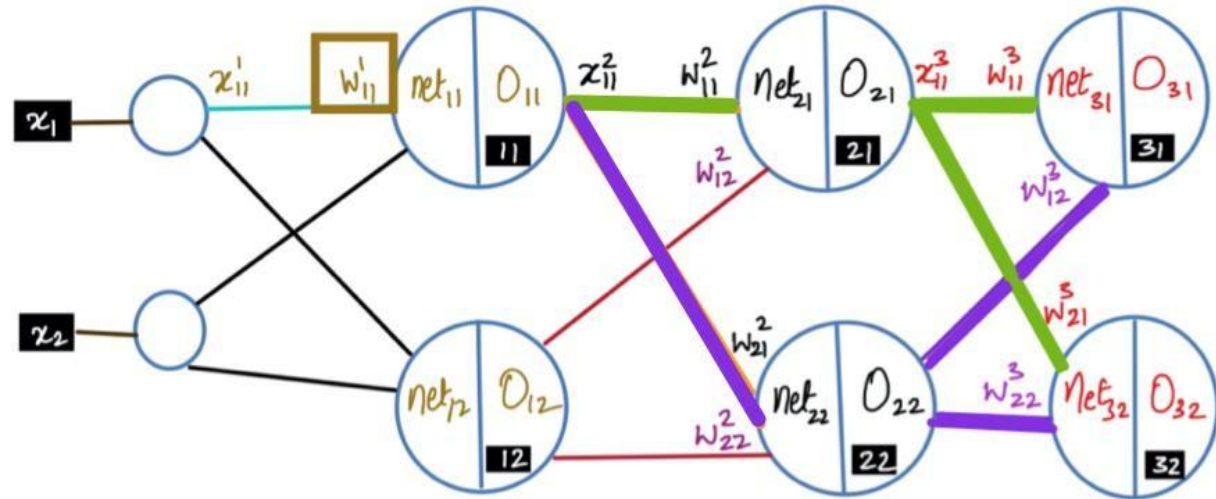
$$w^2_{11} = w^2_{11} + \Delta w^2_{11},$$

$$\Delta w^2_{11} = -\eta \frac{\partial E_d}{\partial w^2_{11}}$$

$$\frac{\partial E_d}{\partial w^2_{11}} = \frac{\partial E_d}{\partial O_{31}} \frac{\partial O_{31}}{\partial net_{31}} \frac{\partial net_{31}}{\partial O_{21}} \frac{\partial O_{21}}{\partial net_{21}} \frac{\partial net_{21}}{\partial w^2_{11}} + \frac{\partial E_d}{\partial O_{32}} \frac{\partial O_{32}}{\partial net_{32}} \frac{\partial net_{32}}{\partial O_{21}} \frac{\partial O_{21}}{\partial net_{21}} \frac{\partial net_{21}}{\partial w^2_{11}}$$

## Unit 2 - ANN

### Updating Weight at Hidden Layer 1



$$w'_{11} = w'_{11} + \Delta w'_{11}$$

$$\Delta w'_{11} = -\eta \frac{\partial E_d}{\partial w'_{11}}$$

$$\frac{\partial E_d}{\partial w'_{11}} = \frac{\partial E_d}{\partial O_{31}} \frac{\partial O_{31}}{\partial net_{31}} \left[ \frac{\partial net_{31}}{\partial O_{21}} \frac{\partial O_{21}}{\partial net_{21}} \frac{\partial net_{21}}{\partial O_{11}} \frac{\partial O_{11}}{\partial net_{11}} \frac{\partial net_{11}}{\partial w'_{11}} + \frac{\partial net_{31}}{\partial O_{22}} \frac{\partial O_{22}}{\partial net_{22}} \frac{\partial net_{22}}{\partial O_{11}} \frac{\partial O_{11}}{\partial net_{11}} \frac{\partial net_{11}}{\partial w'_{11}} \right] + \frac{\partial E_d}{\partial O_{32}} \frac{\partial O_{32}}{\partial net_{32}} \left[ \frac{\partial net_{32}}{\partial O_{21}} \frac{\partial O_{21}}{\partial net_{21}} \frac{\partial net_{21}}{\partial O_{11}} \frac{\partial O_{11}}{\partial net_{11}} \frac{\partial net_{11}}{\partial w'_{11}} + \frac{\partial net_{32}}{\partial O_{22}} \frac{\partial O_{22}}{\partial net_{22}} \frac{\partial net_{22}}{\partial O_{11}} \frac{\partial O_{11}}{\partial net_{11}} \frac{\partial net_{11}}{\partial w'_{11}} \right] +$$

# Backward Propagation Derivation Rules



### Updating weights to lower the error

Error computation for a single training instance considering error at all neurons in output layer.

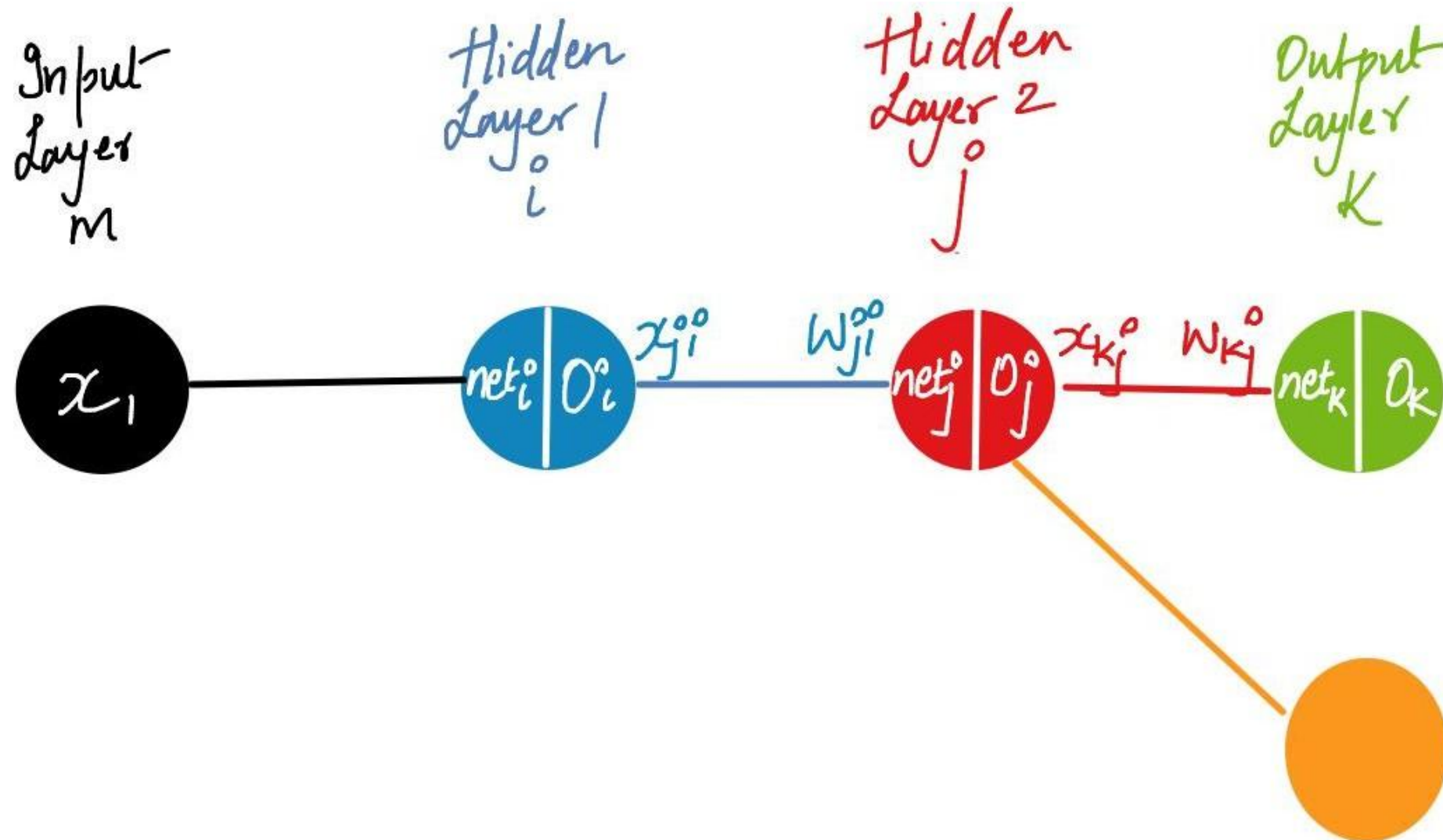
$$\longrightarrow E_d(\vec{w}) \equiv \frac{1}{2} \sum_{k \in \text{outputs}} (t_k - o_k)^2$$

Weights are updated as using the gradient descent method.

$$\longrightarrow w_{ji} \leftarrow w_{ji} + \Delta w_{ji} \quad \text{where} \quad \Delta w_{ji} = -\eta \frac{\partial E_d}{\partial w_{ji}}$$

Notice the weight can influence the rest of the network only through  $net_j$ . Hence, we use the chain rule :

$$\longrightarrow \frac{\partial E_d}{\partial w_{ji}} = \frac{\partial E_d}{\partial net_j} \frac{\partial net_j}{\partial w_{ji}}$$





## Unit 2 - ANN

### Training rule for output unit weights



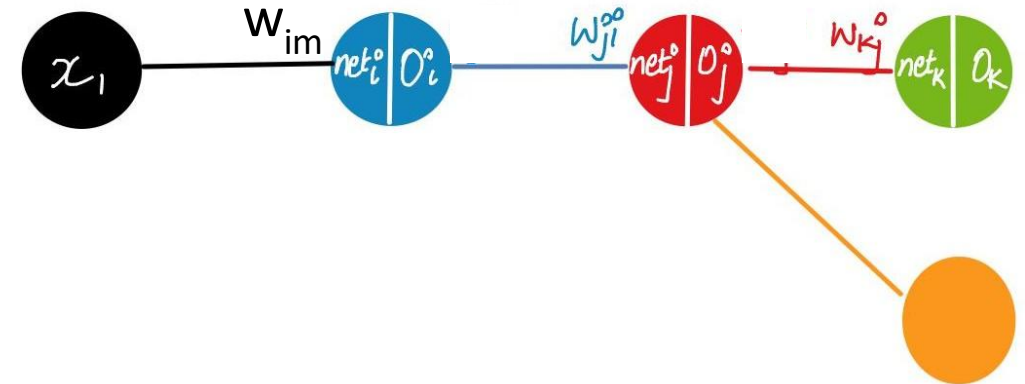
$$w_{kj} = w_{kj} + \Delta w_{kj}$$

$$\text{we know, } \Delta w_{kj} = -\eta \frac{\partial E_d}{\partial w_{kj}}$$

$$\Rightarrow \Delta w_{kj} = -\eta \frac{\partial E_d}{\partial O_k} \times \frac{\partial O_k}{\partial \text{net}_k} \times \frac{\partial \text{net}_k}{\partial w_{kj}}$$

$$\text{net}_k = \sum_j w_{kj} O_j$$

$$\boxed{\frac{\partial \text{net}_k}{\partial w_{kj}} = O_j}$$



## Unit 2 - ANN

### Training rule for output unit weights



$$E_d = \frac{1}{2} \sum_{k \in \text{outputs}} (t_k - O_k)^2$$

$$\frac{\partial E_d}{\partial O_k} = -(t_k - O_k)$$

$$O_k = \sigma(\text{net}_k)$$

$$\frac{\partial O_k}{\partial \text{net}_k} = \sigma(\text{net}_k) (1 - \sigma(\text{net}_k))$$

$$\frac{\partial O_k}{\partial \text{net}_k} = O_k (1 - O_k)$$

$$\Delta w_{kj} = -\eta \frac{\partial E_d}{\partial O_k} \times \frac{\partial O_k}{\partial \text{net}_k} \times \frac{\partial \text{net}_k}{\partial w_{kj}}$$

$$y = \sigma(x) = \frac{1}{1 + e^{-x}}$$

$$\frac{dy}{dx} = \frac{d\sigma(x)}{dx} = \frac{d}{dx} \left\{ \frac{1}{1 + e^{-x}} \right\}$$

$$= -(1 + e^{-x})^{-2} \frac{d}{dx} (1 + e^{-x})$$

$$= -(1 + e^{-x})^{-2} \left( \frac{d(1)}{dx} + \frac{d(e^{-x})}{dx} \right)$$

$$= -(1 + e^{-x})^{-2} \times -e^{-x}$$

$$= \frac{e^{-x}}{(1 + e^{-x})^2} = \frac{1}{1 + e^{-x}} \left( 1 - \frac{1}{1 + e^{-x}} \right)$$

$$= \sigma(x) (1 - \sigma(x)) = y(1 - y)$$

### Training rule for output unit weights

$$\Delta w_{kj} = -\eta \frac{\partial E_d}{\partial O_k} \times \frac{\partial O_k}{\partial \text{net}_k} \times \frac{\partial \text{net}_k}{\partial w_{kj}}$$

$$\Delta w_{kj} = -\eta (- (t_k - O_k)) \times (O_k (1 - O_k)) \times (O_j)$$

$$\Delta w_{kj} = \eta (t_k - O_k) O_k (1 - O_k) O_j$$

$$\delta_k \leftarrow -\frac{\partial E_d}{\partial \text{net}_k}$$

$$\delta_k \leftarrow (t_k - O_k) O_k (1 - O_k)$$

$$\Delta w_{kj} = \eta \delta_k O_j$$

## Unit 2 - ANN

### Training rule for hidden layer 2 weights

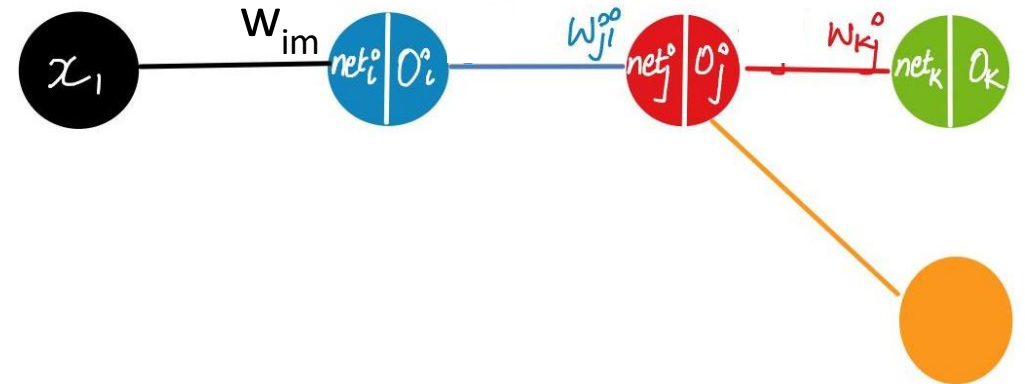
$$w_{ji} = w_{ji} + \Delta w_{ji}$$

we know,  $\Delta w_{ji} = -\eta \frac{\partial E_d}{\partial w_{ji}}$

$$\Rightarrow \Delta w_{ji} = -\eta \frac{\partial E_d}{\partial net_j} \times \frac{\partial net_j}{\partial w_{ji}}$$

$$net_j = \sum_i w_{ji} O_i$$

$$\boxed{\frac{\partial net_j}{\partial w_{ji}} = O_i}$$



### Training rule for hidden layer 2 weights

$$\frac{\partial E_d}{\partial \text{net}_j} = \sum_{k \in \text{downstream}(j)} \frac{\partial E_d}{\partial O_k} \frac{\partial O_k}{\partial \text{net}_k} \frac{\partial \text{net}_k}{\partial O_j} \frac{\partial O_j}{\partial \text{net}_j}$$

$$\text{net}_k = \sum_j O_j w_{kj}$$

$$\frac{\partial \text{net}_k}{\partial O_j} = w_{kj}$$

$$\frac{\partial E_d}{\partial \text{net}_j} = \sum_{k \in \text{down}(j)} -\delta_k w_{kj} O_j (1 - O_j)$$

$$[ \text{where } \delta_k = (t_k - O_k)(O_k(1 - O_k)) ]$$

### Training rule for hidden layer 2 weights

$$\Delta w_{ji} = -\eta \times \left[ \sum_{k \in \text{down}(j)} -\delta_k w_{kj} O_j (1 - O_j) \right] \times O_i$$

$\Rightarrow -\delta_j$

$$\Delta w_{ji} = \eta \delta_j O_i$$

where

$$\delta_j = \sum_{k \in \text{downstream}(j)} \delta_k w_{kj} O_j (1 - O_j)$$

$$\delta_k = (t_k - O_k) O_k (1 - O_k)$$



## Unit 2 - ANN

### Training rule for hidden layer 1 weights

$$w_{im} = w_{im} + \Delta w_{im}$$

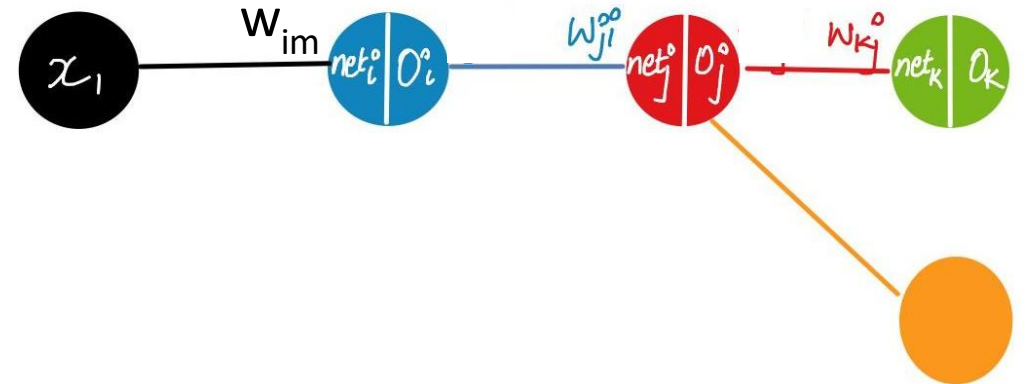
we know,  $\Delta w_{im} = -\eta \frac{\partial E_d}{\partial w_{im}}$

$$\Rightarrow \Delta w_{im} = -\eta \frac{\partial E_d}{\partial net_i} \times \frac{\partial net_i}{\partial w_{im}}$$

$$net_i = \sum w_{im} x_m$$

$$\frac{\partial net_i}{\partial w_{im}} = x_m$$

$$\frac{\partial E_d}{\partial net_i} = \sum_{k \in \text{downstream}(i)} -\delta_k w_{ji} O_i (1 - O_i)$$



$$\Delta w_{im} = \eta \delta_i x_m$$

where  $\delta_i = \sum_{k \in \text{downstream}(i)} \delta_j w_{ji} O_i (1 - O_i)$

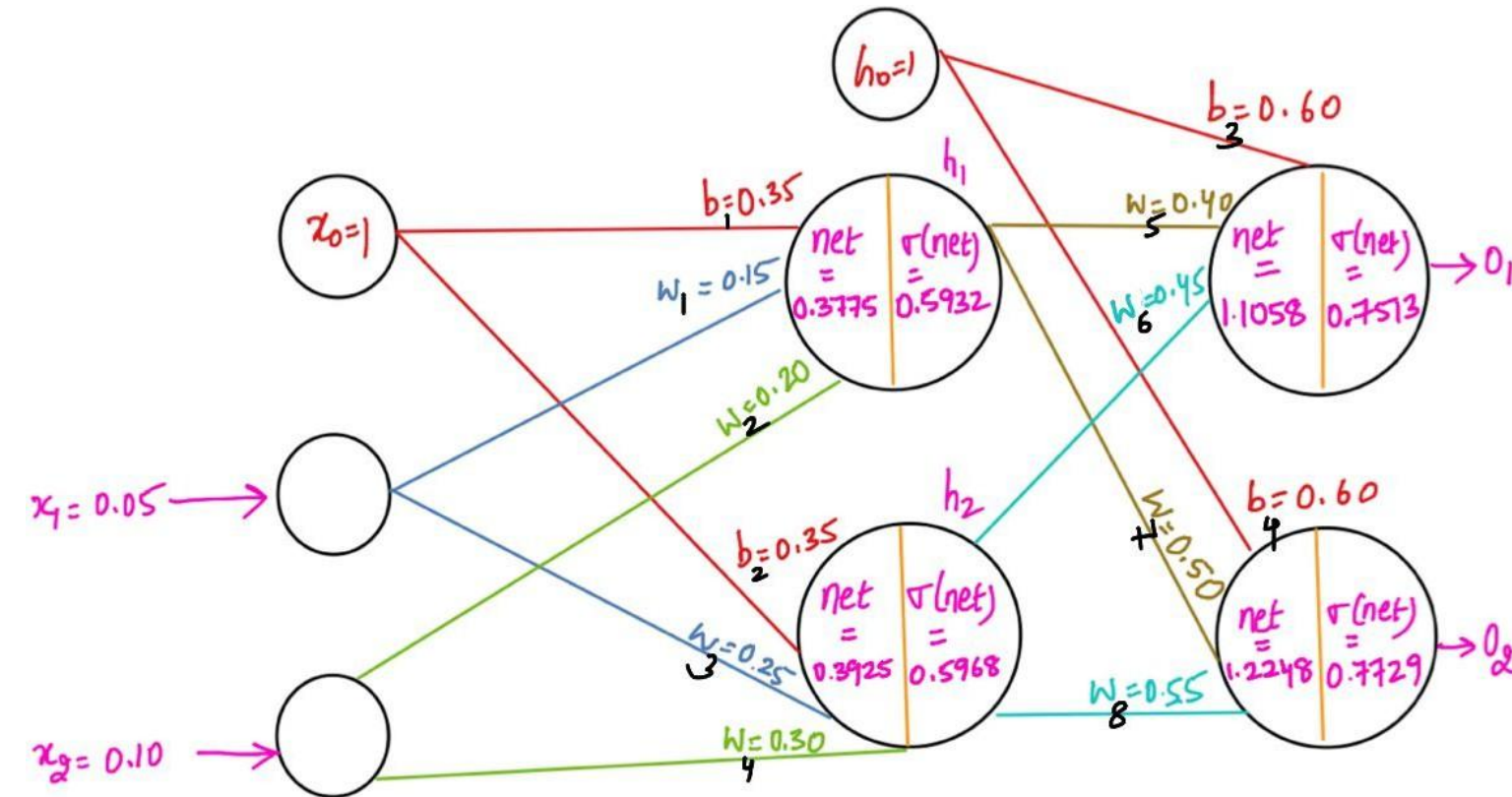
$$\delta_j = \sum_{k \in \text{downstream}(j)} \delta_k w_{kj} O_j (1 - O_j)$$

$$\delta_k = (t_k - O_k) O_k (1 - O_k)$$

# Backward Propagation Problem

## Unit 2 - ANN

### Using Backward propagation update w5, w7, w1

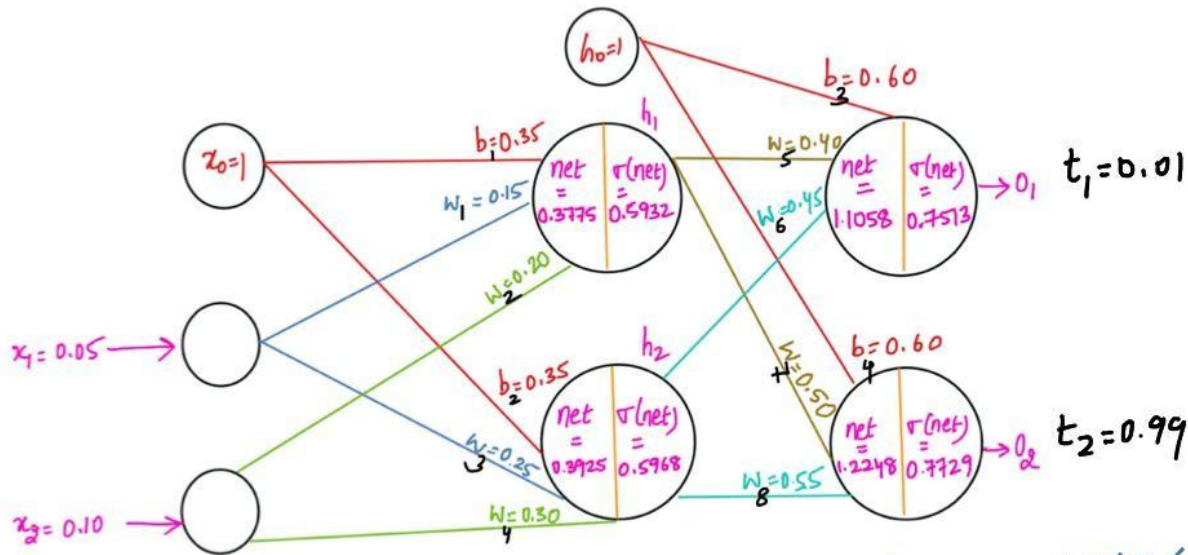


| Neuron # in Output layer (k) | Target values at output neuron (t) | Predicted values (O) | Error in prediction ( $t_k - O_k$ ) |
|------------------------------|------------------------------------|----------------------|-------------------------------------|
| 1                            | 0.01                               | 0.7513               | -0.7413                             |
| 2                            | 0.99                               | 0.7729               | 0.2171                              |

$$E_d(\vec{w}) \equiv \frac{1}{2} \sum_{k \in \text{outputs}} (t_k - o_k)^2$$

## Unit 2 - ANN

### Updating w5 (output layer)



Output layer :-

$$\Delta w_{kj} = \eta * (t_k - o_k) * o_k (1 - o_k) * x_{kj}$$

$$\begin{aligned} \Delta w_5 &= 0.5 * (0.01 - 0.7513) * 0.7513 (1 - 0.7513) * 0.5932 \\ &= 0.5 * -0.7413 * 0.7513 * 0.2487 * 0.5932 \\ &= 0.5 * -0.0821 \\ &= -0.04105 \end{aligned}$$

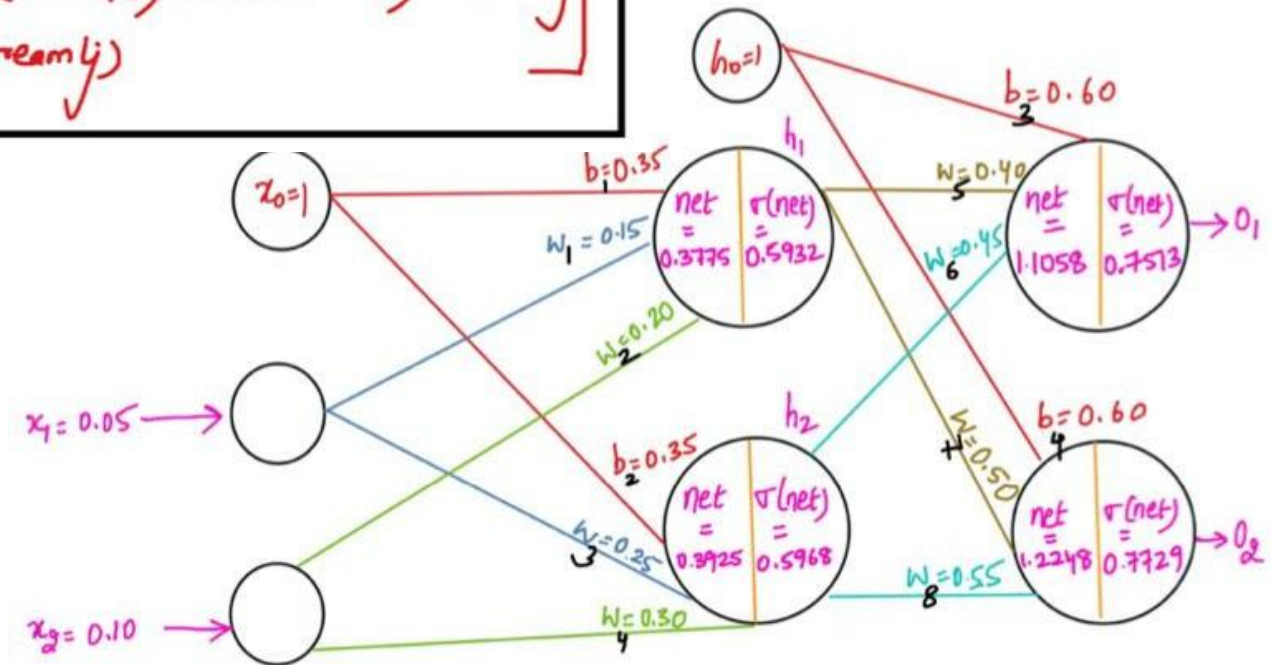
$$\begin{aligned} \Rightarrow w_5 &= w_5 + \Delta w_5 \\ &= 0.40 + (-0.04105) \\ &= 0.35895 \end{aligned}$$



## Unit 2 - ANN

### Updating w1 (hidden layer)

$$\Delta w_{ji} = -\eta * o_j(1-o_j) * x_{ji} * \left[ \sum_{k \in \text{Downstream}} -(t_k - o_k) * o_k(1-o_k) * w_{kj} \right]$$



$$\Delta w_1 = -0.5 * 0.5932(1-0.5932) * 0.05 * \left[ \begin{aligned} &-(0.01 - 0.7513) * 0.7513(1-0.7513) * 0.40 \\ &+ \\ &-(0.99 - 0.7729) * 0.7729(1-0.7729) * 0.50 \end{aligned} \right]$$

$$= -0.00021$$



### Vanishing Gradients Problem with Sigmoid

---

#### Vanishing Gradients

- As the backpropagation algorithm advances downwards (or backward) from the output layer towards the input layer, the **gradients often get smaller and smaller and approach zero** which eventually leaves the weights of the initial or lower layers nearly unchanged.
- As a result, the gradient descent never converges to the optimum. This is known as the *vanishing gradients* problem.

### Overfitting – Analogous to Decision Trees case

---

- With several weights to update over several iterations, the backpropagation algorithm tends to overfit.
- Overfitting occurs because the weights are being tuned to fit noise of the training examples that are not representative of the general distribution of examples.

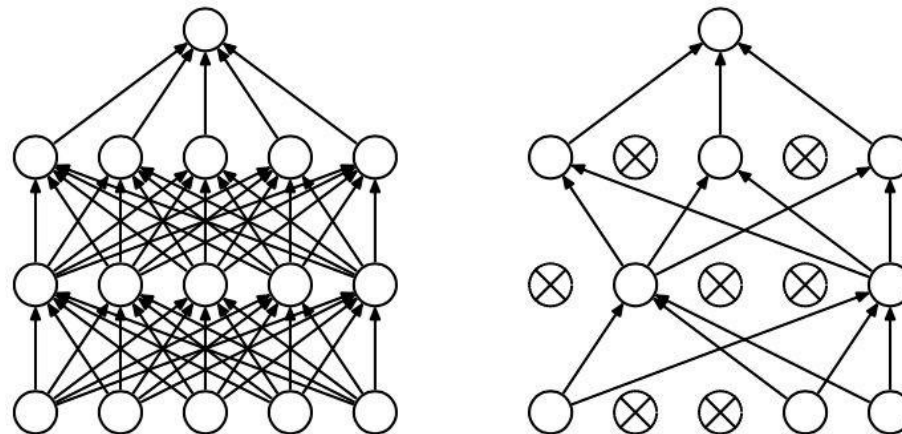
#### **Possible solutions:**

- Maintain a separate validation set, apart from training and test sets.
- Measure the accuracy over the validation set at intervals.
- Choose the set of weights that produce the least error on the validation set.

### Overfitting – Some methods to avoid overfitting in Neural Networks

#### Dropout:

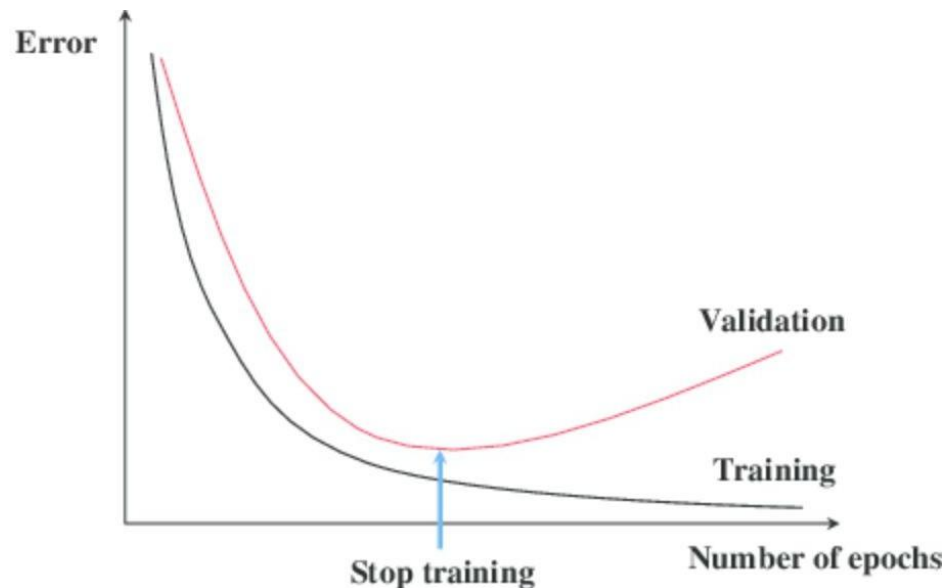
- It is a regularization technique that prevents the neural network from overfitting.
- It randomly drops neurons from the neural network during training which is equivalent to training different neural networks.
- The different networks will overfit differently, so the net effect of the dropout regularization technique will be to reduce overfitting so that our model will be good for predictive analysis.



### Overfitting – Some methods to avoid overfitting in Neural Networks

#### Early termination:

- This regularization technique updates the model to make it better fit the training data with each iteration.
- After a certain number of iterations, new iterations improve the model.
- After that point, however, the model begins to overfit the training data.
- Early stopping refers to stopping the training process before that point.



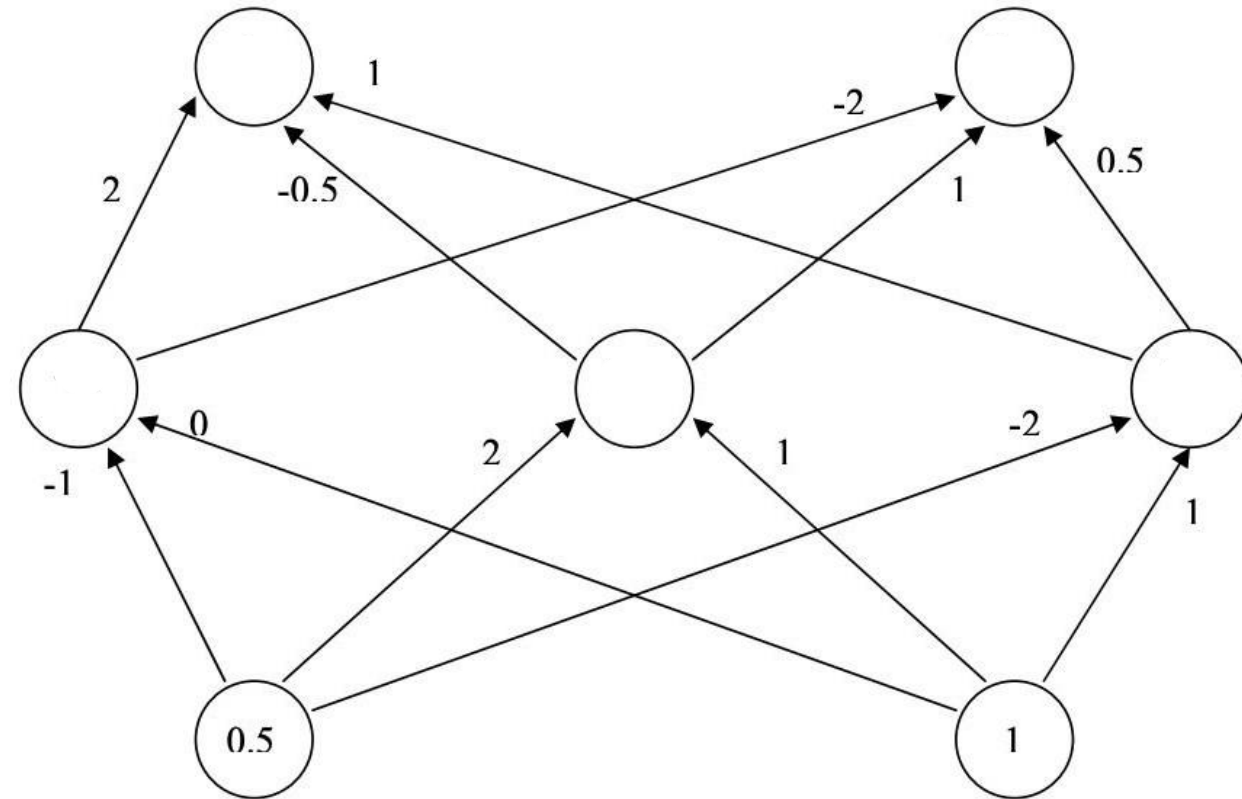
# Practice Questions

Question on Linear Neurons:

The following is a network of linear neurons, that is, neurons whose output is identical to their net input,  $o_i = \text{net}_i$ . The numbers in the circles indicate the output of a neuron, and the numbers at connections indicate the value of the corresponding weight.

a) Compute the output of the hidden-layer and the output-layer neurons for the given input (0.5, 1) and enter those values into the corresponding circles.

b) What is the output of the network for the input (1, 2), i.e. the left input neuron having the value 1 and the right one having the value 2? Do you have to do all the network computations once again in order to answer this question? Explain why you do or do not have to do this.



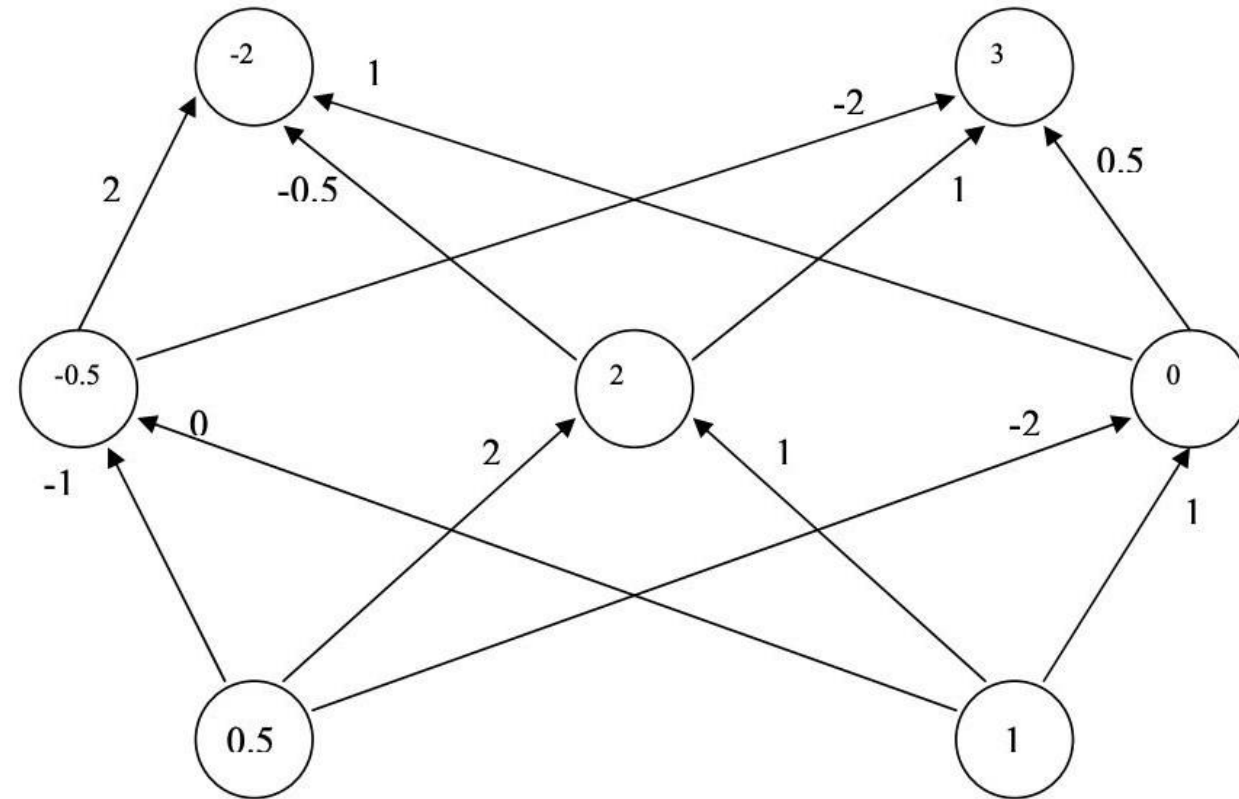


a) Compute the output of the hidden-layer and the output-layer neurons for the given input (0.5, 1) and enter those values into the corresponding circles.

Result: (-2, 3).

b) What is the output of the network for the input (1, 2), i.e. the left input neuron having the value 1 and the right one having the value 2? Do you have to do all the network computations once again in order to answer this question? Explain why you do or do not have to do this.

It's (-4, 6). We do not have to do all the computations, because every neuron computes a linear function on its inputs, which means that the entire network computes a linear function. For such a function, if we double the input, we simply double the output as well.



A perceptron with a unipolar step function has two inputs with weights  $w_1 = 0.5$  and  $w_2 = -0.2$ , and a threshold  $\theta = 0.3$  ( $\theta$  can therefore be considered as a weight for an extra input which is always set to -1). For a given training example  $\mathbf{x} = [0, 1]^T$ , the desired output is 0 (zero). Does the perceptron give the correct answer (that is, is the actual output the same as the desired output)?

A. Yes.

B. No.

Unipolar or Binary step function:- This function can be defined as,

$$f(x) = \begin{cases} 1, & \text{if } x \geq \theta \\ 0, & \text{if } x < \theta \end{cases}$$

where  $\theta$  represents the threshold value. This function is most widely used in single layer nets to convert the net input to an output that is binary (1 or 0).

## Unit 2 - ANN

### Practice Questions



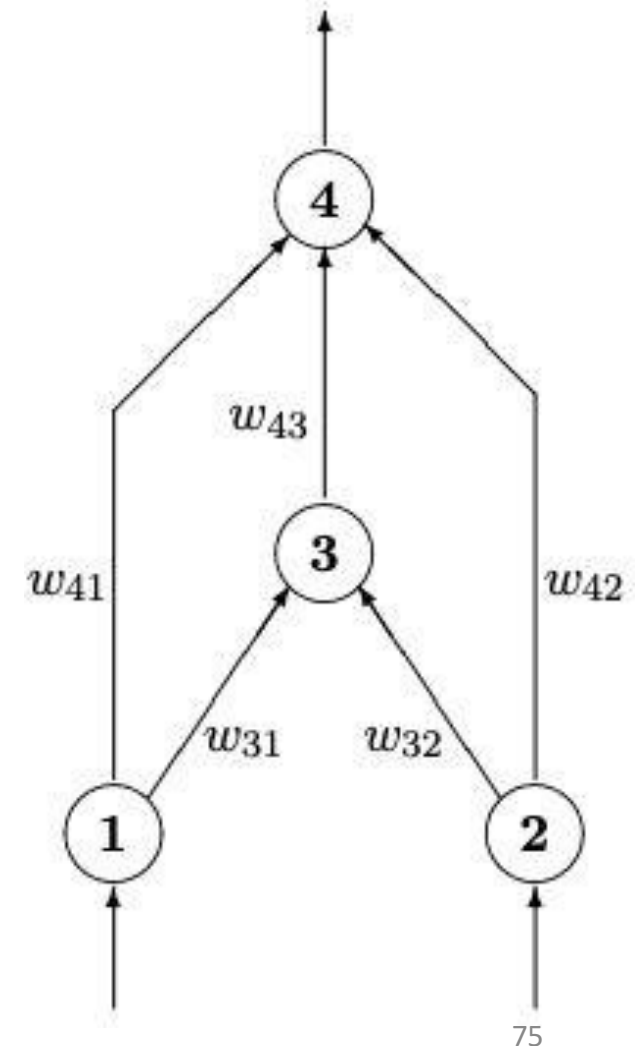
The following network is a multi-layer perceptron, where all of the units have binary inputs (0 or 1) and binary outputs (0 or 1).

The weights for this network are  $w_{31} = 1$ ,  $w_{32} = 1$ ,  $w_{41} = -1$ ,  $w_{42} = -1$  and  $w_{43} = 3$ .

The threshold of the hidden unit (3) is 1.5 and the threshold of the output unit (4) is  $-0.5$ .

The threshold of both input units (1 and 2) is 0.5, so the output of these units is exactly the same as the input. Which of the following Boolean functions can be computed by this network?

AND, OR, XOR or none?



Sigmoid Neuron, Gradient Descent : <http://www.cse.iitm.ac.in/~miteshk/CS7015/Slides/Handout/Lecture3.pdf>

Backpropagation : <https://www.youtube.com/watch?v=XN5IRqtFhOY>

Numerical : <https://www.youtube.com/watch?v=0e0z28wAWfg>



## UE24CS352A: Machine Learning

**Dr. Shylaja S S**

Department of Computer Science & Engineering

[shylaja.sharath@pes.edu](mailto:shylaja.sharath@pes.edu)



## **UE24CS352A: Machine Learning**

---

**Dr. Shylaja S S**

Professor

Department of Computer Science & Engineering

Acknowledgement : Dr. Rajanikanth K (Academic Advisor, PESU)  
Teaching Assistant (Arya Rajiv Chaloli)



# Activation Functions

- An activation function is a **non-linear transformation** that we apply on our input before propagating it to the next layer of neurons.
- Put simply, they decide whether a neuron should be activated or not.

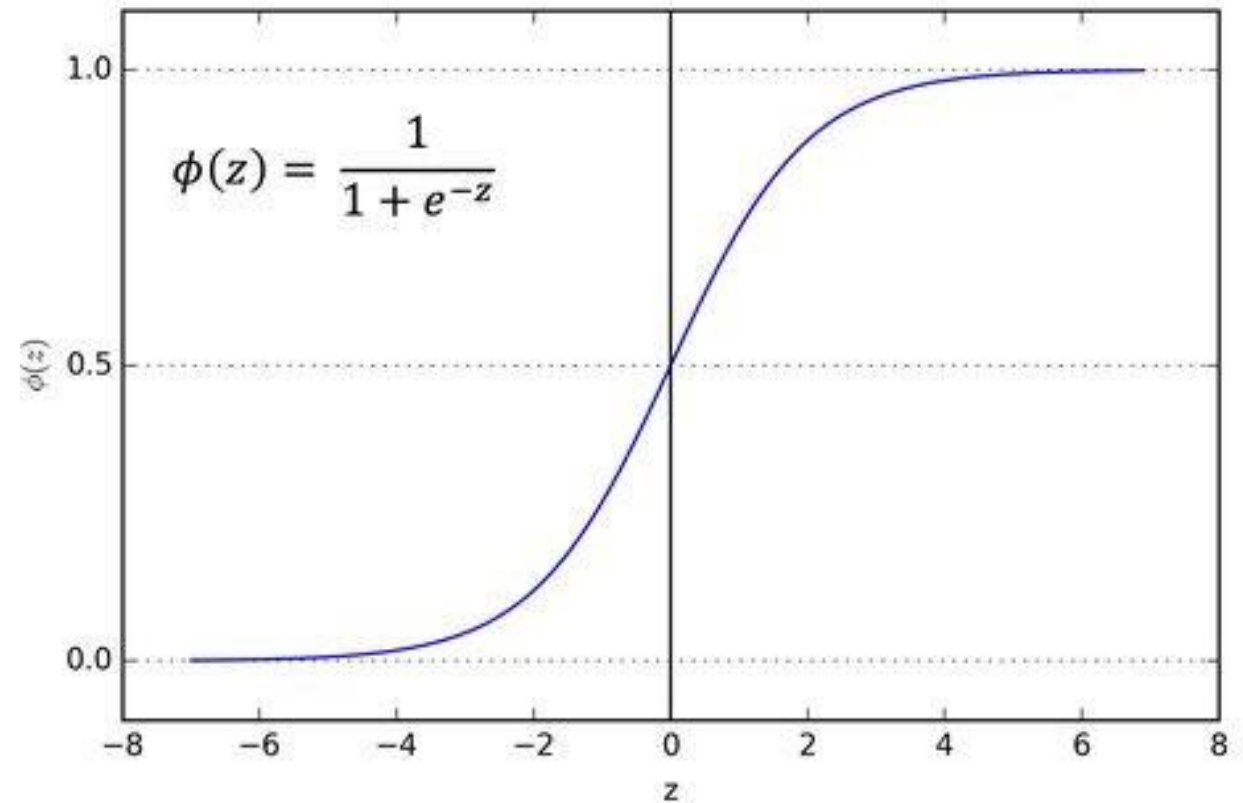
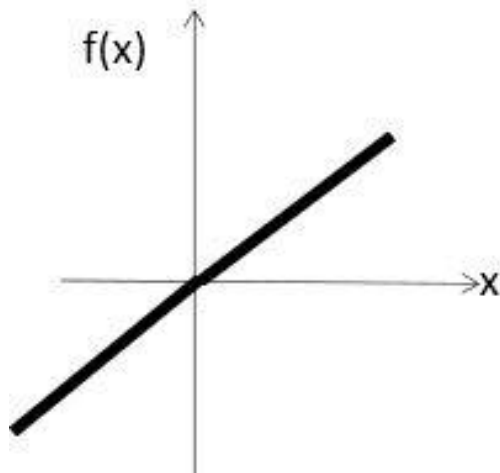
$$Y = \text{Activation}(\Sigma(\text{weight} * \text{input}) + \text{bias})$$

- Imagine a neural network without the activation functions. In that case, every neuron will only be performing a linear transformation on the inputs using the weights and biases.
- Although linear transformations make the neural network simpler, this network would be less powerful and will not be able to learn the complex patterns from data.
- A neural network without an activation function is essentially just a linear regression model.
- Thus we use a non linear transformation to the inputs of the neuron and this non-linearity in the network is introduced by an activation function.

There are a number of options available:

- Sigmoid
- tanh
- ReLU
- Softmax

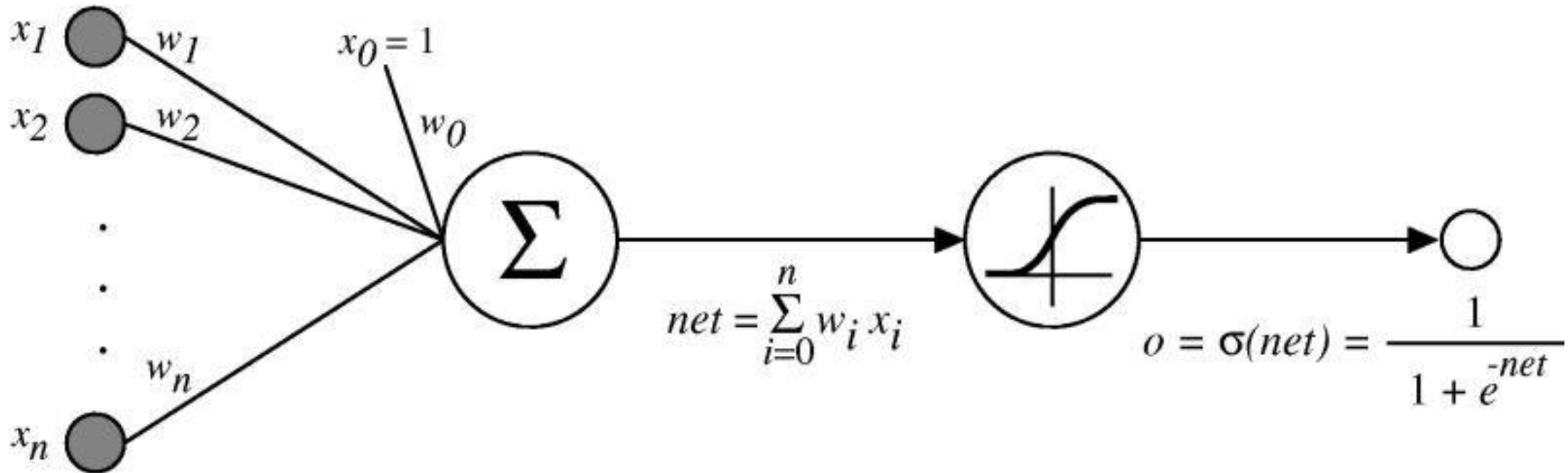
- Equation :  $f(x) = \frac{1}{1 + e^{-x}}$
- Range :  $[0, 1]$



## Example of an activation function -- Sigmoid

- While there are many activation functions, **sigmoid** is commonly used when # of hidden layers = 1.

$$f(x) = \frac{1}{1 + e^{-x}}$$



- The function is **differentiable**.
- One of the most commonly used activation functions.

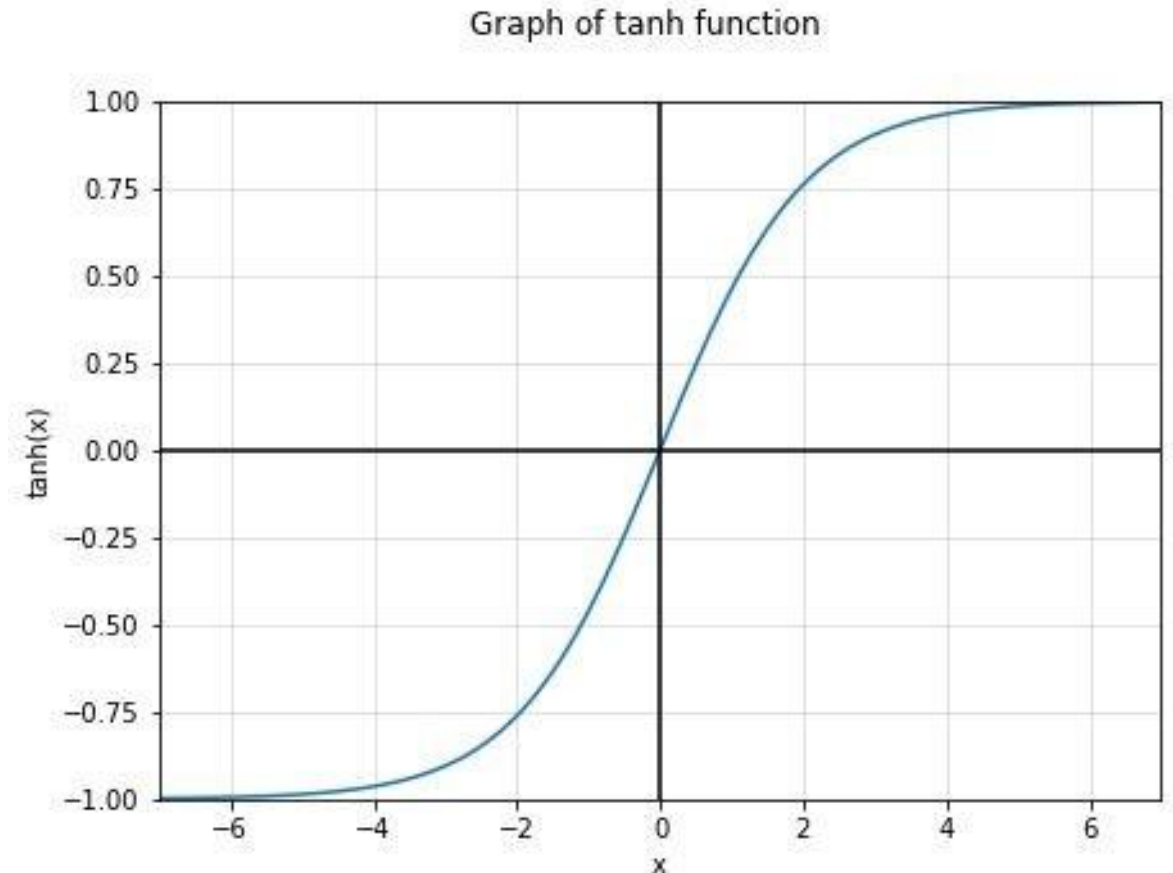
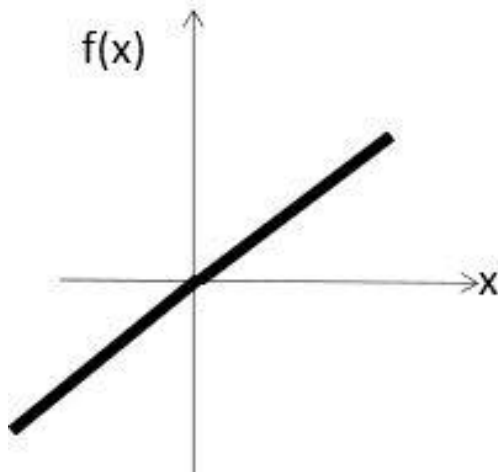
We know the range of sigmoid function is  $[0, 1]$ . What else do you know of that lies between 0 and 1?

### **Probability.**

Thus, sigmoid is used for models where we have to predict the probability as an output as we know the probability of anything exists only in  $[0, 1]$ .



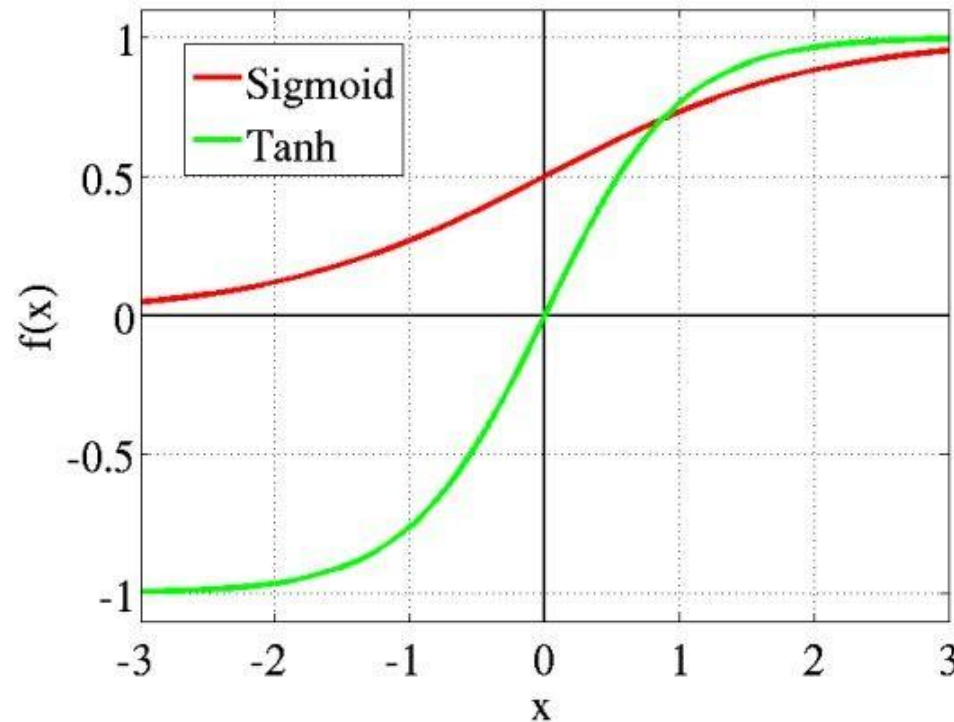
- Equation :  $f(x) = \frac{1 - e^{-2x}}{1 + e^{-2x}}$
- Range :  $[-1, 1]$



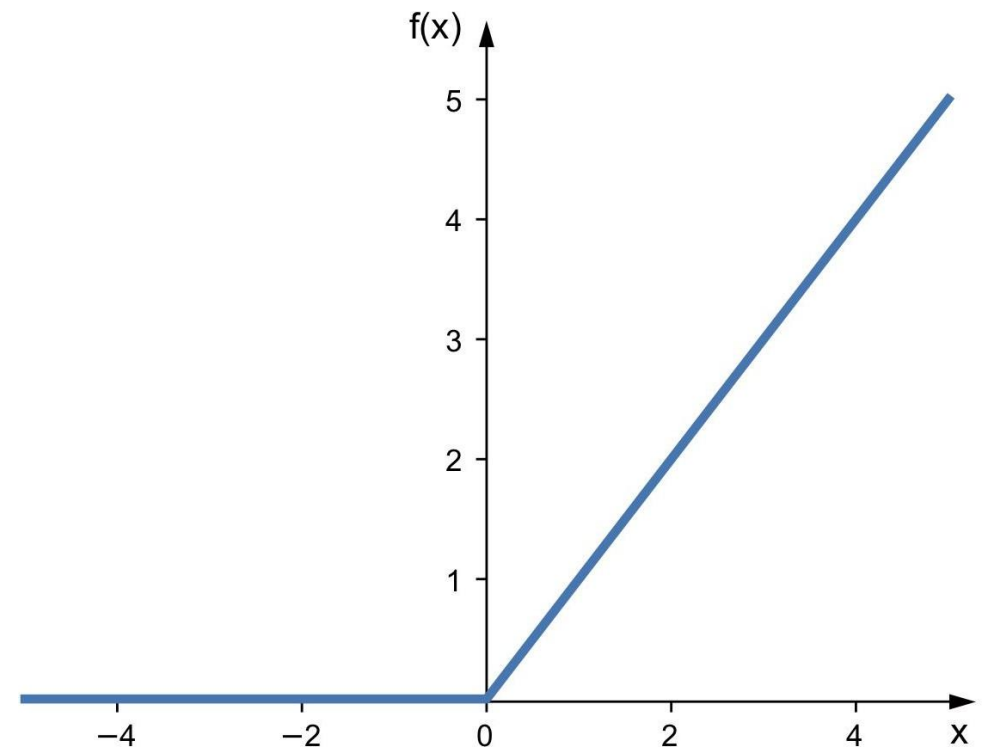
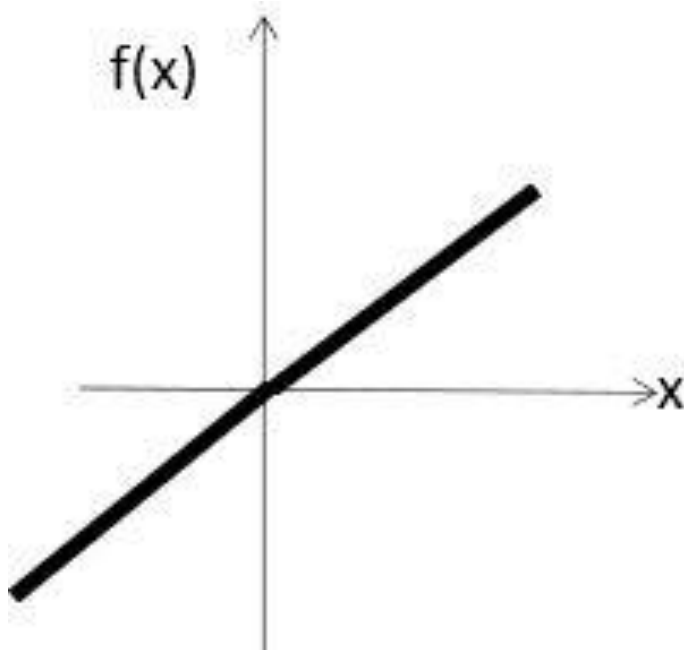
- The function is **differentiable**.
- The tanh function is mainly used for classification between two classes.

## Comparing tanh with sigmoid

- tanh is also like sigmoid but better.
- The advantage being that the negative inputs will be mapped strongly negative and the zero inputs will be mapped near zero in the tanh graph.



- **Equation :**  
 $f(x) = x$  when  $(x > 0)$   
 $= 0$  when  $(x \leq 0)$
- **Range :**  $[0, \text{infinity})$



- Very commonly used activation function in deep neural networks.

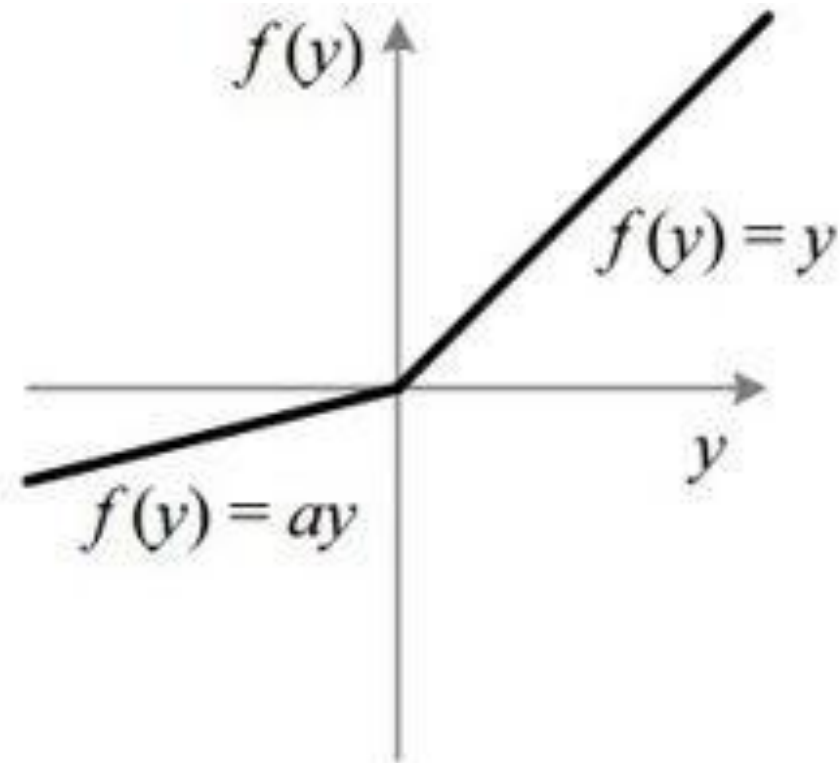
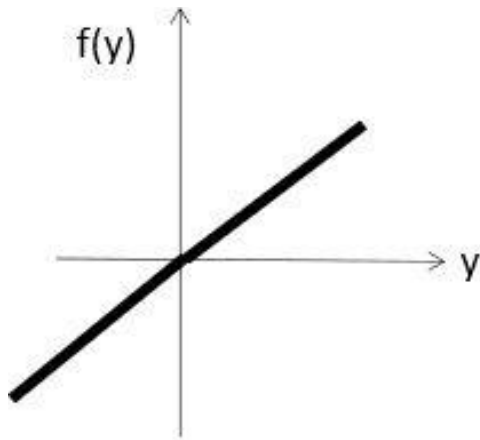
### What is dying ReLU?

One issue is that all the negative values become zero immediately which decreases the ability of the model to fit or train from the data properly.

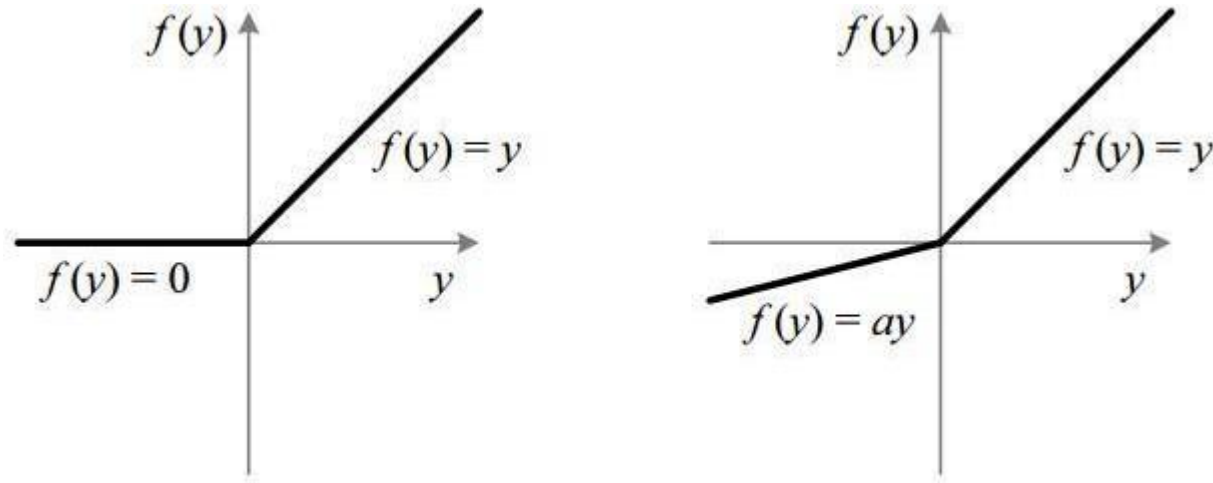
That means any negative input given to the ReLU activation function turns the value into zero immediately in the graph, which in turns affects the resulting graph by not mapping the negative values appropriately.

## How to deal with dying ReLU? Leaky ReLU

- Equation :  
 $f(y) = y$  when  $(y > 0)$   
 $= ay$  when  $(y \leq 0)$
- Range :  $[-\infty, \infty)$



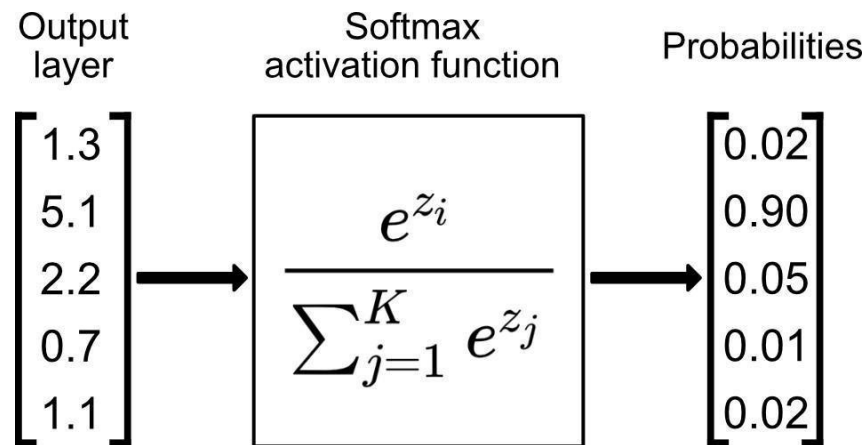
- The leak helps to increase the range of the ReLU function. Usually, the value of  $a = 0.01$ .
- Instead of setting some default value of  $a$ , we can set them as a parameter of a neural network and the network can be trained to learn the optimal value for it!



ReLU vs Leaky ReLU



- It is a generalization of the sigmoid function i.e. when the number of classes is two, it becomes the same as the sigmoid function.
- **Used in the final layer** of an ANN while performing multi-class classification tasks.
- The Softmax activation function calculates the relative probabilities of each class for being the output. Thus, the sum of the softmax values will always be equal to 1.



z represents the values from the neurons of the output layer.

## Which activation functions to use?

---

- It would be incredibly difficult to recommend an activation function that works for all use cases. Basically, there is no concept of “one size fits all.”
- Some points to be weary of:
  1. We are no strangers to the fact that calculus slows things down. Hence, we must consider how difficult it is to compute the derivative, if at all it is differentiable.
  2. How quickly a network with the chosen activation function converges.
  3. How smooth it is.
  4. Whether it satisfies the conditions of the Universal Approximation Theorem.
- It does not make sense to consider so many points before making a choice, hence we follow certain general guidelines.

- Sigmoid works better in case of binary classification problems. Continues to be popular with 1 hidden layer network.
- ReLU is widely used in deep neural networks as it yields better results.
- If we encounter a case of dead neurons in our network, leaky ReLU is the best choice.
- ReLU should only be used in the hidden layers. It simply sets everything that's negative to zero.



## UE24CS352A: Machine Learning

---

**Dr. Shylaja S S**

Department of Computer Science & Engineering

[shylaja.sharath@pes.edu](mailto:shylaja.sharath@pes.edu)



## **UE24CS352A: Machine Learning**

---

**Dr. Shylaja S S**

Professor

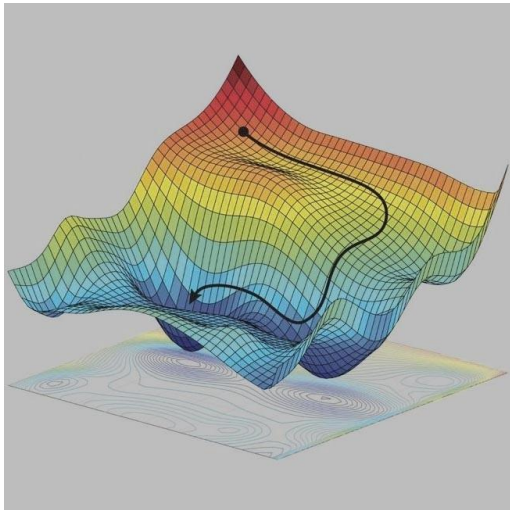
Department of Computer Science & Engineering

Acknowledgement :

Teaching Assistant (Anashua Kritika Dastidar)

*Optimization is a process of finding optimal parameters for the model, which significantly reduces the error function.*

Optimization algorithms used for training of deep models differ from traditional optimization algorithms in several ways. In most machine learning scenarios we care about some performance measure say ,  $P$  but we optimise  $P$  only indirectly by reducing a different cost function  $J(\theta)$  in the hope that doing so will improve  $P$ .



The following are the optimization algorithms discussed in this course :

- Mini Batch gradient descent
- Stochastic gradient descent
- Gradient descent with momentum
- RMS Prop
- Adam

# A Recap of Gradient Descent

Gradient Descent searched the hypothesis space of all possible weight vectors to find the best fit for all training examples. Say we have a cosy function  $J(w)$  and we wish to minimise this cost function:

1. We randomly initialize the weight vectors
2. We calculate the loss and update the weight vectors in the direction of steepest descent.

The equation to do so is :

$$W_{\text{new}} = W - \alpha * \frac{\delta J(W)}{\delta W}$$

1. We repeat till we converge at a minimum

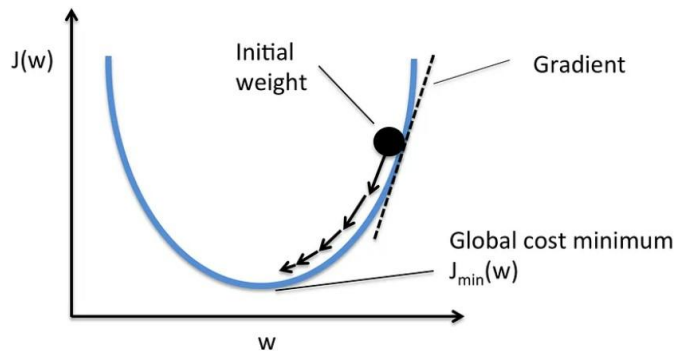
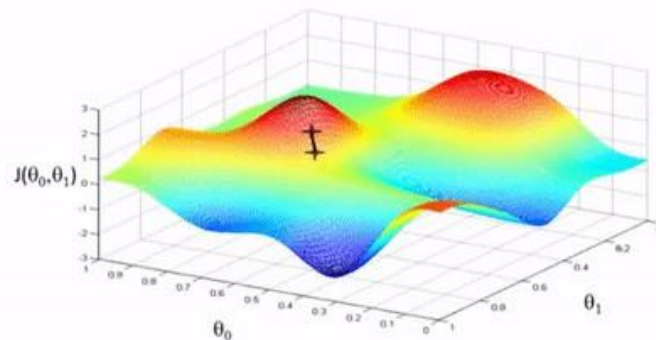


Figure 2: Example of minimizing  $J(w)$ ; (Source: [MLextend](#))





# Batch Gradient Descent

---

Batch gradient descent is a variation of the gradient descent algorithm that calculates the error for each example in the training dataset, but only updates the model after all training examples have been evaluated

## Upsides

- Fewer updates to the model means this variant of gradient descent is more computationally efficient than stochastic gradient descent.
- The decreased update frequency results in a more stable error gradient and may result in a more stable convergence on some problems.
- The separation of the calculation of prediction errors and the model update lends the algorithm to parallel processing based implementations.

## Downsides

- The more stable error gradient may result in premature convergence of the model to a less optimal set of parameters.
- The updates at the end of the training epoch require the additional complexity of accumulating prediction errors across all training examples.
- Commonly, batch gradient descent is implemented in such a way that it requires the entire training dataset in memory and available to the algorithm.
- Model updates, and in turn training speed, may become very slow for large datasets.

# Stochastic Gradient Descent

---

Stochastic gradient descent, often abbreviated SGD, is a variation of the gradient descent algorithm that calculates the error and updates the model for each example in the training dataset.

## Upsides

- The frequent updates immediately give an insight into the performance of the model and the rate of improvement.
- This variant of gradient descent may be the simplest to understand and implement, especially for beginners.
- The increased model update frequency can result in faster learning on some problems.
- The noisy update process can allow the model to avoid local minima (e.g. premature convergence).

## Downsides

- Updating the model so frequently is more computationally expensive than other configurations of gradient descent, taking significantly longer to train models on large datasets.
- The frequent updates can result in a noisy gradient signal, which may cause the model parameters and in turn the model error to jump around (have a higher variance over training epochs).
- The noisy learning process down the error gradient can also make it hard for the algorithm to settle on an error minimum for the model.

# Mini Batch Gradient Descent

---

Mini-batch gradient descent is a variation of the gradient descent algorithm that splits the training dataset into small batches that are used to calculate model error and update model coefficients. Mini-batch gradient descent seeks to find a balance between the robustness of stochastic gradient descent and the efficiency of batch gradient descent. It is the most common implementation of gradient descent used in the field of deep learning.

## Upsides

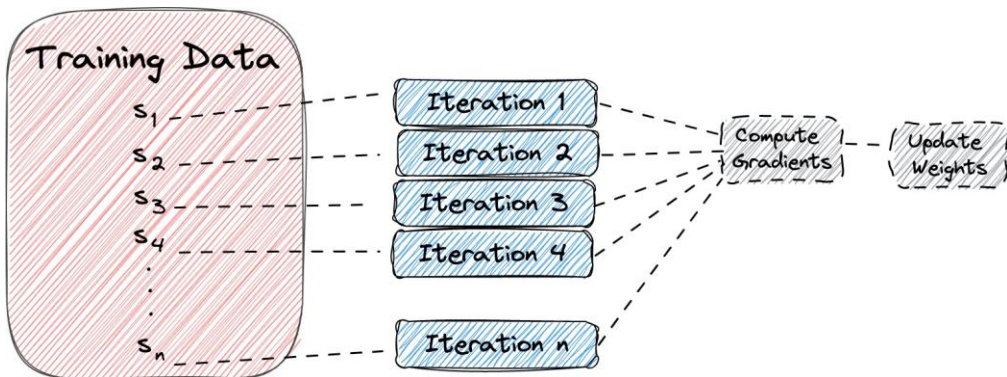
- The model update frequency is higher than batch gradient descent which allows for a more robust convergence, avoiding local minima.
- The batched updates provide a computationally more efficient process than stochastic gradient descent.
- The batching allows both the efficiency of not having all training data in memory and algorithm implementations.

## Downsides

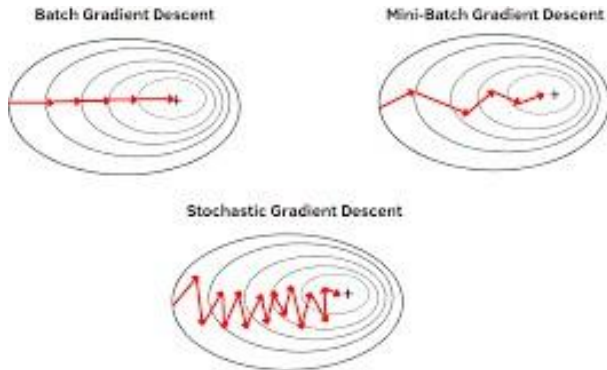
- Mini-batch requires the configuration of an additional “mini-batch size” hyperparameter for the learning algorithm.
- Error information must be accumulated across mini-batches of training examples like batch gradient descent

# How to choose the best value for Mini Batch size

- If there is a small training set batch gradient descent is preferred.
- Mini batch size is a hyperparameter and it is a good idea to experiment with a range of values to get the best fit for your model.
- Mini-batch sizes, commonly called “batch sizes” for brevity, are often tuned to an aspect of the computational architecture on which the implementation is being executed. Such as a power of two that fits the memory requirements of the GPU or CPU hardware like 32, 64, 128, 256, and so on.



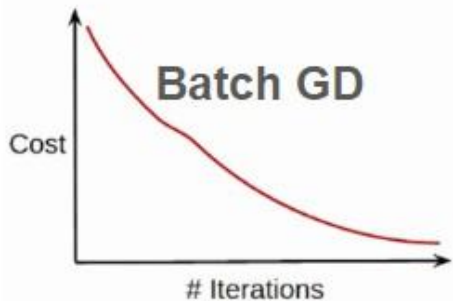
# Convergence in different versions of Gradient Descent



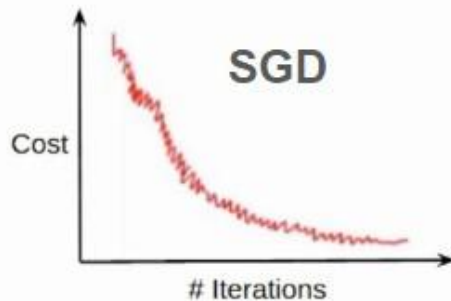
- Batch gradient descent takes small steps towards the minima and will converge to a minima.
- While Stochastic gradient descent is noisy and oscillates near the minima . It never actually converges to the minima

## Comparison: Cost function

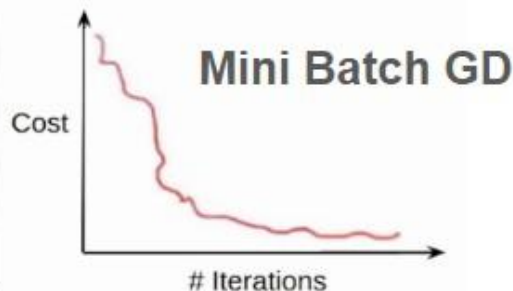
- Cost function reduces smoothly



- Lot of variations in cost function



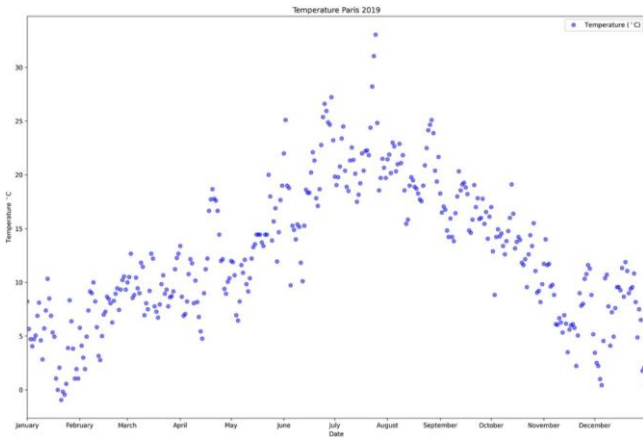
- Smoother cost function as compared to SGD



# Exponentially Weighted Averages

The Exponentially Weighted Moving Average (EWMA) is commonly used as a smoothing technique in time series. However, due to several computational advantages (fast, low-memory cost), the EWMA is behind the scenes of many optimization algorithms in deep learning, including Gradient Descent with Momentum, RMSprop, Adam, etc.

Let's understand with an example, say we have some data for temperatures across multiple days in a city and we wish to approximate the next day's temperature from this data. Thus let the estimated temperature be  $V_t$ , let the previous estimate be  $V_{t-1}$ ,  $O_t$  be the temperature for day  $t$  and  $\beta$  be a hyperparameter.



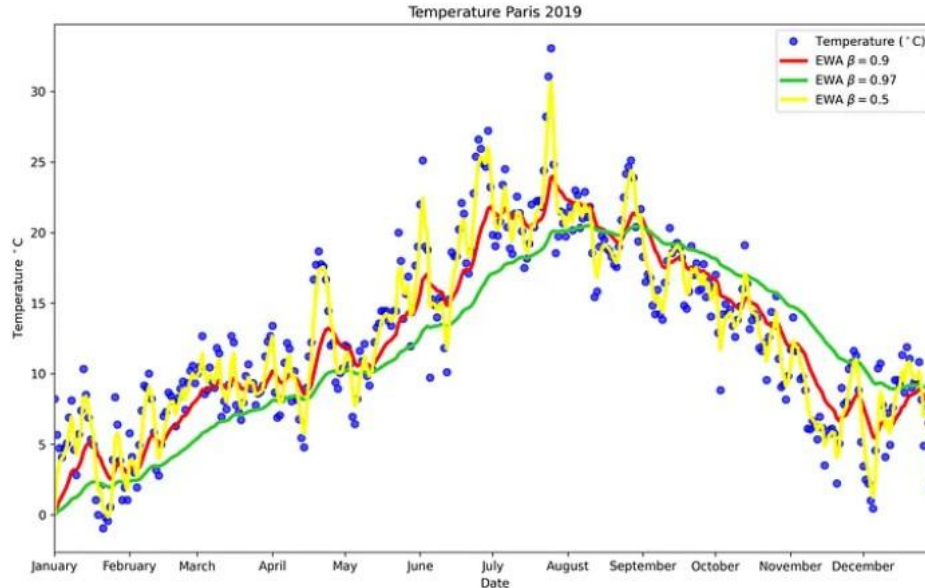
$$V_t = \beta * V_{t-1} + (1 - \beta) * O_t$$

Here,  $\beta$  determines how important the previous value is (the trend), and  $(1 - \beta)$  how important the current value is.

# Exponentially Weighted Averages

Here ,  $V_t$  is calculated by approximating over  $1/(1-\beta)$  days of temperature . Thus if  $\beta$  is chosen to be 0.9 then we approximate over the last 10 days if 0.5 then the last 2 days and so on.

| $\beta$ | Days |
|---------|------|
| 0.9     | 10   |
| 0.98    | 50   |
| 0.5     | 2    |



**Note :** As the value of  $\beta$  increases the curve becomes smoother and has less noise.

However with large values of  $\beta$  as we are averaging over a larger window the formula adapts more slowly to changes in data as high weightage is given to older values



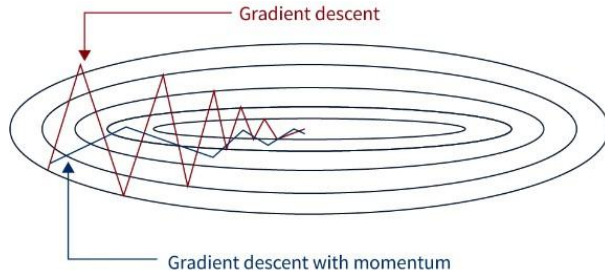
# Gradient Descent with Momentum

*In this method we compute the exponentially weighted average of the gradients and use this new value to calculate the weights instead*

A problem with the gradient descent algorithm is that the progression of the search can bounce around the search space based on the gradient. For example, the search may progress downhill towards the minima, but during this progression, it may move in another direction, even uphill, depending on the gradient of specific points (sets of parameters) encountered during the search.

This can slow down the progress of the search, especially for those optimization problems where the broader trend or shape of the search space is more useful than specific gradients along the way.

One approach to this problem is to add history to the parameter update equation based on the gradient encountered in the previous updates



# Gradient Descent with Momentum

1. First we calculate  $dW$  (where  $w$  is the weight) for the current mini batch

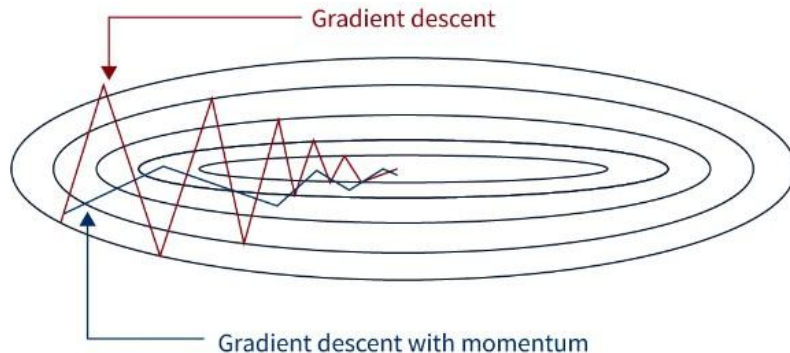
1. Then we compute

$$V_{dW} = \beta * V_{dW} + (1-\beta) * dW$$

1. Then we update our weights as

$$W_{\text{new}} = W - \alpha * V_{dW}$$

Using a very high value for  $\beta$  we can dampen out the oscillations which arise in gradient descent !



# Gradient Descent with Momentum

---

## Upsides:

1. Momentum has the effect of damping down the change in the gradient and, in turn, the step size with each new point in the search space.
1. Momentum is most useful in optimization problems where the objective function has a large amount of curvature (e.g. changes a lot), meaning that the gradient may change a lot over relatively small regions of the search space.
1. It is also helpful when the gradient is estimated, such as from a simulation, and may be noisy, e.g. when the gradient has a high variance.
2. Finally, momentum is helpful when the search space is flat or nearly flat, e.g. zero gradient. The momentum allows the search to progress in the same direction as before the flat spot and helpfully cross the flat region.

## Downsides :

1. It can overshoot the global minimum and converge to a local minimum instead.
2. Another disadvantage is that the momentum term can cause the optimization process to oscillate around the global minimum.

# Root Mean Square or RMS prop

We know that while implementing gradient descent we end up with lots of oscillations  
 Let the horizontal direction be  $w$  and the vertical direction be  $b$ . In this case we wish to speed up learning in the  $w$  direction and slow down learning in the  $b$  direction.  
 for iteration  $t$ :

1. Calculate the derivative  $dW$  for the current mini batch ‘
2. Calculate an exponentially weighted average of the squares of derivatives

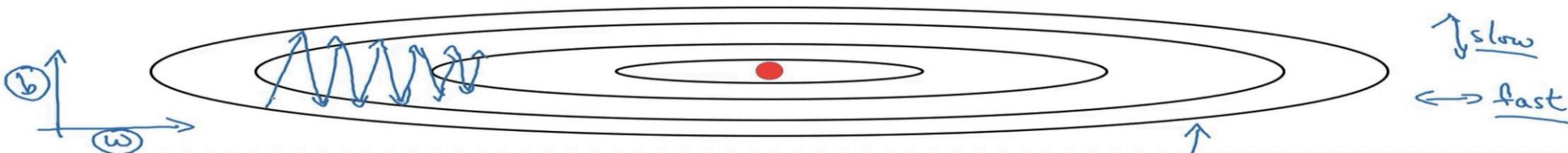
$$S_{dW} = \beta_2 * S_{dW} + (1 - \beta_2) * (dW)^2$$

$$S_{db} = \beta_2 * S_{db} + (1 - \beta_2) * (db)^2$$

NOTE :  $\beta_2$  and  $\beta$  are separate hyperparameters

1. Then we update the weights as:  $W_{\text{new}} = W - \alpha * \frac{dW}{\sqrt{S_{dW}}}$  and  $b_{\text{new}} = b - \alpha * \frac{db}{\sqrt{S_{db}}}$

## RMSprop



# RMS prop intuition

- In this example we wish to dampen out oscillations in the  $b$  direction and to learn faster in the  $x$  direction.
- Thus as  $S_{dW}$  is a small number while  $S_{db}$  is a large number.

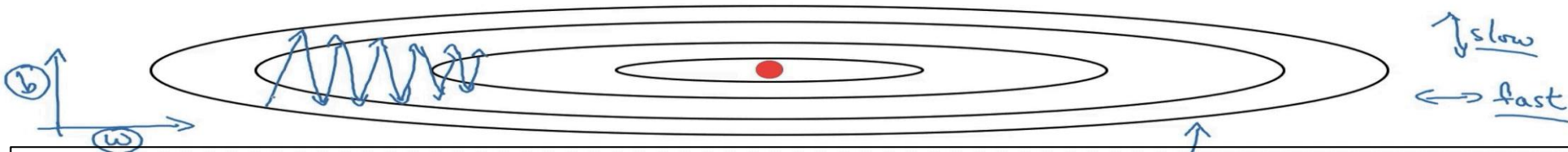
$$S_{dW} = \beta_2 * S_{dW} + (1 - \beta_2) * (dW)^2 \leftarrow \text{SMALL}$$

$$S_{db} = \beta_2 * S_{db} + (1 - \beta_2) * (db)^2 \leftarrow \text{LARGE}$$

$$W_{\text{new}} = W - \alpha * \frac{dW}{\sqrt{S_{dW}}} \text{ and } b_{\text{new}} = b - \alpha * \frac{db}{\sqrt{S_{db}}}$$

- Thus by dividing by a small  $S_{dW}$  we can increase the magnitude of update in the  $W$  direction and while dividing with a large  $S_{db}$  we can dampen the oscillations in the  $b$  direction.

## RMSprop



Note : if  $S_{dW}$  is very close to zero in order to avoid the  $W$  term becoming very large we add a small term  $\epsilon$  to the denominator

$$W_{\text{new}} = W - \alpha * \frac{dW}{\sqrt{S_{dW} + \epsilon}}$$

# RMS Prop

---

## Upsides:

1. Faster convergence: RMSprop can converge faster than SGD by adapting the learning rate based on the magnitude of the gradients for each parameter.
2. Robustness to different learning rates: RMSprop is more robust to different learning rates for different parameters, which can be helpful in deep learning models with many parameters.
3. Adaptive learning rate: RMSprop adaptively scales the learning rate based on the history of the squared gradients, which can improve the convergence speed and stability of the optimization process

## Downsides:

1. RMSProp can sometimes lead to slow convergence.
2. It is sensitive to the learning rate.

# Adam optimizer

Adam or Adaptive moment estimation optimizer is a combination of gradient descent with momentum and the RMSprop optimizer

1. First we initialize  $V_{dW} = 0, V_{db} = 0, S_{dW} = 0, S_{db} = 0$
2. for iteration  $t$  :
  - a. Compute  $dW$  and  $db$  for the current mini batch
  - b. Then compute :
    - i. A momentum like update

$$V_{dW} = \beta_1 * V_{dW} + (1 - \beta_1) * dW$$

$$V_{db} = \beta_1 * V_{db} + (1 - \beta_1) * db$$

- i. A RMSprop like update

$$S_{dW} = \beta_2 * S_{dW} + (1 - \beta_2) * (dW)^2$$

$$S_{db} = \beta_2 * S_{db} + (1 - \beta_2) * (db)^2$$

**Note** : Adam optimization applies bias correction to the first and second moment estimates to ensure that they are unbiased estimates of the true values.

So, the bias corrected terms:

$$V_{dW}^C = \frac{V_{dW}}{1 - \beta_1^t} \quad V_{db}^C = \frac{V_{db}}{1 - \beta_1^t}$$

$$S_{dW}^C = \frac{S_{dW}}{1 - \beta_2^t} \quad S_{db}^C = \frac{S_{db}}{1 - \beta_2^t}$$



# Adam optimizer

Thus the weights will be updated as :

$$W_{\text{new}} = W - \alpha * \frac{V_{dW}^C}{\sqrt{S_{dW}^C} + \epsilon} \quad \text{and} \quad b_{\text{new}} = b - \alpha * \frac{V_{db}^C}{\sqrt{S_{db}^C} + \epsilon}$$

## Adam Configuration Parameters

- **alpha**. Also referred to as the learning rate or step size. The proportion that weights are updated (e.g. 0.001). Larger values (e.g. 0.3) results in faster initial learning before the rate is updated. Smaller values (e.g. 1.0E-5) slow learning right down during training
- **beta1**. The exponential decay rate for the first moment estimates (e.g. 0.9).
- **beta2**. The exponential decay rate for the second-moment estimates (e.g. 0.999). This value should be set close to 1.0 on problems with a sparse gradient (e.g. NLP and computer vision problems).
- **epsilon**. Is a very small number to prevent any division by zero in the implementation (e.g. 10E-8).

Further, learning rate decay can also be used with Adam.

# Adam optimizer

---

## Advantages :

1. **Learning Rates:** Unlike fixed learning rate methods like SGD, Adam optimization provides adaptive learning rates for each parameter based on the history of gradients. This allows the optimizer to converge faster and more accurately, especially in high-dimensional parameter spaces.
1. **Momentum:** Adam optimization uses momentum to smooth out fluctuations in the optimization process, which can help the optimizer avoid local minima and saddle points.
1. **Bias Correction:** Adam optimization applies bias correction to the first and second moment estimates to ensure that they are unbiased estimates of the true values.
1. **Robustness:** Adam optimization is relatively robust to hyperparameter choices and works well across a wide range of deep learning architectures.

# Adam optimizer

---

## Disadvantages :

**Memory intensive:** Adam needs to store moving averages of past gradients for each parameter during training and hence it requires more memory than some other optimization algorithms, particularly when dealing with very large neural networks or extensive datasets.

**Slower convergence in some cases:** While Adam usually converges quickly, it might converge to flawed solutions in some cases or tasks. In such scenarios, other optimization algorithms like *SGD (stochastic gradient descent)* with momentum or *Nesterov accelerated gradient (NAG)* may perform better.

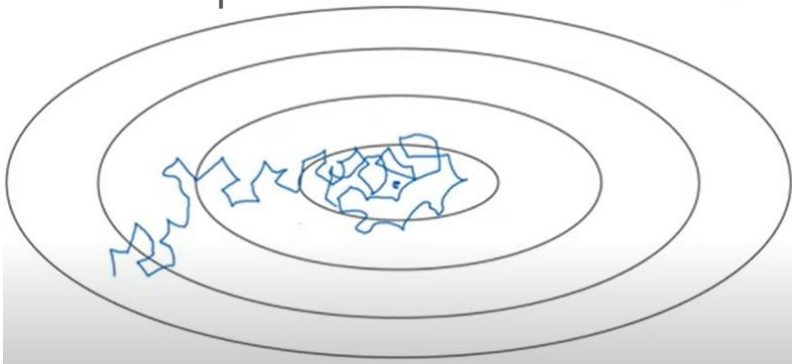
**Hyperparameter sensitivity:** Although Adam is less sensitive to hyperparameter choices than some other algorithms, it still has hyperparameters like the learning rate,  $\beta_1$ , and  $\beta_2$ . Choosing inappropriate values for these hyperparameters could impact the performance of the algorithm.

# Learning Rate Decay

Learning rate decay is a technique used in machine learning models to train modern neural networks. It involves starting with a large learning rate and then gradually reducing it until local minima is obtained.

→ Say, we are implementing mini batch gradient descent with a relatively small batch size and the gradient takes noisy steps as shown below. It also does not converge at the minima but oscillates around the minimum value.

→ In such a situation slowly reducing the learning rate ( $\alpha$ ) value as we approach the minima could be advantageous as the final value would oscillate closer to the minima if small steps are taken .



This could be done by setting :

$$\alpha = \frac{1}{1 + \text{decay rate} * \text{epoch number}} * \alpha_0$$

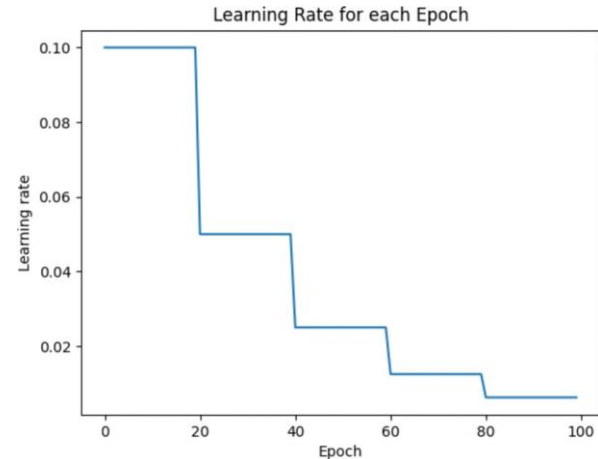
Here decay rate is yet another hyperparameter while is  $\alpha_0$  the initial learning rate value.

# Other learning rate decay methods

1. Exponential decay:  $\alpha = \text{decay rate}^{\text{epoch number}} * \alpha_0$

2. Epoch number Based decay:  $\alpha = \frac{\text{constant}}{\sqrt{\text{epoch number}}} * \alpha_0$

3. Discrete Step decay : In this method learning rate is decreased in some discrete steps after every certain interval of time , for example you are reducing learning rate to its half after every 10 secs..



4. Manual decay : In this method practitioners manually examine the performance of algorithm and decrease the learning rate manually day by day or hour by hour etc.

# Resources

---

- <https://www.scaler.com/topics/momentum-based-gradient-descent/>
- <https://medium.com/mlearning-ai/exponentially-weighted-average-5eed00181a09>
- <https://medium.com/nerd-for-tech/optimizers-in-machine-learning-f1a9c549f8b4>
- <https://www.analyticsvidhya.com/blog/2021/03/variants-of-gradient-descent-algorithm/>
- <https://machinelearningmastery.com/gradient-descent-with-momentum-from-scratch/>
- <https://machinelearningmastery.com/gentle-introduction-mini-batch-gradient-descent-configure-batch-size/>
- [https://youtube.com/playlist?list=PLkDaE6sCZn6Hn0vK8co82zjQtt3T2Nkqc&si=7Z\\_y1LQXzn7fZF1I](https://youtube.com/playlist?list=PLkDaE6sCZn6Hn0vK8co82zjQtt3T2Nkqc&si=7Z_y1LQXzn7fZF1I)
- <https://arxiv.org/pdf/1412.6980.pdf>



## UE24CS352A: Machine Learning

---

**Dr. Shylaja S S**

Department of Computer Science & Engineering

[shylaja.sharath@pes.edu](mailto:shylaja.sharath@pes.edu)





## **UE24CS352A: Machine Learning**

---

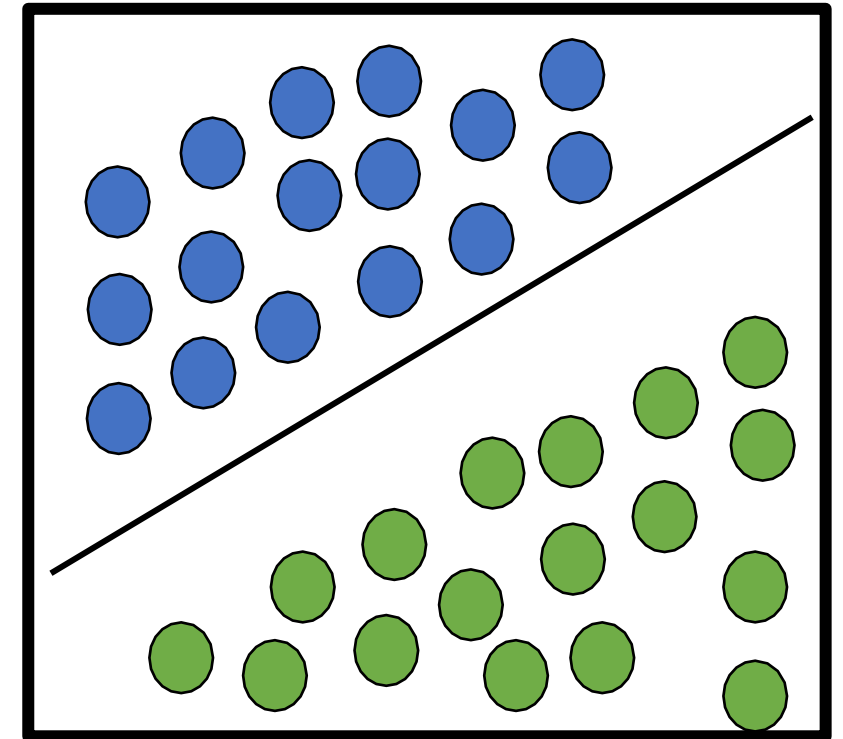
**Dr. Shylaja S S**

Professor

Department of Computer Science & Engineering

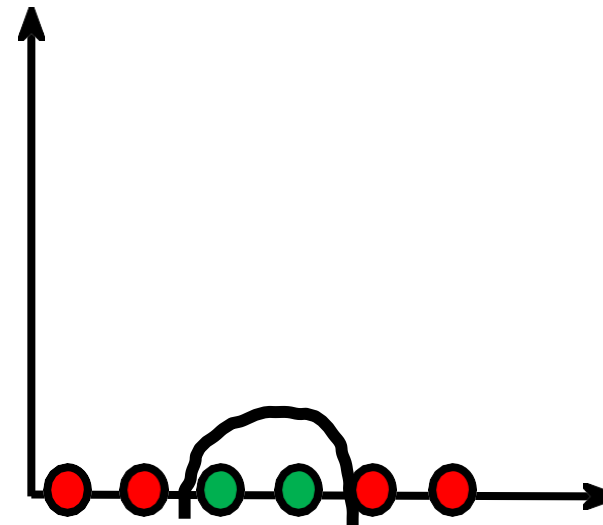
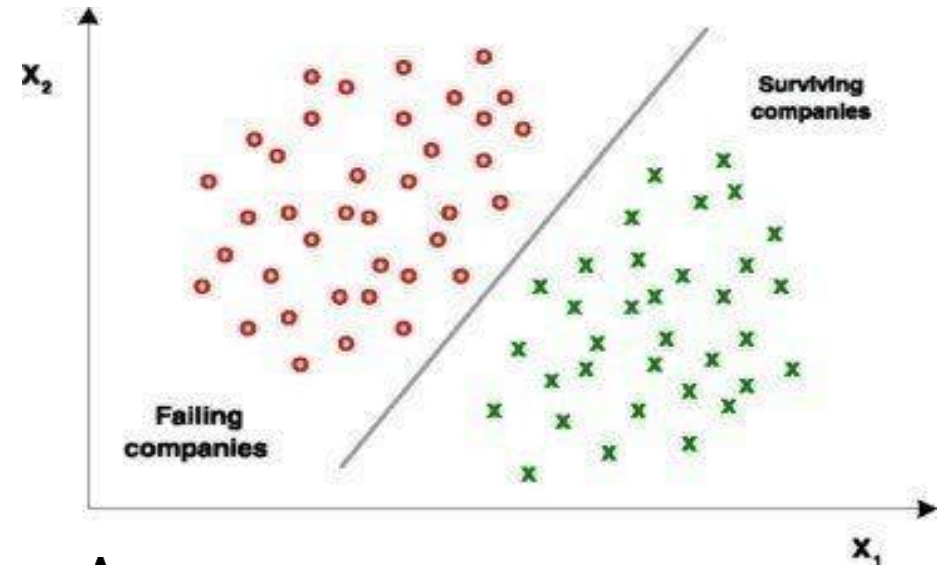
Acknowledgement : Dr. Rajanikanth K (Academic Advisor, PESU)  
Teaching Assistant (Arya Rajiv Chaloli)

- By default SVM works as a linear classifier.
  - A data set is said to be linearly separable if we can have a hyperplane that can separate the data into required number of classes.
    - In 2 dimensions - The line that separates them is the decision boundary.
    - In a 3 dimensions that would be a plane.
    - In general in  $d$  dimensions, the separator is a  $(d-1)$  dimensional hyperplane - although it is pretty much impossible to visualize for 4 or more dimensions.
- ( $d$  dimensions means there  $d$  features that describe an input instance,  $\Rightarrow$  input vector  $X \in \mathbb{R}^d$ )

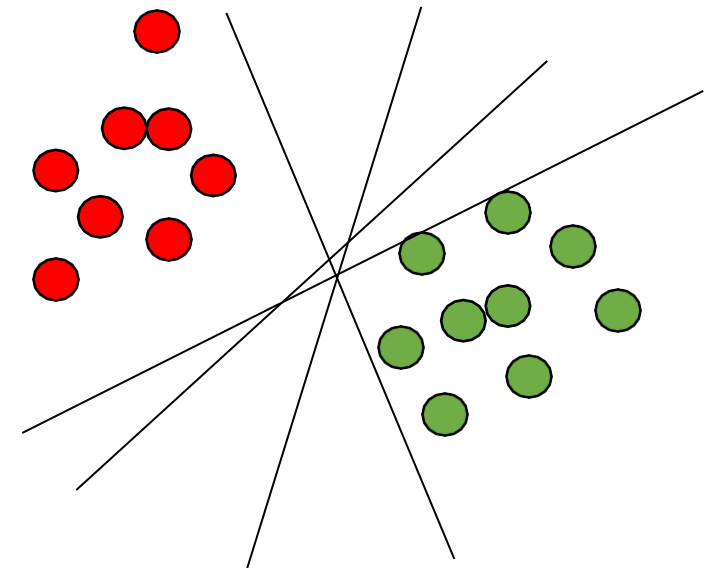


1. Applications of SVM :
  - Classification (Our Focus)
  - Regression
2. SVM can work on both kinds of datasets :
  - Linearly separable
  - Not linearly separable (Kernel Trick)
3. SVM can be used for both :
  - Binary classification (Our Focus)
  - Multi-class classification

The concept of Hard Margin SVM is presented in the slides.

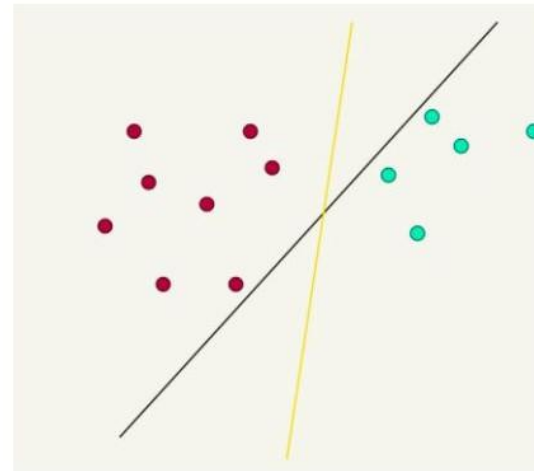


- When we think all ML algos are found and there can be no more
- Vapnik in 90's came up with idea of SVM for handwritten recognition for Bell labs ,about which he started thinking in early 60's
- The idea of Vapnik goes something like this.
- How do we separate these points .....is by drawing a straight line
- BUT WHICH LINE???



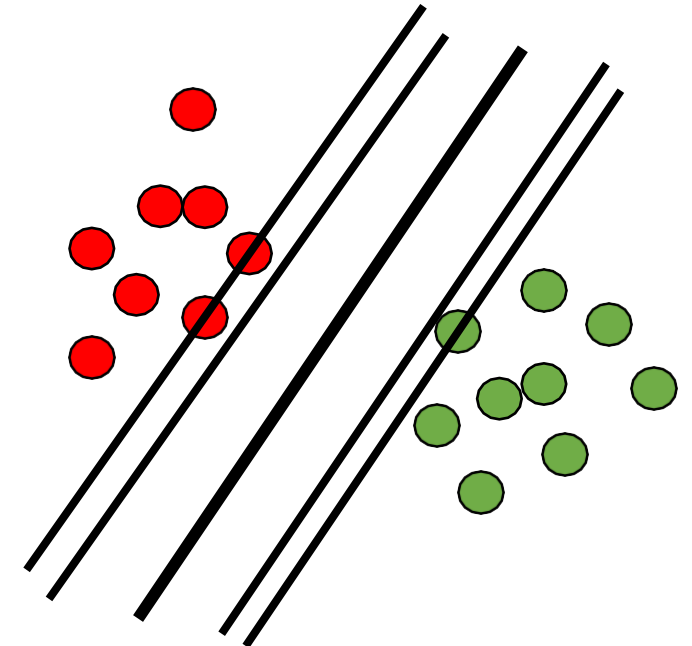
### How to pick an Optimal Hyperplane

- But: Which points should influence optimality?
  - All points?
    - Linear regression
    - Neural nets
  - Or only “difficult points” close to decision boundary
    - Support vector machines

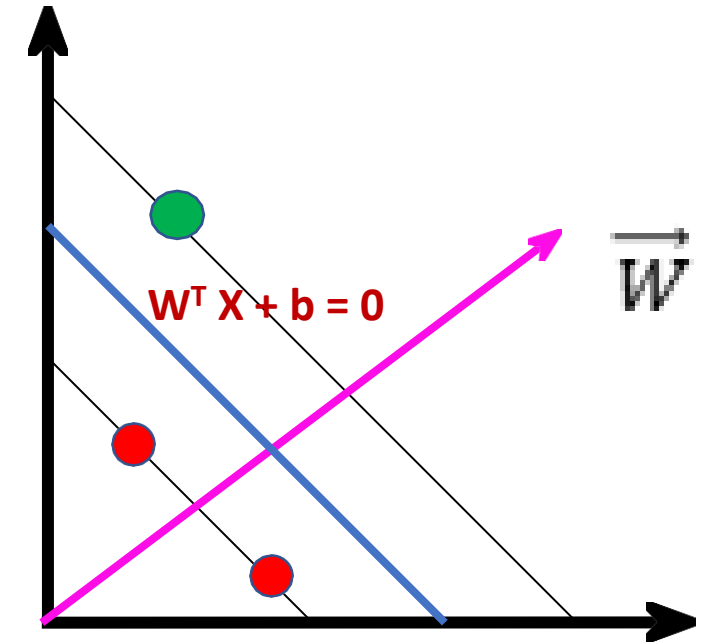


The problem of finding the optimal hyperplane is an optimization problem and can be solved by optimization techniques (we use **Lagrange multipliers** to get this problem into a form that can be solved analytically).

- When we think all ML algos are found and there can be no more
- Vapnik in 90's came up with idea of SVM for handwriting recognition for Bell labs ,about which he started thinking in early 60's
- The idea of Vapnik goes something like this.
- How do we separate these points .....is by drawing a straight line
- BUT WHICH LINE???
- And this line is drawn with a view toward putting in the widest street that separates positive examples from negative
- So this is the way SVM puts a decision boundary in such a way that the street is wide as possible



- The three hyper planes are parallel to each other
  - $W$  can be same.
- The general equation for a hyperplane is  $W^T X + b = 0$  where  $W$  is a vector that is normal to the hyperplane)





### Equation of Median Line --- Recall from Perceptron

Equation of the median line:

$$W^T X + b = 0$$

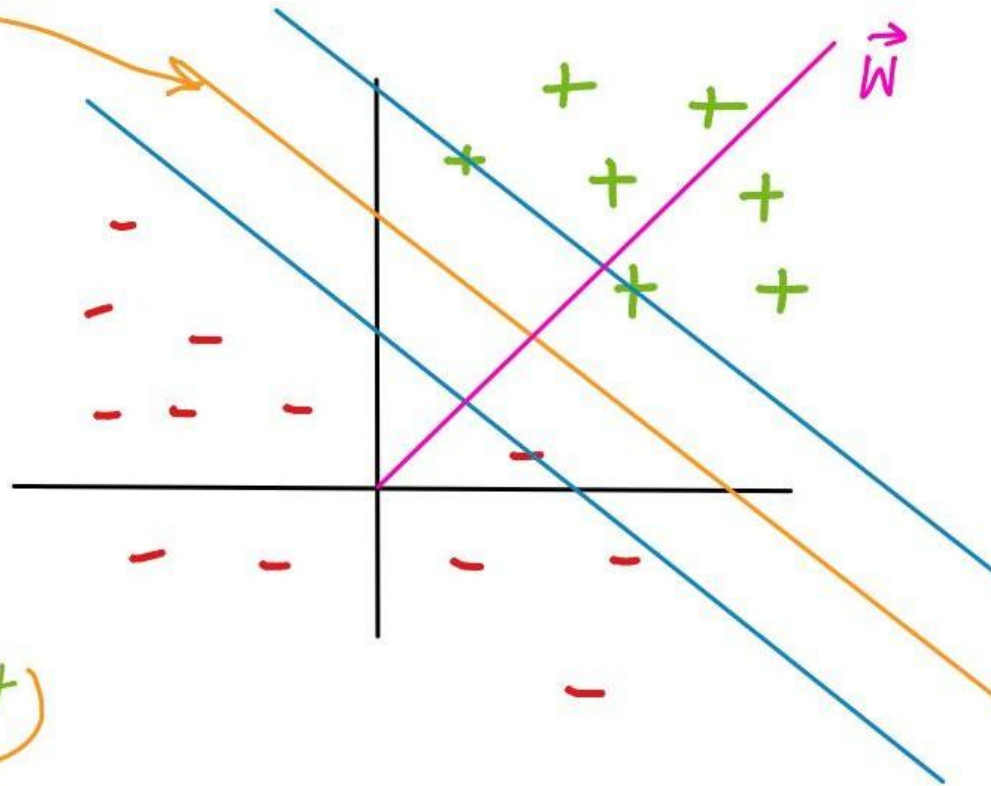
logic :-

if  $W^T X + b \geq 0$

$X$  is positive ( $X^+$ )

if  $W^T X + b < 0$

$X$  is negative ( $X^-$ )



## Unit 2 - SVM

### Equation of Hyperplanes



Let the hyper planes be represented by general equations:

$$W^T X + b_1 = 0 \text{ and } W^T X + b_2 = 0$$

Equation of the hyperplanes

Case 1 :  $W^T X^- + b_1 = 0$

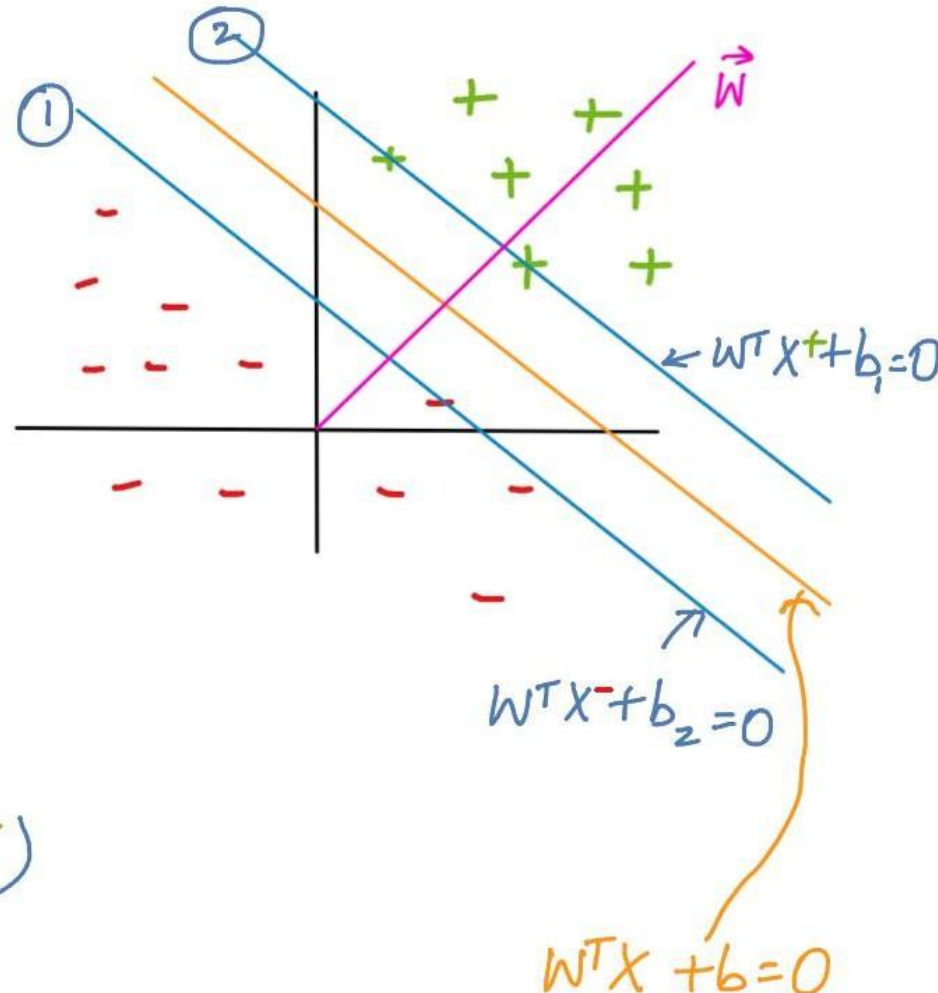
if  $W^T X^- + b_1 \leq 0$

$X$  is negative ( $X^-$ )

Case 2 :  $W^T X^+ + b_2 = 0$

if  $W^T X^+ + b_2 \geq 0$

$X$  is positive ( $X^+$ )

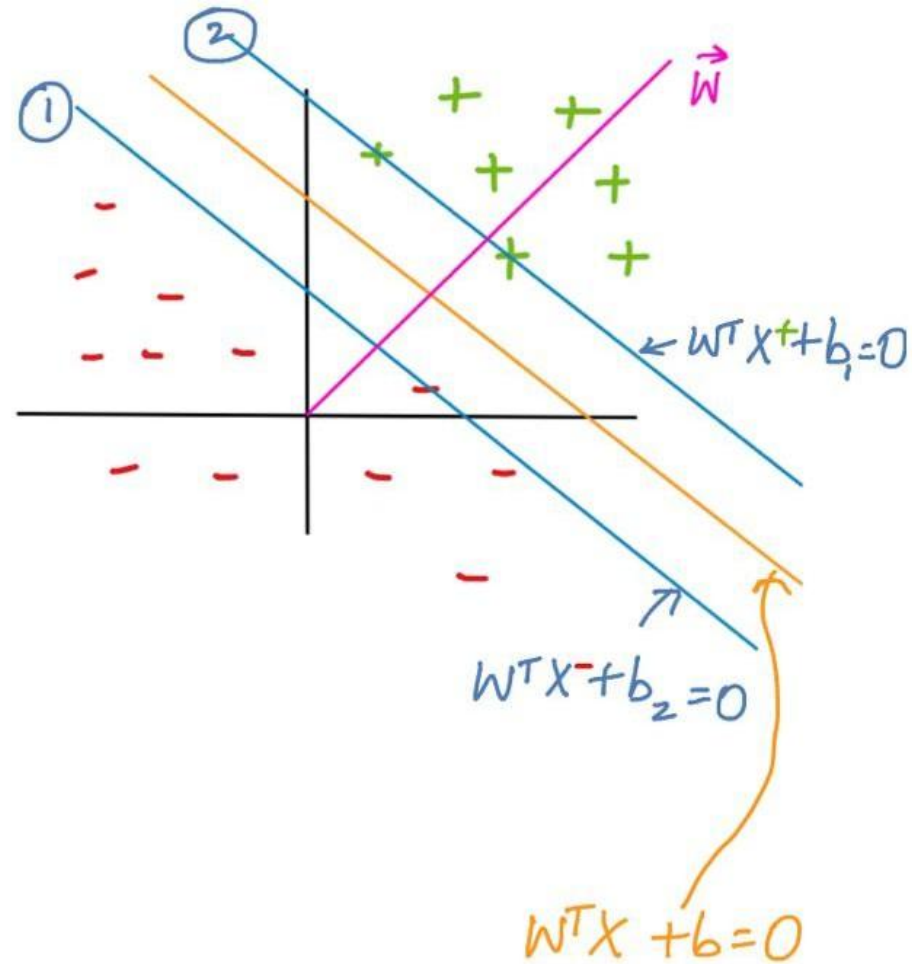


Decision rules / logic :-

if  $W^T X + b \geq 0$   
 $X$  is positive

① if  $W^T X^- + b_1 \leq 0$   
 $X$  is negative

② if  $W^T X^+ + b_2 \geq 0$   
 $X$  is positive



### Another way of representing the decision rule for hyperplanes

Decision rules / logic :-

if  $W^T X + b \geq 0$   
 $X$  is positive

① if  $W^T X^- + b_1 \leq 0$   
 $X$  is negative

② if  $W^T X^+ + b_2 \geq 0$   
 $X$  is positive

Can be  
written as  
→

①  $W^T X^- + b \leq -1$

②  $W^T X^+ + b \geq 1$

## Unit 2 - SVM

### Proof -- Both ways are equivalent



Proof :-

Given  $W^T X^- + b_1 = 0$  ,  $W^T X^+ + b_2 = 0$

Let's define  $C = \frac{b_1 + b_2}{2}$  and  $D = \frac{b_1 - b_2}{2}$

$\Rightarrow b_1 = C + D$  and  $b_2 = C - D$

Substituting the values of  $b_1$  and  $b_2$  into the hyperplane equations we get

|                                |                               |
|--------------------------------|-------------------------------|
| $W^T X^- + C + D = 0$          | $W^T X^+ + C - D = 0$         |
| $\Rightarrow W^T X^- + C = -D$ | $\Rightarrow W^T X^+ + C = D$ |

Divide both Sides by  $D$

|                                                    |                                       |
|----------------------------------------------------|---------------------------------------|
| $\Rightarrow \frac{W^T}{D} X^- + \frac{C}{D} = -1$ | $\frac{W^T}{D} X^+ + \frac{C}{D} = 1$ |
|----------------------------------------------------|---------------------------------------|

Let  $M = \frac{1}{D}(W)$  &  $b = \frac{C}{D}$

|                              |                 |
|------------------------------|-----------------|
| $\Rightarrow M X^- + b = -1$ | $M X^+ + b = 1$ |
|------------------------------|-----------------|

rename vector  $M$  as  $W$

|                              |                 |
|------------------------------|-----------------|
| $\Rightarrow W X^- + b = -1$ | $W X^+ + b = 1$ |
|------------------------------|-----------------|

Decision rules / logic :-

if  $W^T X + b \geq 0$   
 $X$  is positive

① if  $W^T X^- + b_1 \leq 0$   
 $X$  is negative

② if  $W^T X^+ + b_2 \geq 0$   
 $X$  is positive

Can be written as  
 $\rightarrow$

①  $W^T X^- + b \leq -1$

②  $W^T X^+ + b \geq 1$



### A compact/single notation for representing hyperplanes decision rule

Decision rule for hyperplanes

① Negative points

$$W^T X^- + b \leq -1$$

② Positive points

$$W^T X^+ + b \geq 1$$

} can be combined into a single rule

Median line :-

$$W^T X + b \geq 0 \quad (\text{Positive})$$

$$W^T X + b < 0 \quad (\text{Negative})$$

define  $y_i = \begin{cases} +1 & \text{for Positive samples} \\ -1 & \text{for negative samples} \end{cases}$

$$y_i (W^T X_i + b) \geq 1$$

$$\Rightarrow y_i (W^T X_i + b) - 1 \geq 0$$

### A compact/single notation for representing hyperplanes decision rule

Note:-

$$y_i (w^T x_i + b) \geq 1$$

if  $x_i$  is *positive*,  $y_i = 1$

$$\Rightarrow w^T x^+ + b \geq 1$$

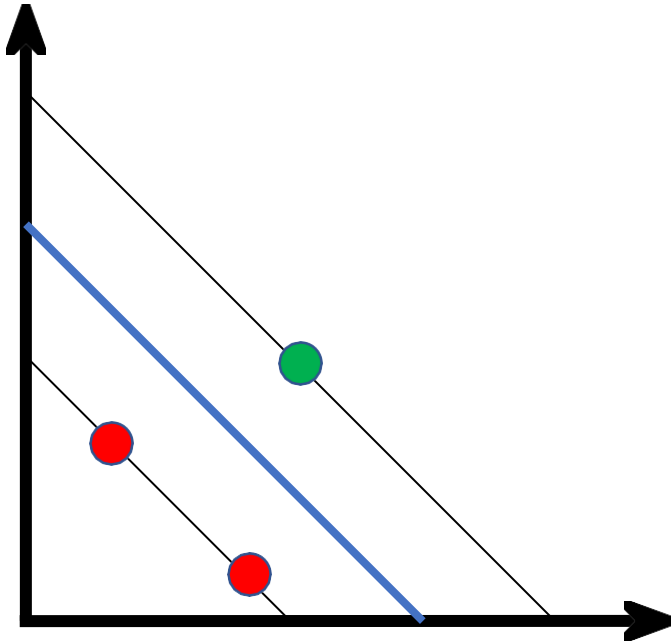
if  $x_i$  is *negative*,  $y_i = -1$

$$\Rightarrow (-1) * (w^T x^- + b) \geq 1$$

Multiply both sides by -1

$$\Rightarrow w^T x^- + b \leq -1$$





$$y_i(\vec{W} \cdot \vec{X} + b) - 1 \geq 0$$

$$y_i(\vec{W} \cdot \vec{X} + b) - 1 = 0, \text{ for } \vec{X}_i \text{ in gutter}$$

## Unit 2 - SVM

### How to classify a new point U??



Let us imagine that by some magic we got that decision boundary i.e. values of vector  $W$  and bias  $b$ .

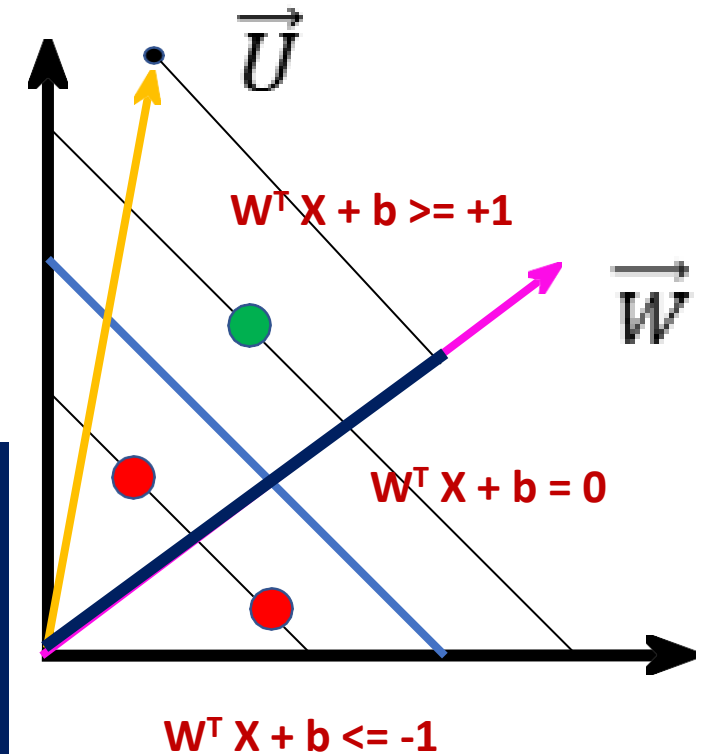
Let us say we have some unknown new point  $U$  which we have to classify.

**Use the following decision rule to classify  $U$  (new point)  
(HARD MARGIN)**

**if  $W^T U + b \geq +1 \Rightarrow U$  is predicted positive with high confidence**

**if  $W^T U + b \leq -1 \Rightarrow U$  is predicted negative with high confidence**

**if the value falls in the gutter/canal --- we cannot make a decision  
with high confidence (SOFT MARGIN)**



## Unit 2 - SVM

### Calculating width of the Canal



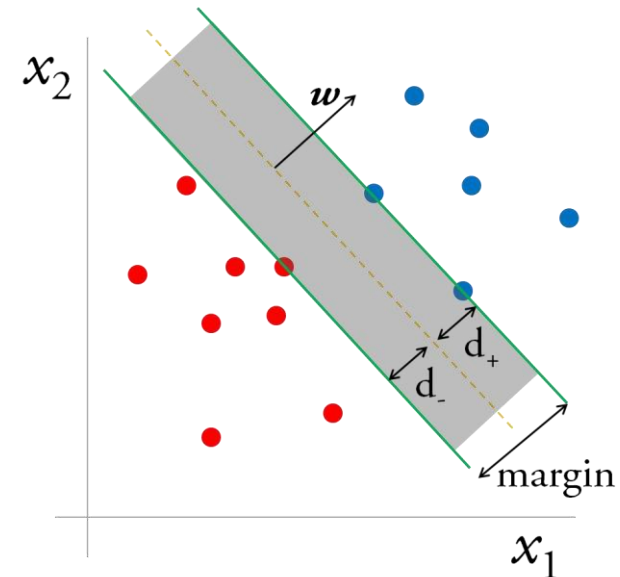
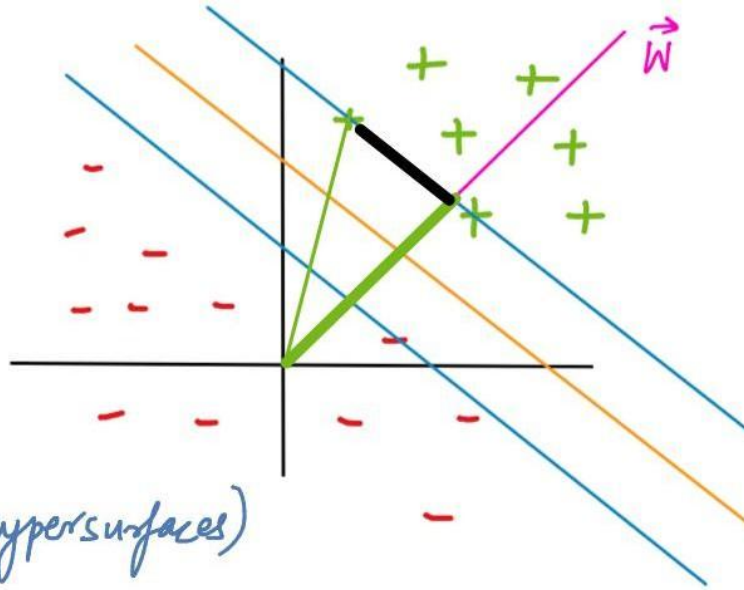
1) Project a +ve positive point  $x^+$  on the hyperplane  $\vec{w}^T x + b = 1$  on the weight vector  $\vec{w}$ .

(vector  $\vec{w}$  is orthogonal to the hypersurfaces)

2) find the length of the projection.

$$\| \text{Projection}_w(x^+) \| = \hat{w} \cdot \vec{x}$$

↑  
Unit vector  $\hat{w} = \frac{w}{\|w\|}$



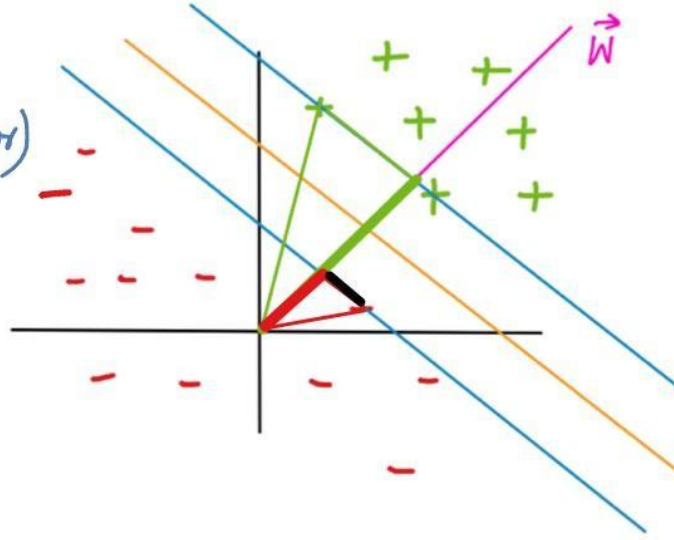
Calculating width of the Canal

3) Project a -ve point  $X^-$  (Input vector)  
on the weight vector  $\vec{W}$ .

4) Find the length of the  
projection

$$\| \text{Projection}_w(\vec{X}^-) \| = \hat{W} \cdot \vec{X}^-$$

5) width of the canal = Difference in the projection length  
 $= \| \text{Projection}_w(X^+) \| - \| \text{Projection}_w(X^-) \|\$   
width =  $\hat{W} \cdot \vec{X}^+ - \hat{W} \cdot \vec{X}^-$



## Unit 2 - SVM

### Calculating Width of the Canal



$$\text{width} = \hat{w} \vec{x}^+ - \hat{w} \vec{x}^-$$

We know the equation of the hyperplanes:-  
 $w \vec{x}^+ + b = 1$  &  $w \vec{x}^- + b = -1$

$$w \vec{x}^+ = 1 - b$$

$$w \vec{x}^- = -1 - b$$

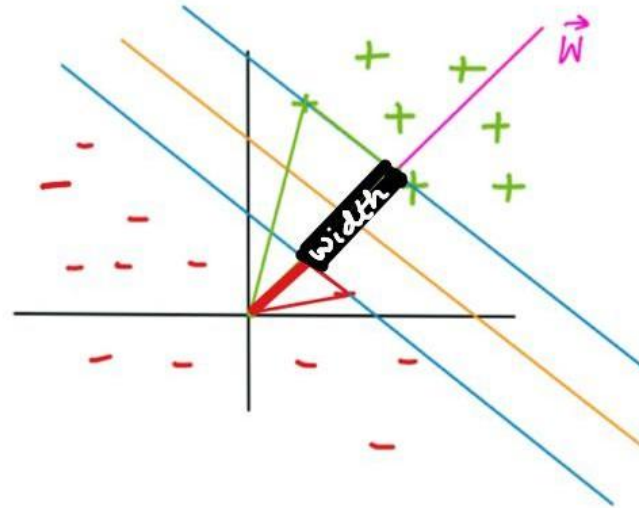
Divide both sides by  $\|w\|$

$$\frac{w \vec{x}^+}{\|w\|} = \frac{1-b}{\|w\|}$$

$$\hat{w} \vec{x}^+ = \frac{1-b}{\|w\|}$$

$$\frac{w \vec{x}^-}{\|w\|} = \frac{-1-b}{\|w\|}$$

$$\hat{w} \vec{x}^- = \frac{-1-b}{\|w\|}$$



$$\Rightarrow \text{width} = \frac{1-b}{\|w\|} - \left( \frac{-1-b}{\|w\|} \right)$$
$$= \frac{1-b + 1+b}{\|w\|}$$

$$\text{width} = \frac{2}{\|w\|}$$

Primal problem

Maximize

$$f(w, b) = \frac{2}{||w||}$$

larger the margin the lower the  
generalization error of the classifier

Subject to constraints

$$y_i (w^T \vec{x}_i + b) - 1 \geq 0$$

for  $i = 1$  to  $m$

where,

$m$  is the number of training instances

**Formulating the problem:**

- Maximize  $2 / ||W||$
- This is same as minimize:  $||W|| / 2$
- Same as Minimize  $(1 / 2) * ||W||^2$
- Add the constraints.

**Problem:**  $\min \frac{1}{2} ||W||^2$

subject to the given constraints:  $y_i (w^T x_i + b) \geq 1$   
for  $i = 1$  to  $m$

(where,  $m$  is the number of training instances)

**Convex** (linear constraints) **Quadratic** (Objective function)

**Optimization:** A standard problem!

It can be solved by the **Lagrangian multiplier method**



$$\min_{w,b} \frac{1}{2} ||w||^2$$

$$\text{s.t. } y_i(w^T x_i + b) \geq 1, \quad i = 1, \dots, m$$

This optimization problem can be solved using commercial quadratic programming(QP) code or (better) with duality formulation.

We will use Lagrange duality to solve the above constrained convex optimization problem.

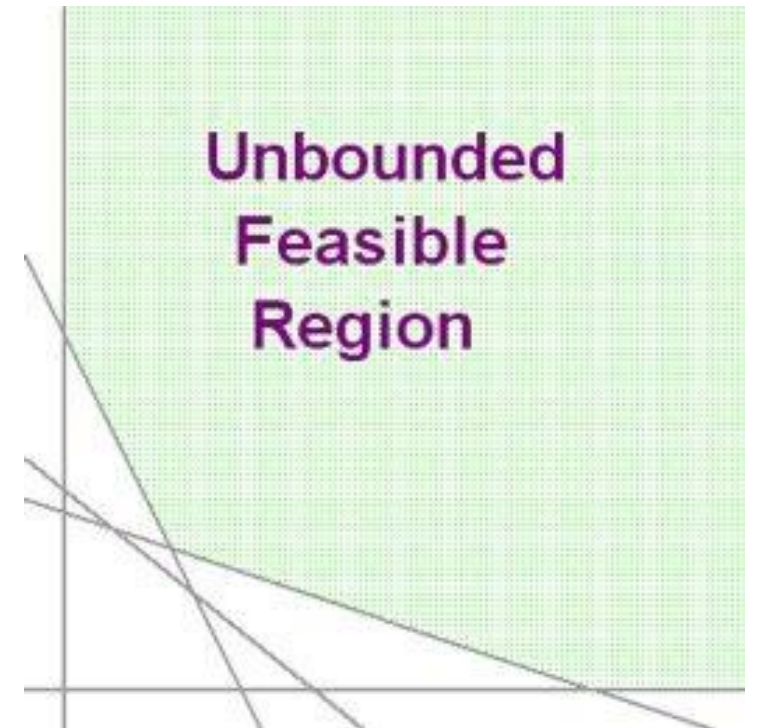
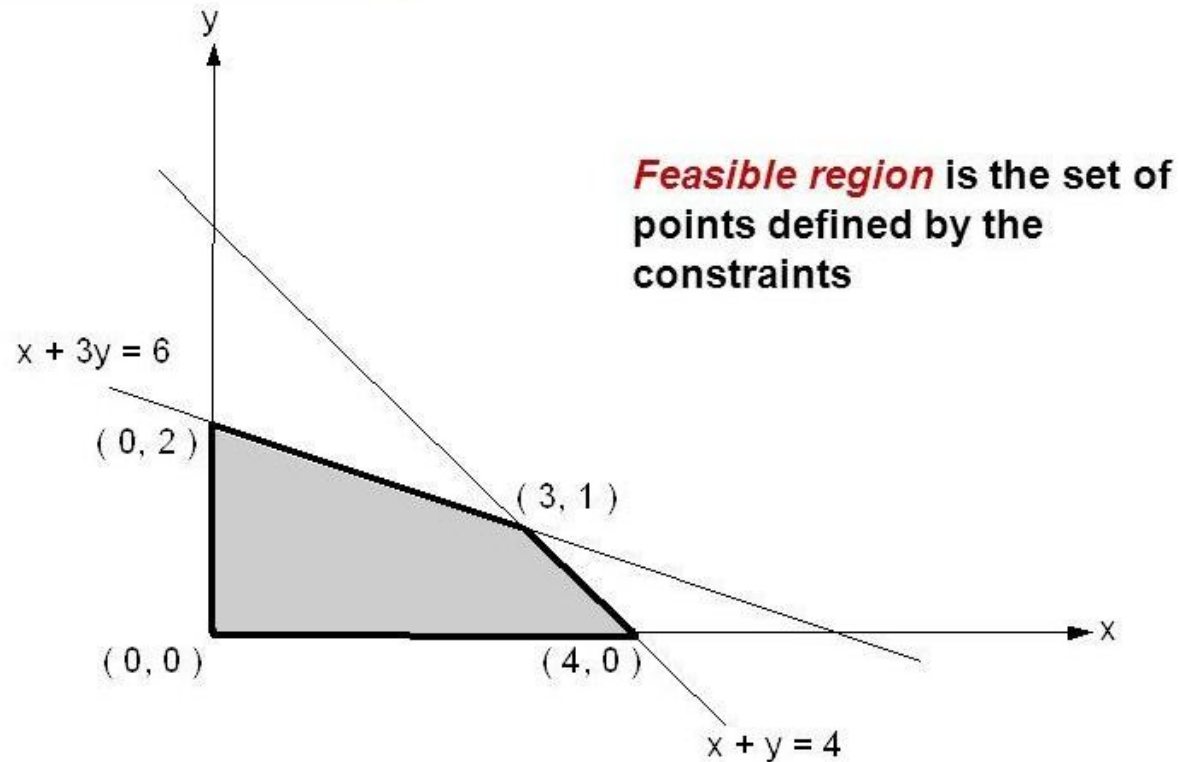
This will allow the use kernels, and it is also more efficient.



### Lagrange Multipliers --- The idea

maximize  $3x + 5y$   
subject to  $x + y \leq 4$   
 $x + 3y \leq 6$   
 $x \geq 0, y \geq 0$

## Example



### Lagrange Multipliers --- The idea

---

Let us say we wish to maximize the function value  $f(x_1, x_2)$  subject to a single equality constraint  $g(x_1, x_2) = c$ .

**Max  $f(x_1, x_2)$**

**subject to constraint  $g(x_1, x_2) - c = 0$  [Popular way of writing the constraint]**

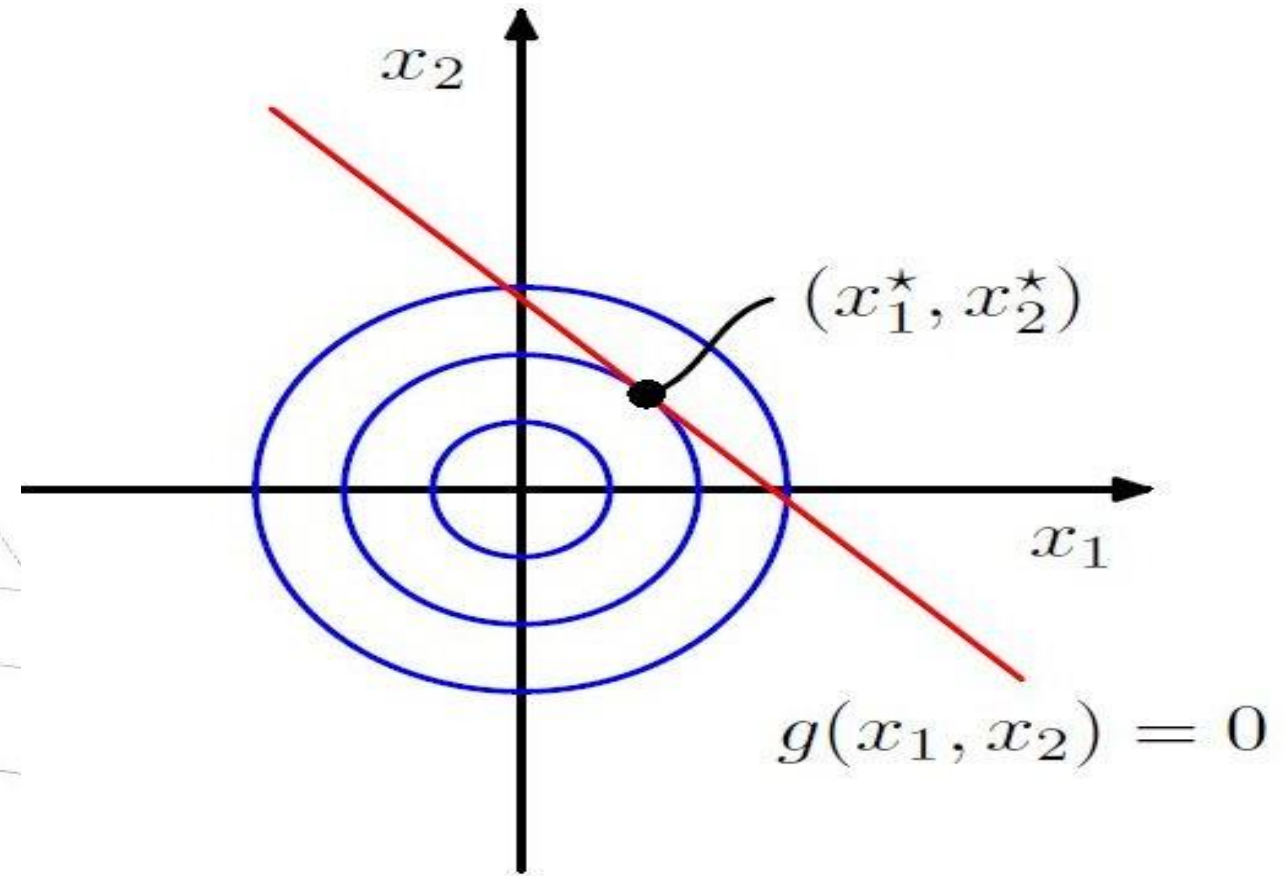
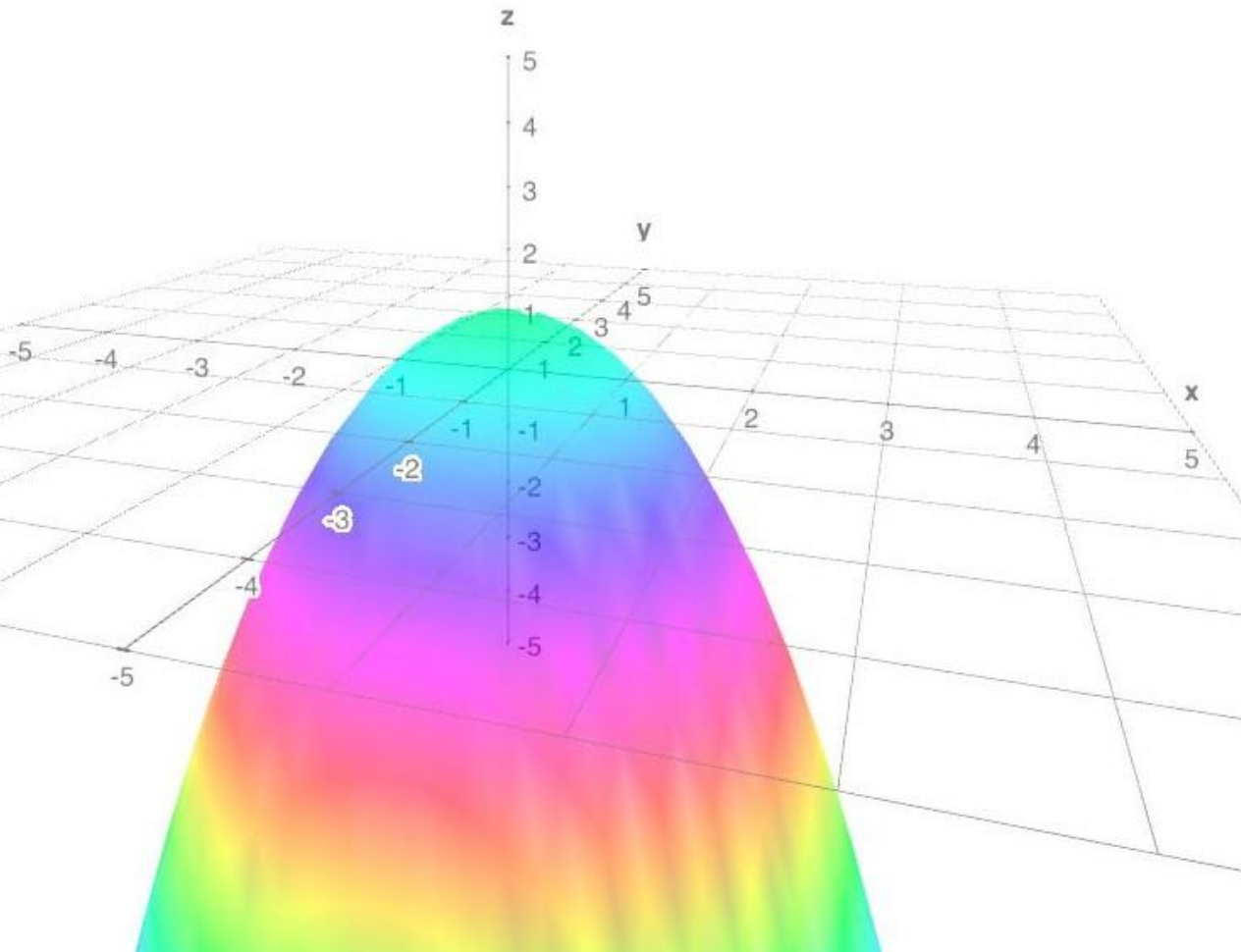
(Remember in our SVM problem is a minimization problem with multiple inequality constraints ( $\geq 0$ ))

However, for minimization same ideas will follow )

## Unit 2 - SVM

### Example

maximize  $f(X) = 1 - x_1^2 - x_2^2$ , subject to  $g(X) = x_1 + x_2 - 1 = 0$



### Lagrange Multipliers --- The idea

---

We know, gradient of a function, points at the direction of steepest ascent. So in order to maximize  $f(x_1, x_2)$  we must move in the direction of the gradient.

While moving along the gradient, we cannot leave the constraint surface. Throughout the constraint surface, i.e. for any point on the constraint surface,  $g(x_1, x_2) - c = 0$ .

We can say, all the points on the constraint surface form a level surface(because value  $g(x_1, x_2) - c = 0$  for any point  $x_1, x_2$  on the constraint surface..

The optimal point (where value of  $f(x_1, x_2)$  is maximum must remain on the constraint surface.

That means for that optimal point  $(x_1^*, x_2^*)$  also, the value of  $g(x_1^*, x_2^*) - c = 0$ .

Let us take an example :

**maximize :  $f(X)$**

**subject to constraint :  $g(X) = 0$**

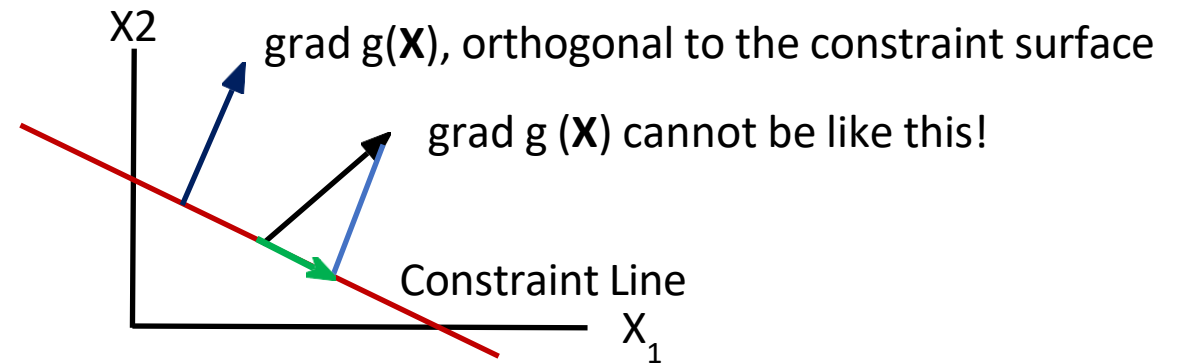
(equality constraint only for now, inequalities will be considered soon)

Assume  $X$  is  $d$ -dimensional, then  $g(X) = 0$  is a  $d - 1$  dimensional surface.

Let us take an example :

**maximize :  $f(\mathbf{X})$**

**subject to constraint :  $g(\mathbf{X}) = 0$**



Gradient  $\nabla g(\mathbf{X})$  is orthogonal to the constraint surface at any point on the constraint surface.

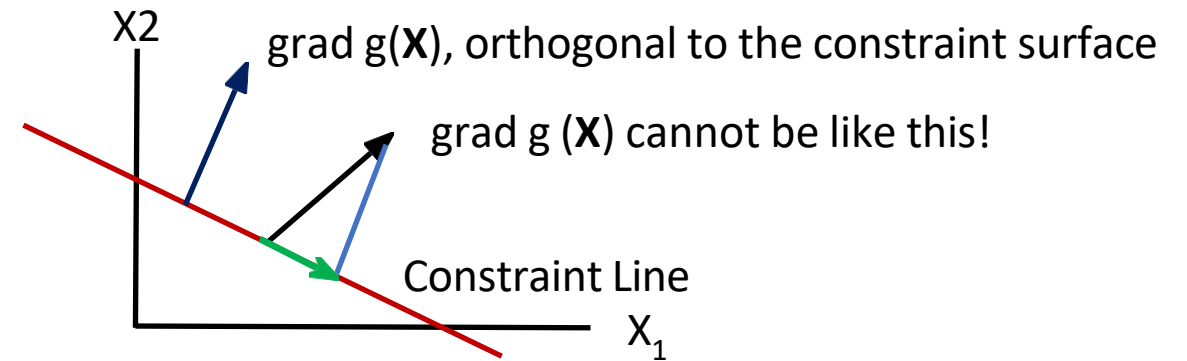
Else, projection of  $\nabla g(\mathbf{X})$  onto the constraint surface would be non-zero, implying the value of  $g(\mathbf{X})$  would increase moving along the direction of that projection. This is not possible, as  $g(\mathbf{X})$  remains unchanged on the constrained surface.

Let us take an example :

**maximize :  $f(X)$**

$X$  is a vector of input values  $(x_1, x_2, x_3, \dots, x_n)$

**subject to constraint :  $g(X) = 0$**



Let  $X$  and  $X + \epsilon$  be nearby points on the constraint surface.

$\epsilon$  is a vector located wholly on the constraint surface.

Using Taylor Series,

$$g(X + \epsilon) \approx g(X) + \epsilon^T \nabla g(X)$$

$$\text{But, } g(X + \epsilon) = g(X) \Rightarrow \epsilon^T \nabla g(X) \approx 0 \Rightarrow \nabla g(X) \text{ is orthogonal to the constraint surface } g(X) = 0$$



Let us take an example :

**maximize :  $f(X)$**

**subject to constraint :  $g(X) = 0$**

Let  $X^*$  be the point on the constraint surface  $g(X) = 0$  which maximizes  $f(X)$ .

Following the same logic, we conclude  $\nabla f(X^*)$  is orthogonal to the constraint surface.

Else, projection of  $\nabla f(X^*)$  onto the constraint surface would be non-zero, implying the value of  $f(X)$  would increase moving along the direction of that projection. This is not possible, as we are claiming  $X^*$  is the optimal point where max of  $f(X)$  occurs.

Thus at  $X^*$  the  $\nabla g(X^*)$  and  $\nabla f(X^*)$  are parallel to each other. (same or opposite direction)

## Unit 2 - SVM

### Parallel Vectors -- Recap



Two vectors are parallel if they are scalar multiples of one another.

If  $u$  and  $v$  are two non-zero vectors and are parallel then,  
$$u = cv$$

if  $c$  is +ve, both vectors are in same direction

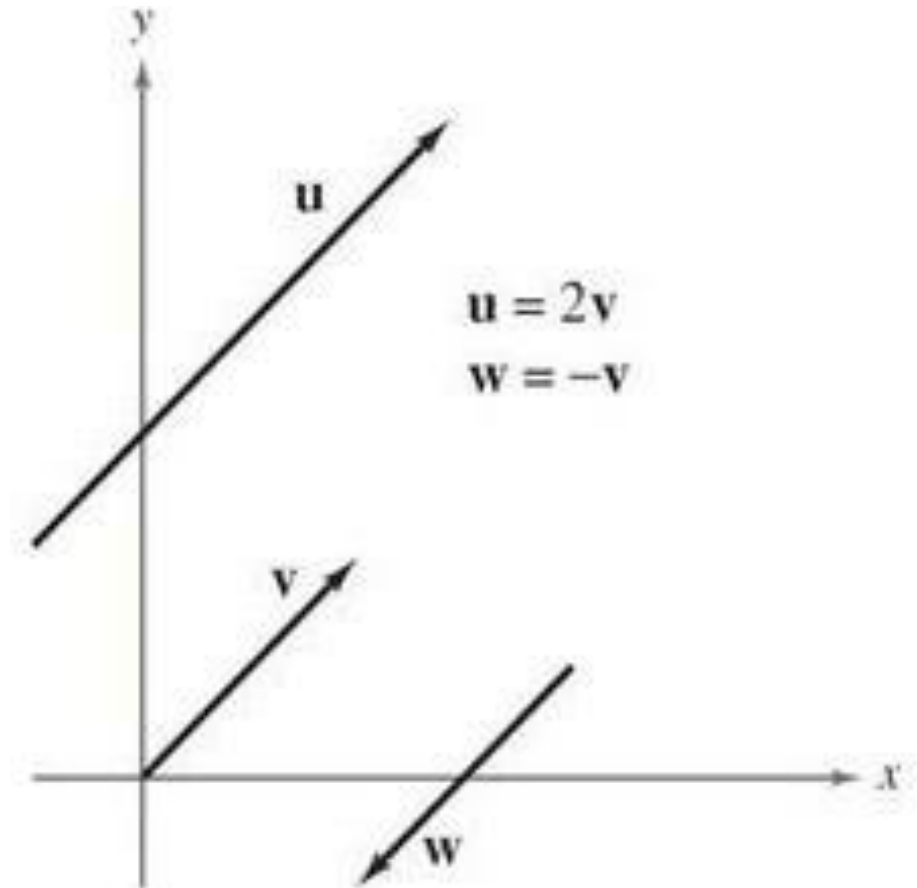
if  $c$  is -ve, both vectors are in different direction

Also,  $u - cv = 0$

In the given diagram, vectors  $u$ ,  $v$  and  $w$  are parallel because

$$u = 2v \text{ and}$$

$$w = -v$$



Since at optimal point  $X^*$  the  $\nabla g(X^*)$  and  $\nabla f(X^*)$  are parallel to each other, we can say they are scalar multiples of one another i.e.,

$$\nabla f(X^*) = \lambda \nabla g(X^*)$$

where  $\lambda$  is a scalar and is called Lagrange Multiplier, **could be +ve or ve but not zero.**

Lagrange wrote down a special new function which takes in all the same input variables as  $f$  and  $g$ , along with  $\lambda$ , thought of now as a variable rather than just a constant/scalar:

$$L(X, \lambda) = f(X) - \lambda g(X)$$

find  $X^*$  by partially differentiating  $L$  wrt to  $X$  and  $\lambda$  and set it equal to zero.

We will have  $d+1$  equations,  $d+1$  unknowns.

if not interested in value of  $\lambda$  do not solve for it. (Method of Undetermined Multipliers!!)

Few sources use the equation as:

$$L(X, \lambda) = f(X) - \lambda g(X)$$

Few sources use the equation as:

$$L(X, \lambda) = f(X) + \lambda g(X)$$

Both ways are fine.

## Unit 2 - SVM

### Example 1



Solve:

$$f(X) = 1 - x_1^2 - x_2^2$$

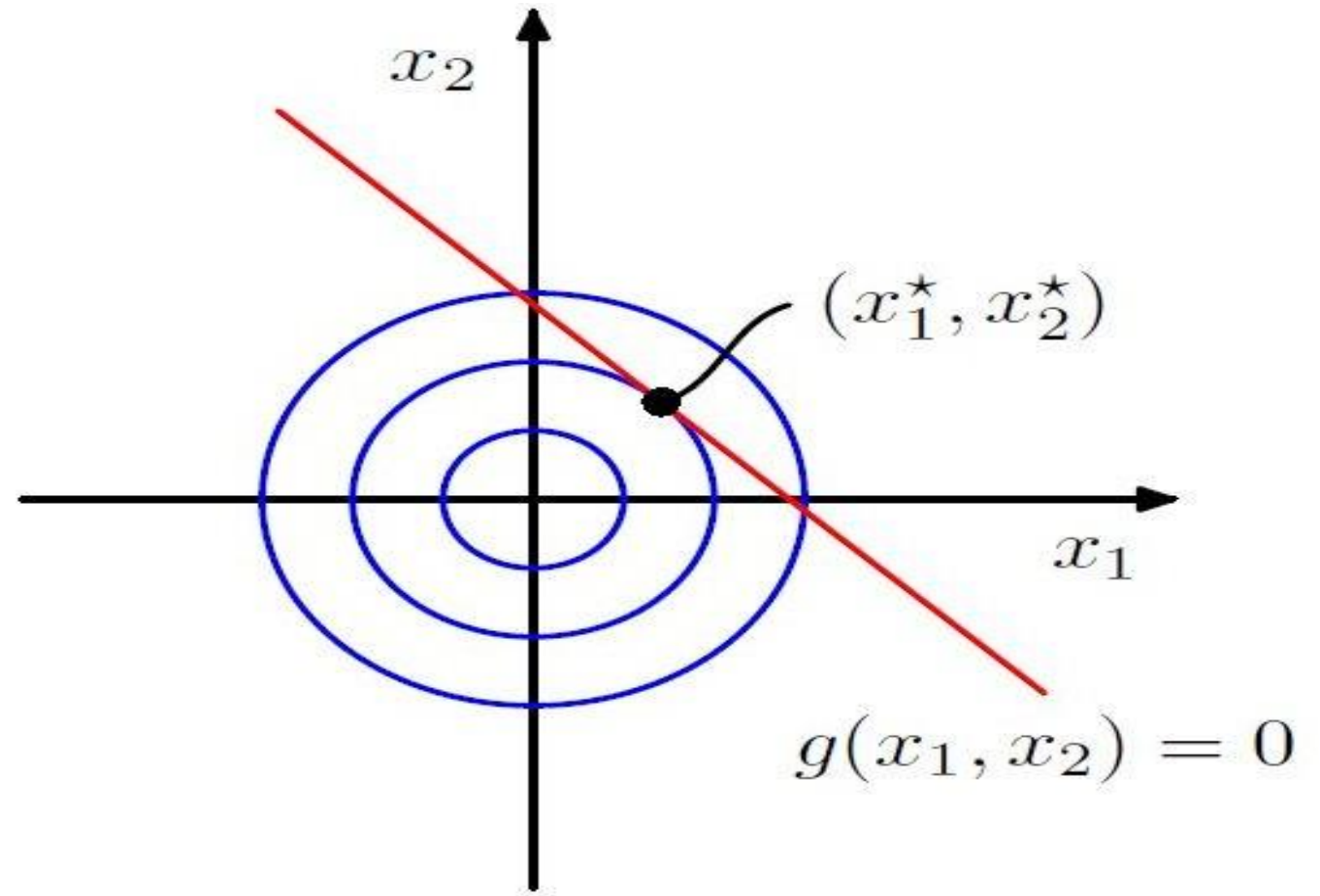
$$\text{subject to } g(X) = x_1 + x_2 - 1 = 0$$

Solution :

Define

$$L(X, \lambda) = f(X) - \lambda g(X)$$

$$L(X, \lambda) = 1 - x_1^2 - x_2^2 - \lambda(x_1 + x_2 - 1)$$



## Unit 2 - SVM

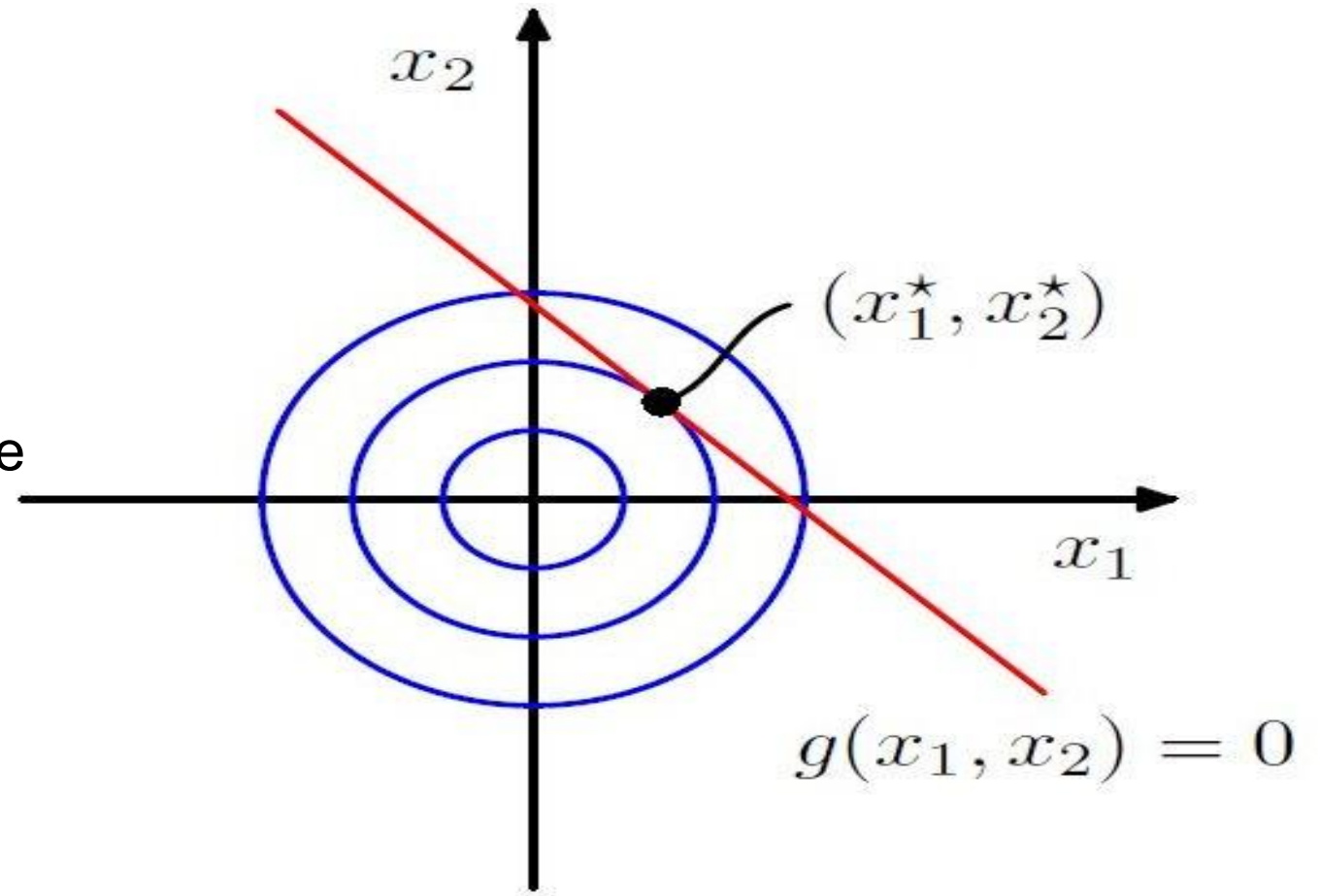
### Example 1

Solve:

$$f(X) = 1 - x_1^2 - x_2^2$$

$$\text{subject to } g(X) = x_1 + x_2 - 1 = 0$$

In case there was no constraint  $g(X)$ , the maximum value of  $f(X)$  would be 1 and the optimal point would have been (0,0).



## Unit 2 - SVM

### Example 1 Solution



Equations for finding the stationary point of the Lagrangian Function:

$$-2x_1 - \lambda = 0 ;$$

$$-2x_2 - \lambda = 0 ;$$

$$-x_1 - x_2 + 1 = 0$$

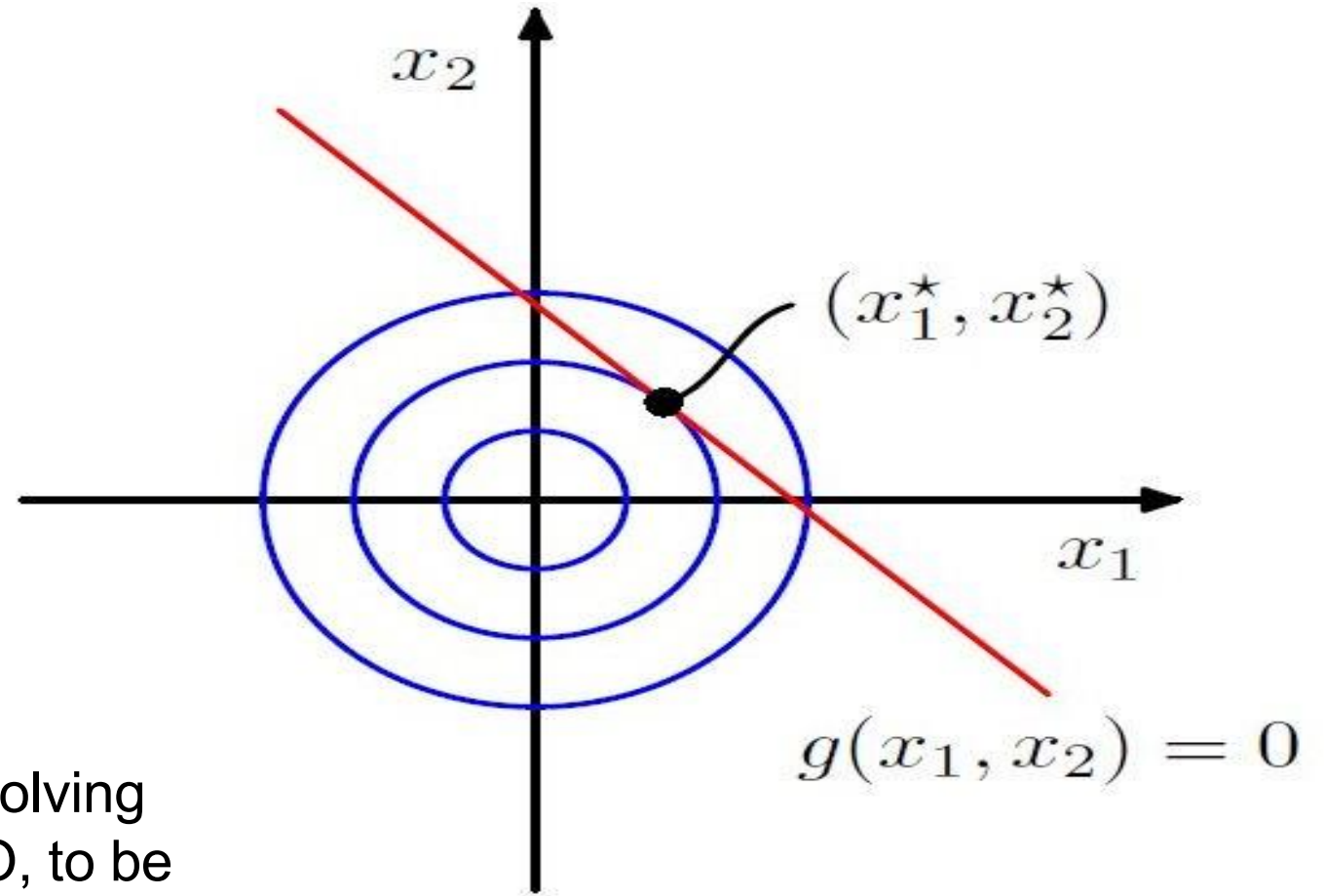
Solution:

$$x_1 = 1/2;$$

$$x_2 = 1/2; \lambda = 1;$$

$$f(X^*) = f(1/2, 1/2) = 1/2$$

With a very large number of equations, solving these equations can be a challenge! SMO, to be discussed in the next course, is a popular iterative algorithm for the SVM Problem.





**Example** Maximize  $f(x_1, x_2) = x_1x_2$  subject to  $h(x_1, x_2) \equiv x_1 + 4x_2 = 16$ .

Solution: Form the Lagrangian

$$L(x_1, x_2) = x_1x_2 - \lambda(x_1 + 4x_2 - 16)$$

The first order conditions are

$$\frac{dL}{dx_1} = x_2 - \lambda = 0$$

$$\frac{dL}{dx_2} = x_1 - 4\lambda = 0$$

$$\frac{dL}{d\lambda} = x_1 + 4x_2 - 16 = 0$$

From the first two equations we have

$$x_1 = 4x_2.$$

Plugging this into the last equation we have that  $x_2 = 2$ , which in turn implies that  $x_1 = 8$ , and that  $\lambda = 2$ . This states that the only candidate for the solution is  $(x_1, x_2, \lambda) = (8, 2, 2)$ .

Maximize  $f(X) = x + 2y$  subject to constraints  $3x^2 + 4y^2 - 3 = 0$

$$\begin{aligned} L(x, y, \lambda) &= f(x, y) - \lambda g(x, y) \\ &= (x + 2y) - \lambda (3x^2 + 4y^2 - 3) \end{aligned}$$

$$\frac{\partial L(x, y, \lambda)}{\partial x} = 1 - 6x\lambda = 0 \Rightarrow x = \frac{1}{6\lambda}$$

$$\frac{\partial L(x, y, \lambda)}{\partial y} = 2 - 8y\lambda = 0 \Rightarrow y = \frac{1}{4\lambda}$$

$$\frac{\partial L(x, y, \lambda)}{\partial \lambda} = -3x^2 - 4y^2 + 3 = 0 \Rightarrow 3x^2 + 4y^2 = 3$$

Substituting  $x = \frac{1}{6\lambda}$  and  $y = \frac{1}{4\lambda}$  in the above eqn.

$$3\left(\frac{1}{6\lambda}\right)^2 + 4\left(\frac{1}{4\lambda}\right)^2 = 3$$

Maximize  $f(X) = x + 2y$  subject to constraints  $3x^2 + 4y^2 - 3 = 0$

Solving  $3\left(\frac{1}{6\lambda}\right)^2 + 4\left(\frac{1}{4\lambda}\right)^2 = 3$

$$\Rightarrow \frac{3}{36\lambda^2} + \frac{4}{16\lambda^2} = 3$$

$$\Rightarrow \frac{1}{12\lambda^2} + \frac{1}{4\lambda^2} = 3$$

$$\Rightarrow \frac{4}{12\lambda^2} = 3 \Rightarrow \frac{1}{3\lambda^2} = 3$$

$$\Rightarrow 9\lambda^2 = 1 \Rightarrow \lambda = \sqrt{\frac{1}{9}}$$

$$\Rightarrow \lambda = \pm \frac{1}{3}$$

## Unit 2 - SVM

### Example 3



Maximize  $f(X) = x + 2y$  subject to constraints  $3x^2 + 4y^2 - 3 = 0$

$$\lambda = \frac{1}{3}$$

$$\Rightarrow x = \frac{1}{6\lambda} = \frac{3}{6} = \frac{1}{2}$$

$$\Rightarrow y = \frac{1}{4\lambda} = \frac{3}{4}$$

$$\left(\frac{1}{2}, \frac{3}{4}\right)$$

$$\lambda = -\frac{1}{3}$$

$$\Rightarrow x = \frac{1}{6\lambda} = -\frac{3}{6} = -\frac{1}{2}$$

$$\Rightarrow y = \frac{1}{4\lambda} = -\frac{3}{4}$$

$$\left(-\frac{1}{2}, -\frac{3}{4}\right)$$

Calculate value of  $f(x, y) = x + 2y$

$$f(x, y) = \frac{1}{2} + 2 \times \frac{3}{4}$$

$$f(x, y) = 2$$

$$f(x, y) = -\frac{1}{2} + 2 \times \left(-\frac{3}{4}\right)$$

$$f(x, y) = -2$$

$\Rightarrow$  max value of  $f$  occurs at  $\left(\frac{1}{2}, \frac{3}{4}\right)$

### Handling Multiple Constraints

---

Multiple constraints can be handled in a similar fashion by introducing the required number of Lagrange Multipliers. (One for each constraint.)

$$L(X, \lambda) = f(X) - \lambda_1 g_1(X) - \lambda_2 g_2(X) - \lambda_3 g_3(X) - \dots - \lambda_k g_k(X)$$

Function minimizations works in the same way.

We are finding a **stationary point** only when we take first order derivatives! To check that it is maximum or minimum, we need second-order derivatives! We will not get in to that!!!

We are still dealing with only Equality Constraints. We will look at Inequality constraints next.

- Assume the inequality constraint is of the form:  $g(X) \geq 0$  [like in our SVM problem]
- Lagrangian is still:  $L(X, \lambda) = f(X) - \lambda g(X)$
- Constrained stationary point  $X^*$  may lie in the region of  $g(X) > 0$  or it may lie on the surface of  $g(X) = 0$ .

(We do not as yet know which is the case! Let us consider each possibility.)

Case 1:  $X^*$  is in the region  $g(X) > 0$ :

- The constraint is **inactive!** We need to solve only  $\nabla f(X) = 0$ .
- Same as solving the Lagrangian with  $\lambda$  set to 0.

(As  $\lambda$  is set to 0,  **$\lambda * g(X)$  will also become 0.**)

$$L(X, \lambda) = f(X)$$

Case 2:  $X^*$  is on the surface  $g(X) = 0$ :

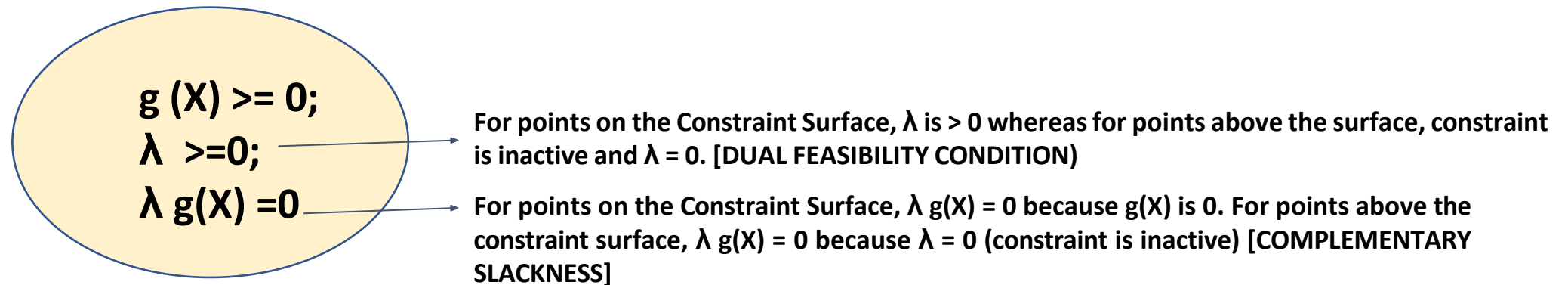
- Same as the case discussed earlier i.e., solve the Lagrangian with  $\lambda$  not equal to 0.
- $\nabla g(X)$  is orthogonal to the surface  $g(X) = 0$  as discussed earlier.
- Again,  $\nabla f(X)$  must be orthogonal to the surface  $g(X) = 0$  but, crucially, it must be opposite in direction from the region  $g(X) > 0$ . (Else,  $f(X)$  can be increased by moving in to the region of  $g(X) > 0$  which we considered as Case 1.)
- Hence  $\nabla f(X) = -\lambda \nabla g(X)$ , with  $\lambda > 0$ .

( $\nabla g(X) \neq 0$ ; so, still  **$\lambda \text{ grad } g(X) = 0$ .  $\lambda * g(X) = 0$** )



### Inequality Constraint (KKT Conditions)

- We must solve the Lagrangian as earlier (Partial derivatives of  $L$  wrt  $X$  as well as  $\lambda$  must be set to 0 and the resulting  $D+1$  equations must be solved) but with some additional constraints:



**Karush – Kuhn – Tucker (KKT) Conditions.**

**Multiple constraints are handled as discussed already but with KKT conditions added.**

### The SVM Lagrangian problem (Dual Problem) -- Linearly separable case

- We saw previously that the SVM optimization problem is:

$$\begin{aligned} & \underset{w, b}{\text{minimize}} \quad \frac{1}{2} \|w\|^2 \\ & \text{subject to} \quad y_i(\vec{W} \cdot \vec{X}_i + b) - 1 \geq 0 \quad i = 1, \dots, m \end{aligned}$$

- We have one objective function to minimize:

$$f(\mathbf{w}) = \frac{1}{2} \|\mathbf{w}\|^2$$

- Subject to m constraint functions:

$$g(w, b) = y_i(\vec{W} \cdot \vec{X}_i + b) - 1, \quad i=1, \dots, m$$

- We introduce the Lagrangian function:

$$L = f(w) - \sum_{i=1}^m \alpha_i g_i(w, b)$$

$$L = \frac{1}{2} \|w\|^2 - \sum_{i=1}^m \alpha_i [y_i(\vec{W} \cdot \vec{X}_i + b) - 1]$$

$$\mathcal{L}(w, b, \alpha) = \frac{1}{2} ||w||^2 - \sum_{i=1}^m \alpha_i (y_i (w^T x_i + b) - 1)$$

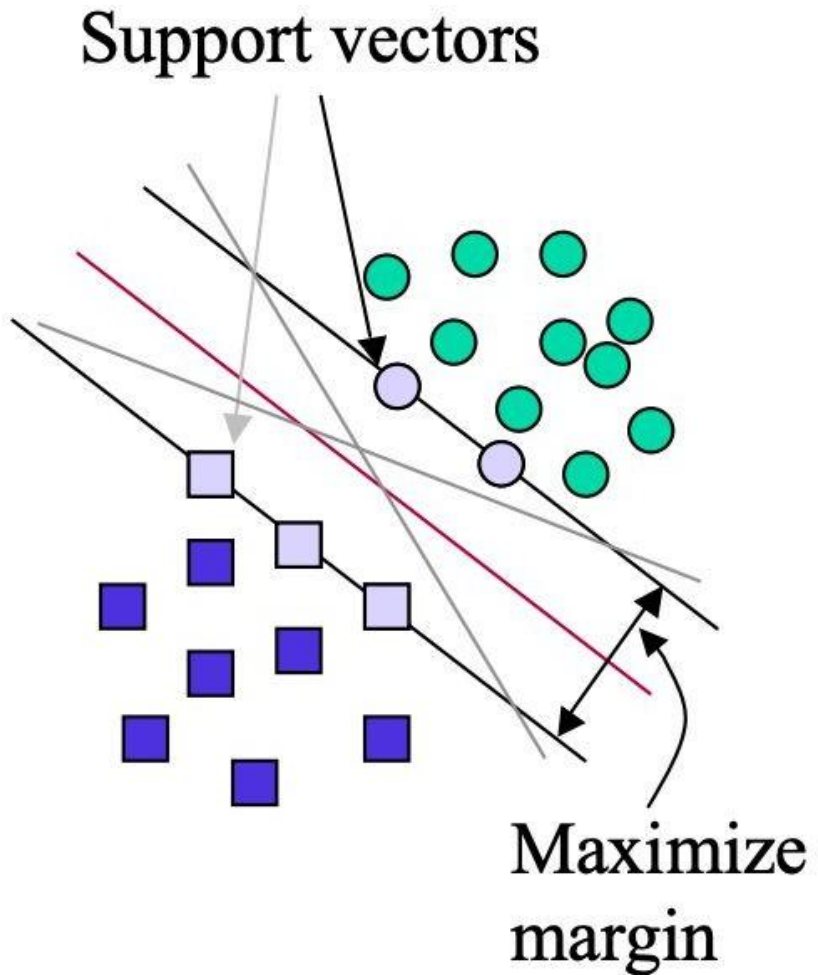
so now we know that  $W$  is going to be linear sum of **some** of the vectors (samples)

Why did I tell **SOME** ??

Because those points/instances which do not lie on gutter ,they will have Lagrangian multiplier=0

and those vectors who have Lagrangian Multiplier  $\neq 0$  will lie on gutter and **guess what???**

## THESE ARE CALLED SUPPORT



Differentiate  $\mathcal{L}$  with respect to  $w, b$ , and set the differential to zero:

- For  $w$ :

$$\frac{\partial}{\partial w} \mathcal{L}(w, b, \alpha) = w - \sum_{i=1}^m \alpha_i y_i x_i = 0$$

$$\Rightarrow w = \sum_{i=1}^m \alpha_i y_i x_i$$

- For  $b$ :

$$\frac{\partial}{\partial b} \mathcal{L}(w, b, \alpha) = 0 - \sum_{i=1}^m \alpha_i y_i$$

$$\Rightarrow \sum_{i=1}^m \alpha_i y_i = 0$$

If we can solve for  $\alpha$ s (dual problem), then we have a solution for  $w, b$  (primal problem)

$$\begin{aligned}\frac{\partial \frac{1}{2} w^T w}{\partial w} &= \left[ \frac{\partial \frac{1}{2} w^T w}{\partial w_1}, \frac{\partial \frac{1}{2} w^T w}{\partial w_2}, \dots, \frac{\partial \frac{1}{2} w^T w}{\partial w_m} \right] \\&= \left[ \frac{\frac{\partial}{\partial w_1} [w_1, w_2, \dots, w_m]^T [w_1, w_2, \dots, w_m]}{\partial w_1}, \frac{\frac{\partial}{\partial w_2} [w_1, w_2, \dots, w_m]^T [w_1, w_2, \dots, w_m]}{\partial w_2}, \dots, \frac{\frac{\partial}{\partial w_m} [w_1, w_2, \dots, w_m]^T [w_1, w_2, \dots, w_m]}{\partial w_m} \right] \\&= \left[ \frac{\frac{\partial}{\partial w_1} (w_1^2 + w_2^2 + \dots + w_m^2)}{\partial w_1}, \frac{\frac{\partial}{\partial w_2} (w_1^2 + w_2^2 + \dots + w_m^2)}{\partial w_2}, \dots, \frac{\frac{\partial}{\partial w_m} (w_1^2 + w_2^2 + \dots + w_m^2)}{\partial w_m} \right] \\&= [2 * \frac{1}{2} w_1, 2 * \frac{1}{2} w_2, \dots, 2 * \frac{1}{2} w_m] = 2 * \frac{1}{2} * [w_1, w_2, \dots, w_m] = 2 * \frac{1}{2} * w = w\end{aligned}$$

Similarly in other cases, we always differentiate a scalar function over a vector  $w$  here, which results in a vector of derivatives over each element  $w_j, 1 \leq j \leq m$ .



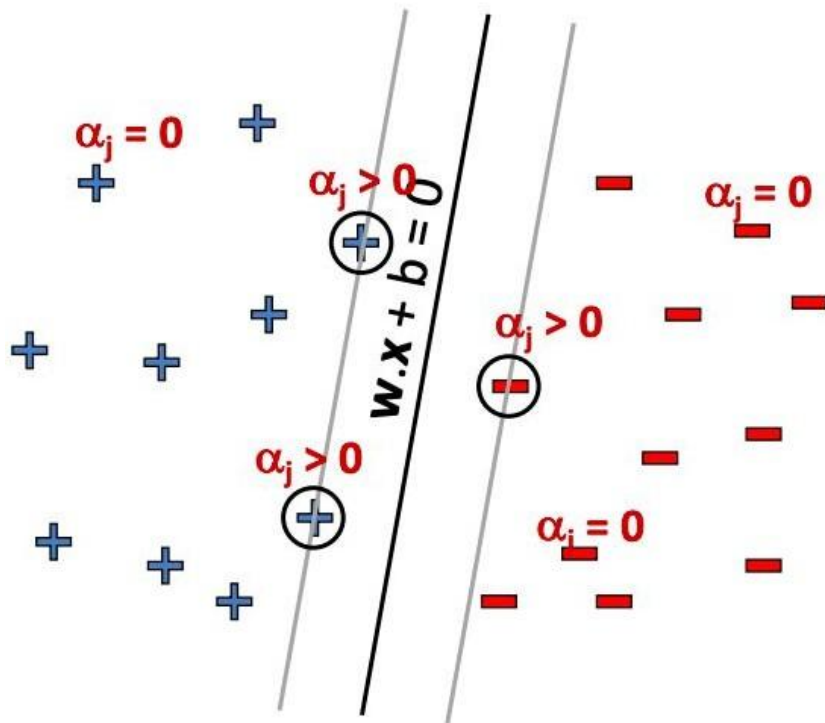
### Lagrange Dual Problem for SVM Primal Problem

**Rewrite the Lagrangian objective.** Let's put these results back into  $\mathcal{L}$  equation in order to eliminate  $w, b$ :

$$\begin{aligned}
 w &= \sum_{i=1}^m \alpha_i y_i x_i \\
 \mathcal{L}(w, b, \alpha) &= \frac{\|w\|^2}{2} - \sum_{i=1}^m \alpha_i (y_i (w^T x_i + b) - 1) \\
 &= \frac{1}{2} (w^T w) - \sum_{i=1}^m \alpha_i y_i w^T x_i - \sum_{i=1}^m \alpha_i y_i b + \sum_{i=1}^m \alpha_i \\
 &= \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j x_i^T x_j - \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j x_i^T x_j + \sum_{i=1}^m \alpha_i \\
 \mathcal{L}(w, b, \alpha) &= \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j x_i^T x_j
 \end{aligned}$$



## Dual SVM: Sparsity of dual solution



$$\mathbf{w} = \sum_j \alpha_j y_j \mathbf{x}_j$$

Only few  $\alpha_j$ s can be non-zero : where constraint is active and tight

$$(\mathbf{w} \cdot \mathbf{x}_j + b) y_j = 1$$

**Support vectors** – training points  $j$  whose  $\alpha_j$ s are non-zero

#### SVM-DUAL OPTIMIZATION PROBLEM

$$\max_{\alpha} W(\alpha) = \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j x_i^T x_j$$

Subject to Constraints:  
for all  $i = 1$  to  $m$  (where  $m$  is the number of Training instances)

Constraint 1 ( $C_1$ ):  $\alpha_i \geq 0$

Constraint 2 ( $C_2$ ):  $\sum_{i=1}^m \alpha_i y_i = 0$

Constraint 3 ( $C_3$ ):  $w^T x^+ + b = +1$

$$w = \sum_{i=1}^m \alpha_i y_i x_i \quad \left[ \sum_{i=1}^m \alpha_i y_i x_i \cdot x \right] + b = +1$$

Constraint 4 ( $C_4$ ):  $w^T x^- + b = -1$

$$\left[ \sum_{i=1}^m \alpha_i y_i x_i \cdot x \right] + b = -1$$

### Karush-Kuhn-Tucker conditions

---

- We previously stressed on equality constraints when introducing Lagrange Multipliers.
- But our SVM is a case of inequality constraints
- When dealing with inequality constraints, there is an additional requirement. The solution must also satisfy the **Karush-Kuhn-Tucker (KKT)** conditions.
- The KKT conditions are first-order necessary conditions for a solution of an optimization problem to be optimal.

$$\begin{aligned}g(X) &\geq 0; \\ \lambda &\geq 0; \\ \lambda g(X) &= 0\end{aligned}$$

- Stationarity condition:

$$\nabla_{\mathbf{w}} \mathcal{L} = \mathbf{w} - \sum_{i=1}^m \alpha_i y_i \mathbf{x}_i = \mathbf{0}$$

$$\frac{\partial \mathcal{L}}{\partial b} = - \sum_{i=1}^m \alpha_i y_i = 0$$

- The stationary condition tells us that the selected point must be a stationary point.
- It is a point where the function stops increasing or decreasing.
- When we have constraints, we use the gradient of the Lagrangian.

### Primal and Dual feasibility condition

---

- Primal feasibility condition:

$$y_i(\mathbf{w} \cdot \mathbf{x}_i + b) - 1 \geq 0 \quad \text{for all } i = 1, \dots, m$$

Looking at this condition, you should recognize the constraints of the primal problem. It makes sense that they must be enforced to find the minimum of the function under constraints.

- Dual feasibility condition:

$$\alpha_i \geq 0 \quad \text{for all } i = 1, \dots, m$$

This tells only those instances which lie on the gutter will have multipliers greater than 0 and for rest of the instances, the value of the Lagrange Multiplier will be 0.

- Complementary slackness condition:

$$\alpha_i [y_i(\mathbf{w} \cdot \mathbf{x}_i + b) - 1] = 0 \quad \text{for all } i = 1, \dots, m$$

From the complementary slackness condition, we see that either  $\alpha_i = 0$  or  $y_i(\mathbf{w} \cdot \mathbf{x}_i + b) - 1 = 0$ .

- Complementary slackness requires only one of the two following cases hold true for a constraint:
  - The constraint is binding (i.e. equality)
  - The Lagrangian multiplier of this constraint is zero

**Support vectors** are examples having a positive Lagrange multiplier. They are the ones for which the constraint  $y_i(\mathbf{w} \cdot \mathbf{x}_i + b) - 1 \geq 0$  is **active**. (We say the constraint is active when  $y_i(\mathbf{w} \cdot \mathbf{x}_i + b) - 1 = 0$ ).



## So why solve the dual SVM?

- There are some quadratic programming algorithms that can solve the dual faster than the primal, (specially in high dimensions  $d \gg n$ )
- But, more importantly, the **“kernel trick”**!!!

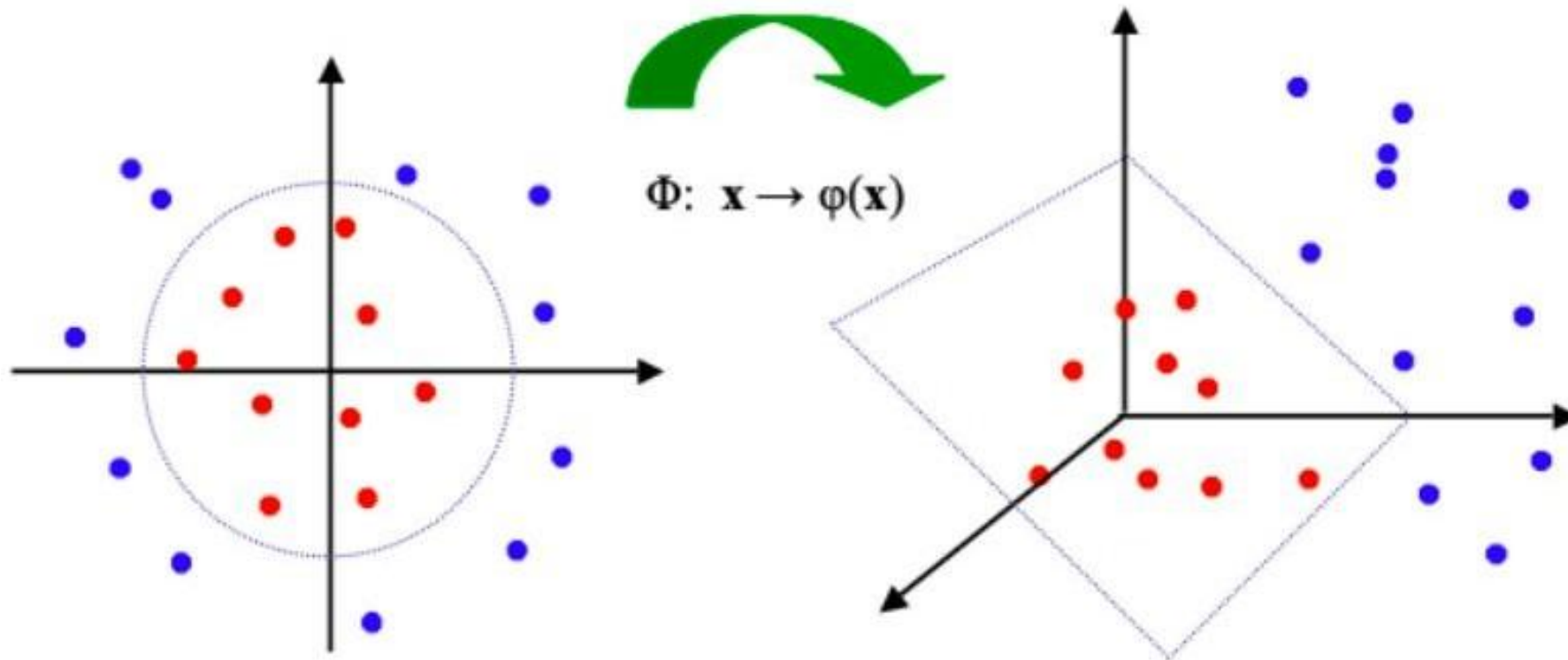


- The classifier is a separating hyperplane.
- Most “important” training points are support vectors; they define the hyperplane.
- Quadratic optimization algorithms can identify which training points  $x_i$  are support vectors with non-zero Lagrangian multipliers.
- Both in the dual formulation of the problem and in the solution training points appear only inside dot products.

### SVM-DUAL OPTIMIZATION PROBLEM

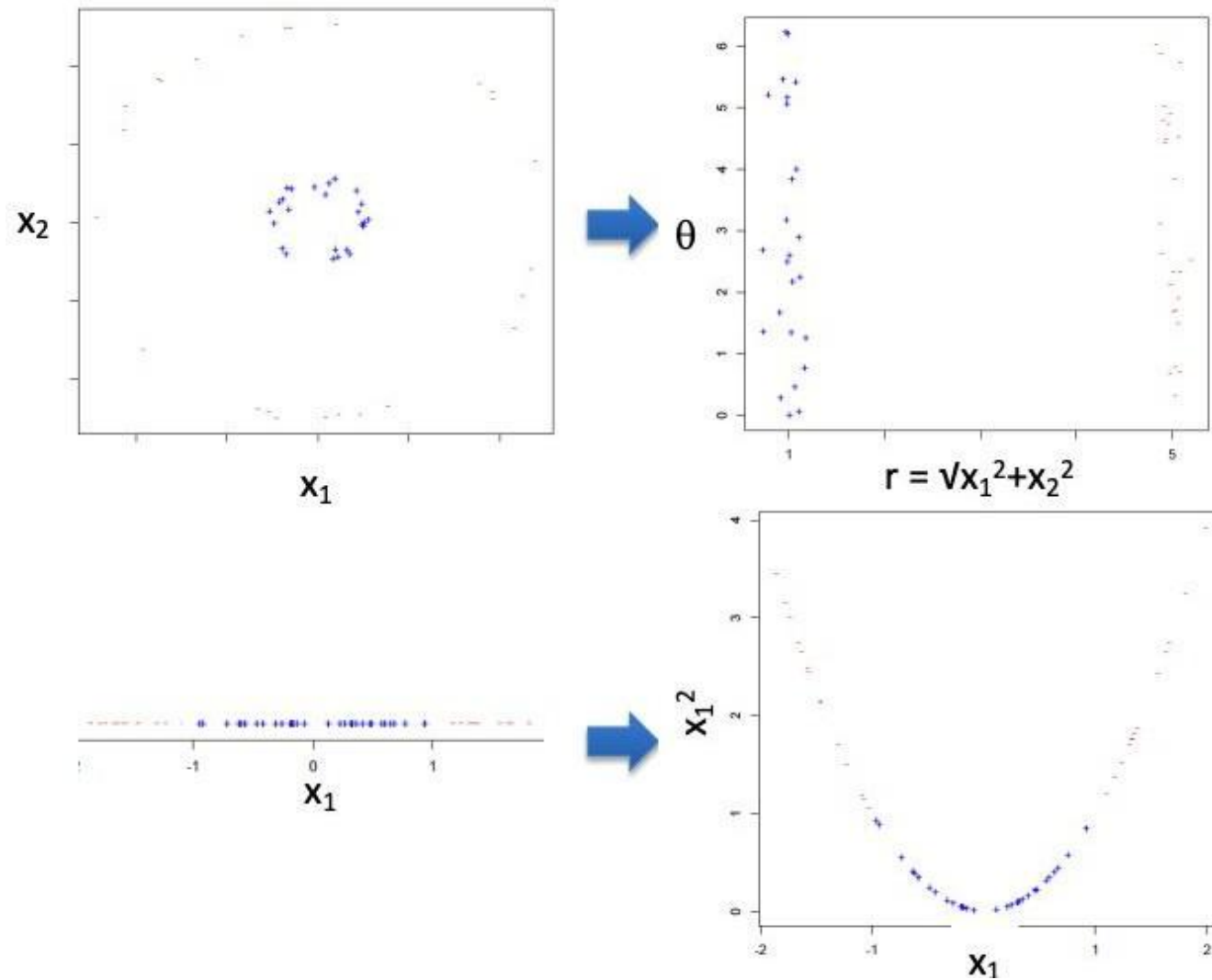
$$\max_{\alpha} W(\alpha) = \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j x_i^T x_j$$

- The original input space can always be mapped to some higher-dimensional feature space where the training set is separable
  - Kernel Trick



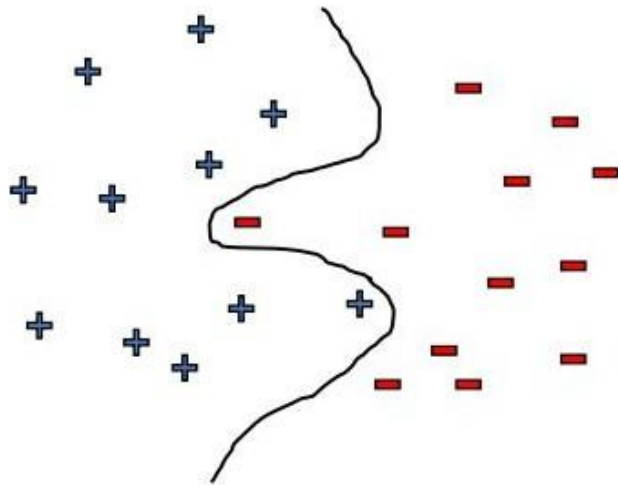
## Unit 2 - SVM

### Non-Linear SVMs



# What if data is not linearly separable?

Use features of features  
of features of features....



$$\Phi(\mathbf{x}) = (x_1^2, x_2^2, x_1x_2, \dots, \exp(x_1))$$

Feature space becomes really large very quickly!

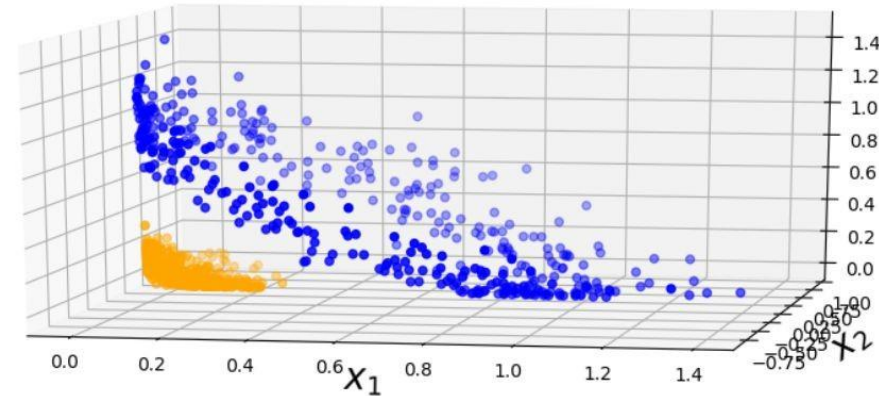
### How to classify non-linearly separable data?

- Even if your original data is in two dimensions, you can transform it before feeding it into the SVM.
- One possible transformation would be to transform every 2D vector into a 3D vector.
- For example, we can do what is called a polynomial mapping by applying the function defined by

$$\phi(x_1, x_2) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)$$

$$\phi : \mathbb{R}^2 \rightarrow \mathbb{R}^3$$

We can see that the data is separable in three dimensions!



You aren't forced to transform data into 3 dimensions.  
It could be 5, 10, or 100 dimensions.

### How to classify non-linearly separable data?

---

- The transformation could also be in the same dimension.
- Let us Construct a support vector machine that computes the XOR function.
- We will use values of +1 and -1 (instead of 1 and 0) for both inputs and outputs, so that an example looks like  $([-1, 1], 1)$  or  $([-1, -1], -1)$ .
- Map the input  $[x_1, x_2]$  into a space consisting of  $x_1$  and  $x_1x_2$ .

### How to classify non-linearly separable data?

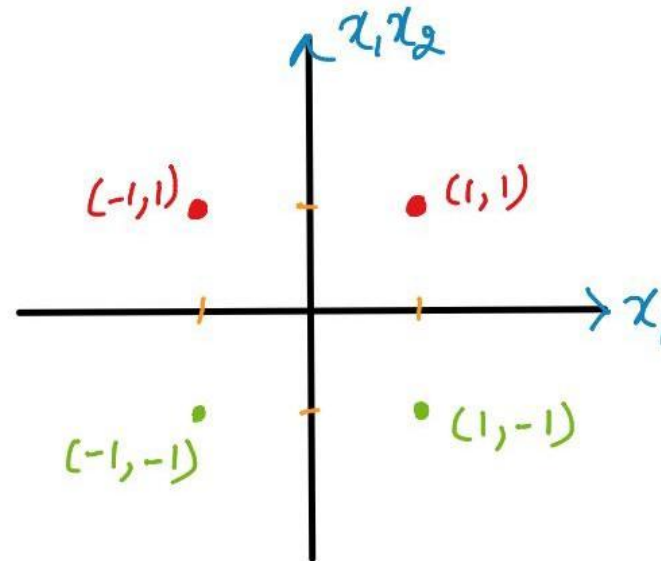
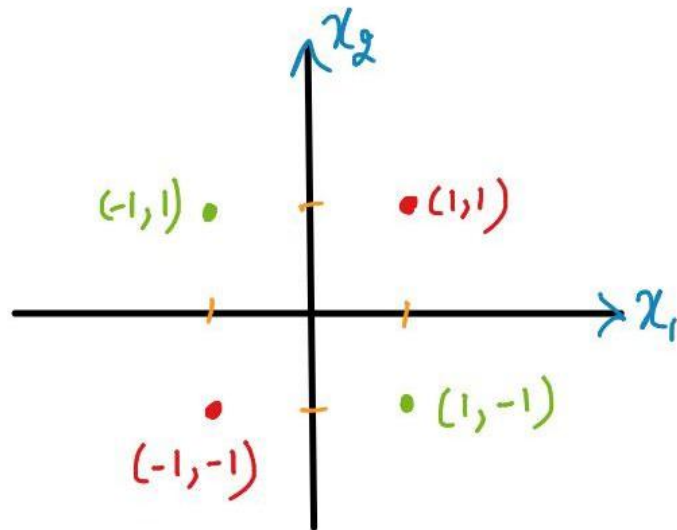
The examples map from  $[x_1, x_2]$  to  $[x_1, x_1 x_2]$  coordinates as follows:

$[-1, -1]$  (negative) maps to  $[-1, +1]$

$[-1, +1]$  (positive) maps to  $[-1, -1]$

$[+1, -1]$  (positive) maps to  $[+1, -1]$

$[+1, +1]$  (negative) maps to  $[+1, +1]$



Thus, the positive examples have  $x_1 x_2 = -1$  and the negative examples have  $x_1 x_2 = +1$ . The maximum margin separator is the line  $x_1 x_2 = 0$ , with a margin of 1.



$$\mathbf{x} = (x_1, x_2, x_3)^T$$
$$\mathbf{y} = (y_1, y_2, y_3)^T$$

Here  $\mathbf{x}$  and  $\mathbf{y}$  are 2 data points in 3D. Let's assume that we need to map  $\mathbf{x}$  and  $\mathbf{y}$  to a 9D space.

We need to do the following calculations to get the final result, which is just a scalar.

$$\phi(\mathbf{x}) = (x_1^2, x_1x_2, x_1x_3, x_2x_1, x_2^2, x_2x_3, x_3x_1, x_3x_2, x_3^2)^T$$
$$\phi(\mathbf{y}) = (y_1^2, y_1y_2, y_1y_3, y_2y_1, y_2^2, y_2y_3, y_3y_1, y_3y_2, y_3^2)^T$$

$$\phi(\mathbf{x})^T \phi(\mathbf{y}) = \sum_{i=1}^3 \sum_{j=1}^3 x_i y_j y_i y_j$$

The computational complexity is  $O(n^2)$ .

If we use the kernel function  $k(x, y)$ , we reach the same result within the 3D space by calculating the dot product of  $x$ -transpose and  $y$  instead of doing the complicated computations in the 9D space.

$$\begin{aligned} k(\mathbf{x}, \mathbf{y}) &= (\mathbf{x}^T \mathbf{y})^2 \\ &= (x_1 y_1 + x_2 y_2 + x_3 y_3)^2 = \sum_{i=1}^3 \sum_{j=1}^3 x_i y_j y_i y_j \end{aligned}$$

The computational complexity is  $O(n)$ .

The kernel trick offers a more efficient and less expensive way to transform data into higher dimensions.

The question is : Why one must take the trouble of projecting the data in  $d$ -dimension to  $m$ -dimension where  $m \gg d$  and then take the dot product and return a scalar??

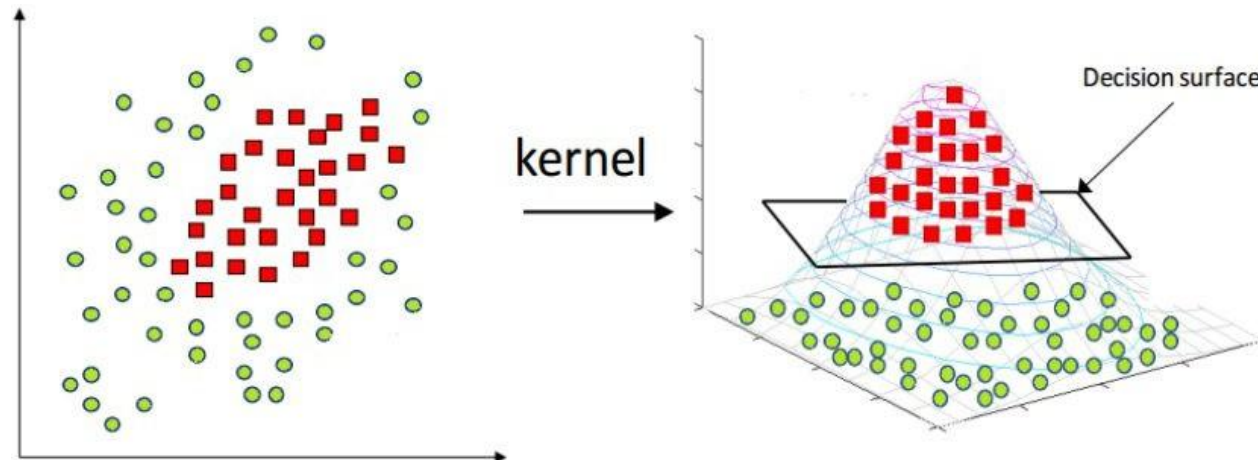
Why not take a function which accepts the vectors in original dimension, does something and gives back the scalar, which instead has the effect of projecting the data to higher dimension and taking the dot product.

This operation is often computationally cheaper than the explicit computation of the coordinates in the higher dimension.. This approach is called the "kernel trick".

Kernel functions ( $K$ ) have been introduced for sequence data, graphs, text, images, as well as vectors.

What is a kernel?

Our kernel function accepts inputs in the original lower dimensional space and returns a scalar that provides the effect of taking the dot product of transformed vectors in the higher dimensional space.



- The “**trick**” is that kernel methods represent the data only through a set of **pairwise similarity comparisons** between the original data observations (with the original coordinates in the lower dimensional space).
- It does this instead of explicitly applying the transformations  $\phi(\mathbf{x})$  and representing the data by these transformed coordinates in the higher dimensional feature space.

#### **Note :**

if you really project the dataset to higher dimension and take the dot product, this value may be different from the one which kernel produces.

Scalar value which the kernel function returns may not be same/close to dot product of scaled input vectors in higher dimension.

Scalar returned by kernel, depends on the type of kernel we are using and how many dimensions we are projecting.

Questions :

- How many dimensions we should transform the data to?
- What should be the functional form of the transformation?
- How do we know which transformation to apply?
- If we do all this, is it guaranteed the dataset will become linearly separable in higher dimension ?

Answers :

- No Clear way of doing this transformation
  - Choosing which transformation to apply depends a lot on your datasets.
  - Unfortunately, there is no perfect recipe, and it will come with experience via trial and error.
- 
- Fortunately, most of the real life datasets have proved so far to require 2-3 varieties of transformations to be applied and small number of variation in dimensions to be made.

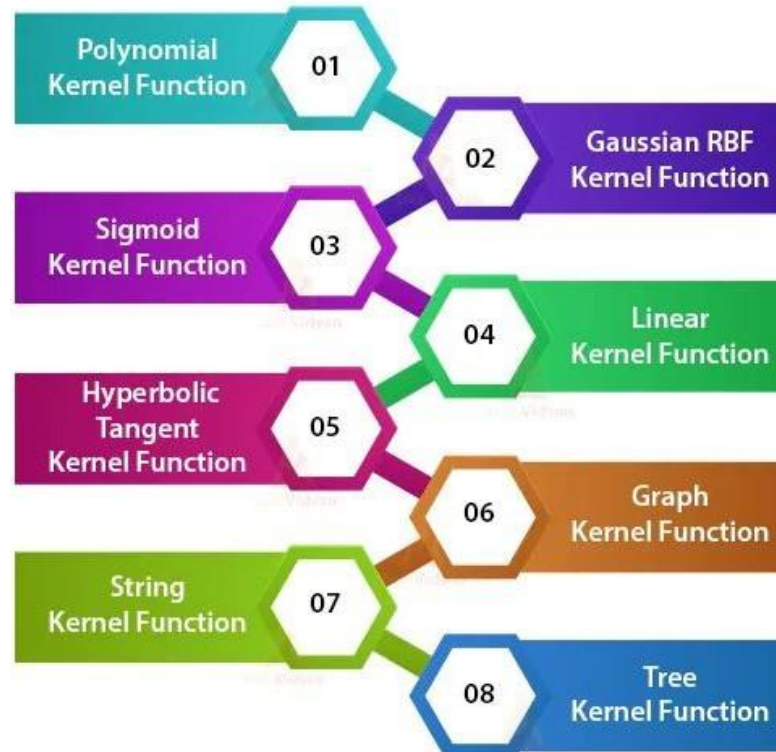
If we define a kernel as  $K(\mathbf{x}_i, \mathbf{x}_j)$ , we rewrite the dual problem as :

$$\underset{\alpha}{\text{maximize}} \quad \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j K(\mathbf{x}_i, \mathbf{x}_j)$$



- A kernel function is some function that corresponds to an inner product in some expanded feature space. Following are few examples of kernel functions:

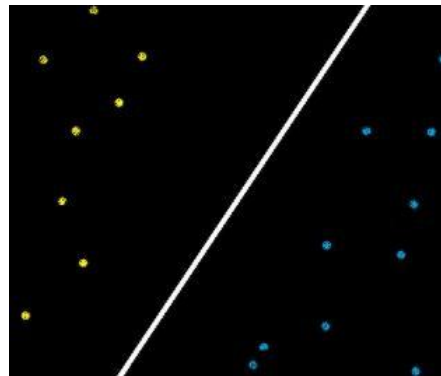
### ***Types of Kernel Functions***



- Used when the data is linearly separable.
- It is mostly used when there are a large number of features in a dataset.
- One of the examples where there are a lot of features is **text classification** where each alphabet is a new feature.

$$K(\mathbf{x}, \mathbf{x}') = \mathbf{x} \cdot \mathbf{x}'$$

$\mathbf{x}$  and  $\mathbf{x}'$  are 2 vectors

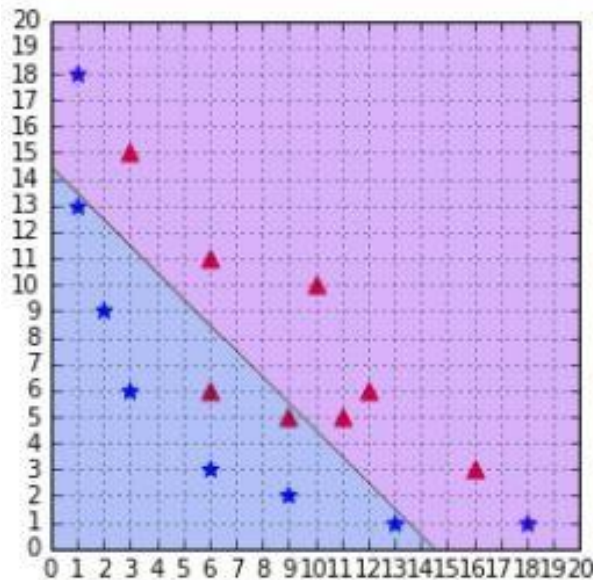


There two set of features **Blue** and **Yellow** features.  
Since they are linearly separable, we can use the linear  
kernel.

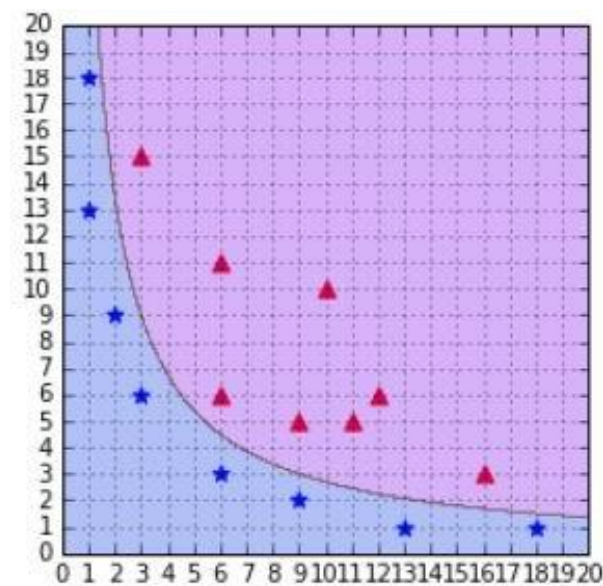
$$K(\mathbf{x}, \mathbf{x}') = (\mathbf{x} \cdot \mathbf{x}' + c)^d$$

c represents a **constant**.

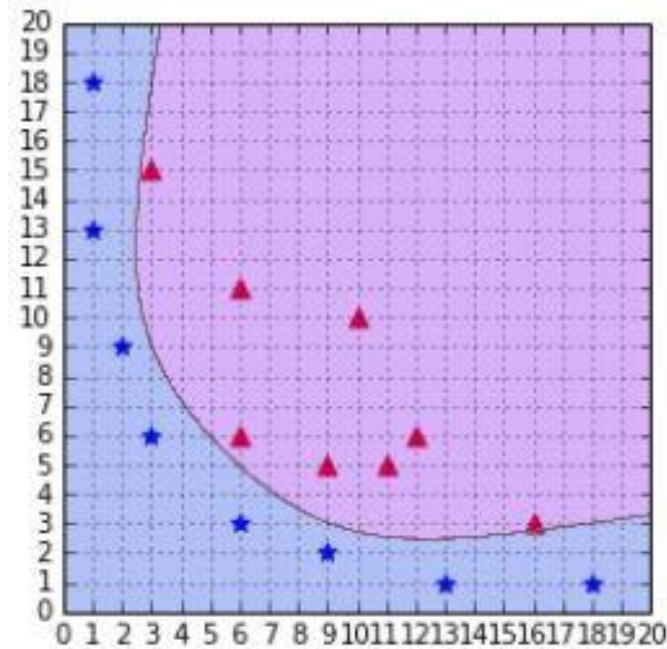
d represents the **degree** of the kernel.



A polynomial of degree 1 and no constant is the linear kernel unable to separate data points.



A polynomial of degree 2 is able to separate data points.

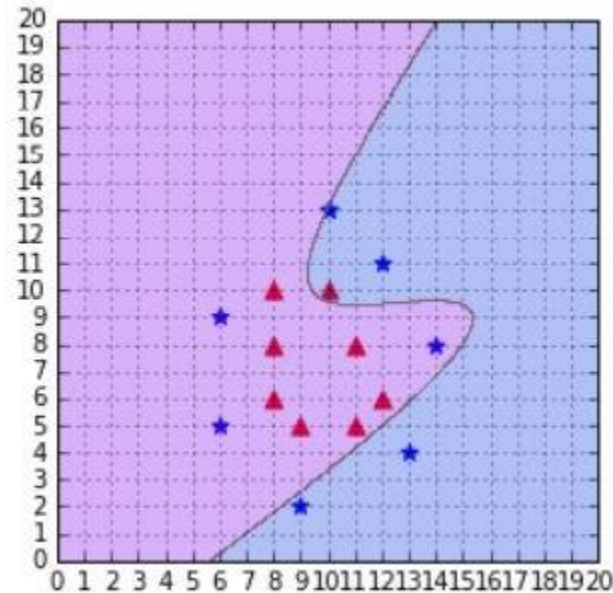
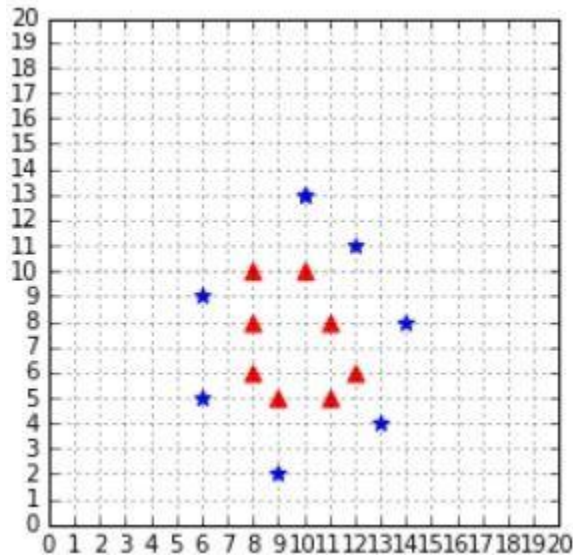


- On increasing the degree of a polynomial kernel, the decision boundary will become more complex.
- It will have a tendency to be influenced by individual data examples.
- Using a **high-degree** polynomial leads to **overfitting**.

## Unit 2 - SVM

### Gaussian / RBF kernel

- Sometimes polynomial kernels are **not sophisticated enough** to work.
- That's when Gaussian kernels come to the rescue.



A polynomial kernel is unable to separate the data (degree = 3, C = 100)

$$K(\mathbf{x}, \mathbf{x}') = \exp(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2)$$

A radial basis functions **areal valued function** whose value depends only on:

- The distance between the input and some fixed point, either the origin, so that  $\varphi(\mathbf{x}) = \varphi(\|\mathbf{x}\|)$
- Or some other fixed point  $\mathbf{c}$  called a center, so that  $\varphi(\mathbf{x}) = \varphi(\|\mathbf{x} - \mathbf{c}\|)$
- Intuitively, the gamma parameter defines how far the influence of a single training example reaches, with **low values** meaning '**far**' and **high values** meaning '**close**'.
- The gamma parameters can be seen as the inverse of the radius of influence of samples selected by the model as support vectors.



- Other than the standard kernel functions, we can define our own kernels as well as combining two or more kernels to another kernel.
- It is interesting to note the following properties. If  $K_1$  and  $K_2$  are two kernels then,

- 1  $K_1 + c$
- 2  $aK_1$
- 3  $aK_1 + bK_2$
- 4  $K_1 \cdot K_2$

are also kernels. Here,  $a$ ,  $b$  and  $c \in \mathbb{R}^+$ .



### Which kernel to use?

---

- The recommended approach is to try a RBF kernel first, because it usually works well.
- However, it is good to try the other types of kernels if you have enough time.
- A kernel is a measure of the similarity between two vectors, so that is where domain knowledge of the problem at hand may have the biggest impact.

## Problems on SVM

### Classification Problems

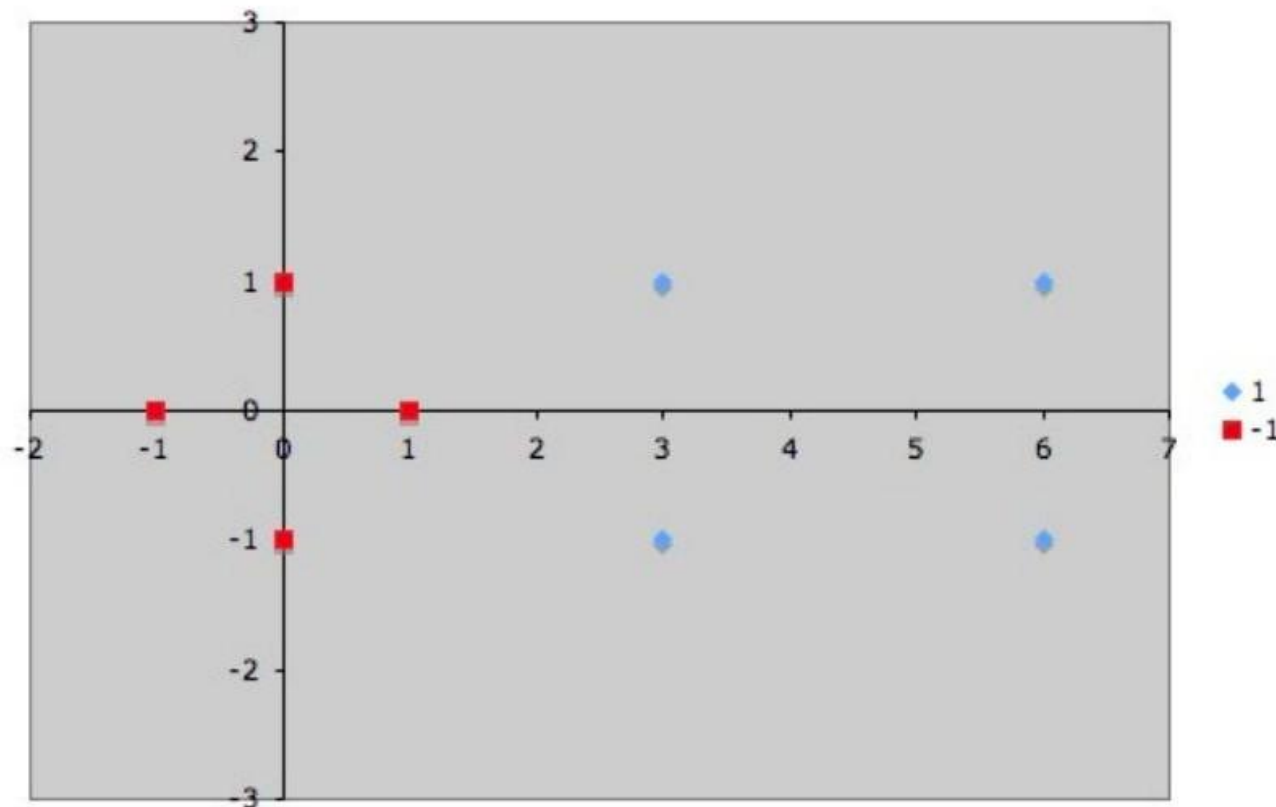
1. Is the data linearly separable?
2. Discover a simple SVM that accurately discriminates the two classes.

| point  | class |
|--------|-------|
| (3,1)  | +     |
| (3,-1) | +     |
| (6,1)  | +     |
| (6,-1) | +     |
| (1,0)  | -     |
| (0,1)  | -     |
| (0,-1) | -     |
| (-1,0) | -     |

### Problem - 1 Solution

1. Is the data linearly separable?

Plot the data and examine whether the data is linearly separable or not. The data provided is linearly separable.



| point  | class |
|--------|-------|
| (3,1)  | +     |
| (3,-1) | +     |
| (6,1)  | +     |
| (6,-1) | +     |
| (1,0)  | -     |
| (0,1)  | -     |
| (0,-1) | -     |
| (-1,0) | -     |

2. Discover a simple SVM that accurately discriminates the two classes.

**Ideally we must solve the dual problem or primal problem, because we don't know which instances are support vectors. (SMO algorithm gives an efficient way of solving the dual problem of the support vector machine optimization problem --- out of scope)**

But for these toy examples, we are able to manually inspect which are the support vectors, we can use the binding constraints in order to solve for  $w$  and  $b$ :

$$\mathbf{w}^T \mathbf{X} + b = -1$$

$$\mathbf{w}^T \mathbf{X} + b = 1$$

**Two ways to Solve :**

Way 1 : Directly substitute the support vectors in above equations and get  $w$  and  $b$ .

Way 2 : Find Lagrange Multipliers and then find  $w$  and  $b$ .

## Unit 2 - SVM

### Problem - 1 Solution



2. Discover a simple SVM that accurately discriminates the two classes.

**Way 1 - Solving for  $w$  and  $b$  using the binding constraint equations :**

$$w^T X + b = -1$$

$$w^T X + b = 1$$

Here,  $X$  is a support vector.

for our question, we have the following support vectors:-

$$S_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad S_2 = \begin{bmatrix} 3 \\ 1 \end{bmatrix} \quad S_3 = \begin{bmatrix} 3 \\ -1 \end{bmatrix}$$

$$y_1 = -1 \quad y_2 = +1 \quad y_3 = +1$$

we get 3 equations

$$w^T S_1 + b = -1$$

$$w^T S_2 + b = +1$$

$$w^T S_3 + b = +1$$

final equations:

$$w_1 + b = -1$$

$$3w_1 + w_2 + b = 1$$

$$3w_1 - w_2 + b = 1$$

## Unit 2 - SVM

### Problem - 1 Solution



2. Discover a simple SVM that accurately discriminates the two classes.

**Way 1 - Solving for  $w$  and  $b$  using the binding constraint equations :**

$$\mathbf{w}^T \mathbf{X} + b = -1$$

$$\mathbf{w}^T \mathbf{X} + b = 1$$

Here,  $\mathbf{X}$  is a support vector.

final equations:

$$\textcircled{1} w_1 + b = -1$$

$$\textcircled{2} 3w_1 + w_2 + b = 1$$

$$\textcircled{3} 3w_1 - w_2 + b = 1$$

Solving above equations

$$\text{Eqn. } \textcircled{2} + \textcircled{3}$$

$$6w_1 + 2b = 2$$

from eqn.  $\textcircled{1}$  we know  $(w_1 = -1 - b)$

$$\Rightarrow 6(-1 - b) + 2b = 2$$

$$\Rightarrow -6 - 6b + 2b = 2$$

$$\Rightarrow 4b = -8$$

$$\Rightarrow \boxed{b = -2}$$

Using eqn  $\textcircled{1}$  we get

$$w_1 = -1 - b \\ = -1 - (-2)$$

$$\boxed{w_1 = 1}$$

using eqn.  $\textcircled{3}$  we get

$$3w_1 - w_2 + b = 1$$

$$\Rightarrow 3(1) - w_2 + (-2) = 1$$

$$\Rightarrow 1 - w_2 = 1$$

$$\Rightarrow \boxed{w_2 = 0}$$

$$\therefore \mathbf{w} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \text{ and bias} = -2$$



2. Discover a simple SVM that accurately discriminates the two classes.

**Way 2 - Finding Lagrange Multipliers first and then solving for  $w$  and  $b$  :**

Step 1 : Identify the support vectors by manual inspection (could be provided in the question)

Step 2 : Write the system of equations using SVM Constraints (C2, C3, C4)

Step 4 : Solve the system of equations to find the value of Lagrange Multipliers and bias.

Step 5 : Use the Lagrange Multipliers to find the weight vector using the formula :

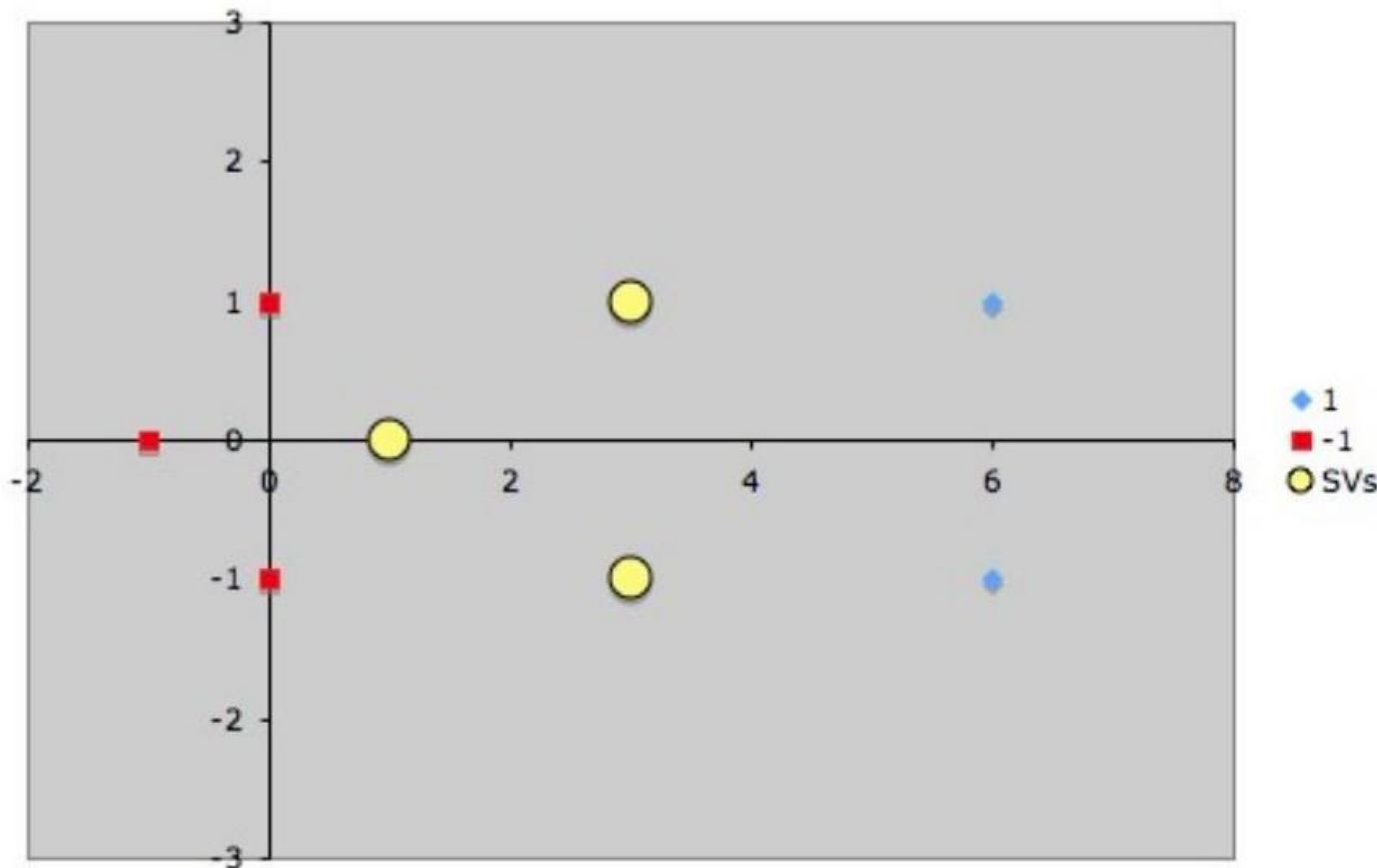
$$w = \sum_{i=1}^m \alpha_i y_i x_i$$

Step 6 : Report the weight vector and bias value as the Solution.

Step 7 : Given any new instance  $U$  to classify, plug the weight and bias value in the equation  $Z = w^T U + b$  and provide your prediction for the label of  $U$  as +ve if  $Z \geq 1$ , else if  $Z \leq -1$ , predict  $U$  as belonging to the -ve class.

2. Discover a simple SVM that accurately discriminates the two classes.

Step 1 : Identify the support vectors by manual inspection



| point  | class |
|--------|-------|
| (3,1)  | +     |
| (3,-1) | +     |
| (6,1)  | +     |
| (6,-1) | +     |
| (1,0)  | -     |
| (0,1)  | -     |
| (0,-1) | -     |
| (-1,0) | -     |

## Unit 2 - SVM

### Problem - 1 Solution



2. Discover a simple SVM that accurately discriminates the two classes.

Step 2a : Write the system of equations using SVM constraints  $c_2, c_3, c_4$

Subject to Constraints:  
for all  $i = 1$  to  $m$  (where  $m$  is the number of Training instances)  
Constraint 1 ( $C_1$ ):  $\alpha_i \geq 0$

Constraint 2 ( $C_2$ ):  $\sum_{i=1}^m \alpha_i y_i = 0$

Constraint 3 ( $C_3$ ):  $w^T x^+ + b = +1$

$w = \sum_{i=1}^m \alpha_i y_i x_i$   $\left[ \sum_{i=1}^m \alpha_i y_i x_i \cdot x \right] + b = +1$

Constraint 4 ( $C_4$ ):  $w^T x^- + b = -1$

$\left[ \sum_{i=1}^m \alpha_i y_i x_i \cdot x \right] + b = -1$

| point  | class |
|--------|-------|
| (3,1)  | +     |
| (3,-1) | +     |
| (6,1)  | +     |
| (6,-1) | +     |
| (1,0)  | -     |
| (0,1)  | -     |
| (0,-1) | -     |
| (-1,0) | -     |

2. Discover a simple SVM that accurately discriminates the two classes.

Step 2a : Plug the support vectors into the constraint equations c2, c3, c4

Because for other instances, Lagrange multiplier is Zero.

- $-\alpha_1 s_1.s_1 + \alpha_2 s_1.s_2 + \alpha_3 s_1 s_3 + b = -1$
- $-\alpha_1 s_2.s_1 + \alpha_2 s_2.s_2 + \alpha_3 s_2 s_3 + b = 1$
- $-\alpha_1 s_3.s_1 + \alpha_2 s_3.s_2 + \alpha_3 s_3 s_3 + b = 1$
- $-\alpha_1 + \alpha_2 + \alpha_3 + 0.b = 0$

| Support Vector | point  | class |
|----------------|--------|-------|
| s1             | (1,0)  | -1    |
| s2             | (3,1)  | +1    |
| s3             | (3,-1) | +1    |

2. Discover a simple SVM that accurately discriminates the two classes.

Step 3 : Solve the equations to find the value of Lagrange Multipliers and bias

- $-\alpha_1 + 3\alpha_2 + 3\alpha_3 + b = -1$
- $-3\alpha_1 + 10\alpha_2 + 8\alpha_3 + b = 1$
- $-3\alpha_1 + 8\alpha_2 + 10\alpha_3 + b = 1$
- $-\alpha_1 + \alpha_2 + \alpha_3 + 0.b = 0$

| Support Vector | point  | class |
|----------------|--------|-------|
| s1             | (1,0)  | -1    |
| s2             | (3,1)  | +1    |
| s3             | (3,-1) | +1    |

2. Discover a simple SVM that accurately discriminates the two classes.

Step 3 : Solve the equations to find the value of Lagrange Multipliers and bias

$$-\alpha_1 + 3\alpha_2 + 3\alpha_3 + b = -1 \quad \text{---(1)}$$

$$-3\alpha_1 + 10\alpha_2 + 8\alpha_3 + b = 1 \quad \text{---(2)}$$

$$-3\alpha_1 + 8\alpha_2 + 10\alpha_3 + b = 1 \quad \text{---(3)}$$

$$-\alpha_1 + \alpha_2 + \alpha_3 = 0 \quad \text{---(4)}$$

Eqn. (2) - (3)  $2\alpha_2 - 2\alpha_3 = 0 \Rightarrow \alpha_2 - \alpha_3 = 0 \Rightarrow \alpha_2 = \alpha_3$

Substitute  $\alpha_2 = \alpha_3$  in (4)  $\Rightarrow \alpha_1 = \alpha_3 + \alpha_3 \Rightarrow \alpha_1 = 2\alpha_3$

Substitute  $\alpha_1 = 2\alpha_3$  ,  $\alpha_2 = \alpha_3$  in equation (1) & (2)

Eqn. 1 :  $-2\alpha_3 + 3\alpha_3 + 3\alpha_3 + b = -1 \Rightarrow 4\alpha_3 + b = -1 \quad \text{---(5)}$

Eqn. 2 :  $-6\alpha_3 + 10\alpha_3 + 8\alpha_3 + b = 1 \Rightarrow 12\alpha_3 + b = 1 \quad \text{---(6)}$

Eqn. (5) - (6)  $-8\alpha_3 = -2 \Rightarrow \alpha_3 = 1/4$

$$\alpha_1 = 1/2, \alpha_2 = 1/4, \alpha_3 = 1/4, b = -2$$

2. Discover a simple SVM that accurately discriminates the two classes.

Step 3 : Solve the equations to find the value of Lagrange Multipliers and bias

solving this equations with 4 unknowns we get

- $b = -2$
- $\alpha_1 = \frac{1}{2}$
- $\alpha_2 = \frac{1}{4}$
- $\alpha_3 = \frac{1}{4}$

| Support Vector | point  | class |
|----------------|--------|-------|
| s1             | (1,0)  | -1    |
| s2             | (3,1)  | +1    |
| s3             | (3,-1) | +1    |



2. Discover a simple SVM that accurately discriminates the two classes.

Step 4 : Find the weight vector

$$w = \sum_{i=1}^m \alpha_i y_i x_i$$

$$w = \frac{1}{2} \times -1 \times \begin{pmatrix} 1 \\ 0 \end{pmatrix} + \frac{1}{4} \times 1 \times \begin{pmatrix} 3 \\ 1 \end{pmatrix} + \frac{1}{4} \times 1 \times \begin{pmatrix} 3 \\ -1 \end{pmatrix}$$

$$= \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

| Support Vector | point  | class |
|----------------|--------|-------|
| s1             | (1,0)  | -1    |
| s2             | (3,1)  | +1    |
| s3             | (3,-1) | +1    |

Final solution : Provide weight vector and bias value.

## Unit 2 - SVM

### Problem - 2



Discover a simple SVM that accurately discriminates the two classes of the given data set.

Use the Model to Classify the following query points  
(0.3,0.3), (0.5,0.5) ,(0.8,0.8)

Lets assign

- + as 1
- - as -1

Support vectors : (Through Visual Inspection)

$s_1=(0.38,0.47)$      $s_2=(0.49,0.61)$

| $A_1$ | $A_2$ | $y$ |
|-------|-------|-----|
| 0.38  | 0.47  | +   |
| 0.49  | 0.61  | -   |
| 0.92  | 0.41  | -   |
| 0.74  | 0.89  | -   |
| 0.18  | 0.58  | +   |
| 0.41  | 0.35  | +   |
| 0.93  | 0.81  | -   |
| 0.21  | 0.10  | +   |

## Unit 2 - SVM

### Problem - 1 Solution



2. Discover a simple SVM that accurately discriminates the two classes.

**Way 1 - Solving for  $w$  and  $b$  using the binding constraint equations :**

$$\mathbf{w}^T \mathbf{X} + b = -1$$

$$\mathbf{w}^T \mathbf{X} + b = 1$$

Here,  $\mathbf{X}$  is a support vector.

$$\mathbf{S}_1 = \begin{bmatrix} 0.38 \\ 0.47 \end{bmatrix} \quad \mathbf{S}_2 = \begin{bmatrix} 0.49 \\ 0.61 \end{bmatrix}$$

$$y_1 = +1 \quad y_2 = -1$$

$$\mathbf{w}^T \mathbf{S}_1 + b = 1$$

$$\mathbf{w}^T \mathbf{S}_2 + b = -1$$

$$0.38w_1 + 0.47w_2 + b = 1 \quad | \quad 0.49w_1 + 0.61w_2 + b = -1$$

Eliminate  $b$  from equation

$$0.38w_1 + 0.47w_2 = 1 \quad | \quad 0.49w_1 + 0.61w_2 = -1$$

Solving these equations

$$w_1 = -6.94$$

$$w_2 = -8.83$$

and

$$b = 7.79$$

## Unit 2 - SVM

### Problem 2 - Way 2 - finding Lagrange Multipliers and then solving for w and b

$$S_1 = \begin{bmatrix} 0.38 \\ 0.47 \end{bmatrix} \quad S_2 = \begin{bmatrix} 0.49 \\ 0.61 \end{bmatrix}$$
$$y_1 = +1 \quad y_2 = -1$$

Constraint      Equation

$$C2: \alpha_1 y_1 + \alpha_2 y_2 = 0 \Rightarrow \alpha_1 - \alpha_2 = 0 \Rightarrow \alpha_1 = \alpha_2$$

$$C3: \alpha_1 y_1 S_1^T \cdot S_1 + \alpha_2 y_2 S_2^T \cdot S_1 + b = 1$$

$$\Rightarrow \alpha_1 \begin{bmatrix} 0.38 & 0.47 \end{bmatrix} \begin{bmatrix} 0.38 \\ 0.47 \end{bmatrix} - \alpha_2 \begin{bmatrix} 0.49 & 0.61 \end{bmatrix} \begin{bmatrix} 0.38 \\ 0.47 \end{bmatrix} + b = 1$$

$$\Rightarrow 0.3653\alpha_1 - 0.4729\alpha_2 + b = 1$$

$$C4: \alpha_1 y_1 S_1 \cdot S_2 + \alpha_2 y_2 S_2 \cdot S_2 + b = -1$$

$$\Rightarrow \alpha_1 \begin{bmatrix} 0.38 & 0.47 \end{bmatrix} \begin{bmatrix} 0.49 \\ 0.61 \end{bmatrix} - \alpha_2 \begin{bmatrix} 0.49 & 0.61 \end{bmatrix} \begin{bmatrix} 0.49 \\ 0.61 \end{bmatrix} + b = -1$$

$$\Rightarrow 0.4729\alpha_1 - 0.6122\alpha_2 + b = -1$$

| $A_1$ | $A_2$ | $y$ |
|-------|-------|-----|
| 0.38  | 0.47  | +   |
| 0.49  | 0.61  | -   |
| 0.92  | 0.41  | -   |
| 0.74  | 0.89  | -   |
| 0.18  | 0.58  | +   |
| 0.41  | 0.35  | +   |
| 0.93  | 0.81  | -   |
| 0.21  | 0.10  | +   |

final equations:

$$\textcircled{1} \quad \alpha_1 = \alpha_2$$

$$\textcircled{2} \quad 0.3653 \alpha_1 - 0.4729 \alpha_2 + b = 1$$

$$\textcircled{3} \quad 0.4729 \alpha_1 - 0.6122 \alpha_2 + b = -1$$

Substituting  $\alpha_2 = \alpha_1$  in  $\textcircled{2}$  &  $\textcircled{3}$  we get

$$0.3653 \alpha_1 - 0.4729 \alpha_1 + b = 1 \Rightarrow -0.1076 \alpha_1 + b = 1$$

$$0.4729 \alpha_1 - 0.6122 \alpha_1 + b = -1 \Rightarrow -0.1393 \alpha_1 + b = -1$$

$$\alpha_1 = \alpha_2 = 63.09, \quad b = 7.7886$$

| $A_1$ | $A_2$ | $y$ |
|-------|-------|-----|
| 0.38  | 0.47  | +   |
| 0.49  | 0.61  | -   |
| 0.92  | 0.41  | -   |
| 0.74  | 0.89  | -   |
| 0.18  | 0.58  | +   |
| 0.41  | 0.35  | +   |
| 0.93  | 0.81  | -   |
| 0.21  | 0.10  | +   |



finding weight vector

$$w = \sum_{i=1}^m \alpha_i y_i s_i$$

$$= \alpha_1 y_1 s_1 + \alpha_2 y_2 s_2$$

$$= 63.09 \begin{bmatrix} 0.38 \\ 0.47 \end{bmatrix} - 63.09 \begin{bmatrix} 0.49 \\ 0.61 \end{bmatrix}$$

$$= 63.09 \begin{bmatrix} 0.38 - 0.49 \\ 0.47 - 0.61 \end{bmatrix}$$

$$= \begin{bmatrix} -6.9399 \\ -8.8326 \end{bmatrix}$$

| $A_1$ | $A_2$ | $y$ |
|-------|-------|-----|
| 0.38  | 0.47  | +   |
| 0.49  | 0.61  | -   |
| 0.92  | 0.41  | -   |
| 0.74  | 0.89  | -   |
| 0.18  | 0.58  | +   |
| 0.41  | 0.35  | +   |
| 0.93  | 0.81  | -   |
| 0.21  | 0.10  | +   |

## Unit 2 - SVM

### Problem 2 - Solution



Using the Model to Classify the following query points (0.3,0.3), (0.5,0.5), (0.8,0.8)

$$w = \begin{bmatrix} -6.9399 \\ -8.8326 \end{bmatrix}, b = 7.7886$$

Classifying query points :  $W^T X + b$

$$X_1 = \begin{bmatrix} 0.3 \\ 0.3 \end{bmatrix} \quad X_2 = \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix} \quad X_3 = \begin{bmatrix} 0.8 \\ 0.8 \end{bmatrix}$$

Finding  $Z = W^T X + b$

$$Z = 3.05685$$

$$Z = -0.09765$$

$$Z = -4.8294$$

Predicted Class

$$\because Z > 1$$

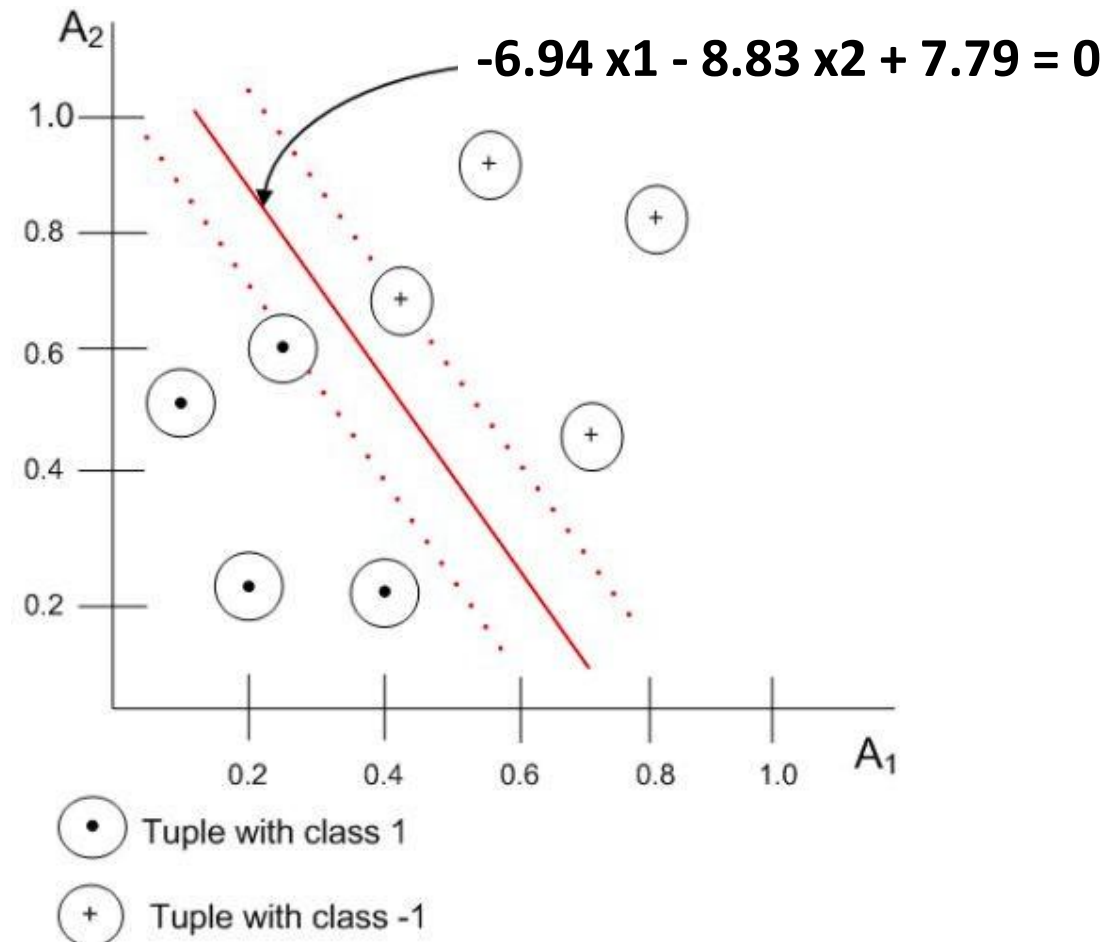
$X_1$  is positive

$$\because -1 < Z < 1$$

$X_2$  falls in the canal  
 $X_2$  predicted -ve with low confidence

$$\because Z < -1$$

$X_3$  is negative



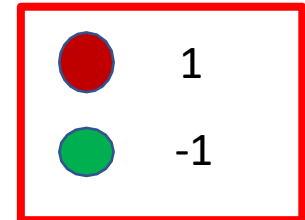
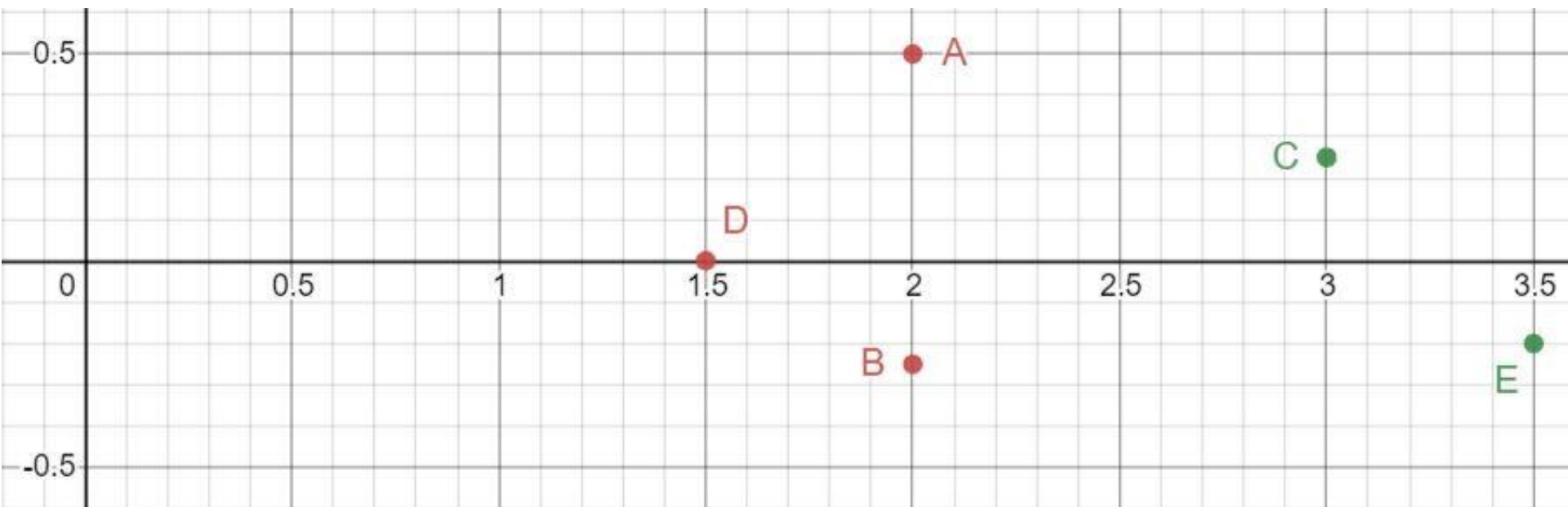


## Unit 2 - SVM

### Problem - 3 -- Homework



Consider the points A, B, C, D, E. Find  $\mathbf{w}$  and  $\mathbf{b}$ .



## Unit 2 - SVM

### Problem - 3 -- Homework

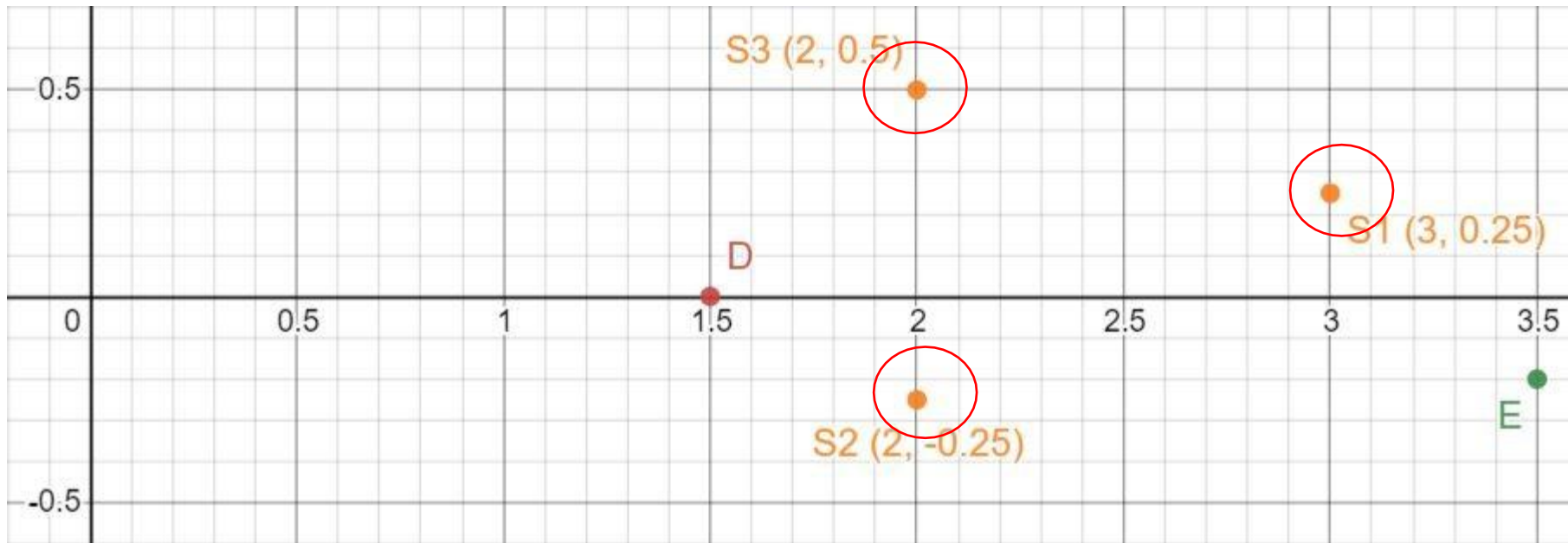


We have 3 support vectors

Lets assign

- + as 1
- - as -1

| Point | Label |
|-------|-------|
| S1    | -     |
| S2    | +     |
| S3    | +     |



## Unit 2 - SVM

### Problem 4 - Non - Linear Data



Given the data ,

$\begin{pmatrix} 1 \\ 1 \end{pmatrix} \begin{pmatrix} -1 \\ 1 \end{pmatrix} \begin{pmatrix} -1 \\ -1 \end{pmatrix} \begin{pmatrix} 1 \\ -1 \end{pmatrix}$  belongs to +ve class

$\begin{pmatrix} 2 \\ 0 \end{pmatrix} \begin{pmatrix} 0 \\ 2 \end{pmatrix} \begin{pmatrix} -2 \\ 0 \end{pmatrix} \begin{pmatrix} 0 \\ -2 \end{pmatrix}$  belongs to -ve class

- a) Plot the data. Is it linearly separable?
- b) Apply the following transformation if not linearly separable

$$\phi(X) = \begin{cases} 6 - x_1 + (x_1 - x_2)^2 \\ 6 - x_2 + (x_1 - x_2)^2 \end{cases} \text{ if } \sqrt{x_1^2 + x_2^2} \geq 2$$

$x_1, x_2$  otherwise

- c) Discover a simple SVM <sup>(no change)</sup> model to perform the classification.

## Unit 2 - SVM

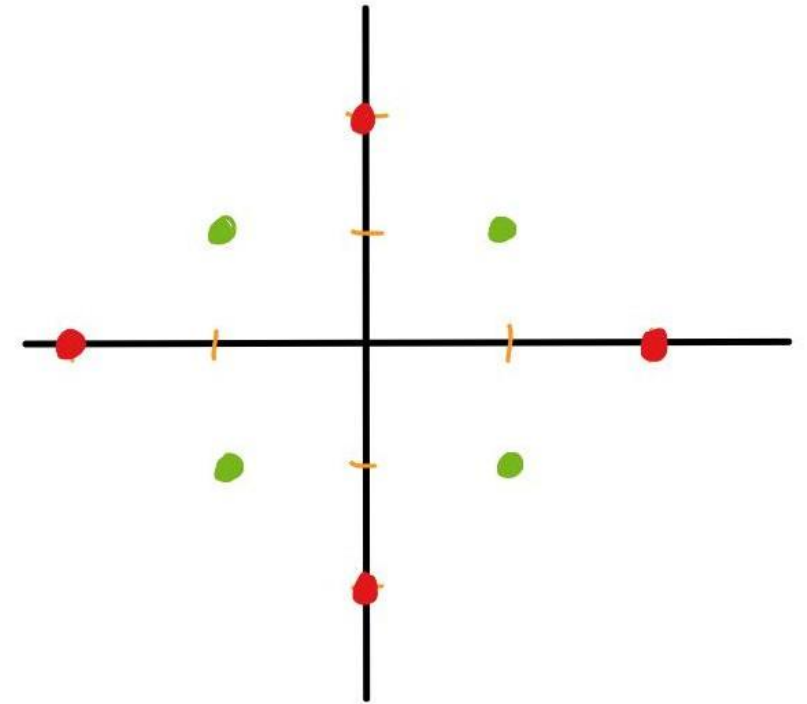
### Problem 4 - Non - Linear Data



Given the data ,

$\begin{pmatrix} 1 \\ 1 \end{pmatrix}$   $\begin{pmatrix} -1 \\ 1 \end{pmatrix}$   $\begin{pmatrix} -1 \\ -1 \end{pmatrix}$   $\begin{pmatrix} 1 \\ -1 \end{pmatrix}$  belongs to +ve class

$\begin{pmatrix} 2 \\ 0 \end{pmatrix}$   $\begin{pmatrix} 0 \\ 2 \end{pmatrix}$   $\begin{pmatrix} -2 \\ 0 \end{pmatrix}$   $\begin{pmatrix} 0 \\ -2 \end{pmatrix}$  belongs to -ve class



## Unit 2 - SVM

### Problem 4 - Non - Linear Data



Given the data,

$\begin{pmatrix} 1 \\ 1 \end{pmatrix} \begin{pmatrix} -1 \\ 1 \end{pmatrix} \begin{pmatrix} -1 \\ -1 \end{pmatrix} \begin{pmatrix} 1 \\ -1 \end{pmatrix}$  belongs to +ve class

$\begin{pmatrix} 2 \\ 0 \end{pmatrix} \begin{pmatrix} 0 \\ 2 \end{pmatrix} \begin{pmatrix} -2 \\ 0 \end{pmatrix} \begin{pmatrix} 0 \\ -2 \end{pmatrix}$  belongs to -ve class

Negative points are transformed as:

$$\phi\left(\begin{pmatrix} 2 \\ 0 \end{pmatrix}\right) = \begin{pmatrix} 6 - 2 + 2^2 \\ 6 - 0 + 2^2 \end{pmatrix} = \begin{pmatrix} 8 \\ 10 \end{pmatrix}$$

$$\phi\left(\begin{pmatrix} 0 \\ 2 \end{pmatrix}\right) = \begin{pmatrix} 6 - 0 + (-2)^2 \\ 6 - 2 + (-2)^2 \end{pmatrix} = \begin{pmatrix} 10 \\ 8 \end{pmatrix}$$

$$\phi\left(\begin{pmatrix} -2 \\ 0 \end{pmatrix}\right) = \begin{pmatrix} 6 + 2 + (-2)^2 \\ 6 - 0 + (-2)^2 \end{pmatrix} = \begin{pmatrix} 12 \\ 10 \end{pmatrix}$$

$$\phi\left(\begin{pmatrix} 0 \\ -2 \end{pmatrix}\right) = \begin{pmatrix} 6 - 0 + 2^2 \\ 6 + 2 + 2^2 \end{pmatrix} = \begin{pmatrix} 10 \\ 12 \end{pmatrix}$$

∴ In

Apply Transformation

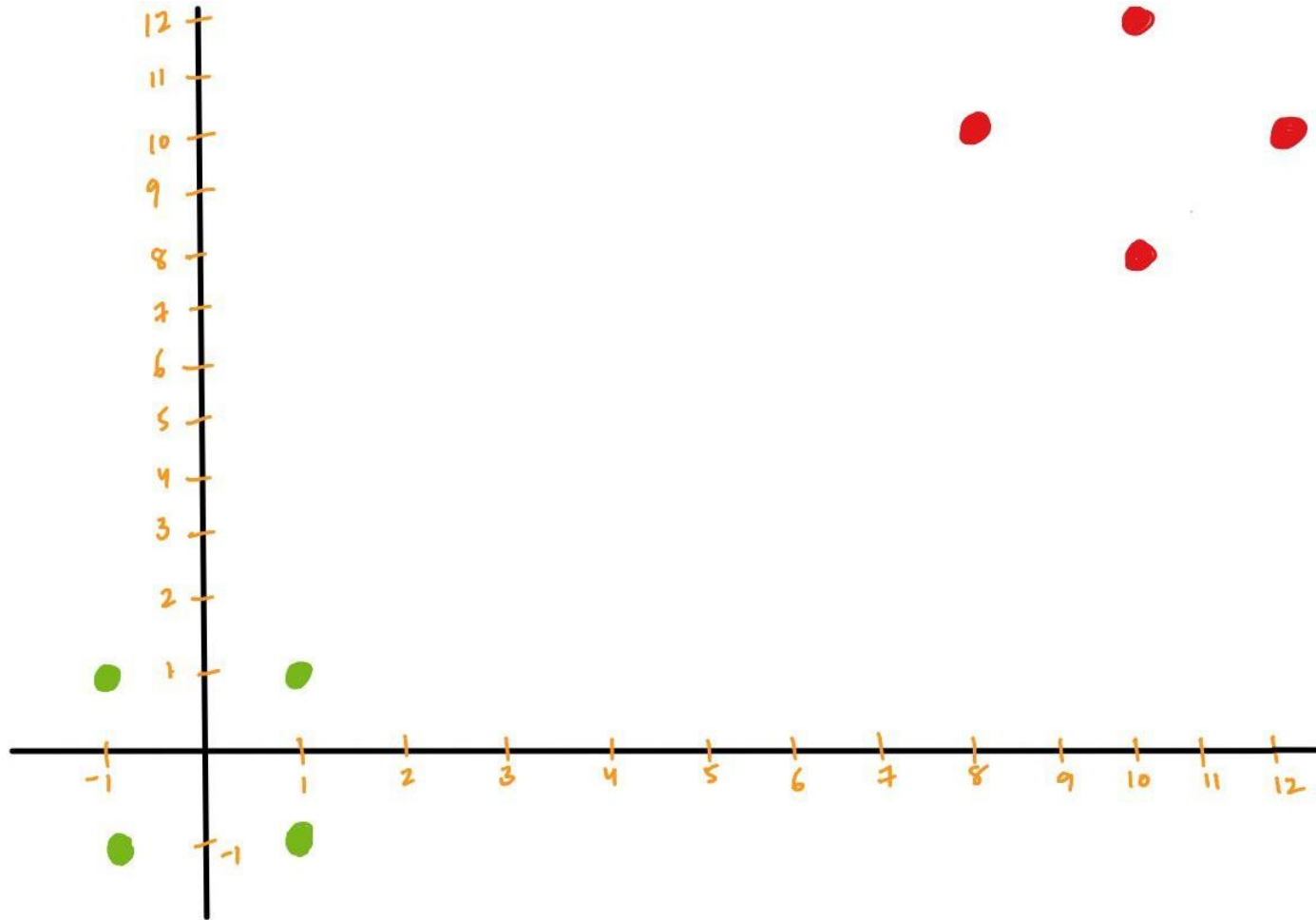
$$\phi(X) = \begin{cases} \begin{pmatrix} 6 - x_1 + (x_1 - x_2)^2 \\ 6 - x_2 + (x_1 - x_2)^2 \end{pmatrix} & \text{if } \sqrt{x_1^2 + x_2^2} \geq 2 \\ x_1, x_2 & \text{otherwise} \end{cases}$$

Positive points remain unchanged as

$$\sqrt{x_1^2 + x_2^2} \neq 2$$

## Unit 2 - SVM

### Problem 4 - Non - Linear Data



Transformed Data

Positive points :  $\begin{pmatrix} 1 \\ 1 \end{pmatrix} \begin{pmatrix} -1 \\ 1 \end{pmatrix} \begin{pmatrix} -1 \\ -1 \end{pmatrix} \begin{pmatrix} 1 \\ -1 \end{pmatrix}$

Negative points :  $\begin{pmatrix} 8 \\ 10 \end{pmatrix} \begin{pmatrix} 10 \\ 8 \end{pmatrix} \begin{pmatrix} 12 \\ 10 \end{pmatrix} \begin{pmatrix} 10 \\ 12 \end{pmatrix}$



Either solve using Way 1 or Way 2, you should get the following values

$$\alpha_1 = \frac{1}{64}$$

$$\alpha_2 = \frac{1}{128}$$

$$\alpha_3 = \frac{1}{128}$$

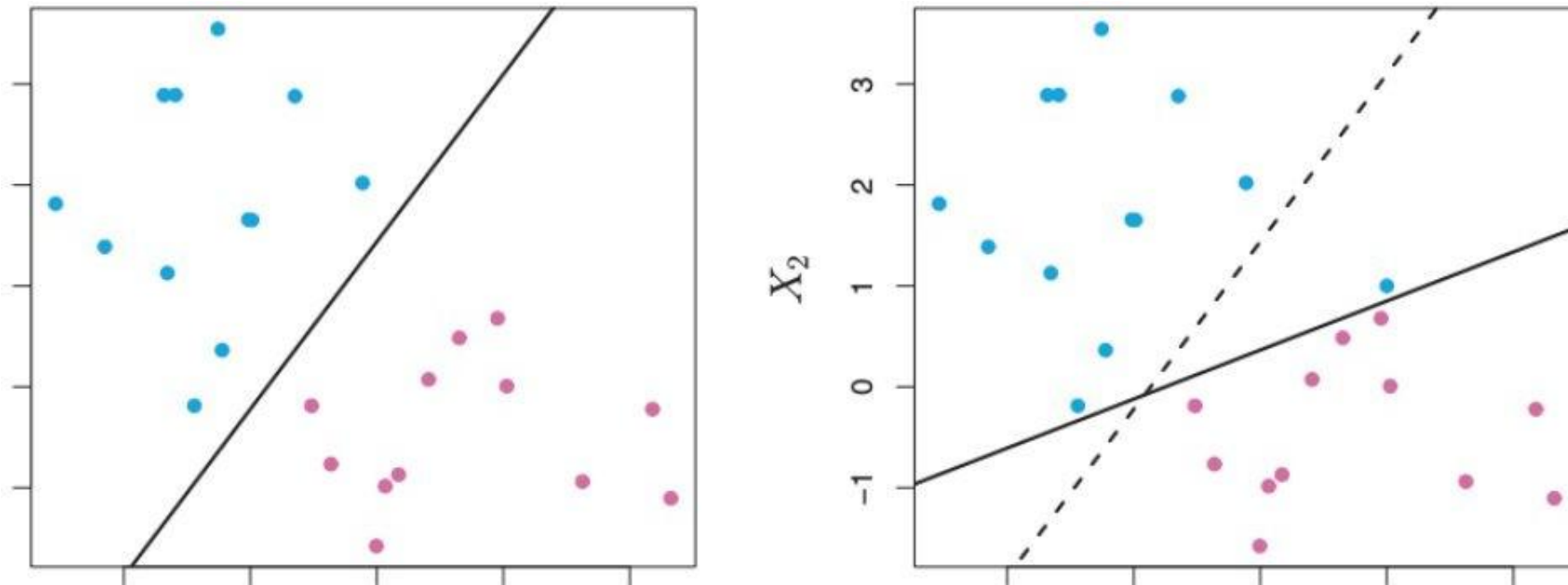
$$w = \begin{bmatrix} -\frac{1}{8} \\ -\frac{1}{8} \end{bmatrix}$$

$$b = \frac{5}{4}$$



### Hard Margin SVM is sensitive to Outliers

Example where one outlier moves the hard-margin SVM decision boundary a lot.



Solution : Allow some points to violate the margin

### Hard Margin SVM is sensitive to Outliers/Noise

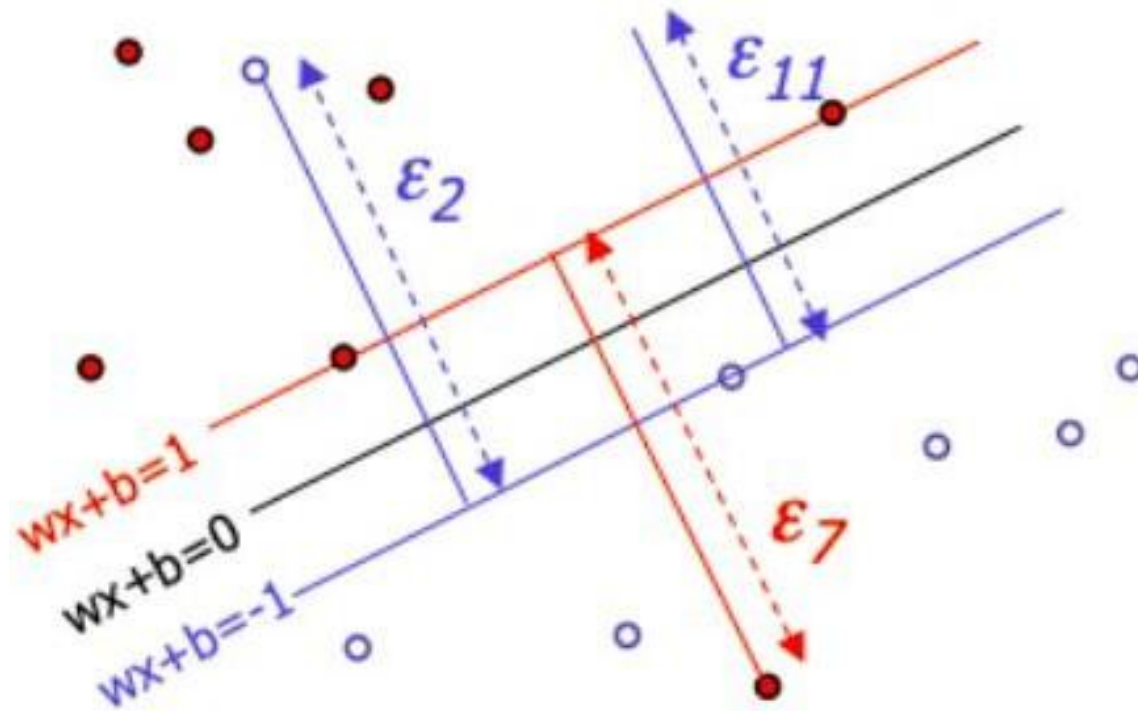
---

Hard Margin SVM requires that all data points are classified correctly  
What if the training set is noisy?

**OVERFITTING !!**

### Hard Margin SVM is sensitive to Outliers/Noise

Slack variables  $\epsilon_i$  can be added to allow misclassification of difficult or noisy examples.



Soft Margin SVM  
(out of scope)

**Support vector guideline 1:** To see if a point  $\vec{x}_j$  is a support vector, imagine deleting it and see if you would draw a different SVM boundary. If you would draw a different SVM boundary, the point is a support vector ( $\alpha_j > 0$ ). If you would draw the same boundary, even if the point were deleted, the point isn't a support vector ( $\alpha_j = 0$ ).

**Support vector guideline 2:**

- Only points on the gutter can be support vectors; if a point isn't on the gutter, it's not a support vector.
- If a point is on the gutter, it *might or might not* be a support vector—you can determine whether it is using Support vector guideline 1.

**Support vector guideline 3:** You may find it useful to think of the  $\alpha_i$  as measuring “supportiveness”. This means, for example, that:

- $\alpha_i$  is zero for non-support vectors, i.e. training points that do not determine the boundary, and which would not affect the placement of the boundary if deleted.
- When you compare two separate SVM problems, where the first has support vectors that are far from the boundary, and the second has support vectors very close to the boundary, the latter support vectors have comparatively higher  $\alpha_i$  values.

Few project idea ...

- Simple numerical data classification -Predicting quality of wine ([DATASET](#))
- MASK DETECTION - Detect if a given picture contain face with mask or no mask
- Text classification



## **UE24CS352A: Machine Learning**

---

**Dr. Shylaja S S**

Department of Computer Science & Engineering

[shylaja.sharath@pes.edu](mailto:shylaja.sharath@pes.edu)



## UE24CS352A: Machine Learning

---

**Dr. Shylaja S S**

Professor

Department of Computer Science & Engineering

Acknowledgement : Dr. Rajanikanth K (Academic Advisor, PESU)  
Teaching Assistant (Arya Rajiv Chaloli)



- In **hard-margin SVM**, the model tries to find a hyperplane that perfectly separates the classes with the maximum margin.
- But in real-world data:
  - Classes may **not be perfectly separable** (overlaps, noise, outliers).
  - A hard-margin constraint (no misclassification allowed) is too strict.

This leads to the idea of the **soft margin**.

- Instead of requiring all points to be classified correctly, SVM allows **some violations of the margin**.
- These violations are controlled by **slack variables**  $\xi_i$  one for each data point.
  - $\xi_i = 0$  → Point is correctly classified and outside the margin.
  - $0 < \xi_i < 1$  → Point is inside the margin but on the correct side.
  - $\xi_i > 1$  → Point is misclassified.

### Objective Function in Soft SVM:

Let us see how we can modify our objective to achieve the desired behavior. In this new setting, we would aim to minimize the following objective:

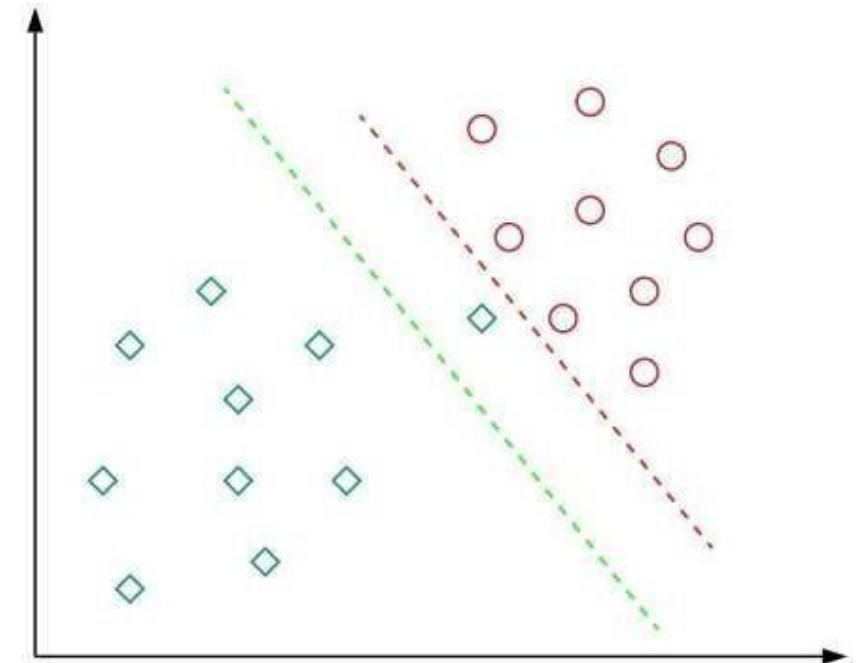
$$L = \frac{1}{2} \|w\|^2 + C(\# \text{ of mistakes})$$

where **C** is a Regularization Parameter

This differs from the original objective in the second term.

Here, **C** is a hyperparameter that decides the trade-off between maximizing the margin and minimizing the mistakes.

When **C** is small, classification mistakes are given less importance and focus is more on maximizing the margin, whereas when **C** is large, the focus is more on avoiding misclassification at the expense of keeping the margin small.



Note : We will discuss more about the parameter C little later.

### Slack Variables in Soft Margin SVM:

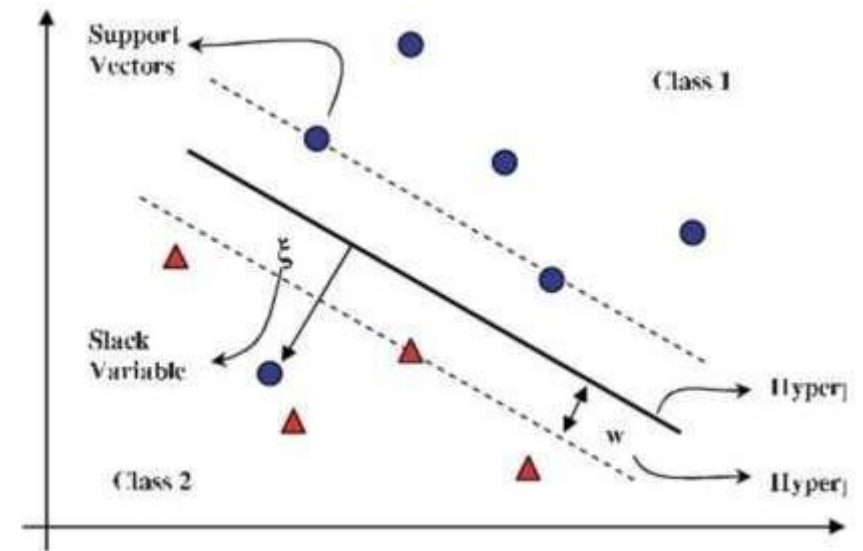
Soft SVM relaxes the linear separability constraint by introducing **slack variables** in the constraint equations .

$$y_i(\vec{w} \cdot \vec{x}_i + b) \geq 1 - \xi_i$$

Here, the left-hand side of the inequality could be thought of like the confidence of classification.

Confidence score  $\geq 1$  suggests that classifier has classified the point correctly.

However, if confidence score  $< 1$ , it means that classifier did not classify the point correctly and incurring a linear penalty of  $\xi_i$ .



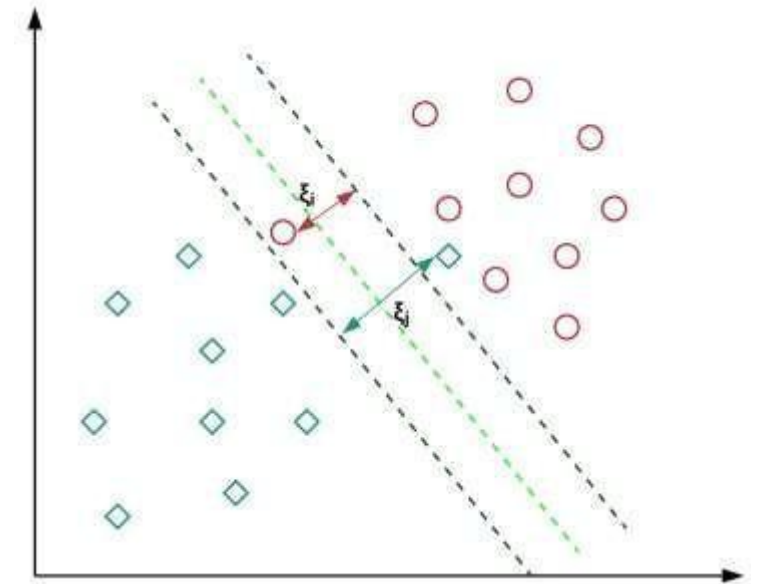
### More about the slack variables in Soft Margin SVM:

$$L = \frac{1}{2} \|\mathbf{w}\|^2 + C(\# \text{ of mistakes})$$

Note: Not all mistakes are equal. Data points that are far away on the wrong side of the decision boundary should incur more penalty as compared to the ones that are closer.

Let's see how this could be incorporated with the help of the diagram.

$$y_i(\vec{w} \cdot \vec{x}_i + b) \geq 1 - \xi_i$$

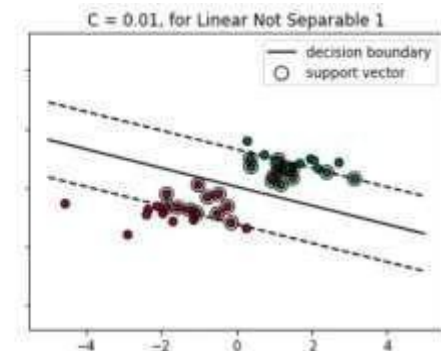
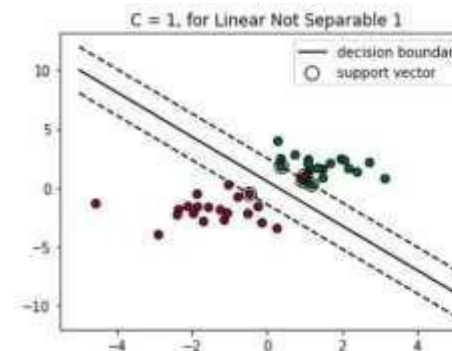
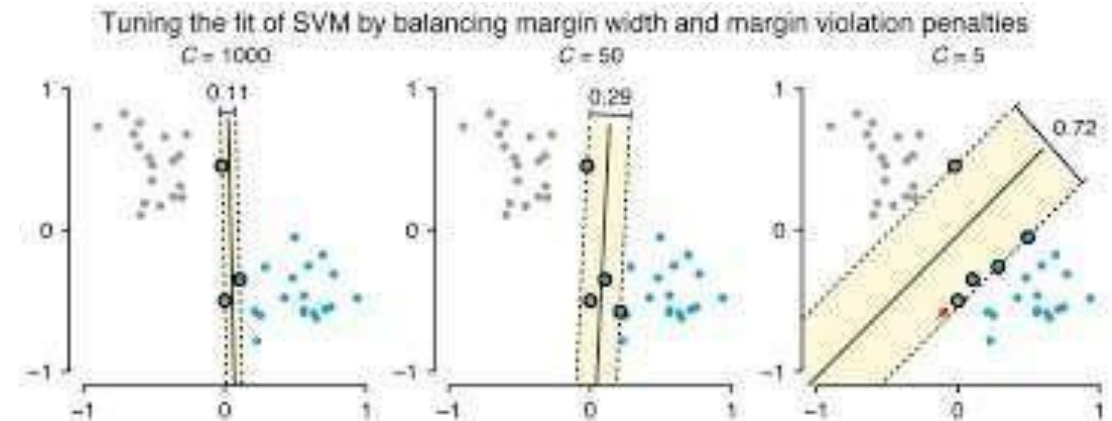


### Relation between Regularization parameter (C) and SVM:

As the value of C increases the margin decreases thus Hard SVM.

– If the values of C are very small the margin increases thus Soft SVM.

– Large value of C can cause overfitting therefore we need to select the correct value using Hyperparameter Tuning.



The primal problem is:

$$\text{Min}_{w, b} \quad \frac{1}{2} \|w\|^2 + C \sum_{i=1}^m \xi_i$$

s. t. c.

$$y_i (w^T x_i + b) \geq 1 - \xi_i \quad \forall (x_i, y_i)$$

$$\text{and } \xi_i \geq 0$$

Writing the corresponding Lagrangian function by introducing two lagrangian multipliers  $\alpha_i$  and  $\beta_i$  one per constraint, we get

$$\text{Min}_{w, b, \xi} L = \frac{1}{2} w \cdot w + C \sum_{i=1}^m \xi_i - \sum_{i=1}^m \alpha_i [y_i (w^T x_i + b) - 1 + \xi_i] - \sum_{i=1}^m \beta_i \xi_i$$



Minimizing of  $L$  w.r.to  $w, b, \xi$  is same as Maximizing  
w.r.to.  $\alpha$  and  $\beta$ .

$$(a) \quad \min_{w, b, \xi} \quad \max_{\alpha \geq 0, \beta \geq 0} \quad L(w, b, \xi, \alpha, \beta)$$

Subject to the KKT conditions for the solutions are given as:

1.  $\nabla_w L = 0, \nabla_b L = 0, \nabla_{\xi_i} L = 0$  (Stationary condition)
2.  $y_i(w^T x_i + b) - 1 + \xi_i \geq 0$  (Primal Feasibility condition)
3.  $\alpha_i (y_i (w^T x_i + b) - 1 + \xi_i) = 0$  (Complementary Slackness condition)
4.  $\alpha_i \geq 0, \beta_i \geq 0$  (Dual Feasibility condition)
5.  $y_i (w^T x_i + b) - 1 + \xi_i = 0$
6.  $\beta_i \xi_i = 0$



## **UE24CS352A: Machine Learning**

---

**Dr. Shylaja S S**

Department of Computer Science & Engineering

[shylaja.sharath@pes.edu](mailto:shylaja.sharath@pes.edu)